

Agent Systems

Lucas Kim

English Edition

August 2026

Preface

This book is about designing, building, evaluating, and running agent systems on top of large language models.

The last few years have buried this field in material. Most of it sits at one of two ends.

At one end is writing that teaches a particular framework. It gets something running fast, and it goes stale the day the framework changes. At the other end are research papers, each proposing one technique. They are exact, and they do not tell you how the whole system goes together.

What belongs between them is missing. *Why you need a given part, when not to use it, and what you pay when you choose it.* This book tries to fill that place.

Who this is for

This book is for readers who want the shape of an agent system whole. It assumes you have called a language model API yourself at least once. Three facts are enough. A call does not remember the call before it, so you resend the entire history every time. You are billed for what you send and for what comes back. And when you hand the model a list of functions, the model only picks a name and fills in arguments. Running the function is your code's job.

Reading Python helps, but there is no code in this book. It leans on machine learning pipeline analogies now and then, and you do not need those deeply either.

What this book holds to

1. **Concepts before frameworks.** Put the concept up first and you will recognize it in any library, under whatever name that library gave it. Memorize names first and nothing survives the next framework. The last chapter walks through real tools, and not to compare features. It asks *which of the decisions built up earlier in this book that tool has already made for you.*
2. **Every choice has a price, and the price is written down.** When a technique appears, what it improves and what it adds appear with it. Each chapter closes with a section on the conditions under which you should *not* use it.

I hope this helps anyone trying to see agent systems whole.

The Map of This Book

This book is about one system. The model puts out an action, the environment hands back an observation, and an assembly step takes that observation along with the other ingredients and builds the next context to feed the model. That is the cycle. Outside it sits the control that owns it.

There are a handful of places in this structure to take charge of, and the seven parts take one each. They do not all sit on the same level. Some are inside the cycle. Some stand off to the side, facing the whole of it.

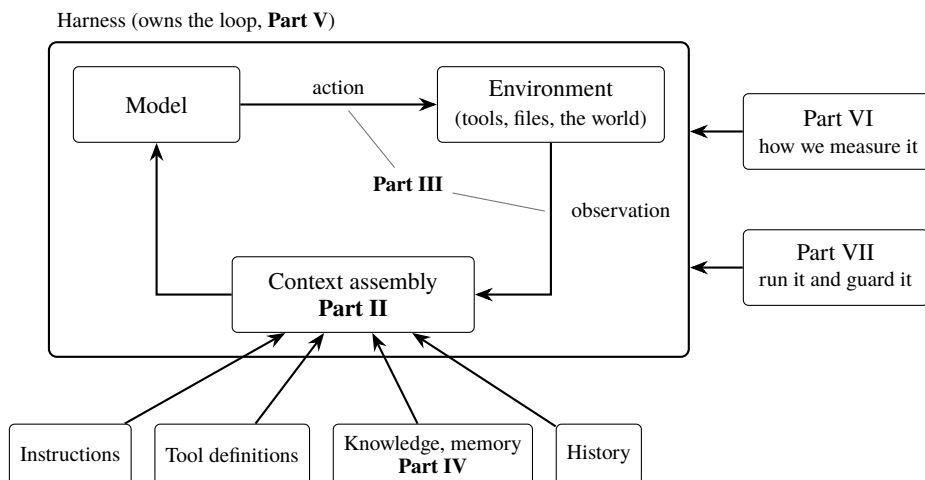
The figure below is that structure with the parts laid on top of it.

On a first pass, read the parts in order. Each part uses what the part before it set up.

That order is a reading order, though, not the system's order. No stage waits for the one before it to finish. All of this is alive at the same time during a run, and each piece has hold of the others.

So when a problem shows up at work, do not start over from page one. Work out which part the problem belongs to and open there. The table at the end of this map is what helps you do that, and its last column lists the symptoms.

It is fine if the figure means little right now. Chapter 1 builds this structure in its own place (Figure 1.1), and every chapter after it speaks on top of the structure. Read that chapter, come back here, and the same figure will stand up again.



Four things about the figure.

First, the part numbers are a reading order, not an execution order. The size and position of a box claim nothing about weight either. Neither do the two boxes standing off to the side. Evaluating the system and running and guarding it do not arrive after the system is up. They face the whole thing from the start, and the arrows are that direction.

Second, the four boxes under assembly are ingredients. Only one of them, the history, does not come from outside. The cycle piles that one up itself.

Third, the passage and the harness are different parts. The action going out and the result coming back are Part III. The harness that owns the cycle from outside is Part V. The ingredients stand first and the mechanism that turns them comes later. Only once instructions and tools and knowledge are all standing can you say what owning the loop owns.

Fourth, one part is not in the figure. Part I does not belong to any single box. It builds the structure itself, so the whole figure is its place. The cycle and the shape of the loop both stand up there. Six parts stand in the figure, and the first part puts the figure up.

The last chapter, Chapter 23, belongs to Part VII but has a different character. It does not take charge of one more place in this structure. It points at the things people actually use at work and asks which parts of this map those things fill in for you. *The first twenty-two chapters draw the map. The last one carries the map out in front of the real thing.*

Part	What it takes charge of, and its question	Symptoms that send you here
Part I What is an agent	The whole structure and the shape of the loop. What counts as an agent, who decides the next step, and how that decision changes the shape of the loop	You cannot name what you are building. You stood up an agent where a workflow would have done. You cannot guess how many steps the loop will run
Part II Working with context	The unit of the budget and the inside of assembly. What one token is and what length costs, and what to load into a context that is finite and gets worse as it grows, when to load it, and with how much certainty	Cost and latency grow faster than the task does. Answers get worse as the context gets longer. Instructions you put in go unfollowed. You changed models and failures appeared that were not there before
Part III Tools and the outside world	The passage out to the environment. Tools as the vocabulary of action, what they return, the route that supplies capability, and the shape of the output. What you let the model act with, and what makes what comes back worth trusting	The model picks the wrong tool or invents a usage that does not exist. Adding tools makes the picking worse. The shape of the output keeps breaking. The tool reports success and the answer is still wrong
Part IV Knowledge and memory	Knowledge, as one of the ingredients of assembly. The mechanism that finds what you left outside and brings it in, and memory that crosses sessions. How you tell the model the things it does not know and the things it knows out of date	It states a stale value with confidence. It cannot find an answer that is in the documents. You retrieved it and loaded it and the answer still came from somewhere else. The answer is wrong and you cannot point at which piece of knowledge was wrong

Part	What it takes charge of, and its question	Symptoms that send you here
Part V Running the loop	The control that owns the cycle from outside. When to stop, what to leave behind, how to start again, and what to trust. And when there is more than one loop, when to divide and what joins the divided pieces	The loop does not stop, or it stops oddly early. A dropped session means starting over. You hit a task that does not fit in one context. The seats you divided work on different assumptions
Part VI Evaluation and observability	Evaluating this system. How you know it got better, and what a run has to leave behind for you to look at later	You changed something and cannot judge whether it improved. Failures do not reproduce. There is no record to trace a wrong answer back through
Part VII Operations, safety, governance	Running this and guarding it. Who stands in front of an action you cannot undo, how you take in text that came from outside, and how much of this is already built	You are nervous about putting it in front of real users. Text from outside acts like an instruction. You have to decide how much to permit. You have to split what you build from what you take off the shelf

One note on citations. Citations in the text give author and year, and primary sources come first. Where the source is an industry document, the publisher and the date are given with it. *This field moves fast, so read the details of any particular product or specification as belonging to the moment it was cited.*

Contents

Preface	iii
The Map of This Book	v
I What Is an Agent	1
1 What Is an Agent	3
1.1 The classical definition: a cycle of perceiving and acting	3
1.2 Why now	3
1.3 Workflows and agents	4
1.4 What an agent is made of	6
1.5 What the loop is made of	7
1.6 When not to use an agent	8
1.7 How this book is laid out	8
1.8 Summary	10
1.9 Exercises	10
1.10 On a second reading	11
1.11 Further reading	11
2 A Map of Reasoning and Planning Patterns	13
2.1 What patterns are there	13
2.2 The decision procedure	15
2.3 How patterns fail	16
2.4 When the cause sits outside the pattern	18
2.5 Summary	20
2.6 Exercises	21
2.7 On a second reading	21
2.8 Further reading	21
II Working with Context	23
3 Tokens: The Unit of the Budget	25
3.1 Tokenization: what gets counted	25
3.2 The same content costs differently in different languages	25
3.3 What this chapter leaves out	26
3.4 Summary	26
3.5 Exercises	27
3.6 On a second reading	27

3.7	Further reading	28
4	Context Engineering	29
4.1	Inside the assembly function	29
4.2	What to load	30
4.3	When to load it	31
4.4	Why not just put everything in	32
4.5	Summary	35
4.6	Exercises	36
4.7	On a second reading	36
4.8	Further reading	36
5	Prompt Engineering	39
5.1	Where the prompt sits	39
5.2	What the instructions do	39
5.3	Prompts are not robust	41
5.4	When not to edit the prompt again	43
5.5	Summary	45
5.6	Exercises	45
5.7	On a second reading	46
5.8	Further reading	46
6	Compaction and Summarization	49
6.1	The layer that grows	49
6.2	The two purposes of compaction	50
6.3	What can be dropped	51
6.4	The methods of compaction	52
6.5	Compaction is not free	54
6.6	The problem with repeated compaction	56
6.7	When not to compact	57
6.8	Summary	58
6.9	Exercises	59
6.10	On a second reading	59
6.11	Further reading	60
III	Tools and the Outside World	61
7	Tool Design	63
7.1	The anatomy of a tool	63
7.2	The name and the description set the performance	65
7.3	Designing the tool itself	66
7.4	Tool output inside the context	70
7.5	When not to build another tool	74
7.6	Summary	76
7.7	Exercises	77
7.8	On a second reading	77
7.9	Further reading	78

8	The Model Context Protocol (MCP)	79
8.1	Connecting to the outside	79
8.2	MCP	80
8.3	What MCP touches	82
8.4	Designing with MCP	83
8.5	When not to use the protocol	85
8.6	Summary	88
8.7	Exercises	89
8.8	On a second reading	89
8.9	Further reading	90
9	Structured Output and Schema Design	91
9.1	Why fix a shape	91
9.2	What a schema does	92
9.3	How tight to make it	94
9.4	When the model breaks the shape	96
9.5	What a schema cannot cover	98
9.6	Summary	99
9.7	Exercises	101
9.8	On a second reading	101
9.9	Further reading	101
IV	Knowledge and Memory	103
10	Agent Memory: Short-Term, Long-Term, Persistent	105
10.1	What has to be left behind	105
10.2	Three tiers	106
10.3	The moment it gets used	108
10.4	Memory that accumulates wrongly	109
10.5	When not to keep memory	112
10.6	Summary	114
10.7	Exercises	115
10.8	On a second reading	115
10.9	Further reading	116
11	Retrieval-Augmented Generation (RAG)	117
11.1	Using material from outside	117
11.2	What changes when you keep it outside	118
11.3	Retrieval as a step	120
11.4	Chunking, embeddings, vector search	121
11.5	When retrieval fails	127
11.6	What retrieval cannot solve	130
11.7	Summary	131
11.8	Exercises	132
11.9	On a second reading	132
11.10	Further reading	133

12 Hybrid Retrieval	135
12.1 Why overlap two retrievals	135
12.2 Two lists into one	136
12.3 Reranking: standing the candidates up again	137
12.4 Contextual retrieval: attach the context while cutting	139
12.5 What it means for the stages to grow	140
12.6 What stacking stages does not fix	141
12.7 Summary	143
12.8 Exercises	144
12.9 On a second reading	145
12.10 Further reading	145
13 Ontologies and Knowledge Graphs	147
13.1 Problems retrieval does not solve	147
13.2 Knowledge as a graph	148
13.3 Designing the ontology	150
13.4 When to build one	152
13.5 What graphs and ontologies do not solve	153
13.6 Summary	155
13.7 Exercises	156
13.8 On a second reading	156
13.9 Further reading	157
14 When the Document Is Not Text	159
14.1 The assumption that the corpus is text	159
14.2 When it is not text	159
14.3 Working with material that is not text	161
14.4 Other material	163
14.5 Summary	165
14.6 Exercises	166
14.7 On a second reading	166
14.8 Further reading	167
V Running the Loop	169
15 Designing the Agent Harness	171
15.1 The Harness: How the Loop Turns	171
15.2 State: What to Leave Behind	172
15.3 Termination: When to Stop	174
15.4 Restart: How to Begin Again	177
15.5 Verification: What to Trust	178
15.6 When Not to Build More Harness	179
15.7 Summary	182
15.8 Exercises	183
15.9 On a Second Reading	183
15.10 Further Reading	184

16 One Agent or Many	185
16.1 Running a Single Agent	185
16.2 When One Agent Is Not Enough	186
16.3 Multi-Agent	187
16.4 How It Lands in Practice	190
16.5 Summary	192
16.6 Exercises	193
16.7 On a Second Reading	193
16.8 Further Reading	194
17 Communication Between Agents	195
17.1 How Two Agents Interact	195
17.2 What Gets Passed	196
17.3 How They Communicate	197
17.4 Among Parties That Do Not Know Each Other: A2A	199
17.5 What to Watch When Designing	201
17.6 Summary	205
17.7 Exercises	206
17.8 On a Second Reading	206
17.9 Further Reading	206
VI Evaluation and Observability	209
18 How to Evaluate an Agent	211
18.1 After the Design	211
18.2 What Makes an Agent Good	211
18.3 What Gets Counted	212
18.4 How You Run It	213
18.5 Real Cases	216
18.6 Evaluation Done Wrong	218
18.7 Summary	220
18.8 Exercises	221
18.9 On a Second Reading	221
18.10 Further Reading	221
19 Designing Evaluation Metrics	223
19.1 There Is No Standard Metric	223
19.2 Designing a Metric	224
19.3 Metrics in the Wild	226
19.4 Metric Design Done Wrong	228
19.5 Summary	231
19.6 Exercises	232
19.7 On a Second Reading	232
19.8 Further Reading	232

20	Tracing and Observability	235
20.1	Why You Have to Keep It	235
20.2	What to Keep and How	236
20.3	What You Still Cannot Know	237
20.4	How It Is Done in Practice	239
20.5	What Keeping Costs and What Not to Keep	241
20.6	Summary	243
20.7	Exercises	244
20.8	On a Second Reading	244
20.9	Further Reading	244
VII	Operations, Safety, Governance	247
21	Human Approval and Permission	249
21.1	Where to Stand a Person	249
21.2	Ask the Operator	250
21.3	Find What Cannot Be Undone	251
21.4	How Permission Gets Divided	253
21.5	What Remains After the Approval Is Built	254
21.6	Summary	257
21.7	Exercises	258
21.8	On a Second Reading	258
22	Prompt Injection and Guardrails	261
22.1	Where It Comes In	261
22.2	What It Is After	263
22.3	How Guardrails Get Built	263
22.4	How to Confirm You Can Be Broken Through	265
22.5	What Remains After You Block	267
22.6	Summary	269
22.7	Exercises	270
22.8	On a Second Reading	270
22.9	Further Reading	271
23	Going Further	273
23.1	Questions to Think Through Yourself	273
23.2	OpenClaw	274
23.3	Open-Source Frameworks	277
23.4	Agents Run Locally	279
23.5	What to Watch From Here	281
23.6	Closing	282
	Bibliography	283

PART I

What Is an Agent

What Is an Agent

Agent design turns on one decision. *Does the code decide what happens next, or does the model?* That single decision fixes cost, latency, the shape of your failures, and how you evaluate, all at once.

The word *agent* covers that decision instead of showing it. A chatbot that summarizes search results gets the name. So does a system that spends days editing a codebase. Knowing that a system is an agent tells you nothing about what it costs, how it fails, or what you should measure it with.

Is this system an agent is a question you can answer and be no better off. *Does the code decide the next step, or does the model* is a question you answer by opening the system up, and the answer changes your budget and your verification.

1.1 The classical definition: a cycle of perceiving and acting

The idea of an agent is far older than language models. Working in distributed artificial intelligence in the 1990s, Wooldridge and Jennings (1995) listed four properties: autonomy, social ability, reactivity, and pro-activeness. Two of them keep working through this book. Autonomy narrows to one question, *who decides what happens next*. Reactivity survives as the shape of a cycle, *what the environment gives back becomes the next input*.

The standard textbook by Russell and Norvig (2020) puts that cycle compactly. An agent is anything that perceives its environment through sensors and acts on that environment through actuators, and its behavior is described by a function from the percept sequence to an action. The definition never asks what the thing is made of, so a language model drops into the frame unchanged.

Write T for the number of times the cycle repeats and call it the stopping time. The name for each place in the cycle comes in Section 1.5.

1.2 Why now

If the idea is thirty years old, why did it only become practical recently? Three capabilities arrived, and then one condition. They are also the four places to look when the system you are building falls apart.

The first is **putting the intermediate reasoning outside the model**. Wei et al. (2022) showed that prompting a model to produce intermediate reasoning steps improves performance on arithmetic, commonsense, and symbolic reasoning tasks. What matters here is not the accuracy. It is that the model's plan comes out as text. Without it you can hold a wrong answer in your hand and have no place to point at where it went wrong.

The second is **putting reasoning and acting together**. ReAct (Yao et al., 2023b) has the model generate reasoning traces and actions in an interleaved way. The reasoning updates the plan for acting, and the acting fetches the information the reasoning needs. Without it, a plan is made once at the start and that is the end of it. An observation that arrives mid-run cannot correct the plan. This is the point where the cycle in Section 1.1 actually closes. Most agent loops in use today are a variant of ReAct or a reaction against it (Chapter 2).

The third is **tool calling you can rely on**. Schick et al. (2023) showed that a model can teach itself which API to call, when to call it, and what arguments to pass. After that, tool calling stopped being a prompting trick and became a basic feature of models and their APIs. The space of actions moved from free text to a set of structured calls. Without it you fail at the very first job, which is working out what the model is trying to do.

The fourth is not a capability. It is a condition. Context windows got wide, and the window limit is no longer the first constraint you hit. The condition comes with a trap attached. A wider window lets you hold a longer history, and it also tempts you into a design that just puts everything in. So the shape of failure moved. It used to fail because things did not fit. Now they fit and the quality falls off. Chapter 4 takes up why.

1.3 Workflows and agents

The frame in Section 1.1 is still too wide. Drop a fixed two-step pipeline into it and the pipeline is an agent.

The most useful line comes from Anthropic (2024a), and it turns on *who owns the control flow*. When the path of execution is written into code ahead of time and the model is only a part that generates text at each step, you have a *workflow*, or in that document’s words, “systems where LLMs and tools are orchestrated through predefined code paths.” Each step’s action is fixed to one thing, and the stopping time T is a constant that does not depend on the input. When the model decides what to do next and when to stop, and the code carries those decisions out, you have an *agent*: “systems where LLMs dynamically direct their own processes and tool usage.” The action is one the model picks out of the tool set $\mathcal{T}$, and T is set during the run by the stopping rule ρ .

Why ownership? Splitting on intelligence or on task complexity looks more natural, and neither draws a line. The same model serves as a part inside a workflow and as the policy of an agent, so *how smart is it* does not separate the two structures. A complex task yields to an elaborate workflow and to a simple agent alike. Ownership is different. Whether the code or the model picks the next step is settled on the spot, by opening the system up.

What actually splits here is predictability. A workflow’s call count is not identical on every run either. Retries get added, a rate limit makes it wait, a step that fails a check runs again. The spread is far narrower, so you can estimate cost and latency in advance, and the failure points are fixed, which makes debugging easy. An agent gives up that narrow spread and gets back the paths you could not enumerate ahead of time.

Workflows have names too

Call a workflow *the thing that is not an agent* and you cannot design with it. The shapes that keep showing up in practice each have a name, and the names are your options. Here are the five from Anthropic (2024a), laid over the job of handling one customer inquiry.

- **Prompt chaining.** Break the task into a fixed sequence of steps, each call taking the output of the one before. You can slip a check written in code between the steps. Use it when the task “can be easily and cleanly decomposed into fixed subtasks.” Three steps here: pull out what was asked for, draft the reply, then rewrite the draft in the house voice.
- **Routing.** Classify the input and send it to a specialized followup. This keeps one prompt from collapsing under the weight of every case at once. It works “for complex tasks where there are distinct categories that are better handled separately.” The place where refunds first get split from technical problems.
- **Parallelization.** *When you know in advance what to split into*, split it, run the pieces at the same time, and put the results together in code. Draft the reply, judge the urgency, and check for a policy violation all in one go.
- **Orchestrator-workers.** *When you cannot tell how many branches there are until you look*, a central call does the splitting and gathers the results. Three tangled orders make three branches, one makes one. Use it for “complex tasks where you can’t predict the subtasks needed.”
- **Evaluator-optimizer.** One call produces and another evaluates and sends it back. It only works when you can write down in words what a good result is. Use it when you have “clear evaluation criteria” and iterative refinement “provides measurable value.” The place where you check whether the required policy wording was left out and send it back.

What separates the third from the fourth is not scale but timing. Reply, urgency, and the policy check can be written down as three before you open the inquiry. How many tangled orders there are is something you learn by opening it.

These five are what you try before reaching for an agent. Only when none of the five solves the problem do you have a reason to hand the control flow to the model. The same source puts the rule plainly: find the simplest solution possible, and only increase complexity when needed.

The rungs in between

Real systems sit somewhere between the two ends, because there is more than one decision you can hand over. What the output says. Which branch to take. Which tool, with which arguments. When to stop. Whether to spawn another agent. You can hand these over one at a time, and each one moves the system a rung toward agent. Hold them all and you have a single call. Hand them all over and you have multiple agents. The space between is not empty.

Table 1.1 is those rungs. Going down, the model owns one more decision at each row, and the right column sums up the price in T . Knowing the stopping time in advance is the same thing as knowing the cost in advance.

The spread in the right column is felt as wobble. As long as the action is a draw, the same task finishes in 3 steps on one run and 30 on another. Going down, that range widens, which is why you budget with a distribution and not an average.

Example 1.1 (One task, two implementations). A system that finds answers in internal documents. The workflow implementation embeds the query, retrieves the top few chunks, packs those chunks and the query into one prompt, and calls the model once. $T = 1$, and the cost is very nearly constant.

The agent implementation hands the model a search tool, lets it write its own query, lets it reword the query and search again when the results come back thin, and lets it write the answer when it judges it has enough. T is 1 on some runs and 8 on others.

Table 1.1: The spectrum of autonomy. Going down, the model owns more decisions, and the spread in cost and the burden of verification grow with it.

Level	What the model decides	What T is like
Single call	Only what the output says	$T = 1$
Fixed chain	What each step outputs	T constant
Routing	Which branch to take	T constant, path varies
Tool-use loop	Which tool to call, with which arguments	T varies, capped
Planning agent	How to break the task down, and in what order	T varies, wide spread
Multiple agents	Which agent to spawn or end, and when	T varies, very wide spread

The second is clearly better on questions that take several documents to answer, and on questions where you only reach the right document by rewording the query. It also spends several times the tokens on questions the first one already handles. So the design question is not *which one is better*. It is *what percentage of my query distribution needs the second one*.

1.4 What an agent is made of

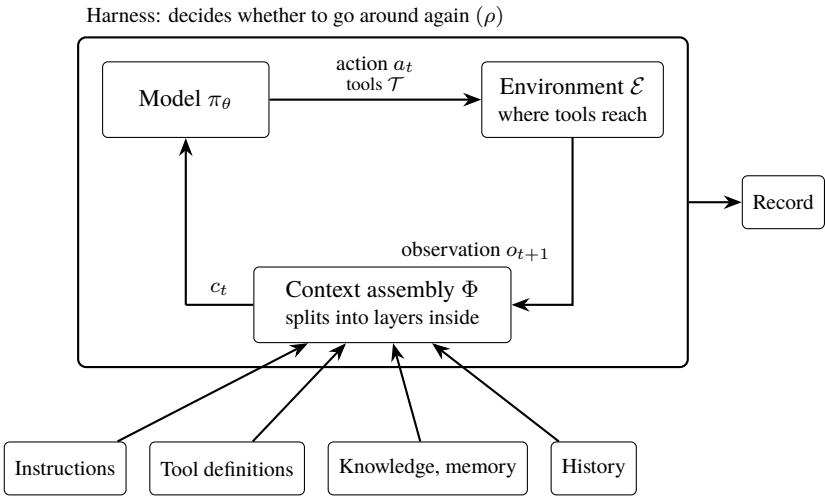


Figure 1.1: The places in an agent system. Model and environment turn in a cycle, and what rides in that cycle is settled by assembly, which takes in the four below. Two things sit outside the cycle: the harness, which decides whether to go around again, and the record, which goes out to people.

Some of the places in Figure 1.1 are not boxes. The arrow going out to the environment, the arrow coming back to the model, the label on the harness box. Count those places too and there are six, and the six take one part of this book each.

Take them away one at a time and you see why there are six. With nothing drawing the action there is no system at all. With a poor view of things, every step draws with its eyes shut.

With no passage out, the system only talks. With no stopping rule, it does not stop. Throw away the state that lives outside and every session ends by starting over. Leave nothing behind and you cannot tell which of the six broke.

1. **The model** π_θ . The policy that produces the action. Chapter 3 opens this box as far as design needs and no further. The cost structure is what matters most.
2. **The context** c_t . Everything the model sees on a step. It is finite, it is expensive, and it gets worse as it gets longer (Chapter 4). The note in the figure that the assembly box splits into layers points at Part II, which handles those layers.
3. **The tools** $\mathcal{T}$. The point is that this is a passage and not a box. It is the only way the model acts on the environment, so the names of the tools and the shape of their arguments become, in effect, the model of action for that domain. Get this design wrong and no prompt recovers it (Part III).
4. **The loop and the stopping condition** ρ . When to stop. The most common accident in practice is not a wrong answer. It is *not stopping*, and it is the central subject of harness design (Chapter 15).
5. **Memory**. State that stays outside the context and lasts across sessions. It is a different problem from the knowledge sitting in the same box (Chapter 10).
6. **The record**. Where you leave what you loaded, what you called, and what came back, on every step. None of the five above absorbs it. The model does not see it, so it is not context. The next run does not use it, so it is not memory. *You leave it for a person to read later.*

1.5 What the loop is made of

Write down in form the places the last two sections pointed at in words, and one turn of the loop splits into four. The parenthesis at the end of each item is what Figure 1.1 calls that place.

1. **Thread everything that has happened so far into one line.** The history is a list of observations and actions stacked in alternation, and the model has exactly one entrance, a sequence of tokens. What to keep, what to drop, and in what order to lay it out are all settled here. (Context assembly Φ)
2. **Cut that line into tokens.** You do not get to pick the cutting rule. It comes attached the moment you pick a model. The same history gives a different token count under a different model. (Tokenizer enc)
3. **The model draws the next action.** A draw, not a value. Put the same thing in and nothing guarantees the same thing comes out. (Language model π_θ)
4. **The environment hands back the next observation.** This side is not a fixed value either. Send the same command and the network drops, the search results change. The environment carries the traces of earlier steps, so what comes back hangs on the whole history up to that point, not on the action you just took. (Environment $\mathcal{E}$)

The loop repeats these four until the stopping rule fires or it hits a step cap you set in advance.

The first two are kept apart because they have different owners. The model does not decide the action alone. The side that decides what to load, the tokenizer that cuts it, and the model that draws the next action decide it together. Of those three, only the first is our decision, and the other two come bundled with the single decision of which model to use. Lump the three

together and you cannot see why the same model yields two completely different agents. Ours does not mean do as you like, though. Part II shows how narrow the space around that decision really is.

The point. An agent's performance is a property of the system, not a property of the model. The room to improve is usually not in the model but in *the place that decides what goes into the context*.

Notation

Later chapters keep calling the names in Figure 1.1. Three get added here. The set of tools is T . Everything the model can pick out of those tools is the action space. The stopping rule is ρ .

Note that two items in the list above say *draw*. Any guarantee that running the same task twice gives the same step count, the same cost, and the same result disappears right there. Why the same content comes to a different token count in different languages is Chapter 3.

1.6 When not to use an agent

Knowing when not to use a technique is half of knowing it. If any one of the following four is *yes*, try a workflow before an agent.

1. **Can you write the path down on paper?** If the steps and branches fit in five lines, writing those five lines as code is always cheaper and steadier.
2. **Are there actions in the mix that you cannot undo?** The more autonomy, the higher the chance of running an action nobody reviewed beforehand. There are two things you can do here, and they are different decisions. Make the action impossible, or leave it possible and stand a person in front of it. The first lowers the ceiling on what the system can do. The second pays a cost on every run. Which to use where is Chapter 21.
3. **Is a user waiting in front of a screen?** In a conversational interface a multi-step loop passes what a user will sit through very quickly. How long that is depends on what you show them while they wait, so the question is not whether they are waiting but how long they can wait.
4. **Can you judge success automatically?** If you cannot judge it, you have no way to improve it. In that case the first thing to build is not an agent. It is an evaluation.

The fourth is ignored most often. Most agent projects that fail in practice do not fail because the model fell short. They fail because they started without saying what a good result is.

1.7 How this book is laid out

Part I is two chapters. This one puts up the structure, and Chapter 2 puts up the five axes you choose the shape of the loop along. That much answers what you are building.

The arithmetic of cost goes to Part II. Chapter 3 establishes that the token is the unit of the budget and that length is cost, and what to load into the context on top of that is the subject of

the other three chapters. Chapter 2 spends the results of that arithmetic ahead of time, so if you hit a place where you wonder where a value on an axis came from, you can open Chapter 3 first.

Each chapter is written to be read on its own, with Chapter 3 as the exception. The arithmetic of tokens and length is the ground under nearly every chapter after it, so read that one in order.

1.8 Summary

In one sentence. An agent is a system, not a model, and which decisions you hand to the model grows both the range of tasks the system can take on and the uncertainty in its cost.

What is now settled.

- The policy you actually run is Φ and enc and π_θ composed, and of the three, only Φ is our decision (Figure 1.1). The room to improve is mostly on that side, and Part II shows how narrow that place is.
- What separates a workflow from an agent is not intelligence but ownership of the control flow. Code fixes the choice of action and the stopping time, and you have a workflow. The model fixes them, and you have an agent.
- There are five workflows to try before going to an agent. Prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer.
- Autonomy is a spectrum and not a dichotomy, and as the model owns more decisions the spread in cost and the burden of verification grow together (Table 1.1).

What is still open.

- enc and the stopping rule ρ have only been named. Why the same content comes to a different token count in different languages goes to Chapter 3, and where the stopping rule belongs goes to Chapter 15.
- How fast the cost climbs as the loop runs longer has not been counted. That arithmetic and the cache go to Chapter 3, and the compaction and isolation that cut the accumulation itself go to Chapters 6 and 16.
- Quality falls off as the context gets longer, said without evidence here. Chapter 4 takes it up.
- No evaluation means no improvement, said here without saying how you build one. Part VI takes it up.

Try this today. Open the code that builds your next call and *look* at whether the branch is written in the code or read off the model’s output. If it is written in the code, most of the rest of this book is not needed yet (Section 1.3). Next, *point* at the row in Table 1.1 that matches what you are building now. The lower the row, the wider the range of step counts, and that range is the uncertainty in your budget. Last, *hold* the four in Section 1.6 against your own task. If even one comes back *yes*, try a workflow first.

1.9 Exercises

1. A router where the model picks one of three predefined branches: workflow or agent? Argue whether this borderline case is a flaw in the distinction or a middle ground worth having.
2. In the last two rows of Table 1.1, why does the spread in step count widen? To narrow it, which decision would you have to take back from the model?
3. For the two designs in Example 1.1, express the condition under which the agent imple-

mentation has higher expected utility than the workflow one, as a function of the query distribution and the unit price.

4. Of the four questions in Section 1.6, which do you think gets ignored most often in practice? Is the reason organizational or technical?

1.10 On a second reading

- Read Section 3.1, come back to Table 1.1, and you see why a spread in step count turns into money. Every step rereads the whole history, so each step you add costs more than the step before it.
- Read where Section 4.4 says performance falls before the ceiling, come back to *quality falls off as it gets longer*, and that phrase has turned into a measurable quantity called the effective context length. Quality has a limit on length of its own, apart from cost.
- Read Chapter 2, come back to Table 1.1, and each row, which was only a level, fills in with a concrete loop shape. Alternating reasoning and acting is the basic form of the tool-use loop, and planning before executing is a choice to move the spread forward to planning time.
- Read Chapter 15, come back to the stopping rule ρ , and what was written down as a name inside the loop turns out to be a decision outside it. That move is the harness's first decision.

1.11 Further reading

The classical formulations of the agent concept are in Wooldridge and Jennings (1995) and Russell and Norvig (2020). The latter gives the background in search, planning, and decision theory that this book does not cover.

The direct lineage of language model agents comes down to three papers, Wei et al. (2022), Yao et al. (2023b), and Schick et al. (2023). All three are short and worth reading in the original.

The workflow and agent distinction and the five patterns are in Anthropic (2024a). The follow-up, Anthropic (2025c), calls Φ *context engineering* (Chapter 4).

A Map of Reasoning and Planning Patterns

Chapter 1 split workflows from agents by who owns the control flow. Once you have decided to hand it over, one question follows. *What shape do you run that loop in?*

The shapes have names. chain of thought, plan-and-execute, ReAct, self-reflection, best-of-N, tree search. Variants get added every year, and the skeleton is these six.

Memorizing names decides nothing. A list goes stale as fast as the list grows. So what this chapter does is not introduce the six. It puts up the questions that separate them. Once those stand, you can ask the same things of the seventh that arrives next year.

The order of the list has a rule too. *Cheapest first*. Every pattern buys capability by adding calls. The one to try first is always the one that adds less, and the one that adds more earns its reason only when the cheaper one fails (Anthropic, 2024a).

2.1 What patterns are there

chain of thought. Have the model put its thinking out as text before it answers. Wei et al. (2022) showed that generating intermediate reasoning steps ahead of the final answer improves performance on arithmetic, commonsense, and symbolic reasoning tasks. Back then you had to ask for it in the prompt. Now it mostly happens without asking. The call count does not change. Only the output gets longer. *When to use it.* Nearly always. This is less a pattern of its own than an ingredient in the five below. What ReAct calls reasoning is this, and the thoughts tree search uses as nodes are this too.

plan-and-execute. The first call writes a list of things to do and the remaining calls cross them off one at a time. Wang et al. (2023) sorted what chain of thought alone leaves behind into “calculation errors, missing-step errors, and semantic misunderstanding errors,” and went after the missing steps by first “devising a plan to divide the entire task into smaller subtasks.” The paper calls it plan-and-solve. The same structure laid on a loop is plan-and-execute. *When to use it.* When the structure of the subtasks is visible before the run and you have to write down in advance how many steps it takes. Note that a plan is strong against omission. If the task was misunderstood, the misunderstanding moves into the plan. It does not go away.

ReAct. One thought, one action, in turn. Yao et al. (2023b) had the model generate reasoning traces and actions in an interleaved manner. The reasoning builds and updates the plan for acting, and the acting fetches information from outside for the next round of reasoning. This is also the first shape that meets Chapter 1’s condition for an agent: the model picks the action, and stopping is decided during the run. *When to use it.* When there is something to exchange with the environment and you cannot know in advance how many exchanges it will take. *The other three are layers you lay on top of this base.*

self-reflection. Look back over a failed attempt, write it up as one piece of prose, clear the context, and start again carrying only that. Reflexion (Shinn et al., 2023) has agents “verbally

reflect on task feedback signals, then maintain their own reflective text in an episodic memory buffer” for the next trial to draw on. Inside it a ReAct loop usually turns. *When to use it.* When you can judge success automatically and a retry is allowed. With no judgment there is nothing to reflect on.

best-of-N. Solve the same task several times over from the start, separately, and pick one. The branches do not split partway. They split at the root and never meet again. *When to use it.* When you have a way to pick and the attempts do not touch each other’s state. Of the two patterns that fan out, this is the cheaper one, so try it before you think about tree search.

tree search. Open several candidates from one state, pick a promising one and go down, and come back to another branch when you hit a wall. Yao et al. (2023a) starts from the observation that language models are “confined to token-level, left-to-right decision-making processes during inference,” so they fall short on tasks that need “exploration, strategic lookahead, or where initial decisions play a pivotal role.” A tree of thoughts takes “coherent units of text (thoughts) that serve as intermediate steps toward problem solving” as its nodes and searches over them. LATS (Zhou et al., 2024) widens the structure to actions as well, and brings the environment’s observations into how a branch is scored rather than the model’s self-evaluation alone. *When to use it.* When the early decisions decide what follows, when actions can be undone or kept apart, and when you have a standard for scoring a branch. Miss any one of the three and this is not the thing to pick.

What separates the six

All six buy the same thing. Capability. And all six pay with the same thing. Calls. So when you meet a new pattern, what you have to ask narrows to five.

1. **What is the unit you count, and when does that number get fixed?** chain of thought counts steps, and that number is already fixed when you write the code. tree search counts nodes, and you learn that number by running it.
2. **What does a step leave behind?** What it leaves becomes the input to the next step and gets reread on every call. So the amount one step leaves is what one step costs.
3. **What gets held up against the limit on length?** The whole loop, one attempt, or one path. The length at which a model still does useful work has a limit shorter than the declared window, and which thing runs into that limit first differs by pattern.
4. **Do you leave the run of leading tokens untouched?** Leave it and the cost of rereading it drops sharply. Disturb it and everything after that point is void.
5. **What survives a failure?** Material to diagnose with. A stretch that left nothing is gone for good.

The first one differs from the rest. What a pattern changes is not only the value of T but its size, its spread, and *when it gets fixed*. A number you know before the run, a number you know once the first step ends, and a number you know only when the loop ends are three completely different designs to budget against. Figure 2.1 puts the six side by side: what shape each one grows the calls into, and whether that count is knowable before the run.

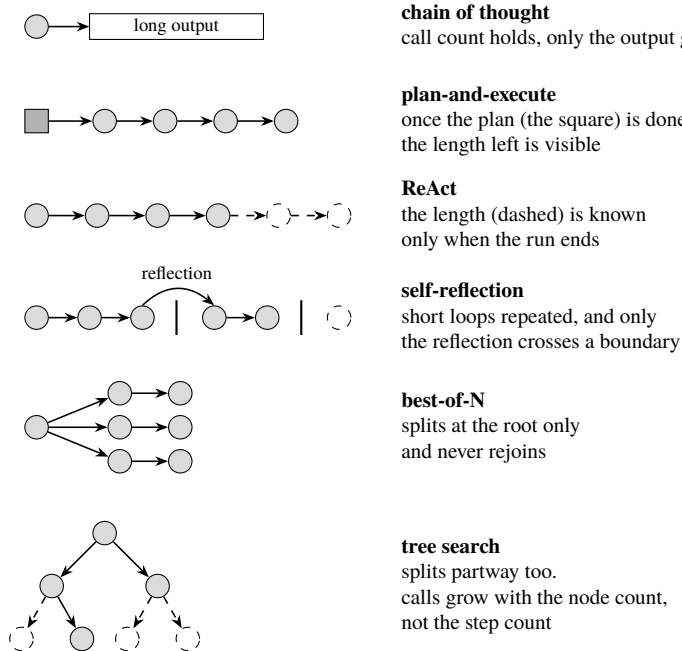


Figure 2.1: The shape each pattern gives to the calls. One circle is one model call. A dashed circle is a call whose count you cannot know before the run.

2.2 The decision procedure

The first branch comes before any pattern. *Would a workflow do?* Every pattern buys capability by adding steps or nodes. If you can enumerate the execution path for the task in advance, there is nothing to buy. Writing that path as code is always cheaper (Section 1.6). Skipping this branch and picking a pattern first is the most common and most expensive mistake in practice. And if there is nothing to exchange with the environment, what comes next is not a question about the loop either. It is a question about one call.

Figure 2.2 is the whole procedure. Here is what each branch buys and what it pays.

1. **Lay on plan-and-execute.** It buys you moving the spread in step count forward to planning time. It pays with the risk of a rigid plan, plus either the room a near-stable block takes up, if you keep the plan in the context, or an extra step to read the plan in, if you keep it outside. Decide where the plan lives at the same time.
2. **Lay on self-reflection.** It buys you turning failure into an asset. It pays with total steps multiplied by the attempt count, and with a dependence on J . If all you want to buy is room in the length, this layer is not what you need.
3. **Fan out with best-of-N.** You set the attempt count yourself, so the budget is fixed at design time, and running them in parallel keeps latency close to one attempt. Try this before going any further down.
4. **Fan out with tree search.** Pick it only when actions can be undone or kept apart, when you have a way to score a state, and when you can carry a budget counted in nodes. If

Table 2.1: Where the six patterns sit on the five axes. A cell whose value is not settled by the pattern alone is a cell the rest of this chapter takes up.

	Unit counted, and when fixed	What it leaves	Held up against length	Leading run	What a failure leaves
chain of thought	one step, at design time	depends on the setting	one call	appends at the end only	depends on the setting
plan-and-execute	plan and subtasks, at planning time	a near-stable block the size of the plan	where you put the plan decides	where you put the plan decides	the plan
ReAct	step count varies, at end of run	thoughts, actions, observations	the whole loop	appends at the end only	the whole alternating record
self-reflection	restart per attempt, attempts capped	only the reflection carries over	one attempt	kept within an attempt only	the reflection
best-of-N	attempt count fixed, at design time	nothing shared	one attempt	only the stable prefix is shared	everything, discarded attempts included
tree search	node count, during the run	piles up on the path only	one path	siblings share it	the whole tree

any one of those is no, this is not the thing to pick.

There is a reason the procedure is a run of *lay it on* and not *pick one*. Two options are exclusive only if they push the same variable in different directions. What plan-and-execute moves is when the step count gets fixed. What self-reflection moves is what the length limit is held up against. What tree search moves is the unit you count. *Different variables cannot collide*. LATS (Zhou et al., 2024) is the working example. It joins reasoning, acting, and planning and takes in self-reflection as well, which is this procedure’s layers all stacked in one system.

When you lay a layer on, write down the condition for taking it off. A layer you laid on does not fall off by itself. Let the judging signal weaken until self-reflection spins in place, and the reflection loop stays there, still billing you the attempt count in steps. Let the subtask structure disappear from the task, and the planning step stays, still loading a near-stable block. If you laid the layer on because the procedure’s question was *yes*, then the moment that answer turns to *no* is the moment to take it off. Nobody but the person who laid it on is watching for that.

2.3 How patterns fail

Section 1.4 said the most common accident in practice is not a wrong answer but not stopping. Each pattern changes the odds of that accident in its own way.

- **chain of thought fails fluently.** A plan fixes only the missing steps. Calculation errors and misunderstandings stay right there on the chain. The worst form is when the reasoning becomes a story that justifies the conclusion instead of checking it. Plenty of prose left, little signal in it.
- **plan-and-execute fails between rigidity and thrash.** An observation knocks the plan’s premise out and the plan is held to anyway, and it fails diligently. Turn it around and rewrite the plan on every observation, and the cost of planning is paid again each time. In a design that keeps the plan in the context, the invalidation repeats too, and what you have is an expensive ReAct loop. *The design variable that survives is not the quality of the plan but the rule that sets how often you replan.* What gets used in practice is usually one of three. Only when an observation breaks a premise the plan named. Only when

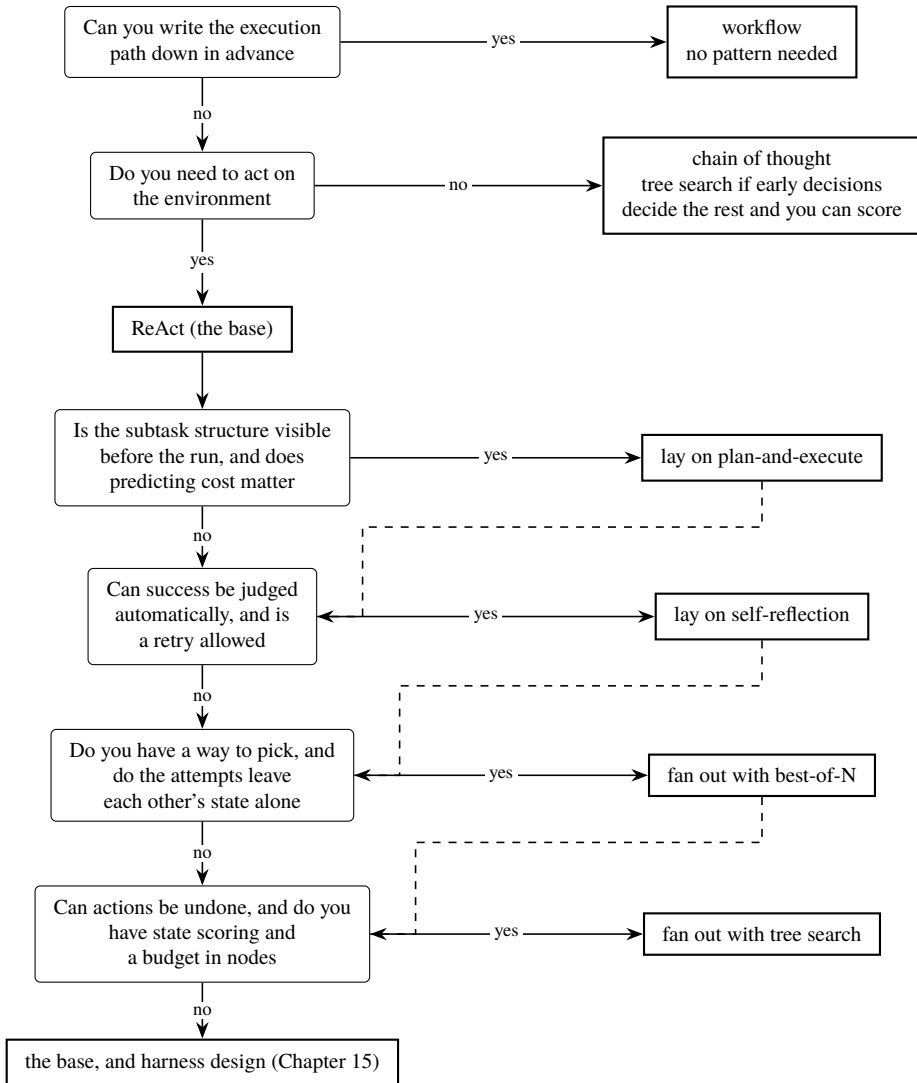


Figure 2.2: The procedure for picking a pattern. The boxes on the right are not terminals but layers that stack on the base, and after laying one on you follow the dashed line down to the next question.

the same subtask has failed a set number of times. Only when the plan's items run out with no stopping decision reached.

- **ReAct walks in place.** It calls the same tool with the same arguments again and confirms a fact it has already confirmed. The decline lengthens the loop and the lengthened loop deepens the decline. A loop past the limit on length does not get slightly worse. It accelerates.
- **self-reflection is the first case that cannot stop.** When *J* reads a failure as a success it stops early, and when it reads a success as a failure it repeats a task that was already done.

The error rate of the judgment is the error rate of the pattern. When the reflection blames the wrong cause, the memory grows and the behavior does not. And if the reflection grows with each attempt, every attempt starts bigger, and the same feedback plays out again at the level of attempts.

- **tree search is the second case that cannot stop.** The model scoring a branch is the model that made it, so an overrated branch keeps looking promising. And a tree can always grow one more node, and the expected gain from expanding is rarely exactly zero. Only two endings can be trusted. You find a solution, or the budget runs out.

Why the judgment has to come from outside shows up if you picture someone grading their own exam paper at the desk where they wrote it. The same understanding that got the answer wrong also writes the grading key. Misread the arithmetic and you will call the same arithmetic right when you grade it, so the more wrong the answer, the more it looks correct to the person who wrote it. They are not failing to find a reason to fix it. They are standing where the reason cannot be found. The same thing happens when you leave best-of-N's picking to the same model.

For self-reflection there is more than a structural argument, and *the fork is whether feedback comes from outside*. Huang et al. (2024) calls it intrinsic self-correction when “an LLM attempts to correct its initial responses based solely on its inherent capabilities, without the crutch of external feedback,” and reports that on reasoning tasks “LLMs struggle to self-correct their responses without external feedback, and at times, their performance even degrades after self-correction.” What struggled is the case where the model both judges and reflects. So what the citation shows is not that the pattern is useless but what it *depends on*.

The five failures share a direction. All of them fail by *lengthening* the loop. And not stopping is not solved inside a pattern. Patterns are good at manufacturing reasons to continue, and the reason to stop has to come from outside. The stopping rule ρ and the structure around it are that outside, and they are Chapter 15's subject. So picking a pattern includes picking that pattern's stopping condition. Lay on self-reflection and a cap on attempts joins the list of stopping conditions that moment. Fan out with tree search and a budget in nodes joins it.

2.4 When the cause sits outside the pattern

Sometimes changing the pattern improves nothing, because the cause was never in the shape of the loop. If any one of the following five holds, pick the simpler side, not the more elaborate pattern.

1. **The simple one already solves most of the distribution.** What fraction of queries does the elaborate pattern win? What multiple do you pay on all the rest to buy that fraction? Those two are the only comparison that means anything (Example 1.1).
2. **You have no way to judge success.** self-reflection and tree search eat judgment as input. With no signal to reflect on and no standard for scoring a branch, they do not even get off the ground. The build order is evaluation before pattern (Section 1.6).
3. **Actions cannot be undone.** tree search is out and retries get dangerous. What is left is best-of-N, whose branches share no state, and only when each branch reads and does not write. Human-in-the-loop design comes before any pattern that adds autonomy.
4. **The latency budget is tight.** self-reflection multiplies latency by the attempt count, and tree search multiplies it by the depth even when you run branches in parallel. In a con-

versational interface where a user waits, that multiple is usually not survivable.

5. **The tools are ambiguous or retrieval does not find things.** Lay a pattern on a system like that and the failures do not vanish. They get elaborate. And the more elaborate the pattern, the more observation and evaluation it demands.

Run through the five and the common thread shows. Picking a pattern looks like a decision inside the loop, and every input to the judgment sits outside it. The query distribution, the way you judge, whether the environment can be undone, the latency budget, the quality of tools and retrieval. *Making a pattern more elaborate produces none of them.*

2.5 Summary

In one sentence. Whatever the pattern, capability is bought by adding steps or nodes, so the first thing to ask is always whether this task is worth that payment.

What is now settled.

- You judge a new pattern on five axes. The unit counted and when it gets fixed, what a step leaves, what gets held up against the limit on length, the leading run, and what a failure leaves (Table 2.1). The shape of the loop is not the only thing that moves those axes. A request setting for how deeply the model thinks moves three of them without changing the structure at all, so that cheaper side comes first.
- chain of thought does not change the call count. It only lengthens the output. But the values on the second and fifth axes are not properties of the pattern. They *depend on the setting*. plan-and-execute is the same: whether you keep the plan in the context or outside it splits the values on the third and fourth axes.
- Restarting cuts cost, because the history reread on every step gets shorter. But a mechanism that trims the history at a threshold and reloads it buys the same room without restarting, so what self-reflection uniquely buys is the case where solving the task again from scratch makes sense.
- Fanning out has to be counted in nodes, and cost and latency come apart there (Section 2.1). But tree search is not the only way to fan out. best-of-N splits at the root and never meets again, so it has no depth to explode along, and because the branches share no state it never raises the demand to undo.
- The patterns fail in a common direction, by lengthening the loop, and self-reflection and tree search have no natural ending. Picking a pattern includes picking the stopping conditions its failure mode demands (Figure 2.2).

What is still open.

- The outside that the reason to stop must come from has only been named. The stopping rule, the structure around it, and what a run cut off at a cap hands back to the user go to Chapter 15.
- The arithmetic for when the memory carried between attempts grows goes to Chapter 15, and the summarization that holds that memory to a fixed size goes to Chapter 6.
- Choosing and trimming what a step leaves, to cut the increment per step, goes to Chapters 4 through 6. Chapter 3 lays the ground for that arithmetic.
- tree search was a tree fanned out inside one context. Fanning out into several contexts, each given different work, goes to Chapter 16.
- The judging signal J was treated only as an input. Designing evaluation goes to Part VI.

Try this today. Put one of the six names on the loop you are running now and *count* how many layers sit on top of the base (Section 2.1). Next, open your budget and *check* that its unit matches that name. If you are running best-of-N or tree search and the budget is written in steps

or latency, rewrite it in attempts or nodes (Section 2.1). Last, *attach* one line to each layer that is on, naming the condition for taking it off. That line records the moment the question that was *yes* when you laid it on turns to *no* (Section 2.2).

2.6 Exercises

1. chain of thought does not change the call count, and leaving the thinking in the history still makes the loop hit the limit on length sooner. Write that path out in a paragraph, and alongside it write what each of the five axes gains and loses in a design that does not keep the thinking.
2. Restarting cuts cost only while the reflection stays the same size. If the reflection grows a little on every attempt, how do the total cost and the steps each attempt can afford change? A design that summarizes the reflection to hold its size pays on which axis?
3. With the node budget fixed, which task property decides whether a wide shallow tree or a narrow deep one wins? For a task without that property, why is best-of-N better than tree search?

2.7 On a second reading

- Read Chapter 15, come back to the reflection carried between attempts, and the premise of a reflection that does not grow has acquired the name handoff and become a design object in its own right. If the handoff grows with each attempt, the cost that restarting saved comes back by the same amount, so the handoff has to be rewritten rather than accumulated. That is the conclusion of that chapter.
- Read Part II, come back to the five axes, and you see that the whole craft of working with context was moving one axis, the second. Prompting, context engineering, and compaction are hands of different depths on the same increment per step, and all three move inside the ceiling on that increment that the choice of pattern already set.
- Read Chapter 6, come back to self-reflection, and you see that throwing the history away whole at an attempt boundary and keeping only the reflection was a coarse form of compaction. That chapter's standard, drop the source text and keep the conclusion, applies to the reflection just the same.
- Read Chapter 16, come back to the two patterns that fan out, and the line between fanning out branches inside one context and splitting the context itself into several becomes visible. The two here share their leading run. Isolated seats do not share even that.

2.8 Further reading

The original papers behind chain of thought and ReAct, Wei et al. (2022) and Yao et al. (2023b), were already recommended in Chapter 1. Read them again and you see what the two papers do not talk about, which is the axis of cost.

For planning, see Wang et al. (2023). What is borrowed here is the structure. The paper's value lies in sorting the failures of chain of thought into three kinds, and the sense of which remedy works on which failure comes out of that sorting.

self-reflection starts with Shinn et al. (2023). Confirm the dependence on judgment that Section 2.3 pointed at directly in the paper's own construction. For the limit, Huang et al. (2024) covers it, and you have to read it holding the case with external feedback apart from the case without. Miss that split and you will misread it as a refutation of self-reflection altogether.

Two papers are enough for tree search. Yao et al. (2023a) gets past the constraint of left-to-right decision making by searching over units of thought, and Zhou et al. (2024) adds acting, planning, and external feedback to it. Read them counting what pays for the tree as it grows, with the eyes of Section 2.1.

PART II

Working with Context

Tokens: The Unit of the Budget

The loop in Chapter 1 pays every time it goes around. There is a unit that cost is counted in, and this chapter puts up that one unit and nothing else.

Not one character of the context you send is free. The system prompt costs. The tool descriptions cost. The file a tool just handed back costs. The indentation you added to make it look tidy costs.

Building an agent fails on cost about as often as it fails on design. So this chapter says three things. Why you have to know the unit. That the same content costs differently in different languages. And that on the day you change models you have to count it all again. None of the three is theory. All three are about the bill, and all three are already settled before you write one character of a prompt.

3.1 Tokenization: what gets counted

The bill does not count characters. It does not count words either. So sizing up a prompt by eye misses from the start. There is a separate unit, and we are not the ones who set it.

Start with what gets counted. What rides along on one call is not just the sentence the user typed. The system prompt rides. The tool definitions ride whole. Every observation and action exchanged so far rides. The 300-line file a tool handed back is in there. Everything loaded gets counted.

And it does not get counted once. *A model call does not remember the call before it.* So the tenth step is the job of rereading everything piled up through the ninth. The twentieth step rereads through the nineteenth. The amount read in one pass grows with the step count, and that grown read is repeated as many times as there are steps.

Make the task four times longer and the bill is eight times bigger. The whole context is loaded again on every step. Double the step count and the cost lands near four times. This describes a loop that rewrites nothing between steps, and knocking that premise over goes to Chapter 6.

3.2 The same content costs differently in different languages

Petrov et al. (2023) ran widely used tokenizers over the same text translated into many languages and found the lengths come apart badly. “The same text translated into different languages can have drastically different tokenization lengths, with differences up to 15 times in some cases.” It did not go away in tokenizers built to support many languages. Call the ratio of token counts

against the English version the *token inflation factor*. It is not a property of a language but of a language and a tokenizer together, and it is already settled the moment you pick a model.

One decision comes out of this. *Write the text only the model reads in English. The text a user reads cannot be.* Start with what gets loaded again on every step. The system prompt and the tool descriptions do, and that is where the factor hurts first and hardest. A 2,000-token system prompt written in a language with a factor of 1.5 becomes 3,000 tokens, and over twenty steps that alone adds 20,000 tokens. The other side is not negotiable. The final response has to be in the user's language, and the model decides what language it thinks in on the way to an answer. Rewriting is not free either. The language of the instructions pulls the language of the output along, so you have to add a separate instruction to answer in the user's language, and how well that gets followed differs by model (Chapter 5).

3.3 What this chapter leaves out

With tokens, the real problem is how far in you need to go. For readers who want to go further, here is one line on each and where to look.

- **How attention computes.** Every token gathers material from the other tokens in the context to fill out its own representation. Which pairs of tokens turn out to be related cannot be known before you compare their contents, so in the basic form every token meets every token, and the number of meetings goes as the square of the length. How the queries and keys and values are built is in Vaswani et al. (2017).
- **How position gets in.** Attention on its own does not know the order of the tokens. Order has to be supplied separately, and how far each way of supplying it holds up on long inputs differs. Su et al. (2024) covers the one in wide use today.
- **Causal masking.** To train a model to predict the next token, no token may see a token that comes after it. That blind is causal masking, and it is why output gets made one token at a time. The same literature covers it.
- **Training.** This book assumes the parameters of π_θ are frozen. What you can touch is not the model's weights but what you load and what you permit. The background is in Goodfellow et al. (2016).

This book takes exactly one thing from that list. Length costs, and on the provider's hardware that cost grows as the square of the length. Length is the only part of it we can touch.

3.4 Summary

In one sentence. Not one character of the context you send is free, and the unit it is counted in comes attached to the model rather than chosen by us, so the story of the budget starts with what makes one token.

What is now settled.

- One call carries the system prompt, the tool definitions, and the whole history so far, and every step loads all of it again. So a task four times longer bills eight times as much (Section 3.1). The size of the stable prefix and the increment per step are

two numbers you can count straight out of the log of the loop you are running now.

- The ratio of token counts against English is the token inflation factor, and it is a property of a language and a tokenizer together rather than of the language. One decision comes out of it. Write the text only the model reads in English, and accept that the text a user reads cannot be.
- Changing models changes the meaning of every decision written down in tokens, without changing one line of code. The threshold stays where it was while the amount it used to hold back shrinks, and the same task costs more without any edit to the prompt.

What is still open.

- The premise that every step rereads all of it can be knocked over. The mechanism that skips recomputing what was already read, and what that mechanism still cannot remove, go to Chapter 6.
- Cutting what gets loaded in the first place, which means handling the context in layers, goes to Chapter 4.
- That length raises more than cost, that quality falls too, was not counted here. Chapter 4 takes it up.
- The arithmetic on the factor assumed the step count does not depend on the language. If a change of language makes the model take more steps on the same task, that effect swamps a constant multiple.

3.5 Exercises

1. The arithmetic on the inflation factor assumed the factor applies equally to the context and to the actions. If the arguments of your tool calls are mostly JSON and file paths, how does that assumption break? How should the effective inflation factor be defined then?
2. The arithmetic in Section 3.1 assumes nothing is rewritten between steps. If the leading stretch already read on the previous call could be reused as is, what in that arithmetic changes and what stays? What kind of edit to the context makes that reuse impossible?
3. Rewriting the system prompt in a language with a lower factor cuts tokens and may move the task success rate. What experiment settles that trade? Write down the quantities you would have to measure first.
4. Say you moved to a new model and left the constants written in tokens where they were. What symptom shows up first? How would you confirm from the log that the symptom came from the model change?

3.6 On a second reading

- Read Chapter 4, come back to the arithmetic here, and the size of the stable prefix and the increment per step turn out to be handles rather than numbers you merely measure. That chapter splits into layers how you shrink each of them.

- Read Chapter 6, come back to *every step loads all of it again*, and you see there were two ways to knock that premise over. Lowering the unit price of the tokens you reread, and slowing the growth in how many get counted at all.
- Read Chapter 5, come back to the inflation factor, and you see that the constant multiple does not hurt evenly across the context. The system prompt is the place that reloads on every step, so the factor gets multiplied by the step count there. It takes both chapters laid over each other to finish the arithmetic that gives 20,000 tokens.

3.7 Further reading

Two papers are enough for the lineage of tokenization, Sennrich et al. (2016) and Kudo and Richardson (2018). The first gives the idea of subwords itself, the second covers a language-independent implementation and the ability to recover the original text. The line that takes the byte as the base unit is Wang et al. (2020).

Section 3.2 rests on Petrov et al. (2023). If you need the cost structure of your own languages as evidence, check how the paper measures and run the same measurement on your own corpus. The place to actually count is Anthropic (2026i). What to take from it is not the shape of the call but two notes. The number that comes back is an estimate, and what it counts by is the model's own tokenizer.

If you are curious about the inside of the model, start with the literature attached to Section 3.3.

Context Engineering

Most of what an agent needs to do its work does not exist until the run has started. A fact that has to be retrieved, an observation a tool handed back, the handoff from an earlier session, a condition the user just added. All of it comes into being outside the text you write down beforehand.

What this chapter covers is that remainder, the *decisions made at run time*.

Anthropic (2025c) puts it in one line. The goal is to “find the smallest possible set of high-signal tokens that maximize the likelihood of some desired outcome.” *Smallest* because length costs. *High-signal* because what is unrelated pushes out what is related. *Set* because this is picking, not filling. The first two words name two different axes, and that split is the skeleton.

4.1 Inside the assembly function

What the model sees on a step is one long string. Chapter 1 called making that string assembly, and the thing that does it is the assembly function.

What rides in it is not only the instructions we wrote down before the run. The names and arguments of the tools the model may call, the conversation exchanged so far, the documents retrieved, and the result a tool just handed back all get joined into one block of text and go in one pass.

And on the next step the same thing goes again, whole. To attach one new line of observation you load everything before it again. It looks like a conversation carrying on. From the model’s side it is reading from the beginning every time. What to load and what to leave out, and in what shape, are all the design freedom this function has.

The share settled before the run is the *stable prefix*. The share the run produces is the *variable tail*. The exact definition and the name for the stable prefix come in Chapter 5 (Definition 5.1). The rest, deciding at run time what to load out of the history and the other information available at each moment, and in what shape, is *context engineering*.

The name comes from industry. Anthropic (2025c) defines it as “the set of strategies for curating and maintaining the optimal set of tokens (information) during LLM inference,” and what it curates includes the system instructions, the tools, external data, and the message history together.

Context moves performance too, so there is no option called not designing it. The real question is not whether to do it but *whose decision is loading what, right now*.

Table 4.1: The context broken into layers. The right column is the reading map for Part II and onward.

Layer	When settled	What sets its size	What shrinks it, and where it is covered
Instruction prefix	before the run	how many kinds of task it tries to cover (Section 5.3)	discipline about what the instructions say (Chapter 5), skills pulled in on demand (Chapter 7)
Tool definitions	before the run	how many tools, and how detailed the schemas	tool design and managing what the definitions cost, going around them with code execution (Chapter 7)
Retrieved knowledge	at run time	the query and the retrieval settings	the quality of retrieval (all of Part IV), judging whether to load it at all (Chapter 11)
History and observations	the run makes it	the step count, and the per-step increment that the volume a tool returns sets	designing what tools return (Chapter 7), compaction (Chapter 6), isolating contexts (Chapter 16)
Handoff artifact	at a session boundary	the size of the task left	a handoff that gets rewritten (Chapter 15, Section 15.2)

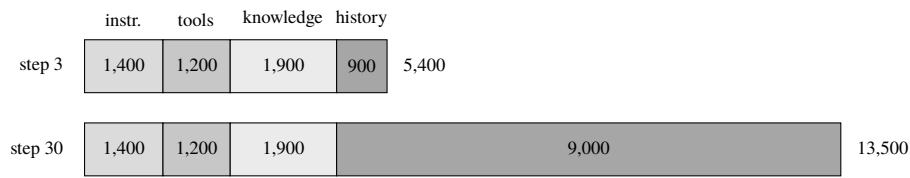


Figure 4.1: One step’s context cut into layers. Both rows are on the same scale. The first three layers hold their width and only the last one grows. The handoff artifact loads only when a session is picked up, so it is left out. The numbers are an example from one system, and your own come out of your step log.

4.2 What to load

Context is not one uniform block. It is the sum of layers with different characters, and every layer differs in when its content gets settled, in the law by which it grows, and in what shrinks it.

First, *only one layer grows with the steps*. History and observations is the only layer the run makes, and this layer grows on its own if you leave it alone. The other layers have their size tied to the task or to a setting, and once set they mostly stay. That is why compaction gets a chapter of its own in Part II.

Second, *which layer dominates the budget differs by system*. A system with many tools is dominated by tool definitions, a system that works with documents by retrieved knowledge, a system running long loops by history. That is not a number you read out of a table but one you measure in your own step log, and once measured it draws like Figure 4.1.

What to read off that figure is the proportion. History takes 17% at step 3 and 67% at step 30. Over the same stretch, instructions and tool definitions together fall from 48% to 19%. They did not shrink. What sits beside them grew. So the gain from trimming tool definitions is large at early steps and small at late ones, and if you rank the layers after measuring short tasks only, you will have it wrong for long ones.

Third, *tool definitions get written before the run and tool returns are made by the run*. Different layers can also have different owners. The person who edits the instruction prefix, the person who manages the tool definitions, the person holding the retrieval settings, and the person setting the rules for compacting history can all be different. Each grows their own layer without knowing what share of the total budget it takes. The share has to be measured before you can show each layer's owner what they are using. Without it, every layer grows a little and nobody is responsible.

On tools in particular, Anthropic (2025c) recommends cutting back bloated tool sets with overlapping function. "If a human engineer can't definitively say which tool should be used in a given situation, an AI agent can't be expected to do better." Compacting knowledge and history beyond the tools is what the following chapters describe.

4.3 When to load it

There are five layers and two kinds of decision.

One kind is what we write down before the run. The system prompt is that. The tool definitions are that. We decide what to write, the size is settled once it is written, and that size follows along from the first step to the last.

The other kind is what you keep outside the context and carry in during the run. The result a tool handed back, the document retrieved, the handoff the last session left. Outside here means the file system, the retrieval index, the world the tools reach. When the run starts these are not in the context. They come in at the moment they are needed.

Then one question stands up. *Why keep it outside? Why not write it all down up front?* Defining a tool is the same puzzle. You could just write down in the context what the tool does, so why make it a name that can be called?

This chapter starts the answer and the later parts finish it. Why tools get defined as a vocabulary, and what shape what they return should come back in, go to Part III. Why knowledge gets kept outside and fetched, and where and how the last session's work gets left, go to Part IV.

Before that there is one thing to count here. Whether you kept it outside or wrote it down up front, to use it you eventually reach the moment it loads into the context. *When you place that moment sets the cost*. Loading it from the start is *loading ahead*. Fetching it at the moment it is needed is *fetching on demand*.

People work this way too. You do not memorize a whole archive. You keep an index, a table of contents, and file names, and you go find things when you need them. Only what you are using sits on the desk. Fetching has the same shape. Rather than processing the data ahead of time, the agent holds "lightweight identifiers (file paths, stored queries, web links, etc.) and uses these references to dynamically load data into context at runtime" (Anthropic, 2025c).

4.4 Why not just put everything in

Putting everything in has a logic of its own. You cannot know before the run what the model will need, so if you put it all in, at least nothing fails for want of it. Underneath sits an assumption that more information is better and that the model will ignore what is unrelated. It does not work that way.

The ceiling: there is a limit on how much fits

Every model has an upper bound on the tokens it takes in one call. That is the *context window*. Go past it and the request is refused or the front gets cut off.

Of the four reasons, this one is the weakest constraint. The number has grown with every generation, and what is too tight today may fit in the next one. A context problem ends with “use a bigger model.” Filling the maximum context, of course, guarantees nothing about performance.

Rot: performance falls before the ceiling

Fill half the ceiling and it already gets worse. As the tokens loaded go up, the ability to pull the right thing out of them goes down (Anthropic, 2025c). Not because anything was cut but because it was buried.

There are two kinds of bad context. One is long. Everything loaded touches the task, and the volume runs to hundreds of pages, so the work goes blurry with nothing extraneous in it. The other is short with the wrong thing mixed in. Every relevant chunk is loaded, and the answer reaches for a number out of an unrelated sentence that came along. The first is about bulk and the second about contamination, and fixing one leaves the other.

Length exacts its cost in two ways as well. One is position. Liu et al. (2024) held the content fixed and moved only where the relevant information sat, and found that “performance is often highest when relevant information occurs at the beginning or end of the input context, and significantly degrades when models must access relevant information in the middle of long contexts” (Figure 4.2). The remedy falls right out. *Put what matters at the front and the back*. When an instruction placed in the middle goes unfollowed, suspect the position before the wording.

The other is length itself. Which reveals that *window size* had been naming two different quantities all along. The length that goes in is the number written in the documentation. The length at which the work still gets done is a number you have to measure.

Definition 4.1 (Effective context length). Fix one task and the quality bar that task demands. The longest input length that still meets that bar is the *effective context length*.

Longer inputs usually carry more content too, so when performance falls you cannot tell whether the length did it or the task got harder. Levy et al. (2024) extended the same sample “with padding of different lengths, types and locations” to isolate the effect of length alone. The result: *reasoning performance degrades noticeably at input lengths far shorter than the technical maximum*. Whatever you fill with and wherever you put it, the filling itself costs.

History: this layer alone grows on its own

In document question answering the input length is a number the designer sets. In an agent it is not. The context grows as observations and actions attach on every step, so length is not chosen.

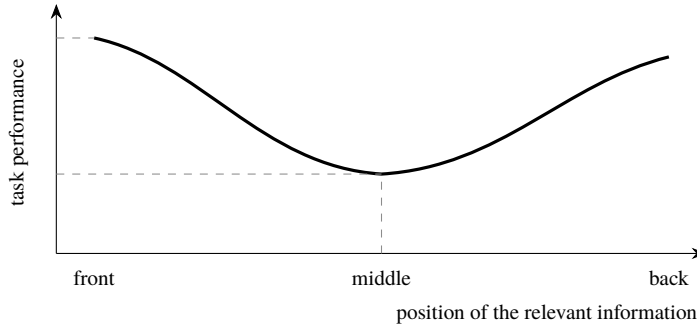


Figure 4.2: Performance against the position of the relevant information. The axes carry no scale on purpose. How deep the curve dips differs by model and by task, and what this figure shows is the shape, not the values.

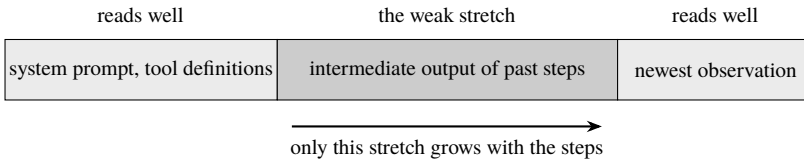


Figure 4.3: The typical layout of an agent's context. Both ends are fixed in size and only the middle grows with the step count.

It is a function of the step count. One line keeps climbing while the effective length stays put, so the two lines have to meet.

Remark 4.1 (An accumulating loop crosses the effective length on its own). If the context accumulates by some amount on every step and the effective context length is finite, there is necessarily a step at which the run enters the degraded region. From then on the context is over the effective length at every step.

The position is bad too. The system prompt is at the very front and the newest observation is at the very back, so what is left in the middle is the intermediate output of past steps. The one part that grows with the step count lands in the part that reads worst (Figure 4.3). And what is asked of it there is not pulling one chunk out but following a thread across several chunks and gathering scattered pieces into one (Table 4.2).

Table 4.2: Tiers of the ability a long context demands. Higher up is easier to test, and lower down is closer to what an agent actually does.

Ability	What it does	What it corresponds to in an agent
Retrieval	finds one chunk and pulls it out	keeps one rule from the system prompt
Multi-hop tracing	follows a thread across several chunks	joins up decisions from earlier steps
Aggregation	gathers what is scattered into one	sets the next action from every observation so far

Table 4.3: Four responses to context decay. The top two touch the terms of the arithmetic and the bottom two touch its premise. The further down, the more structure changes.

Direction	What it does	Where it is covered
Load less before the run	cuts what is already loaded before the loop turns, widening the room you start with	this chapter, Chapter 5
Put in less	cuts what one step leaves	Chapter 5, this chapter, Chapter 6
Keep it outside	stores it in files and indexes instead of the context and fetches it only when needed. The designer takes back the ceiling on how much attaches	Chapter 10, Chapter 11
Divide it	isolates one long context into several short ones. No one of them carries the whole step count	Chapter 16

So it has to be cut

Join the three and one conclusion follows. There is a ceiling on how much fits, it already gets worse before the ceiling, and one of the five layers heads there on its own if you leave it alone. So it has to be cut. There are four ways to push the point of decay back or to make its arrival harmless.

The question in Section 4.3 starts getting its answer here too. Why keep it outside? Because it does not all fit. Why define tools as a vocabulary? Because writing down everything a tool does is itself the put-everything-in design. Loading only the name and taking the result when you call is cheaper.

Cutting loses something, of course. Replace what was loaded with a summary and what is left is not the original. What you lose, how often you should cut, and what happens when you cut what was already cut are Chapter 6's subject.

4.5 Summary

In one sentence. Context is not filled but picked. There is a ceiling on how much fits and it gets worse before the ceiling, so what goes into which layer, and when, has to be decided at run time. You would have to do this even if tokens were free.

What is now settled.

- The goal is the smallest possible set of high-signal tokens that maximize the likelihood of the outcome you want. The line between this and prompting is not a technique but the moment of decision.
- Context splits into five layers along a scale of when things get settled (Table 4.1). Only history grows with the steps, and which layer dominates the budget differs by system. That is not a number you read out of a table but one you measure in your own step log, and once measured, history shows as 17% at step 3 and 67% at step 30 (Figure 4.1).
- There are two kinds of decision. What we write down before the run, and what sits in files and indexes and gets carried in during it. The first is loading ahead and the second is fetching on demand. The question that stands up here is why keep it outside, and Parts III and IV fill in the answer.
- There are four reasons not to put everything in, and they get heavier down the list. There is a ceiling, rot arrives before the ceiling, of the five layers only history heads there on its own (Remark 4.1), and so it has to be cut. The length at which the work gets done is measured, not declared (Definition 4.1), and the stretch that grows with the steps lands, of all places, where reading is worst (Figure 4.3).
- There are four responses to that decay (Table 4.3). The first row, cutting what loads before the run, is cheapest, and the further down, the more structure changes.

What is still open. The numbers in Figure 4.1 are an example rather than a measurement, and measured shares by layer are still homework. What this chapter answered goes as far as what to load. Cutting down what is already loaded goes to Chapter 6, why tools get defined as a vocabulary goes to Part III, and why knowledge and memory get kept outside goes to Part IV. Deciding whether to load at all is Chapter 11, and designing the record you need in order to measure the shares by layer is Chapter 20.

Try this today. Pick one step out of the step log and *pull* its whole input. Count separately how many tokens the system prompt is, how many the tool definitions are, and how many the history so far is. Do it once at an early step and once at a late one and you see which one grows (Section 4.2). Next, take the largest single block in that input and *find when it was first used*. If it was loaded from the first step and first used at the twentieth, you threw away nineteen steps' worth of loading it ahead (Section 4.3). Last, *lay* the input lengths of a few failed runs *beside* those of successful ones. If the failures are longer, this system is already rotting (Section 4.4).

4.6 Exercises

1. Design an experiment that separates the two kinds of bad context on your own task. You need one context with the content fixed and only the length grown with padding, and another with the length nearly fixed and content unrelated to the task mixed in. How does the remedy differ between a case that is insensitive to length but sensitive to contamination and the reverse?
2. For the five layers in Table 4.1, measure tokens per layer in your own step log. Measure at an early step and a late one and draw them as two rows like Figure 4.1. Which layer dominates the budget? Write down who in the organization sets the size of that layer, and whether that person knows their layer's share.
3. Take the largest single block you are loading ahead and pull from the log the steps where it actually gets used. Is that number in the log, or a guess? If you moved that block outside and fetched it on demand, what would have to stay in the context in its place for the model to be able to fetch it?
4. Of the four directions in Table 4.3, which does your system need now? Confirm with your own numbers why the first row is cheapest. Then write down what it concretely changes in your system to say that the bottom two rows touch the premise of the arithmetic rather than its terms.

4.7 On a second reading

- Read Chapter 6, come back to *so it has to be cut*, and the one line written here as a conclusion has become a technique with a price on it. Cutting down reduces the loaded volume directly and leaves behind something that is not the original, and cutting badly leaves a stale conclusion that actively hurts. This chapter's first reason, that there is a ceiling on how much fits, turns there into the question of how often and what to drop.
- Read Parts III and IV, come back to *why keep it outside*, and what was only set up as a question here has an answer attached. Part III fills in the tools and Part IV the knowledge and memory. The right column of Table 4.1 is the list of those chapters. The progressive disclosure of Chapter 7 is exactly this structure, loading the name ahead and taking the body on call, applied to the instruction layer.
- Read Chapter 16, come back to Table 4.3, and you see why dividing is the last row. The first three leave the premise that there is one context and handle volume inside it. Dividing throws that premise out. That is also why it costs the most.
- Read Chapter 5, come back to Figure 4.1, and Definition 5.1 gives one name to the first two layers, drawn here only as holding the same width. The left half of this figure is that chapter's entire subject, and the reason the width holds comes from there. It is settled before the run.

4.8 Further reading

On position, see Liu et al. (2024). The design holds the content fixed and moves only the position to draw out the U, so you can see the real shape of what this chapter reduced to one

figure. On length, see Levy et al. (2024). The design that stretches the same sample with padding to isolate the effect of length alone is clean enough to copy directly when you build the same experiment on your own task. Keep the finding that a traditional metric correlated negatively with reasoning performance filed separately, as a warning about proxy metrics.

For an industry write-up there is Anthropic (2025c). It is where the definition of context engineering came from, and it collects a list of practical techniques. Fix your attitude toward it, though. A list of concrete techniques gets refreshed with every model generation. Whatever technique you meet, ask two things. Which layer of Table 4.1 does it touch? And which way does it move something between what you write down before the run and what you keep outside and carry in? The list goes stale. The places stay.

Prompt Engineering

A prompt is text written before the run starts. So nothing you learn during the run can go in it.

Chapter 4 cut one step's context into layers and said that what to load and when is the design freedom. The share of that we write down before the run is this chapter's subject. Retrieved knowledge and what tools hand back go to the later parts.

Three things to answer. Where in the context does the prompt sit? How do you write it so it is clear? And how do you know that an edit made it better?

5.1 Where the prompt sits

Chapter 4 drew one step's input in four layers. Instructions, tools, retrieved knowledge, history. The two layers whose width holds after thirty turns are the first two, and those two are this chapter's subject (Figure 4.1).

Those first two layers split into three.

The system prompt is at the very front. We write it before the run, and it reloads on the first step and the thirtieth without one character of difference.

The user's input comes after it. This side differs run to run. Behind the same system prompt, today's inquiry attaches today and tomorrow's tomorrow.

And there are the tool descriptions. Every tool the model can call carries a sentence naming it, listing its arguments, and saying when it is the tool to use. A sentence like this.

“Returns a customer's recent orders. Use when you have the order number. With only a name, look the customer up first.”

Plenty of people do not count tool descriptions as prompt. They are written in the tool's own code and maintained by whoever built the tool. But in the string the model sees, that sentence is right next to the system prompt. It is settled before the run, reloads every step, and costs every step. By position it is the same object.

The three can go under one name.

Definition 5.1 (Prompt). Whatever the history becomes, the part that loads in the same shape on every step is the *prompt*. The system prompt, the tool descriptions, and the output conventions are most of it, and the frame that sets the order the history is laid out in and the markup that wraps it is the rest. This is the share Chapter 4 called the stable prefix. *Prompt engineering* is designing that stable prefix.

5.2 What the instructions do

What the stable prefix actually carries comes down to three things. First, the definition of the task. What is being asked, and what counts as done. Chapter 15 says a declaration of completion

is an input that triggers verification, and much of the standard that verification checks against has to be written out as sentences in the prompt first. Second, the output conventions. What gets written where and in what structure. Much of that share has moved out of prose and into a format specification attached to the request, and that story goes to Chapter 9. Third, the boundary on behavior. What not to do, what not to run without confirmation, what conditions to hold to when using a tool. The third comes with a note. The boundary is written in the stable prefix, and the sentences that push on that boundary arrive during the run. So the stable prefix is a contract and a line of defense at once, and designing the line of defense goes to Chapter 22.

Mix the three together and none of them gets kept. Anthropic (2025c) recommends splitting the prompt into sections and marking the boundaries, using “XML tagging or Markdown headers to delineate these sections.” Background, instructions, guidance on tool use, and output conventions each in their own place. The skeleton looks like this.

```
<background> This agent handles questions about mobile plans. </background>
<task> Explain in one paragraph why the bill went up. </task>
<tools> Check charges with the lookup tool. Do not rely on memory. </tools>
<output> Give amounts to the cent and mark estimates as estimates. </output>
```

This is not a recommendation about style, and the reason is verification. With sections, you can answer *what are the output conventions* by pointing at one sentence. Without them, you have to reread the whole prompt.

How specific to write

The same material sets up one axis of prompt design. *Specificity*. Two failures sit at the two ends.

One end is a sentence too vague to mean anything. No signal.

“Help the user well.”

It rules out no output, so it is not a contract. A sentence that repeats what the model already knows belongs here too. Write this in the same place and some outputs get ruled out.

“Look up the order status and confirm it before settling on an answer. If you could not look it up, say that you could not confirm it.”

Turn a sentence written as a prohibition around once. Telling the model not to do something permits everything else, so writing the same intent as a positive narrows the target to one. “Answer in connected prose paragraphs” rules out more than “do not answer in a list.”

The other end is a sentence too specific. The prompt becomes code with the branches written out one at a time. In this case do this, in that case do that. The cases you meet in a run are always more than the cases on the list. When one that is not on the list arrives, the model misapplies the clause that looks closest. And the clauses have to be reworked every time the situation shifts. This is writing a workflow in natural language, and if that is the job, the verdict in Chapter 1 says write it in code, cheaper and steadier.

The same failure arrives today by another route, more often. Instead of adding clauses, you raise your voice. Back when models followed instructions less well, writing like this earned its keep.

“You **MUST** use this tool. When in doubt, **ALWAYS** call it.”

Now they follow what is written, so the same sentence calls the tool where it did not need to be called and drags the clause over cases that should have been exceptions. Emphasis neither rules

out more outputs nor makes the sentence more specific. It only widens the range of application. The remedy is not to delete the sentence but to lower the voice.

“Use this tool when an order number is given.”

The clause stays and only the excess goes.

How do you find the middle? Judge one sentence at a time. *Does this sentence rule any output out?* If not, it is the vague kind. *Does this sentence apply only in a particular situation?* Then it is the over-specific kind, and if you want the model to work out when that situation holds, it has to be written as a principle rather than a condition. There is a check that does the same job without reading: hand the prompt as written to a colleague who does not know the background of the task and ask them to do it. If they are confused, the model is confused.

5.3 Prompts are not robust

Sciar et al. (2024) measured sensitivity to prompt formatting, calling it “a quintessential class of meaning-preserving design choices.” Changing separators, changing whitespace and capitalization, edits that do nothing to what the instructions say. The result: “several widely used open-source LLMs are extremely sensitive to subtle changes in prompt formatting in few-shot settings.” The sensitivity “remains even when increasing model size, the number of few-shot examples, or performing instruction tuning.”

At work that sensitivity wears this face. One day you rework the examples section of a prompt. You tidy a clause, unify the markup around the examples while you are in there, and clean up the blank lines between items. You run the evaluation and the success rate is up. The change log says “clarified the instructions.” The next month someone else clarifies the same clause further and the score goes down. If the earlier gain came from the blank lines rather than the clause, the two of you did not run the same experiment twice. You each ran a different experiment once. And from the log alone there is no way to tell which.

Read carefully how it was evaluated. Not an average across many tasks. One task held fixed, the question and the examples and the answer required left alone, and the same metric measured again with only the format changed. And performance by format correlated only weakly between models. A format that suits one model is no guarantee for another.

The sensitivity was not confirmed on the format axis alone. Lu et al. (2022) showed that in few-shot settings, changing only the *order* of the examples “can make the difference between near state-of-the-art and random guess performance.”

What to shake

Write the same instruction two ways and it looks like this.

(a) “Output convention: one item per line.”

(b) `<output convention> one item per line </output convention>`

The meaning is the same. To the model they are different strings. There are six such places, and Table 5.1 is the list.

So you choose. Shake three of the six and leave the other three at the value you use now. The rule for choosing is one line. *Pick the places that repeat many times in the stable prefix.* Markup that appears once makes little difference when shaken. In a prompt with an examples section, the order of the examples usually comes out first.

Table 5.1: Six places you can shake without changing the meaning. Two values at each place gives 64 formats.

Place to shake	One value	The other value
Section boundaries	wrapped in tags	opened with a header
Item separators	a short dash	a number
Between name and value	colon and space	equals sign
Between items	a blank line in	the blank line out
Markup around examples	quotation marks	tags
Order of examples	the current order	reversed

The condition for saying it got better

You judge range against range, not point against point. Say it got better only when the lowest score after the edit clears the highest score before it. Two overlapping ranges mean you do not know yet.

You cannot pay this for every sentence you touch. There are three places where you pay. When you first stand a prompt up, when you rework the stable prefix in a batch, and when you change models. The third gets forgotten most often. A good format does not carry between models, so *changing the model means verifying the prompt again*.

The edits in between you make without evaluating. Just do not write “improved” in the log. Write “changed.” *A prompt edited without evaluation is not improved. It is different.*

That is the prompt’s story. The same problem runs across the whole context, though. When you change what to load, when you change what to drop, when you change the compaction interval. Every one of them has to answer whether it got better, and every one meets the same difficulty, because what counts as success, who judges it, and which tasks stand as the cases are all unsettled.

So context engineering rolls forward together with evaluation. Without evaluating you cannot know what you fixed, and not knowing what you fixed leaves the next decision without ground. How to stand an evaluation up goes to Part VI.

What a prompt cannot fix

A prompt can hold only what is known before the run. The fact that has to be retrieved, the observation a tool returned, the condition the user just added, none of them exist yet. Most of what an agent needs to work does not exist until the run has started. No amount of care narrows this boundary.

So one rule falls out. *A prompt can fix only the problems common to every run.* If the same prompt works on one run and fails on another, the prompt was not one character different in the two. What differed is outside the prompt.

Where this rule gets used at work is diagnosis. Take a few failed runs and ask. Did they all fail at the same place, or at a different place each time? If the same, look at the prompt. If different, look at the tools or the retrieval or the task itself. Skip this and you will keep editing the prompt forever, because it is the part of the system your hand reaches most easily.

Ignore the boundary and the prompt goes down a set path. As the tasks get more varied, one prompt accumulates each task’s rules and exceptions. Removing any one makes that task

Table 5.2: Only the last row can be fixed with the prompt. The four above are other parts' problems flowing in.

Symptom	What to check	Where to fix it
The result splits with the history identical	run the same input several times	checks and retries (Chapter 15)
The histories of a working run and a failing run differ	lay the two contexts side by side	what gets loaded and when (Chapter 4)
Instructions guarding against one tool's misuse keep piling up	show that tool's name and arguments to a person	the tool's definition (Chapter 7)
Answers are invented no matter how many instructions say to check the evidence	look at whether the evidence was loaded into that run's context	retrieval (Part IV)
Every run fails at the same place	look at whether a sentence covering that place is in the stable prefix	the prompt

worse, so removing is hard and adding is easy. A prompt grown that way costs more on every step and clouds the context with clauses unrelated to the task at hand.

Then the answer is not to write the prompt better. Instead of always loading the instructions for every task, pick what this task needs at run time. That is the context engineering Chapter 4 covered, and the structure that pushes the same direction as bundles of instructions pulled in only when needed comes up in Chapter 7.

5.4 When not to edit the prompt again

The prompt is the part of the system your hand reaches most easily. So other parts' problems flow here first. Before editing, work out whether this is a prompt problem at all.

One question sorts most of it. *Does the same prompt work on one run and fail on another?*

If it splits, there are two branches again. Run the same input several times, and if the result still splits with the history identical, that is sampling, and no rewording removes it. You take it with checks and retries, and that is Chapter 15's work. If the histories differed, what differed is the answer. Which document was loaded, what the tool returned, how many steps it ran. The place to fix is what gets loaded and when, and that is Chapter 4's work.

If instructions guarding against one tool's misuse keep piling up, the tool's name and arguments are bad, and a bad tool is not recovered by any prompt. If answers are invented no matter how many instructions say to check the evidence, the evidence never came into the context. A fact the model does not have does not appear because the instruction was written more forcefully.

There are places the table does not hold. A demand that latency is too long or cost too high catches on no row. That is not a problem of runs splitting into failures and successes. Every run is equally slow and equally expensive. The prompt looks like a candidate, and the share you can trim by tightening sentences is small. Changing the model is usually faster.

Example 5.1 (One symptom split into three). An agent that finds documents, summarizes them, and attaches the summary to a ticket succeeds seven times in ten. A request comes in to fix the prompt for the failures. Take ten failed runs and hold them against Table 5.2.

Rerun each three times on the same input. Four succeeded on some run. The histories were identical and the results split, so this is not something a rewording removes.

Lay the remaining six contexts beside a successful run. In five, the document to be summarized was never loaded at all. It wrote an answer with no evidence, so this is not a matter of writing the instruction more forcefully.

One is left. The document was loaded, the summary is right, and the body went into the ticket's title field. And the same field was wrong the same way on successful runs too. That one is a candidate for the stable prefix. *One out of ten.*

5.5 Summary

In one sentence. A prompt is the part settled before the run that reloads in the same place on every step, so you can verify it in advance, and it is neither robust nor able to hold what you learn during a run. Fix it while evaluating and push the rest into decisions made at run time.

What is now settled.

- The prompt sits at the very front of the context. The system prompt and the tool descriptions are there (Definition 5.1). Tool descriptions are easy not to count as prompt because they live in the tool's code, and they are settled before the run, reload every step, and cost every step.
- The stable prefix carries three things. The definition of the task, the output conventions, the boundary on behavior. The reason to write it in sections is not style but verification. You have to be able to answer *what are the output conventions* by pointing at one sentence.
- You judge an instruction one sentence at a time. Does this sentence rule any output out, and does it apply only in a particular situation? Raising your voice catches on neither, and only widens the range of application.
- Prompts are not robust. Format changes that leave the meaning alone, and changes to the order of examples, move performance a great deal, and a good format does not transplant between models. The list of places to shake is Table 5.1, and you start with the places that repeat many times in the stable prefix. So you judge range against range rather than point against point, and what was edited without evaluation is not an improvement but a change.
- A prompt can fix only the problems common to every run. The first question is whether the failed runs all failed at the same place or at a different place each time, and Table 5.2 lays out the path from symptom to the place to fix.

What is still open. Evaluation does not finish in this chapter. What counts as success, who judges it, and which tasks stand as the cases have to be settled before you can draw the range, and that design goes to Part VI. How to write tool descriptions goes to Part III.

Try this today. Open your system's prompt. Opening the system prompt alone is half of it, so pull the tool descriptions out beside it and *count* the two token totals together (Section 5.1). Next, read the system prompt one sentence at a time and *write* beside each what output it rules out. A sentence with nothing beside it is the first candidate (Section 5.2). Last, take one edit request that has come in and *hold* it against Table 5.2. If the failures are landing at a different place each run, that request does not belong to the prompt (Section 5.4).

5.6 Exercises

1. Pull all your tool descriptions into one place and compare their length with the system prompt. Which is longer? Then find out who edited those descriptions last quarter and

how many times, and what they used to judge that an edit made things better. By Definition 5.1 the two are the same object. Were they under the same discipline?

2. Count how many of the six places in Table 5.1 actually exist in your prompt. Build variants starting with the places that repeat most, and ask where in that range the score you report today sits. If the range comes out wider than you expected, which of last quarter’s improvement records does that fact make you reread?
3. Pull every sentence in your system prompt containing “must” or “never.” Rewrite each as “in this case, do this” and see whether the outputs it rules out change. For any sentence where they do not change, write down what that word was doing.
4. In a system with several kinds of task, consider the choice between one prompt covering every kind and separate prompts per kind. Splitting avoids the path Section 5.3 drew, and it multiplies the prompts you maintain and each one has to be evaluated separately. List what you need in order to settle that trade, and argue which side gains as the number of kinds grows.

5.7 On a second reading

- Read Chapter 7, come back to Section 5.1, and what was recorded only as tool descriptions sitting in the same place as the prompt has a design attached. How you choose names and arguments, and how long you write a description, sit there under the same discipline as editing the prompt.
- Read Chapter 22, come back to Section 5.2, and you see the stable prefix’s third role was a trust boundary and not just a contract. Writing down the boundary on behavior and defending that boundary against text that arrived during the run cannot be the same sentence’s job.
- Read Chapters 18 and 19, come back to Section 5.3, and format sensitivity has become a precondition of evaluation design. Here it only said to compare ranges. How many variants and how many tasks you draw that range with, and what you do when two ranges overlap, are those two chapters’ subject.
- Read Chapter 6, come back to the end of Section 5.3, and you see that asking whether it got better was never the prompt’s problem alone. What to cut down and how often meets the same question, and without evaluating you stall in the same place.

5.8 Further reading

The ground for Section 5.2’s advice to split into sections is Anthropic (2025c). It recommends putting background, instructions, guidance on tool use, and output conventions each in its own place, and it collects a list of other practical techniques alongside.

Section 5.3 rests on Sclar et al. (2024) on the format axis and Lu et al. (2022) on the order axis. The six places in Table 5.1 are this book’s reduction of that space of variations to a working prompt, so count the places in your own system by the papers’ definitions instead. Both are experiments on the models of a particular moment, so what to take away is not the size of the sensitivity but the warning that a score from a single setting cannot be trusted.

Lists of prompt-writing tips are well kept in each provider’s current documentation and get refreshed with every model generation. A sentence that earned its keep in one generation

becomes excess in the next, so copying a list across is exactly the mistake this chapter warned about. Whatever tip you meet, ask three things. Which of the stable prefix's three roles does it belong to? What outputs does that sentence rule out? And can its effect be evaluated as a range rather than a point? The list goes stale. These three questions stay.

Compaction and Summarization

Compaction is loss.

Replace a stretch of the history with something shorter and the original leaves the context, usually for good. So the question compaction asks is not *how much to cut* but *what to lose*.

And the loss is not something you can go around. Do not compact and the history grows with every step, and a grown history raises cost and lowers performance. Compact and the growth stops, but what you dropped does not come back. *Carry the performance you lose to a growing context, or carry the loss that comes out of compacting the history*. One of the two gets paid. There is no path where neither does.

That trade is the axis of this whole chapter. Every decision below is a decision about which side of it to pay and how much, and the difference between good compaction and bad is not whether there was loss but what you chose to lose.

6.1 The layer that grows

Chapter 4 cut the context into five layers. Instruction prefix, tool definitions, retrieved knowledge, history and observations, handoff artifact. Of the five, only history grows with the steps. The other four have their size settled before the run or at a session boundary and then sit there. Only history grows as observations and actions attach on every step, and it does not shrink if you leave it alone (Table 4.1).

The position is bad too. The system prompt is at the very front and the newest observation is at the very back, so the stretch that grows is the middle of the context. Of all places, the one that reads worst (Figure 4.3).

So whether to bring compaction in is decided by the trajectory of the step count, not by today's shares. A picture in which tool definitions take half the budget flips once the loop runs long. Only one layer has a slope.

Definition 6.1 (Compaction). Replacing a stretch of the history loaded in the context with a shorter representation of it is *compaction*. The original text of the replaced stretch leaves the context and generally cannot be recovered. Compaction is a lossy transformation. Send the original out of the context before replacing it and the loss is bounded to the inside of the context, and Section 6.6 makes that order a rule.

What the cache leaves behind

One thing to clear away before compaction. The claim that every step rereads the whole context from the start is only half true. Where two calls' token sequences run identically from the beginning, there is no reason to recompute, and that stretch can be stored and reused far cheaper. Call the stretch that runs identically the *leading run*.

Table 6.1: What the cache removes and what it leaves. Not one row has an empty cell in the column of what is left.

Item	What the cache does	What is left	Where it is covered
Rereading the same leading run	does not recompute it	the part newly attached	this section
Tokens loaded and counted	nothing	the whole history, growing each step	this chapter
Output billing	nothing	all of it. Summaries go out here	Section 6.5
The growth rate	cannot lower it	compaction takes it, or isolation does	this chapter, Chapter 16

There is one rule. The leading run has to match exactly, *so the moment you hit the first place that differs, everything after it is void* (Anthropic, 2026a). So put what does not change at the front and what does change at the back. Put a value that differs on every call at the head and one token costs you the whole leading run.

Another split opens here. Reading tokens you were given and producing new ones a token at a time have different bottlenecks and different prices. The cache helps only the reading side. The writing side, the output tokens, gets nothing from the cache and is the most expensive of the three axes. That fact returns in Section 6.5, because making a summary happens on exactly that writing side.

Remark 6.1 (The cache lowers the unit price and leaves the growth rate alone). In a loop where the leading run is preserved each step and new material only attaches at the back, the only thing the cache makes smaller is the unit price of the tokens reread. A token that hit the cache is still loaded in the context and still counted, so the rate at which the counted number grows with the steps is what it was before the cache was on.

6.2 The two purposes of compaction

Four things ride in the context together. The instructions saying what to do, the tool definitions saying what can be called, the documents retrieved, and the history so far. There is one window and the four divide it. So when the history grows, the other three lose share. And what they lose is not only room. The instructions get buried under the pile of history and lose their force, and a document retrieved a moment ago competes for space with a tool result from twenty steps back.

The symptom usually shows up like this. A rule kept reliably on short runs stops being kept around step thirty. Retrieval brings back the right document and it does not reach the answer. The hand goes to the instructions first, and what needs fixing is usually not the instructions but the size of the history. The instructions did not change. Their share did.

So *compaction is not done for the history. It is done for the other three*. Shrinking the history does not improve the history. Shrinking it lets the instructions be read again, lets the tool definitions earn their names, lets what was retrieved do its work. Which is why you evaluate

whether compaction succeeded by what the other three were doing, not by the stretch that was compacted.

The second purpose is quality, and this side has evidence. Jiang et al. (2024) starts from the observation that “LLM performance hinges on the density and position of key information in the input prompt.” It reports raising performance by compacting for density while cutting cost and latency at the same time. That compaction is a performance instrument and not only a cost-saving one is what gets taken from it here.

The two purposes mostly point the same way. Choose well what to drop and the length falls while the density of what remains rises. Where they come apart is where a design fails.

They come apart when the target is set as a compaction ratio. A ratio measures the length axis only. Two contexts cut by the same ratio can be opposites on the relevance axis. One cleared out source text that had gone irrelevant and got denser. The other cleared out the reason for a decision and went slack. The metric reports both as the same number. The loss does not show up on the bill. What was dropped turns out to be needed later and has to be fetched again, and the extra steps take back what one shortened step earned.

The point. The measure of compaction is density, not ratio. How much you cut is a metric of the means, and how related what remains is is the metric of the end. The moment you make a ratio the target, related material gets cut with the rest, and that loss is billed late, in quality and step count rather than in cost.

The grammar of the instruction splits here. Write this in a summarization instruction and the target becomes length.

“Cut the history so far in half.”

What to drop passes entirely to the summarizer’s discretion. Write it the other way around.

“Keep the list of files changed and the names of tests that are still failing, exactly as they are. Fit the length after you have kept those.”

Give the list of what has to stay, and leave length as what follows from keeping that list. The difference between the two sentences is the difference between ratio and density.

6.3 What can be dropped

There are three standards for choosing what to drop.

- **Can it be fetched back?** Start with what you can get again if you dropped it wrongly. What is still in the environment, the contents of a file or the result of a lookup you can call again with the same arguments, comes back with one round trip if dropped by mistake.
- **Did a recent step refer to it?** A stretch nobody has looked at again for several steps is the next candidate. This standard guesses the future from the past, so it is weak on its own. Overlap it with the one before. Something rarely referred to that can be fetched back is the first candidate.
- **Has it already gone stale?** A file’s contents before it was edited, an abandoned plan, an error message from before the fix. What has spoiled does not merely take up room. It misleads. Dropping without a trace is used here and only here.

Move that to the level of steps and the three fold into one sentence. *Can a later step reach the same decision again without it?*

Loss splits into two kinds. Drop the same number of tokens and the price differs.

- **Loss you can fetch back.** The original survives in the environment or in external storage and there is a way to bring it in. The worst case costs no more than one round trip to fetch it. Compaction should touch this kind first.
- **Loss you cannot fetch back.** What existed only in the context. Why this path was chosen, which alternative was rejected, what was tried and failed, what assumptions the current plan stands on. None of it is anywhere in the environment. Making it again means living through those judgments and attempts again.

Cannot be fetched back does not mean do not touch. There is one more fork inside it. Dropping without a trace and cutting down are different things, and what is forbidden for what cannot be fetched back is only the first. Note, though, that once you cut something down, the shortened representation becomes the only copy. So what to keep cannot be the summarizer's discretion. It has to be a list written in advance, and Section 6.6 sets that list up.

The most dangerous thing is the record of failure

Of the losses you cannot fetch back, the most dangerous is the record of failure. Three reasons.

First, failure leaves no artifact. Work that succeeded leaves files and verification results in the environment as traces. A failed attempt does the opposite. Say an agent tried an approach that edited a configuration file, found that it broke another module, and reverted the edit. The file is back as it was and the checks pass again, so the environment holds no trace of the attempt. The fact that the road was taken and was blocked lives in a few sentences of the context and nowhere else. So the cost of fetching a failure record back is not one lookup. It is the cost of living through that failure again.

Second, the record of failure is the very information that moves the loop forward. The self-reflection pattern in Chapter 2 was a structure that summarized failure into language and fed it as input to the next attempt, and the whole value of that pattern hangs on preserving the failure record.

Third, the symptom when you lose it is quiet. No error is raised. Steps that confirm the same dead end again simply accumulate, and those revisits come back as cost and latency through Chapter 3's arithmetic.

The basic direction of compaction falls out here. A stretch of the history is usually made of the source text of an observation and the conclusion drawn from it. If 300 lines of a file are the source text, "the function we are looking for is not in this file" is the conclusion. The source text is large and can be fetched back. The conclusion is small and cannot. *So drop the source text and keep the conclusion.* What Anthropic (2025c) lists as worth preserving when summarizing a history is the same family: it keeps "architectural decisions, unresolved bugs, and implementation details while discarding redundant tool outputs or messages."

6.4 The methods of compaction

There are four things you can actually do. Cut it out, clear a tool result, replace it with a summary, or put it outside and point at it. Table 6.2 lays the four side by side, and below is when to use each.

Table 6.2: The four methods of compaction. The top two can be decided by kind without reading the content, so they are cheap. The further down, the deeper the judgment, and the depth of the judgment is the risk.

Method	What it does	What it loses	When to use it
Clearing tool results	erases the result of a tool already called and keeps the call itself	the result's source text. Call again with the same arguments and it comes back	nearly always the first candidate. No judgment needed and the loss is the fetchable kind
Cutting it out	removes an old stretch whole without reading it	all of that stretch. Source text and conclusion go together	only on spoiled stretches. Stale plans and errors from before a fix
Putting it outside and pointing	moves the stretch to a file or a store and leaves a pointer	immediacy. Seeing it again spends a step	on what is large, fetchable, and may or may not be looked at again
Replacing with a summary	the model summarizes its own history in prose and swaps it for the original	everything the summarizer judged less important. The record of failure wears away first	only on the narrative left over after the other three

Clearing tool results is the first candidate because it needs almost no judgment. Anthropic (2025c) points out that there is no reason to keep showing the raw results of tools already called deep in the history, and names this “one of the safest lightest touch forms of compaction.” It holds by the previous section's standards too. A tool result is usually the fetchable kind of loss. The default is to keep the tool *call* and erase only the result, which is how the record of what has already been tried survives.

In place of judgment there are four values to set. When it turns on, how many of the most recent to keep, how little is too little to clear at once, and which tools' results are never cleared. The last two get left out most often in practice. Clearing a little at a time and often only breaks the cache and gains nothing, so one clearing has to be worth breaking the cache over (Section 6.5), and a tool whose result is itself an artifact belongs on the do-not-clear list. Intermediate reasoning the model produced is handled the same way. It can be lifted out by kind alone, and there is one value to set, how many recent turns to keep.

Cutting it out is the crudest. It does not look at what is being removed, so that stretch's conclusion goes with it. Use it only on what has spoiled. Cut the front of the history just because it is old and you usually cut the most valuable part, because the early steps hold the definition of the task and the first judgments.

Putting it outside and pointing changes who owns the size. The length of a conversation is tied to the step count. The size of a file is tied to the structure of the task. Chapter 15's principle of artifacts rather than records is exactly this (Section 15.2), and the structured artifacts Rajasekaran (2026) names as the means of handoff between sessions are the working version. What you lose is immediacy alone, and seeing it again spends a step.

Replacing with a summary comes last. Its judgment is the deepest, so it has the most room to improve the relevance axis, and the risk sits in exactly that judgment. The summarizer is usually the same model that made the history. The narrative Chapter 2 called a story that justifies the

conclusion, and that Chapter 15 said not to hand a judge, seeps into the summary. Progress gets inflated and failures get smoothed over. The record of failure, which this chapter called the most dangerous loss, is the thing that wears away first, and here is where.

So what you actually design in this method is not an algorithm but the *summarization instruction*. Anthropic (2025c) gives the order for tuning it: “Start by maximizing recall to ensure your compaction prompt captures every relevant piece of information from the trace, then iterate to improve precision.” Write it generously at first so nothing is missed, then take material out of the summaries you get and tighten. Reverse the order and there is no way to find out what went missing, because what went missing is not in the summary. The same material carries a working example. An implementation that, after summarizing the history, continues with that compressed context plus the five most recently accessed files. A lossless handful pinned beside a lossy transformation, and one answer to what you leave to the instruction and what you nail down as a rule.

The four are not exclusive and they have an order. Do what can be decided by kind alone first, move out what can be moved out, and summarize the narrative that is left. Summarize first and the summarizer faces something larger than it needed to, and the larger it is the more judgment it demands, and that judgment is the most expensive thing in this chapter.

Not everything is yours to build

Two of the four are no longer code you write. Clearing out old tool results and replacing a history with a summary come as parameters on the request. The values you pass are the ones set above. The first takes a threshold, a number to keep, a minimum to clear at once, and the tools to leave alone. The second takes one threshold and one instruction. There is a separate place to say what gets attached verbatim after the summary.

Keep what changed apart from what did not. The algorithm has left your hands. The instruction, the preservation list, and the thresholds stay. And one more thing: pass no value at all and a default does the work. Compaction happens in systems where it appears in neither the code nor the prompt. If you did not decide where the cut falls, you decided not to decide.

6.5 Compaction is not free

One compaction costs in three places.

First, *the compaction call itself*. Making a summary is a model call, so it costs input to read the history being compacted and output to write the summary, and this call is billed apart from the loop’s steps. And a summary is output. Output is the most expensive axis, the one the cache cannot help (Table 6.1). The reason compaction stings is in that order. *The summary comes out of the most expensive axis, and the place the summary cleared gives back the cheapest.*

Second, *cache invalidation*. Compaction edits the context in the middle, so the leading run from the replacement point on goes void (Figure 6.1). That cost can be cut by placement. Set a separate cache boundary at the end of the instruction prefix and compaction will not break the prefix’s cache. Only what sits after the replacement point goes void.

Third, *information loss*. Drop the wrong thing and refetching and revisiting add steps. This is the one cost that never appears on the bill.

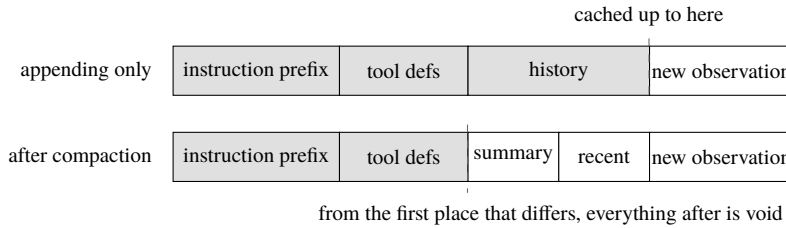


Figure 6.1: Reuse of the leading run, and its invalidation. The shaded part is what gets reused. On a step that only appends, all of the front is reused and only what newly attached is read fresh. Compaction edits the middle, so from the edit on, everything is read fresh. That is why the cost of compaction includes this invalidation on top of the cost of writing the summary.

How often to do it

Point at the two extremes and the shape shows. Compact on every step and you pay the output the cache cannot help with every time, and you edit the middle of the context every time so the leading run goes void every time. Never compact and the context grows with the steps and every step rereads all of it. Both ends lose, and they lose in opposite ways.

Proposition 6.1 (The trade in the compaction interval). *The more often you compact, the more compaction itself costs, and the more rarely you compact, the more rereading the history costs. The two costs move in opposite directions with the interval, so there is one interval between the extremes at which their sum is smallest. Compacting more often is not better, and neither is putting it off.*

Fix the interval at one number and the rest follows. Right after each compaction the context returns to the same size, so the cost per step never crosses a certain line, and so however far the step count runs, the total input cost does not grow faster than the step count. This is what the cache could not do.

Where that interval sits is set by three numbers. How much history attaches per step, how large one summary is, and the ratio between the price of rereading and the price of writing fresh. All three can be measured in today's step log.

Of the three, the summary's size does the most work. A large summary makes one compaction expensive, and expensive compaction has to be done rarely. *A system that writes long summaries compacts that much less often.* That one sentence gets used the most in practice.

There is one more thing to take. *Compute in steps and configure in length.* The place you hang compaction is usually a threshold on length rather than a step count, so a conversion is needed. Add to the size right after a compaction the amount that piles up over one interval and you have the length at the end of that interval. If the stable prefix is 2,000 tokens, the summary is 400 tokens, 300 tokens attach per step, and you decided to compact once every twelve steps, the number to write into the threshold is around 6,000 tokens. Without that conversion, the interval is a number you computed with nowhere to write it down.

If the context just before compaction goes past Chapter 4's effective length, then the interval the cost calls optimal is an interval you cannot use. Put the expected cost of dropping the wrong thing into the arithmetic and one compaction costs more and the optimal interval gets longer. A correction whose direction matches the intuition: the lossier the compaction, the more rarely it should run.

6.6 The problem with repeated compaction

Once compaction repeats, a new problem starts at the second round. The input to the compaction contains the previous compaction's summary.

Run "summarize the history so far" every time the interval comes up and each round takes the previous summary as its material. A summary of a summary of a summary. The length stays well controlled. The trouble is on the loss side. One summary is a selection from the original. Even a wrong judgment is one layer deep, and as long as the original survives you can check what got cut. The tenth summary is a selection from what survived nine rounds of judgment. No round's judgment is made knowing what a later round will need, and what was dropped is not even in the next round's input, so that misjudgment is permanent. Loss only piles up with the rounds. It never comes back. Round after round, the summary stops being a record of what happened and becomes a record of what earlier summaries judged important.

This failure is the exact opposite of Chapter 15's handoff. A handoff that only appends summaries keeps the information and grows by that much every round. Repeated summarization holds the size down and shaves the information every round instead. They also break differently. Grown size shows up on the bill immediately. Shaved information shows up late, as the same failure repeating and as the success rate.

Which of the two you do is set by the task boundary. Chapter 15 split what you can do at a threshold in two. Shrink the history and carry the same session on, or close the session and leave a handoff. If the work at hand is done, closing is better. If it is not done, compact and carry on. What settles the fork is not the threshold but where the task stands.

The answer is not to drop a reasonable amount somewhere between the two extremes. It is to give the two demands to *different mechanisms*. Size is held by a summary that gets rewritten. Items finished inside the summary get deleted, and the size is tied to the size of the live task rather than to the round count. Preserving the information is the job of what sits outside the context. Give that outside place a name.

Definition 6.2 (Origin). The place where the content of a compacted stretch still sits, outside the context, is that stretch's *origin*. With an origin, a summary is a derivative you can always make again. Without one, the summary is the original.

With an origin, the rule for resummarizing settles. *The input to a compaction has to be the origin, not the previous summary*. Summarizing a previous summary stacks loss on loss. Summarize again from the origin and each time you select fresh, with that moment's judgment.

Where the origin sits differs by system, and having none is the default. A mechanism that cuts a history down usually does not keep the pre-cut history for you. What came before the shortened representation is not loaded on the next request, and the next summary gets made on top of what is already shortened. Which means that if you do nothing, a summary of a summary is what happens by default.

So the origin is not far away. Before it is searchable storage or memory that crosses sessions, it is the raw message list the harness is already holding. Keep hold of that and you have an origin. Keep only what goes on the request and drop the rest and you do not. That choice usually does not get made out loud. What has to survive longer goes to Part IV and Chapter 10, but the origin this interval needs is already in hand without waiting for those chapters.

What has to survive regardless of the round is not left to the summarizer's discretion either. The definition of the live task, the list of failures and their reasons, the decisions still open. The list of losses you cannot fetch back becomes the preservation list as it stands. *What cannot be*

fetch back is not a candidate for dropping. It is a candidate for keeping. That is not a rule against cutting it down. It is a rule that, because the shortened representation becomes the only copy, what to keep has to be written in advance.

Compaction is the one standard operation of the loop that leaves no trace. Truncation leaves a handoff artifact, a file edit stays in the environment, and even a wrong action is recorded as an observation. The discipline for an operation you cannot undo is the same everywhere. *Set the original's destination before you run it.* The harness can enforce the order. Send the stretch to be compacted out to a record outside the context first, then replace it. Then compaction is a move rather than a destruction, and the premise of resummarizing from the origin is provided by that same order. What is left to the model's discretion should be writing the sentences of the summary inside that frame, and nothing else.

The largest gap is on the loss side. To know what got cut you have to compare against a run that was not cut, and that run usually does not exist. So saying the interval is set by computation is half true. The cost half is computed and the loss half is estimated. A designer who knows about that asymmetry sets the interval longer than the computed value, which is to say compacts less.

6.7 When not to compact

If any of the following holds, it is not time to bring compaction in.

1. **The history is short.** If the task runs a few steps it ends before the moment to compact arrives. The compaction mechanism bills its own cost and its maintenance complexity and saves nothing. If the context falls well short of the effective length measured on your own task, there is no quality gain either.
2. **There is no way to fetch back what you lose.** With no record of the original anywhere outside the context and no tool to refetch, every loss gets promoted to the kind you cannot fetch back. Reverse the order. First put the mechanism in place that keeps the raw records and artifacts outside, then compact. Compaction is a technique that is safe when there is an origin (Definition 6.2).
3. **There is no way to measure the quality of compaction.** The gain, tokens removed, is visible immediately. The loss, quality and revisits, shows up late and scattered. Without evaluation, compaction always looks like a gain. What has to be evaluated is task success rate, total cost including the cost of compaction, step count, and the frequency of refetch calls. Leave the invalidation loss out of the arithmetic and even the cost can go up (Remark 6.1).

The three share a shape. Compaction is a technique whose gain and whose loss are observed at different moments. The tokens removed get counted on that step. The cost of dropping the wrong thing gets stamped much later, and without compaction's name on it.

So do not finish a compaction mechanism switched on. Bring it in able to be switched off. Chapter 15's remedy of turning a mechanism off to test whether it has expired applies here with more weight. The profit and loss of compaction are only fully observed against a run with the mechanism off. Compaction with no off switch stays forever with only its gain observed. The less immediately you can see the result of an operation, the more you have to bring in the means of doubting it alongside.

6.8 Summary

In one sentence. Compaction is a lossy transformation you cannot undo, so the measure is not how much you cut but what you kept. And more often is not better. Both ends lose, so the best interval sits somewhere between them.

What is now settled.

- Loss is not optional. You carry the performance lost to a growing context or you carry the loss from compacting. One of the two gets paid. The only layer that grows with the steps is history, and the cache shaves the unit price of rereading without lowering the growth rate (Remark 6.1, Table 6.1).
- There is one window and instructions, tool definitions, retrieved knowledge, and history divide it. When history grows the other three lose share. So compaction is done not for the history but for the other three, and you read whether it worked in what the other three were doing rather than in the compacted stretch (Section 6.2).
- The measure of compaction is density, not ratio. A ratio measures the length axis only, so two compactations of the same ratio can be opposites on the relevance axis, and the loss from a bad compaction is billed late, in quality and step count. Write a preservation list into the instruction, not a ratio.
- There are three standards for dropping. Can it be fetched back, did a recent step refer to it, has it already gone stale. In one sentence, *can a later step reach the same decision again without it*, and the direction is to drop the source text and keep the conclusion. The most dangerous loss is the record of failure. It leaves no trace in the environment, so fetching it back costs what living through that failure costs, and losing it raises no error, so the symptom is quiet (Section 6.3).
- There are four methods and they have an order (Table 6.2). Clearing tool results, cutting it out, putting it outside and pointing, replacing with a summary. Do what can be decided by kind alone first and put summarizing last. Two of them now come as parameters on the request, and the values to set, the instruction, and the preservation list stay yours.
- One compaction costs in three places. The output that writes the summary, the cache invalidation from the replacement point on, and what you dropped wrongly. The first two move against each other with the interval, so there is one best interval (Proposition 6.1). What sets it is the increment per step, the summary's size, and the ratio of the prices, all three measured in today's log. Compute in steps and configure in length, and the ceiling on the interval is set by the effective length rather than by cost (Section 6.5).
- Repeated summarization piles loss up round by round. Size is held by a summary that gets rewritten and information is held by the origin outside (Definition 6.2). The input to a compaction has to be the origin, not the previous summary, and the nearest origin is the raw message list the harness is already holding. Do nothing and a summary of a summary happens by default. What cannot be fetched back is a candidate for keeping, not for dropping (Section 6.6).

What is still open.

- The raw message list in hand is enough of an origin for one interval, and it disappears when the run ends. The mechanism that keeps and refetches across runs, which is retrieval, goes to Part IV, and memory that crosses sessions goes to Chapter 10.
- The artifacts that putting it outside and pointing leans on are made by tools. Design on the tool side, having observations come back small and structured from the start, goes to Part III and to Chapter 7 in particular.
- Compaction is a remedy inside one vessel. Dividing the vessel itself so that each context's history stays short goes to Chapter 16.
- What a preserved record of failure is good for outside the loop, which is the run record as evaluation material, goes to Part VI from Chapter 18 on.

Try this today. Take one late step's input and *count it in four parts*. How many tokens are instructions, tool definitions, what was retrieved, and history? If history is past half, the other three are already being pushed out (Section 6.2). Next, take the largest tool result in that history and *find the last step that used it*. The number of steps it has ridden along unread since is what you can save right now (Section 6.4). Last, *look* at one place in the code. If it keeps only what goes on the request and drops the raw message list, this system is cutting down what it already cut down (Section 6.6).

6.9 Exercises

1. Name one thing in your system that compaction might drop wrongly. Does losing it cost one lookup, or does it cost living through the work again? As that cost rises, which way should the interval move, and which section of this chapter holds the means of lowering that cost in advance?
2. The same job of holding the context from growing can be done by compacting inside a session or by truncating the session and handing off (Figure 15.2 in Chapter 15). Write the per-step cost of each side by side and argue what settles the choice.
3. Take one real stretch of history and apply the four methods in Table 6.2 to it in turn. Measure the tokens removed by each, and sort the information that disappeared into loss you can fetch back and loss you cannot. In the log after that stretch, was there a case where the vanished information was needed again?
4. Design a comparison that summarizes the same history with an instruction that names a preservation list and one that does not. How will you measure whether the failed attempts survived into the summary, and the frequency of revisits afterward? Argue by the principle in Section 15.5 whether the same model as the summarizer may judge that summary.

6.10 On a second reading

- Read Part IV and Chapter 10, come back to Definition 6.2, and the origin acquires the name of a mechanism that outlives the run. The two differ only in lifetime.
- Read Chapter 7, come back to Table 6.2, and you see that when a tool returns something close to an artifact from the start, the history never grows as narrative in the first place.

- Read Chapter 16, come back to Proposition 6.1, and you see that the arithmetic assumed one vessel. Isolation solves the same problem with the number of vessels rather than with an interval, and pays the cost of handoffs between vessels instead of the loss of compaction.
- Read Chapter 18, come back to the preservation list, and you see that the record of what was tried and why it failed is itself the raw stone of an evaluation case. The preservation list is a discipline for the session and a collection rule for evaluation material at once.

6.11 Further reading

The two newly cited papers treat compaction as different problems. Jiang et al. (2023) sees it as a problem of allocating a budget, and Jiang et al. (2024) sees it as a problem of performance. The latter is the empirical ground for the claim that compaction can raise quality. The algorithmic detail in both gets refreshed quickly, though, so read the problem setting rather than the technique. The working version of putting it outside and pointing is in Rajasekaran (2026).

PART III

Tools and the Outside World

Tool Design

There are places where fixing the tools pays better than fixing the prompt. What the model can do is set by the tool list, and an act that is not on the list does not happen under any instruction.

Two things to answer here. What structure does one tool ride in on, and what round trip does it make? And how do you fill each place in that structure so performance goes up?

Both questions look at the same four things. The tool name, the tool description, the arguments, and what the tool returns. Where the tools come from goes to Chapter 8, and turning what comes back into a contract goes to Chapter 9.

7.1 The anatomy of a tool

One tool is written down in three places. The tool name, the tool description, and the input schema. The result is written nowhere.

Fill the three places with something real and it looks like this.

Tool name: `search_orders`

Tool description: “Looks up a customer’s orders. Use it when you have an order number or a customer identifier. It returns the last 90 days only, and older orders do not appear. It does not carry payment details or shipment tracking.”

Arguments: `customer_id` is a string, required. `status` is one of `pending`, `shipped`, `delivered`, and means all of them if left out. `limit` is an integer and defaults to 20.

The tool name has a constraint. Letters, digits, underscores, and hyphens only, and no more than 64 characters (Anthropic, 2026j). The description and the input schema have no length limit. Section 7.2 shows that the side with no limit is the side that costs.

That the result is not in the definition is not a detail to wave past. The only way the model knows in advance what comes back and in what shape is the one line of the tool description. Everything else it learns by calling. It also means that however the designer cuts the result down before sending it, there is no place to check that. So the result is not the last thing this chapter counts but the heaviest, and Section 7.4 takes it.

Where these three ride

The three places become text and ride along in the request. What the model sees is not a tool but this text. It cannot see the code and does not know the actual behavior.

The cost starts here too. Hand over even one tool and a notice explaining how to use tools attaches automatically. On a recent model that notice alone is around 300 tokens, and forcing a tool to be used pushes it past 400 (Anthropic, 2026j). On top of that comes each tool’s name and description and input schema. All of it is settled before the run and reloads in the same place on every step.

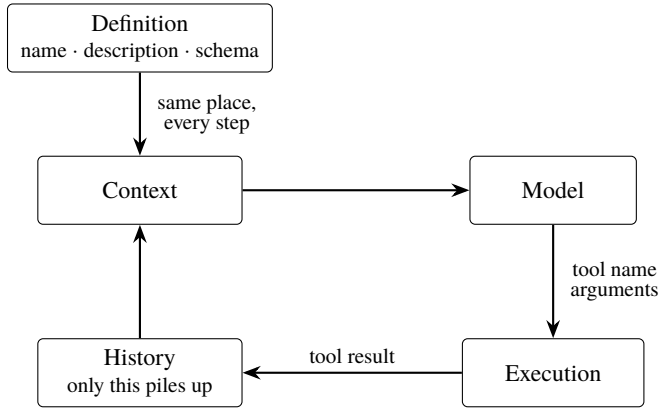


Figure 7.1: The round trip of one tool. Definitions only go in, and results pile up. Add one line to a definition and the same place on every step gets that much longer. Return one large result once and every step after it carries that result along.

The round trip

The model does not run the tool. It puts out one tool name and one set of arguments.

Taking that output and calling the actual function is the harness’s job. The harness has to take the result and put it back into the context before the model sees it. Figure 7.1 is that round trip.

Two places in this round trip have different accounting. A definition reloads at the same size on every step and never accumulates. A result loads once and then rides along forever after. Section 7.2 handles the first place, Section 7.4 the second.

Acts that are not on the list

An agent has two tools, one that reads a whole file and one that writes a whole file. The task is to change 3 lines in a 300-line file.

Whatever you write in the prompt, all this agent can do is read 300 lines and write 300 lines back. There is no way for these two tools to answer “change only what needs changing.” Not because the instruction is bad. Because the act of partial editing does not exist in this system.

Definition 7.1 (Action vocabulary). Two things set what an agent can actually do. Which tools exist, and what arguments each tool accepts. What the harness runs is only a call made of one tool and one set of arguments that passes that tool’s input schema. Those calls, plus a natural-language reply that does not act on the environment, are the system’s *action vocabulary*, and this is the action space of Chapter 1.

Only one kind of failure gets fixed by prompting. The failure where a worse tool gets picked out of the ones that exist. Failures where the needed tool does not exist at all, or where the tools are cut along the wrong lines, survive any sharpening of the instructions.

There is evidence too. Yang et al. (2024) left the model alone and changed only the interface, and reports a large change in how well it handled files, moved around a repository, and ran programs.

Note also that the natural-language reply never leaves the list. You cannot forbid talking. So when the needed tool is missing, the model does not go quiet. It talks its way through. It invents what it could not look up. *A hole in the vocabulary gets filled by invention, not by silence.*

When a run fails, the three things a team suspects are usually the prompt, the model, and the context. Tools are not on that list. The tool list gets set when the system is first built and then becomes background, and because it does not change it does not get suspected. A cause that never reaches the diagnosis list never gets fixed. The first question in a failure analysis should not be *was the instruction bad* but *was there a tool for this task*.

7.2 The name and the description set the performance

All the model has to go on when it picks a tool is the name and the description. So the name and the description are the performance.

The tool name

Scanning the list, the token read first, most often, and most reliably is the name. The description gets read only after the name's expectation lets it through.

And a name does more than settle the choice. It creates an expectation. The model has seen enormous amounts of code in training and learned from it which names go with which behavior. A name drags that whole convention in with it.

Put them side by side and it shows.

Bad names: `get_data`, `process`, `handle_request`, `do_action`, `helper`

Good names: `search_orders`, `cancel_subscription`,
`get_customer_context`, `schedule_event`

The left side does not say in the name what it does, so it pushes the whole choice onto the description. The right side has a verb stuck to an object, so candidates get filtered out before the description is read at all.

When the expectation and the reality disagree, the name usually wins. It is short, it is always in the list, and it has a hardened convention behind it. The clues in the description are long and buried among others. One of the two failures Patil et al. (2024) points at is a tendency to hallucinate the usage of an API call, and a hallucinated usage is not random noise. The model does not get things wrong at random. It gets them wrong plausibly, and the material of that plausibility is exactly the expectation the name created.

Three rules for naming come out of this.

- **Do not fight the convention.** Using a name that convention will read another way and then correcting it in the description is adding sentences to a fight you will lose. What has to change is the name.
- **Name against the list.** Two names that look good alone regularly blur each other inside one list. The test is one line. *If a person cannot say for certain which tool to use, the model cannot either.*
- **Give a group a shared prefix.** Tools that touch the same service or the same resource get a common front. Anthropic (2025g) recommends namespacing tools “by service (e.g., `asana_search`, `jira_search`) and by resource (e.g., `asana_projects_search`,

`asana_users_search`}).” However long the list grows, the name still says which group a thing belongs to.

The same discipline binds argument names. Write `user` and it is not settled whether a name or an identifier is coming. Write `user_id` and it is. The same source puts it directly: “instead of a parameter named `user`, try a parameter named `user_id`” (Anthropic, 2025g).

The tool description

What goes into a description? Anthropic (2025g) gives the frame: “think of how you would describe your tool to a new hire on your team.” Do that and the list falls out. What does the tool do? When do you use it and when do you not? What does each argument mean and how does it change the behavior? And what does it *not* return?

That last item goes missing most often. It is obvious to whoever built the tool, so it does not get written, and the model has no way to know, so it calls the tool believing something is there.

The published contrast, carried over as it stands, looks like this. Two descriptions of the same tool (Anthropic, 2026j).

Poor: “Gets the stock price for a ticker.”

Good: “Retrieves the current stock price for a given ticker symbol. The ticker symbol must be a valid symbol for a publicly traded company on a major US stock exchange like NYSE or NASDAQ. The tool will return the latest trade price in USD. It should be used when the user asks about the current or most recent price of a specific stock. It will not provide any other information about the stock or company.”

The poor one is a single sentence. When this tool is the right one, what it returns, and what it does not return are all left open. The good one is five sentences and closes all four. The standard is readable off that contrast. *Three or four sentences per tool at least, and more for a complicated one.*

7.3 Designing the tool itself

If the name got the act chosen, the arguments get the chosen act filled in. And the input schema is the only constraint a designer can put on the model’s behavior that gets enforced decisively. Everything else lives in the prompt, which is to say in probability.

Write the same constraint in two places and the difference is plain. Put “always name the target identifier when you call this tool” in the prompt and it is mostly followed, occasionally not, and the call that did not follow it runs anyway. Write the same constraint as a required argument and a call with no identifier gets rejected mechanically before it reaches execution.

Remark 7.1 (Enforcement and instruction make a violation a different event). Suppose the harness checks the arguments against the input schema before running a tool call and returns a violating call as an observation of refusal. Write the same constraint as an instruction in the prompt and it only tilts the distribution of behavior. Then (a) a violation of a schema-enforced constraint is blocked before it reaches the environment and is always an observable refusal, and (b) a violation of an instructed constraint runs with nonzero probability, and whether it was violated is known only from the result of the run, and only when the result leaves a trace of the violation.

The weight is less in the difference in probability of a violation than in the difference in its *standing*. An argument violation is an event before execution, so there is nothing to undo, and a refusal stays as an observation the model can fix on the spot. An instruction violation is an event after execution, so its effect stays in the environment.

Making arguments easy to fill

The other failure Patil et al. (2024) points at is not generating accurate input arguments. Four adjustments cut that down.

Use fewer arguments. More arguments means more places to fill, and every place is one more chance to get it wrong. One required argument with one or two optional ones is the easiest shape to fill. Past five is usually a sign that one tool is doing two jobs, so look at splitting it first.

Enumerate what can be enumerated. A free string gets filled by sampling, and sampling wobbles. Write the same place this way and there is nothing left to wobble.

Before: `status` is a string. The description says “write the order status.”

After: `status` is one of `pending`, `shipped`, `delivered`, `cancelled`.

In the first, the model might write “in transit” or it might write “shipping.” In the second, a value outside the four never reaches execution.

Give it a default. If what happens when you leave it out is settled, the model can leave the place empty. One default removes one argument. The `limit` above is that.

Change the shape so it cannot be got wrong. Some arguments get filled wrong no matter how you explain them. Then do not lengthen the explanation. Change the argument. The published case is file paths. A tool took relative paths, and the model kept getting the path wrong after changing directory. Switching to absolute paths only made the error disappear. A relative path is only right if you remember where you are. An absolute path has nothing to remember. Removing the place where you can be wrong beats writing down that you should not be wrong.

Look at the format with the same eye. There are shapes a model finds easy to produce and shapes it does not. In a tool that edits files, a format that takes only the changed lines is right only if you counted the line numbers first. A format that takes the whole file has nothing to count. A format that needs counting hands the failures of that counting to the tool.

An argument the model cannot fill is not an argument

Argument errors come from two sources, and the remedies are opposites.

One is what the input schema did not say. The format is open, the allowed values live only in the prose, the unit is implicit. This side you remove by tightening. The four above do that job.

The other is what the model cannot know. The target’s identifier, a path in the execution environment, values there is no way to know unless they are loaded in the context. Tightening this side makes it worse. A place forced to be required has to be filled before execution is reached, and if the model does not know the value, that place gets filled with an invented one. *Enforcement does not remove absence. It calls up invention.*

And an invented value is quiet. An invented identifier is the right shape, so it passes the argument check and runs. It comes back as an error about a target that does not exist, or worse, as a success on the wrong target.

Table 7.1: The trade in where you cut. Splitting and merging do not remove errors. They move where the errors sit, between choosing and filling.

	A finely split list	A coarsely merged list
Choosing	many candidates, and neighbors look alike, so the wrong one gets picked	few candidates, so choosing is easy
Arguments	few and simple per tool	more arguments and conditional requirements widen the surface for argument errors
Definition layer	more definitions, so the share on every step grows	fewer definitions, but each one longer
Where errors sit	the wrong tool gets picked	the right tool gets filled wrong
Blast radius of a change	changing one act stays inside one tool	changing one act shakes a shared input schema

So one question stands up before you mark anything required. *Can the model know this value from the context at that moment?* If it cannot, the remedy is not in the input schema but in the supply route. A tool that finds that value has to be next to it, and the tools have to be strung together so that one lookup’s result becomes the next call’s argument.

Where to cut the tools apart

So far, one tool. Now, the list.

Cut the acts finely into many tools and each tool gets simple. The names get precise and the arguments get few. In exchange the list gets long and choosing gets hard. Merge them coarsely into one tool and choosing gets easy, but inside it a mode argument and conditional requirements appear and filling gets hard. Neither direction removes the errors. *It moves where the errors happen.* Table 7.1 is that trade.

A trade does not mean the default sits in the middle. The direction is clear. More tools do not make results better, and rather than wrapping every endpoint of an outside system one at a time, it is better to build a few tools aimed at high-value workflows (Anthropic, 2025g). Three published contrasts, carried over as they stand.

Instead of `list_users`, `list_events`, and `create_event`,
one `schedule_event` that finds a free slot and books it.

Instead of `read_logs`, a `search_logs` that returns the relevant log lines with their surrounding context.

Instead of `get_customer_by_id`, `list_transactions`, and `list_notes`,
a `get_customer_context` that gathers a customer’s recent activity at once.

All three do the same thing. A job that takes three calls by hand gets handled inside one tool, and the model makes one decision instead of three choices and three fills.

So the standard for where to cut is not the size of a tool but the *unit of decision*. An act that one decision of the model completes is one tool. Splitting apart acts that are always called together is splitting one decision into several choices. When the cut matches the unit of decision,

choosing and filling improve together. That is the only way out of a trade where one side rises as the other falls.

Ask where the default came from as well. If nobody decides, the tool boundaries get inherited from the outside system's boundaries. Wrap an API with five endpoints and you get five tools. It looks natural, and it is the boundary of an implementation, not the boundary of a decision. A boundary drawn for human developers assumes a user who can carry state between calls. The model's state is the context and nothing else. Did you design the boundary, or inherit it? That is the first question in an audit of a tool list.

Not loading all of the definitions

Some systems have a large list even after the tools are cut down. Then the road left is not to load all of it.

Keep the name and a one-line description loaded at all times, and read the input schema and the full description in only after the need is confirmed. The same mechanism applies to the instruction layer.

Definition 7.2 (Progressive disclosure). Split what can be loaded into the context into levels. Keep the upper level's summary loaded on every step, read the lower level's body in at the moment its summary confirms a need, and for the executable part, bring in only the result of running it rather than the part itself. That way of loading is *progressive disclosure*.

There are three levels (Anthropic, 2025f). The first is the name and the description, loaded from the start and always present. The second is the body of the instructions, read in when the agent judges it relevant. The third is the reference files and scripts the body points at, opened only when the body has been read and something is actually needed. The third level is where this structure gets its power. The rest sits on the file system and the agent opens it when it needs it, so "the amount of context that can be bundled into a skill is effectively unbounded."

Definition 7.3 (Skill). A *Skill* is a unit that bundles instructions on how to carry out one kind of task, together with optional scripts and resources, keeps it outside the context, and loads it by progressive disclosure through its summary.¹

It is not free. Loading everything always costs for certain and is there for certain. Progressive disclosure costs less and has no guarantee of being called up when it is needed.

How selection fails

Three failures. *Absence* is when the summary fails to signal its relation to the task at hand and the body never gets called up. The run goes all the way through without the instructions and only the result differs. *Wrong pick* is the opposite. A bundle that was not needed gets called up and takes budget and attention with it. *Non-compliance* is when the body was loaded and the instructions were not followed, and this is not a loading problem but a prompt problem, under the discipline of Chapter 5. Only the first two are the summary's responsibility, which is why the effort belongs in the summary and not the body. The same holds when you pick which tool definitions to load. What loads is the name and one line, and the discipline of naming that Section 7.2 set up decides whether the selection works.

¹The name and the structure come from Anthropic (2025f). The file layout and conventions of an implementation are left out. They are the part that will go stale first.

Table 7.2: Four kinds, cut by how they load. Going down, ownership of execution moves from the model to the environment and then outside the system.

	Who runs it	What loads	When it loads
Prompt	the model reads and follows	the full text of the instructions	settled before the run, on every step
Skill	the model follows the instructions, the environment runs the executable part	the summary always, the body only when selected, the executable part only as a result	the summary on the first step, the body during the run by the model's choice
Tool	the environment runs it	the definition (name, description, input schema) and the call's result	in the default layout the definition before the run, the result at call time
Protocol	an outside provider runs it	the supplied tools' definitions and results, and resources	the list of definitions itself is set by a connection at run time

Asking what is a Skill and what is a tool by function goes in circles. Ask by loading and it holds still. Three questions. Who runs it, what goes into the context, and when does it go in?

Inside Table 7.2 the grey areas find their places. A Skill carrying a script splits into two components inside the second row, and a tool description swollen with guidance sits in the third row while paying the first row's cost. The last row's working example is the Model Context Protocol of Chapter 8. Every row pays something new. The first pays tokens and verification, the second adds the cost of selection, the third adds execution rights and safety, the fourth adds having to trust a provider. The further down, the wider the range of things you have to take on trust without having reviewed them, and Chapter 22 prices that risk.

7.4 Tool output inside the context

If the definition is the side going in, the result is the side coming out. And the side coming out is heavier. What a tool returns is an observation, observations pile up in the context, and the context sets the next action. *Setting the shape of the result is setting the input of the next step.*

What goes back and forth, in what shape

What the model puts out has this shape. An identifier for one call, the tool name to call, and an arguments object (Anthropic, 2026j).

```
id is toolu_01A09q, name is search_orders,
input is {"customer_id": "C-4471", "status": "shipped"}
```

This is a request, not an execution. The harness finds the real function under that name, passes it the arguments, calls it, takes the result, and sends it back in a paired block. That block carries three things. An identifier saying which call it answers, the body of the result, and a mark for whether it was an error.

```
Success: tool_use_id is toolu_01A09q, content is
"3 orders. C-4471. In transit..."
```

Failure: the same `tool_use_id`, `is_error` is true, `content` is
“Rate limit exceeded. Retry in 60 seconds.”

Three things get settled here. First, the model’s output and the tool’s result are separate messages, and the result must come immediately after that call. Second, a failure is a normal result. This is not a place that throws an exception. It is a place that raises one mark and writes the reason in words. Third, both blocks stay in the history exactly as they are.

The third is where the cost is. Over thirty steps, thirty of these pairs pile up. The tenth step’s result is loaded at the eleventh step and at the thirtieth. One large result returned once does not cost once. It costs as many times as there are steps left.

And this is also where it follows that the only channel for moving a value from one tool to another is the model. For the output of one tool to become the input of the next, that value has to pass in front of the model’s eyes. It is read once as an observation and written once again as an argument. *Data crossing from tool to tool pays twice*. And since writing it again is sampling, the value written in the argument matching the original observation is a probability, not a guarantee.

Length: cut it upstream

A design where a lookup tool returns all the results and a run tool returns the whole log looks diligent. That diligence costs what we just counted. Length is not the only problem either. The unrelated parts mixed into a result bury the signal the next decision needs inside it.

There are four ways to cut it down, and all four need defaults set with them (Anthropic, 2025g). Pagination, range selection, filtering, and truncation. None of them can be left to the discretion of whoever uses the tool. With no default the first call fetches everything, and that one call decides the context.

Putting a ceiling on the size of a result is the same reasoning. In the published implementation, tool responses are restricted “to 25,000 tokens by default.” When you truncate, though, you have to return the fact that you truncated and how to get the rest along with it. A result truncated in silence makes a part look like the whole.

There is also the road of letting the model choose the detail. Open the format of the result as an argument that picks between concise and detailed. In the published contrast the same result was 206 tokens detailed and 72 tokens concise. A call that skims and a call that digs should not cost the same.

Content: so the next call can fill itself

Values the model cannot know call up invention, we said. Most of those values come from an earlier call’s result. So what a result carries sets the quality of the next call’s arguments.

One rule comes out. *Return names a person can read rather than identifiers a machine uses*. Anthropic (2025g) reports that “merely resolving arbitrary alphanumeric UUIDs to more semantically meaningful and interpretable language” improves precision. The reason follows the earlier argument. The model has to write that value again in the next call, and meaningless strings cannot be told apart inside a context and get mixed up as the history grows.

Side by side it looks like this.

Cut down: `uuid`, `mime_type`, `256px_image_url`

Keep: `name`, `file_type`, `image_url`

If the identifier really is needed, return it alongside the name.

Table 7.3: Three grades of failure result. Going down, the number of steps from failure to recovery falls.

Grade	What the result says	The model’s next step	What stays in the loop
Silent failure	the fact of failure is not there at all. An empty or partial result comes back in the shape of a success	carries on normally on a false observation	a wrong conclusion hardens into a premise. The cause is in no log
Bare failure	only the fact of failure. No cause, no direction	guesses the cause and retries. The same call again, or an unrelated repair	steps get burned, and it turns until no-progress detection catches it (Table 15.1)
Guiding failure	what was refused and why, and what to change	a corrected call	the failure ends as a one-step correction

The structure of a result moves performance too. Whether you give the same content as a table, a nested object, or line-oriented text changes the evaluation, and Anthropic (2025g) finds “there is no one-size-fits-all solution.” So this is an item to evaluate on your own task, not an answer to memorize.

Chapter 6 said a stretch of history is made of source text and conclusion and that the direction of compaction is to drop the source text and keep the conclusion. Designing results does the same job upstream. When a tool returns something close to a conclusion, there is no source text in the context to drop in the first place.

The result on failure

What demands more design than the result on success is the result on failure. The next action after a successful step is mostly set by the task. The next action after a failed step is set by *the result*, because the result is all the model knows about the failure.

Say a document search tool could not run the search because of a permissions problem, and it swallows the error and returns an empty list. The observation the model sees is “no results,” and the natural conclusion from there is “no such document exists.” That conclusion is false and consistent with the observation, and the steps that follow stack on top of it. *A result that does not report a failure does not remove the failure. It turns it into a false observation.*

Table 7.3 is three grades, and one axis divides them. How much does the result settle the next step?

Write the three grades out and the difference shows at a glance. Three results for the same failure.

Silent: “0 orders”

Bare: “An error occurred”

Guiding: “You do not have permission to view this customer’s orders. Pass the owning team’s identifier as well, or check the list of accessible customers with the permissions lookup tool first.”

The refusal observation that (a) of Remark 7.1 produces lands automatically at the third grade on this scale. Which argument failed which check is reported mechanically, and the hidden gain of enforcing arguments is here. The first grade, on the other hand, gets built regularly out of a

good intention to handle failure inside the tool. Code that swallows an error gracefully can be a kindness to a human user. To a model, for which the observation is everything, it is showing a world that is not the case.

Trying to move the remedy upstream is the test of a result design. A team suffering from long histories attaches mechanisms downstream. They add a summarizer and tune the compaction interval. All of it is legitimate, and Chapter 6 covered it, and a good part of the source text those mechanisms face was made and sent by a tool. A system that has this order backwards ends up with a summarizer shrinking back down what the tools poured out, paying for the same information twice.

Picking definitions, and keeping results outside

Two costs counted so far. Definitions go out whether they get used or not. Intermediate results go out whenever they get used. The remedies differ.

The cost that goes out unused, you cut by choosing what to load. Section 7.3 was that mechanism.

The cost that goes out on use does not yield to the same remedy. If you leave out a value the next call needs as an argument, the chain breaks, so intermediate results are not candidates for selection. If you cannot cut it down, one road is left. *Change the channel.*

Ask first whether you may. Data that is the object of *analysis* gets read by the model and its content becomes the ground of the next decision. Data that is the object of *manipulation* gets moved and filtered and counted, and its content does not change the model's decision. One question decides it. *Would the next decision be the same if the model never saw this value?* If it would, this is manipulation. If it would not, this is analysis. Data for analysis has to be in the context. Passing data for manipulation through the context pays a cost and adds a chance of error on top.

Definition 7.4 (Code execution). In *code execution*, the model writes code saying what to call and how to weave the results together, and an execution environment runs that code. Instead of loading a tool list into the context and having calls pulled out one at a time, you provide an *execution environment* that can call the tools. Intermediate results stay inside the environment, and what loads into the context is the code and the result of running it.

The solution Anthropic (2025b) proposes has this shape. Present the tools as a code API rather than as objects of direct calls, and filter the results inside the execution environment before handing them to the model. Call a run of five interlocking calls one at a time and five sets of intermediate results all pile into the history. Under code execution, one piece of code and the final result load. The step count falls from five to one. The published figure for one case is a drop “from 150,000 tokens to 2,000 tokens.”

Remark 7.2 (Code execution removes the ceiling on the vocabulary and moves where enforcement stands). The action vocabulary (Definition 7.1) is the set of valid calls that the tool set and the input schemas jointly define, and the harness checks arguments before execution. But the argument of a code execution tool is a program, and the execution environment calls the tools inside it in whatever order and however many times that program says. Then (a) there is no ceiling on how many tool calls, and in what combination, one action can hold, and the action vocabulary stops being an enumerable list. (b) What an argument check can confirm goes as far as the form of the program. Which sequence of calls that program will produce at run time

is not enforced by any check before execution. The place where decisive enforcement can stand moves from the input schema before a call to the boundary of the execution environment.

What code opens up

Code does more than change the channel. The moment a plan takes the form of code, four things open up. Write intermediate results to files and you can *resume*. The whole plan exists as text before any of it runs, so you can *check it before running*. Leave a piece of processing that worked in the environment as a function and you can *use it again*. And data for manipulation stays in the environment, so you can *keep sensitive values away from the model* (Anthropic, 2025b).

In exchange, observation moves back. Call one at a time and every call leaves an observation in the history, and the harness's mechanisms watched that sequence. Code execution folds that sequence into one expression. The sequence of calls inside the environment goes down into the environment's own record rather than the context. It is moved rather than lost, and if there is no recording mechanism where it moved to, it is lost.

The trust boundary opens with it. Running code the model made up needs an execution environment with isolation, resource limits, and monitoring (Anthropic, 2025b), and this operational load did not exist with direct calls. So bringing in code execution is not a decision to add one more tool. It is a decision that expires several of the harness's assumptions. Two things have to stand up on the day you bring it in. A check of the code before it runs, and a record of the execution environment.

Where to draw the boundary and how to hold it is Chapter 22's material. That tool results are mostly text we did not write comes back there too. Web pages, received mail, and outside API responses are all content from outside, and sentences meant to turn the model around may be mixed into them.

7.5 When not to build another tool

Rather than many vaguely defined tools, a few clear ones.

The reason is that the list is the stage on which choosing happens. As the list grows, choosing gets harder. With two similar tools, neither gets called or the wrong one does. A decision to add one tool is not a decision about that one tool. Its definition adds to the list's budget, its name lands on top of the list's distinctions, and choosing is an event over the list.

So the benefit of an addition has to be evaluated on the success rate of the whole list, not of that tool. A team that added a new tool and confirmed only that the tool gets used well has not evaluated what that tool took from its neighbors' chance of being chosen.

Every tool has to justify its own existence. Anthropic (2025g) puts the same standard as making sure "each tool you build has a clear, distinct purpose." If any one of the following three holds, it is not time to build a new tool.

1. **The cause of the failure is how the tools are laid out.** When an existing tool is being misused, adding a new tool that routes around the misuse is a common response and the wrong direction. If the cause is a name creating the wrong expectation, renaming is the remedy (Section 7.2). If the boundary does not match the unit of decision, recutting is (Section 7.3). Trying to solve a layout problem by adding a tool grows the problem and the list together.

2. **The procedure has to run deterministically.** If the same input must produce the same behavior and there is no reason for the model's judgment to enter, that belongs to code or a workflow, not to a tool. A tool is a place where you hand the model a choice, so making a procedure that needs no choice into a tool is handing something certain over to probability.
3. **There is no place it gets used.** A just-in-case tool pays the definition's cost for certain while working only to blur the list. A tool that earns nothing either way.

So how many tools is acceptable? There is no number to give for the question you get asked most often in practice. The threshold is not set by the count alone. It depends together on how far the names are from each other and on how many of them a task actually calls. The published material also only says which direction choosing goes when the list grows. The answer is not a number but an observation. Count calls and misuses per tool in your own system, and when the list starts going bad is told to you by your own log rather than by someone else's number.

7.6 Summary

In one sentence. A tool rides in as a name, a description, and an input schema, and comes back as a result, and that text is all the model sees. So the name sets the choice, the input schema sets the enforcement, the result sets the next step, and tool design becomes behavior design that stands ahead of the prompt.

What is now settled.

- A tool is written in three places and the result is written nowhere. Definitions reload in the same place on every step and results pile up in the history (Figure 7.1). The two places have different accounting, so they get different remedies.
- The action vocabulary is everything the system can actually run, set jointly by the tool list and the input schemas (Definition 7.1). All prompting fixes is the failure where a worse tool gets picked, and a hole in the vocabulary gets filled by invention, not silence.
- All the model has to go on is the name and the description. A name drags in the expectation of a learned convention, and when they disagree the name usually wins. So a name that misfires does not get corrected in the description. It gets renamed. The description says what the tool does, when to use it, and *what it does not return*, in three or four sentences. The last item goes missing most often (Section 7.2).
- Use fewer arguments, enumerate what can be enumerated, give defaults, and remove the places where you can be wrong. But enforcement is a pure gain only on values the model can know. Make a value it cannot know required and absence turns into invention (Remark 7.1).
- Where you cut does not remove errors. It moves them between choice errors and argument errors (Table 7.1). The standard is the unit of decision, and the direction is to merge into high-value workflows rather than wrap endpoints one at a time. When the list grows, one road keeps only the name and one line loaded and reads the rest in later (Definition 7.2), and it pays less and gives up the guarantee of being called up (Section 7.3).
- A result is the next step's input. Give the four means defaults and a ceiling, return names a person can read rather than meaningless identifiers, and let failures follow the three grades that set the steps to recovery (Table 7.3). A result that swallows a failure turns it into a false observation.

What is still open.

- Where tools come from and when that list gets settled, the last row of Table 7.2, goes to Chapter 8.
- Turning the structure of a result into a contract, the general case of output schemas, goes to Chapter 9.
- Designing an evaluation that separates choice errors from argument errors goes to Part VI, and to Chapter 18 in particular.
- Reading the vocabulary as a list of permissions, tool design as restriction of action, goes to Chapter 21.

Try this today. Open the file where the tool definitions live and *count* how many sentences each description has. Where there is a one-sentence description, *write in* when this tool is not the one to use and what it does not return (Section 7.2). Next, point at each argument marked required and *ask* whether the model can know that value at that moment, and for every place it cannot, *check* whether a tool that produces that value is on the list (Section 7.3). Last, take the largest tool result in the log, count its tokens, and *mark* the lines in it that the next decision actually used (Section 7.4).

7.7 Exercises

1. Pick three tools whose descriptions are one sentence, and write into each when it is not the tool to use and what it does not return. Run the same twenty tasks before and after and count which tools got called. If only the description changed and the choices moved, that tool had been getting picked on its name alone.
2. Find a tool in your system that requires a value the model cannot know. Draw the supply route for that value, meaning which tool's result produces it, and if there is no route, design a way to measure how often an invented value passes the argument check and runs.
3. Write down a procedure that separates choice errors (the wrong tool got called) from argument errors (the right tool got filled wrong) in an execution log. Which direction of recutting does the ratio of the two point to in the trade of Table 7.1?
4. Sort your tools' failure results into the three grades of Table 7.3. If there is a silent failure, trace in the log what conclusion the false observation it made hardened into at later steps. Design an experiment that evaluates the effect on average steps per task of raising a grade by one.

7.8 On a second reading

- Read Remark 7.2, come back to the two ends of Table 7.1, and the endpoint of one tool that does everything acquires the name code execution. You see that sharpening the vocabulary and abandoning the vocabulary to give a grammar instead were two sides of the same trade.
- Read Chapter 9, come back to Section 7.4, and the demand to return results in the shape of a conclusion acquires the form of a contract. What stood only as a direction, that the shape sets the next step, gets filled in with a mechanism that enforces that shape at run time.
- Read Chapter 18, come back to Table 7.1, and the row about where errors sit has become an axis for decomposing evaluation. Splitting one lumped task success rate into the layers of choice, arguments, and understanding the result is that chapter's tool evaluation.
- Read Chapter 21, come back to Definition 7.1, and the same vocabulary reads as a list of permissions rather than a list of capabilities. The conclusion that an act not on the list does not happen gets turned over and used as a design that removes from the list the acts that must not happen.

7.9 Further reading

For the practical record of building tools, read Anthropic (2025g) first. Of the concrete material in this chapter, naming and shared prefixes, the direction to merge into high-value workflows with its three examples, the four means of handling result size, and returning names a person can read all came from there. There is one thing it covers that this chapter does not: the procedure for evaluating tool descriptions on real tasks and fixing them from that. The concrete steps will be refreshed, so reading the structure of the demand lasts longer.

The reference for the mechanics, the constraint on tool names, the token count of the tool-use notice, and the shape of the call and result blocks, is Anthropic (2026j). The contrast between the poor and the good stock price description is there as well.

The evidence for why tool use is a hard task is in Patil et al. (2024). What is taken from it here is the sorting of two failures, not generating accurate input arguments and a tendency to hallucinate usage. The numbers in the experiment go stale and the sorting of the failures stays. Read it with an eye to counting those two failures apart in your own log.

Yang et al. (2024), which Chapter 15 takes up, is worth reading alongside. In this chapter's language it is an experiment where redesigning the tool list moved performance while going around any adjustment of the prompt.

The Model Context Protocol (MCP)

You do not have to build every tool yourself. You can pull in what someone else has already built.

The other direction exists too. You can expose what you built for others to use. The team next door's agent, or someone outside your company, puts one of your system's lookups on their own tool list.

What opens both directions at once is one shared contract, and that contract is called the Model Context Protocol. What a contract settles is not capability but the road capability travels on. Attach the protocol and the list of things your system can do is unchanged. What changes is the cost and the procedure of adding one thing to that list, and who wrote the one you added.

This chapter is about that cost and that procedure. It is not about what format the messages take or what the requests are named. That is the part that goes stale fastest, and the specification itself is a document under revision.

8.1 Connecting to the outside

Chapter 7 set up how to write one tool. Name it, describe it, shape its arguments, and cut down what it returns. One question was left unanswered there. Who builds that tool?

Say one organization has three agent applications. An assistant that works on code, a question answering system over internal documents, and an on-call bot watching operational metrics. Four data sources they have to reach. The code repository, the internal wiki, the team calendar, the metrics dashboard. For one application to reach one source, one piece of wiring is needed. Code that gets past that source's authentication, asks in that source's query format, and handles the errors and the paging that source produces. Three applications against four sources makes twelve places to wire up.

Why is this code needed once per pair? Because two conditions overlap. Every source is spoken to differently, and the applications share neither code nor contract with each other. Either one alone does not get you to twelve. Together they do.

Twelve hurts less when you build it than afterward. Change the wiki's authentication and three places have to be fixed separately. Build a fourth application and four more places appear. Bring in a fifth source and three more do. The cost of one move is always the size of the list on the other side. And the twelve pieces of code get written at twelve different levels of quality and go stale on twelve different schedules.

The problem Anthropic (2024c) named when it published the protocol is exactly this. Models are "constrained by their isolation from data, trapped behind information silos and legacy systems," and "every new data source requires its own custom implementation," which makes truly connected systems difficult to scale.

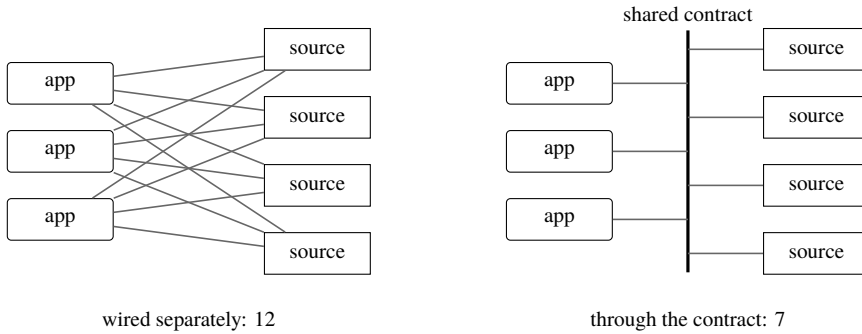


Figure 8.1: Two ways to connect three applications to four data sources. One line is one piece of code you have to write separately, and the vertical bar on the right is the shared contract everyone matches.

8.2 MCP

Think back to attaching devices to one computer. The printer had the printer’s cable and the external drive had its own. One more kind of device meant one more kind of socket on the back of the computer. Then the shape of the socket got settled to one. Whoever makes a device matches that shape, and whoever uses one uses whatever is plugged in the same way. However many kinds of device there are, what the computer has to learn is one.

Do the same between sources and applications. Settle one shared contract for everyone to match, and a source translates how to speak to it into that contract’s form once, and an application learns to read that contract once. For the organization in the last section, twelve places to wire up become seven things to build. One per source makes four, one per application makes three. Figure 8.1 puts the two cases side by side.

That contract is called the Model Context Protocol.

The two sides of the contract

The implementation on the side that offers capability is a *server*. It offers its own system’s capabilities and content in the contract’s form. The implementation on the side that uses them is a *client*. It fetches the list of what a server offers, passes the model’s calls to the server, and takes the result and places it in the context as an observation. There is a third name. An application that carries several clients is a *host*. One client attaches to one server, so an application with ten servers attached has ten clients.

The specification nails down as a design principle that servers should be extremely easy to build. The hard coordination is the host’s job and a server concentrates on its one thing. A constraint follows from that. A server cannot read the whole conversation, and it cannot see what other servers attached to the same application are doing. What a server sees is one call that came to it and that call’s arguments.

Table 8.1: The three kinds a server offers. What divides them is not function but who chooses.

What is offered	What it is	Who chooses	Which layer it loads into
Tool	a capability you call and run. The definition loads first and the result loads later	the model	tool definition layer
Resource	content you read and put into the context. One address points at it	the application	retrieved knowledge layer
Prompt	a bundle of instructions you fill values into	the user	instruction prefix

What gets offered

What a server offers is not only tools. Three kinds. Tools, resources, and prompts, and what divides the three is not function but *who chooses*. The model chooses a tool and calls it, the application chooses a resource and loads it, the user chooses a prompt and uses it. Table 8.1 is that correspondence.

It matters not to read this list as a list you are seeing for the first time. If the protocol had been an invention of capability, the three kinds would be new kinds. It is a standardization of the road, so the three run straight back into the layers that Table 4.1 cut the context into. Which means the arithmetic of loading set up for each layer still applies when the supply route changes.

The direction is not one way either. The specification opens a road for a server to ask back, through the client, to the user. Fill in a missing value, or confirm whether this action may run. The decision to stand a person in front of an action you cannot undo, the decision Chapter 21 will take, comes up here from the provider's side as well. What to show and what to ask stays the application's decision.

How a connection gets made

Three things the specification settles change the design.

First, you cannot know the tool list in advance. The client has to ask the server. The system at design time has no tool list written in it. A connection has to open for the list to come, and the list has to come for the stable prefix to be assembled. And the list can change. A server may send a notification that it changed, but that delivery is not guaranteed, so the client has to ask again, which is why the list response carries a value for how long it may be reused.

Second, the two sides do not sit holding a connection open. The 2026-07-28 revision moved a long way in this direction. Each request now travels on its own, carrying its protocol version, client identity, and client capabilities, so from the server's side it becomes the same job as taking an ordinary web request. You can stand several up and put a load balancer in front. For whoever offers a server this is a welcome change, and in exchange the server has to look after state that spans several calls by itself.

Third, the specification settles authentication and does not settle permission. A server attached over the network takes only tokens issued to it. Taking a token issued to someone else and passing it straight through is one of the few things the specification nails down as forbidden.

A server running as a process on your own machine does not ride this flow. It gets its credentials from the environment, and so it inherits your machine's permissions as they are.

What the specification does not settle is just as clear. It does not settle what to offer, what of what is offered to load, or whether what you loaded can be trusted. Not that it declines to. It cannot. Those belong to the sources and the task, not to the road.

The specification gets revised too

The protocol was published as an open standard in November 2024, together with a stated plan to develop it as a collaborative open-source project (Anthropic, 2024c). In December 2025 it moved to a newly formed foundation under the Linux Foundation as an open standard rather than one company's property, and a revision came out in July 2026. Which means the revision cycle now follows the cycle of an open standard rather than one company's product cycle, and for exactly that reason it means it will keep being revised.

The specification put a deprecation rule in with it. A feature slated for removal stays in the specification for at least twelve months after the notice, a "twelve-month minimum window so you can plan upgrades instead of reacting to them." What a designer has to do here is sort their own decisions into two kinds. Decisions coupled to the message format and the transport go stale with the revisions. Decisions built on the existence of a supply route, the three kinds offered, and the arithmetic of loading cross the revisions.

8.3 What MCP touches

It touches two places. Inside the context, and the loop itself. The first is a matter of what goes out on every step. The second is a matter of time and reproducibility.

Context: a tool you never call still costs

A tool that came from outside also has its definition loaded into the context. The name, the description, and the input schema become text, sit in the front seat, and get reread on every step for the whole run. Attach a server with twenty tools and use two of them, and the descriptions of the other eighteen cost the same on every step.

The protocol did not raise that cost. What it raised is how many you attach. When one integration took days, those days were the threshold, and only connections worth paying for got built. Now that it takes a few lines of configuration, the threshold is gone. So a system that used to attach five attaches twenty. All twenty definitions sit in the front seat.

One published case gives the size. A run that loaded all the tool definitions of the attached servers up front spent 150,000 tokens, and a run that kept the definitions like files and read them only when needed spent 2,000 (Anthropic, 2025b).

The cost is half of it and the other half is choosing. With one tool there is no event called choosing. You call it or you do not. With a hundred you have to pick the one you need out of ninety-nine, and what the model sees while doing it is names and descriptions. The more candidates, and the more similar definitions in the mix, the lower the chance the right tool gets called. The specification requires a tool name to be unique only within one server, so attach three servers and the same name can appear three times.

And you cannot edit those descriptions. Back when you built them yourself, a description that confused the model got fixed. The text of a supplied tool was written by someone else. It

spends your budget and steers your model's choice, and you have no right to change it. Chapter 5 said that what has to be fixed is the tool and not the instruction, and here that exit is blocked.

So the question that approves a connection changes. It used to be whether it was worth building, and a no ended it there. Now there is almost always something worth building, so that question filters nothing. There is one thing to ask. *Is this provider's tool definitions worth loading into the front seat of every run?*

The loop: outside circumstances become yours

Calls go outside, so one more round trip appears. The round trip where the model puts out a tool name and arguments and the harness turns that into a real function is unchanged (Figure 7.1), and that function now runs on someone else's machine. A slow server makes a slow step, and a dead server stops the loop. Who holds the timeout and the retry is the harness's job, and Chapter 15 takes it.

The quieter side is reproducibility. That the list comes at connection time means that list and the definitions in it may be different at the next connection. A definition is text loaded into your run's stable prefix, and the right and the moment to change that text belong to the provider. A revision can happen without your deploy, without notice to you, between yesterday's run and today's. The guarantee that yesterday and today are the same system hangs on the provider's practice rather than on your deployment history.

So how you look for a cause on a day when performance wobbles has to change. A team looks for the cause in the list of what it changed. Did we edit the prompt, did we upgrade the model. A connected definition is not on that list, so it does not even get suspected, and a provider's revision gets misdiagnosed as a degraded model or a prompt problem. Two remedies. Leave the state of the connection in the run record, meaning what was attached and what each definition said (Chapter 20). And in a system where reproducibility matters, pin the definitions as of the moment they passed review, and make their update an engineer's explicit decision rather than the provider's circumstance.

8.4 Designing with MCP

Design goes two ways. Using what others offer, and offering your own. The questions differ.

Using what others offer

Four things to check when you attach a server.

- Where does this server run? Your machine, or someone else's infrastructure? The first inherits your machine's permissions as they are. The second puts latency and availability into someone else's hands.
- What comes, and how many? Did you count the tools? Did you count how many tokens their definitions are in the stable prefix?
- How do you find out when the list changes? Does a notification come, or do you have to ask?
- Can you pin the definitions as of the moment you reviewed them?

One more to add. A server can send annotations describing a tool's behavior. This tool only reads, or this tool cannot be undone. The specification writes that clients "MUST consider tool annotations to be untrusted unless they come from trusted servers." The server is the one attaching the annotation, and there is no mechanism inside the contract that confirms an annotation is true. A design that skips a person's approval on the strength of a "read-only" annotation is a design that trusts whoever wrote that annotation.

All four are questions of loading and reproducibility rather than of connecting. And there is work left after attaching. Treat the supplied definitions not as given facts but as text to review, select, and wrap and rewrite where needed. What makes a good definition, and where Section 7.4 sets out how definitions get picked, are both in Chapter 7.

Offering: inside the company

This is where the protocol pays best. Stand one server per source and the code that gets past that source's authentication, turns its pages, and handles its errors is written once for the whole organization. Grow to a fourth and a fifth application and that code does not grow. The organization in Section 8.1 having seven pieces instead of twelve is this.

But Chapter 7's tool design binds just as hard here, and harder. The people reading the name and the description are no longer your team. Say the team that runs the internal data pipeline offers a tool and names it `run_ingest`. Inside that team everyone knows what goes in where. The team next door's agent does not. The model has nothing but the name and the description to look at, so that tool does not get called where it should be and does get called where it should not. A description written in words only your team knows becomes a defect the moment it is offered.

Four things the offering side has to hold to.

- **The name has to read from outside.** Do not put internal abbreviations and the in-house nicknames of internal systems in a name. The specification only requires a name to be unique within a server, so think of the place where several servers attach together and put a piece naming your service on the front.
- **Write the boundary into the description.** If what it does not return and when it is not the tool to use are not in the description, the using side has no way to find out (Section 7.2).
- **Do not shake the order of the list.** For the same tool set, return the same order. Shake the order and the using side's cache breaks.
- **Return failures in two kinds.** A failure the caller can fix and call again, a date in the wrong format or a value out of range, comes back inside the result with a failure mark on it. The model reads it and fixes it itself. Calling a tool that does not exist, or a malformed request, comes back as a protocol error, because it is not something the model can fix (Table 7.3). The specification draws the same line: tool execution errors "contain actionable feedback that language models can use to self-correct," while protocol errors are "less likely to result in successful recovery."

One server, one job is also a design principle of the specification. If your team handles several things, standing them apart beats putting them all in one big server. The using side can attach only what it needs, and what it did not attach does not load into anyone else's stable prefix.

You can also offer different lists to different callers. The specification blocks the tool set from varying per connection but allows it to vary by what credentials the caller arrived with. A party whose credentials go as far as reading sees only the reading tools. A tool not visible in the list cannot be called, and it does not spend anyone else's budget either.

The structure of results gets settled here too. The principle was that a tool definition does not write down its result, and the specification sets aside a separate place to carry the output's shape as well. Put a shape there and the using side can check the result. Leave it out and the using side learns only by calling. What to write there and how goes to Chapter 9.

Offering: outside the company

Go outside the company and security lands on top of those four. Three questions. Who is calling, how far can they go, and what do you return?

Who is calling. A server opened over the network takes only tokens issued to it. It does not accept a token issued to someone else, and it does not pass a token it received straight through to another system behind it. Do the latter and the record in the system behind you holds the wrong identity rather than yours, and when something goes wrong there is no telling who did what. That is why the specification writes it down as forbidden.

How far can they go. Give permissions starting small. Open reading only at first, and when you reach a place that needs writing, ask back for more at that place. The specification has the server write into that asking-back response what more is needed for this particular job. Demand every permission at once and two things get worse together. The user backs away at the approval screen, and if approval was given, that one token becomes everything you lose when it leaks. Picture the approval screen and you see why. Someone trying to attach one tool that searches internal documents gets twenty-odd lines at once, covering document read and write and delete and changing share settings. What this person needs is reading. The length of the list alone postpones the approval, and a postponed approval mostly does not come back.

What do you return. The text you return loads into someone else's context. If something that should not go out is mixed into a result, it leaves your organization right there. The reverse holds too: on the receiving side, the text you sent is text nobody reviewed. Whichever side you stand on, handling this boundary is Chapter 22's material, and the boundary itself is defined in Definition 22.1.

It binds on holding state as well. Since the two sides do not hold a connection open, a server that needs state across several calls makes a handle, returns it, and takes it back as an argument on the next call. A cart number or a job number. One judgment goes wrong easily here. Treating possession of the handle as proof of identity. A handle is a name, not a key. Unless every call confirms that the caller owns that handle, anyone who learns the number opens someone else's cart. Make handles unguessable, and store them bound to the caller's identity.

Last, the case of offering a server that runs on your own machine. Attach it to standard input and output and only the application that launched it can reach it. Open it on a network port and other processes on the same machine can reach it too, and then you require a token or use a restricted channel.

8.5 When not to use the protocol

If any of the following holds, wiring it directly is the answer rather than the protocol.

Table 8.2: The decisions standardization leaves. The right column is the chapter that takes each one.

The decision left	Why the specification cannot answer it	Covered in
What to offer. The name, the arguments, the description	a definition still has to be written by someone, and its quality sets choice and misuse. The road carries. It does not write the content	Chapter 7
What of what is offered to load	the contract has no mechanism for choosing. Not loading everything that arrived is the application's job	Chapter 7
What form observations and outputs travel in	the contract sets the shape of the envelope. The structure of what goes inside is a design between the tool and the application	Chapter 9
Load knowledge as a resource, or fetch it by retrieval	a resource is one supply route. The trade about where to keep knowledge stays exactly where it was	Part IV, Chapter 11
Whether to trust the provider	ownership of execution sits outside the boundary, and incoming definitions and content are text nobody reviewed	Chapter 22

1. **There are only a few places to wire.** With one application and one or two sources, all that is left is the complexity of a contract in the middle. Two notes. Make this judgment on the expected count rather than today's. The arithmetic flips the moment applications go from one to two. And if someone has already built the server on the source side, all you build is one client. Counting what you will build is the right count.
2. **Control of the definition decides the outcome.** For the core tool that is the system's bottleneck, the one where the quality of choosing and observing directly moves the task success rate, you have to be able to write and tune the definition yourself. A supplied definition cannot be, and a wrapping layer adds cost and complexity.
3. **You cannot widen the trust boundary.** The protocol is a structure that puts ownership of execution outside the boundary. In a system that handles actions you cannot undo or moves sensitive data, bringing in an executor you cannot review and definitions nobody reviewed is not justified by the convenience of connecting.
4. **The tool has to know the conversation.** A server does not see the whole conversation. It sees one call that came to it and that call's arguments. A tool that has to know what was exchanged at earlier steps in order to answer properly has to have that content loaded into its arguments and sent outside, and then it costs twice. Each outgoing round trip carries that many more tokens, and part of your conversation stays in someone else's system. A tool like this is better kept inside the application.

If none of the four catches and you decide to attach, the judgment does not end there. It passes to the questions of Section 8.4. Deciding whether to use it and handling what comes after attaching are different jobs, and passing the first does not excuse you from the second.

And even having decided to attach, there are decisions standardization never finishes. Table 8.2 is that list, and the chapter that takes each one is on the right.

There is one more item that is hard to put in a table. The list of where in your system the

code that touches the specification's details lives. A system where that coupling gathers into one client per application fixes one place per application when the specification is revised. A system where the details have seeped into every branch that handles a tool and every corner that parses a result sweeps every application per revision. Look at what the second one is a repeat of. The world of Section 8.1, where one change to a source became as many repairs as there were copies, has come back to life, this time against the specification itself.

So that list has to get made once. Three things to look for. Code that builds requests and unpacks responses, code that branches on a tool's name, and code that assumes a result will arrive in some shape and reads it against that shape. The first can be gathered into one client. The last two spread easily through application code, and once they have spread, every revision means finding all of those places again. What a revision costs is set not by how much the specification changed but by how long that list is.

Last, this judgment can be made per tool rather than for the whole system. It is not all supplied or all built yourself. The usual answer is a mix: build the few tools that decide the outcome and hold their definitions, and take the many surrounding sources as supplied to cut the places to wire. Which tools are in the few can only be told by someone who has evaluated two numbers, how often it gets called and how much it moves the choice.

8.6 Summary

In one sentence. The protocol standardizes the road of supply rather than capability, which cuts the places you have to wire. It does not settle what to load, so the cheaper connecting gets, the heavier the decision about loading.

What is now settled.

- Three applications reaching four sources need twelve pieces of code wired separately and seven through a shared contract (Figure 8.1). If someone has already built the server, what you build is fewer still.
- A server offers tools, resources, and prompts, and what divides them is who chooses. The model chooses tools, the application resources, the user prompts (Table 8.1). All three run back into the context's layers, so each layer's arithmetic still applies.
- What the protocol raised is not the cost per step but the number you attach. With the threshold gone, a system that attached five attaches twenty, and all twenty definitions sit in the front seat. In one published case a run that loaded everything up front spent 150,000 tokens and a run that read only what it needed spent 2,000.
- The more candidates, the lower the chance the right tool gets called, and the person who wrote those descriptions is outside, so you cannot fix them. So the question narrows to one. Is this provider's tool definitions worth loading into the front seat of every run?
- The tool list comes only when asked and changes during a run. The right to change a definition belongs to the provider, so a revision happens without your deploy, and with no record it gets misdiagnosed as a degraded model (Section 8.3).
- On the offering side, your names and descriptions get read by strangers, the order of your list sets someone else's cache, and the kind of your failures sets whether someone else's model can fix itself (Section 8.4). Go outside the company and three more land on top. Take only tokens issued to you, give permissions starting small, and remember that what you return loads into someone else's context.
- There are four places not to use it. When there are few places to wire, when control of the definition decides the outcome, when you cannot widen the trust boundary, and when the tool has to know the conversation (Section 8.5).

What is still open.

- What to write in the definition and how, and what of what is offered to load, are in Chapter 7.
- The structure of observations and outputs, which is turning a shape into a contract, goes to Chapter 9.
- Whether to load knowledge as a resource or fetch it by retrieval goes to Part IV and Chapter 11.
- Defending the trust boundary that an executor outside it and unreviewed definitions widen goes to Chapter 22.

Try this today. Open the list of servers attached right now and *count* how many tools each

server has (Section 8.3). *Mark* the ones not called even once in the past month, and *write down* the tokens their definitions cost on every step (Section 8.3). Last, if you offer a server of your own, show its tool names and descriptions to someone outside that team and *ask* them what the tool does (Section 8.4).

8.7 Exercises

1. Count the application and source pairs in your organization and write down the number of places wired. If a shared internal connection library already exists, that number was never twelve to begin with. Under what conditions does adopting a standard raise, rather than lower, the number of implementations you maintain?
2. Measure the share of the front seat that the attached servers' tool definitions take, and count separately the share belonging to tools not called even once last month. Hold the case that went from 150,000 tokens to 2,000 against your own system: how far down does it go? If you take those tools off the list, what changes in the chance that the remaining tools get called?
3. Performance is different today than yesterday. What could have changed is the model, the prompt, and the provider's definitions. List the records you need to tell which of the three moved. Write down too what misdiagnosis this question drifts into in a system without those records.
4. Say you offer one of your team's capabilities as a server. Rewrite the tool name and description on the condition that someone outside your team reads them. Then write down what else you have to settle to open that server outside the company, following the three questions of Section 8.4.

8.8 On a second reading

- Read Chapter 7, come back to *the person who wrote those descriptions is outside*, and the weight of that sentence has changed. Once you know the standard for a good name and a good description, you have a ruler for judging supplied definitions, and you can see what to fix when you wrap one and rewrite it.
- Read Chapter 7's progressive disclosure, come back to the difference between 150,000 tokens and 2,000, and you see that the difference was a design object rather than an observation. Keeping the connection while choosing which definitions to load, and going around loading definitions at all, get filled in there.
- Read Chapter 20, come back to recording the state of the connection, and the demand for a per-run record has become a design object called a trace. What was attached and which definitions were visible have to be in the step record before a provider's revision becomes an observable event rather than a misdiagnosis.
- Read Chapter 22, come back to the three questions of Section 8.4, and you catch how the offering side's discipline looks from the other side. The definitions and the content a provider offers are unreviewed text arriving at run time on the receiving side.

8.9 Further reading

The protocol's announcement, Anthropic (2024c), is short, and the diagnosis and the remedy are both in it. The diagnosis of isolated models, information trapped in silos, and a custom implementation per source. The remedy of replacing fragmented integrations with a single protocol. And the three kinds, tools and resources and prompts. Read it sorting which sentence is about the problem of connecting and which is about the problem of loading. That the document speaks only to the first is this chapter's argument.

This chapter did not cover the specification's details. What format the messages take, how authentication and permission attach, what the transport is, are all settled by the specification and left out here. If you plan to build a server, you need those details. They are all in the specification document, so confirm them there. The revision history and the plans ahead are published with it.

The details do get revised, though. What has to be kept is the structure that crosses the revisions, and whatever detail you meet, the question is the same. Is this settled by the shared contract, or is it still mine to design?

Structured Output and Schema Design

Ask a model the same thing twice and it answers twice differently. No amount of polishing the prompt makes that difference go away. But in an agent, what receives that output is code, not a person. One step's output is the next step's input, so when the shape wobbles, the wobble travels through the loop.

What this chapter is about is removing that wobble from the form. There is one method. Carry the shape the receiving side expects along in the request, so that an output outside that shape never gets drawn. If the earlier chapters were about what to put into the context, this is the other side of that work. However well you polish what goes in, if what comes out is a different shape every time, the next step, which puts that output back into the context, has to do the same polishing from the start.

One thing is not enforced. That the values are right. This is not an interpretation. It is the range the feature's own documentation draws narrowly around itself. What is guaranteed goes as far as conformance to the shape, the type of each field, and the presence of the required fields. That the value is right is not on that list (Anthropic, 2026h). So the problem in schema design is not what to enforce. It is *knowing what enforcement does not catch*.

Designing an evaluation of whether the content is right goes to Part VI. Whether an input in the right shape can be trusted, which is the defensive question, goes to Chapter 22.

9.1 Why fix a shape

Say there is a tool that sweeps the system every morning and reports whether anything is wrong. The same request was sent twice and these came back.

“The check is done. Three items looked off, and two of them are the same items as yesterday.”

“The check did not come back clean. There are 3 places that need a look.”

A person reads one report. Code reads two.

The receiving code does three things. It branches on whether anything was wrong, it holds the count against a threshold, and it hands the items to the next step. The values are in both sentences. And there is no getting them out. A check looking for “looked off” is helpless in front of “did not come back clean,” and the rule that pulled a number out of “three items” spins on the day “3 places” arrives. That there are endless ways to say the same thing is the power of a sentence to a person and the whole of the trouble to code.

The trouble stays even when the next step's model receives it. If a value from the previous step's result has to become an argument of the next call, the model has to write that value again. And writing it again is not copying. The model draws fresh every time, so it can differ from the original. And when that value is buried in a sentence, finding it is added on top. You have to

pick A-1183 out of “the order number is A-1183,” and move what you picked without getting it wrong.

Writing it in the prompt does not solve it. “Answer in this format” is well followed. Just not always. A shape that is mostly followed is, in the eyes of the receiving code, a shape that occasionally collapses, and a parser collapses on the rarest phrasing.

And the wobble is normal behavior, not an accident. A model commonly adds a word about what it is about to do before calling a tool, and the phrasing of that word differs every time. The published guidance says not to assume any particular phrasing there. That sentence, a kindness to a person, is a different input every time to code.

Fix no shape and a shape is already there. It is in the receiving side’s code, in the form of a parser. Every rule that pulls a count out of a sentence is an assumption that “the model says it like this,” and all of those assumptions together are this system’s real output convention. And it is written down nowhere. It is scattered across each receiving site, and the producing side does not even know it is bound by it. So on the day you change the model or polish the prompt and the phrasing shifts a little, the mismatch shows up only as the receiving side misbehaving.

Fixing a shape is taking that already existing promise out and writing it in one place.

9.2 What a schema does

There are two places to fix a shape and they have different force.

Write it in the prompt and it is a request to please answer this way. Mostly followed, occasionally not. Write it as a schema and it is different. Before what the model produced goes to the receiving side, code checks it, and if it does not match it stops there.

A schema looks like this.

```
status: one of "ok", "issues", "unknown". Required.  
count: an integer, zero or more. Required.  
items: a list of strings. May be empty.
```

Then the two reports from the last section come back like this.

```
{"status": "issues", "count": 3, "items": [...]}
```

The receiving code’s three jobs drop to one. Pull the values out. No finding things in a sentence, no worrying whether the phrasing changed.

Three things get fixed per field. The name and the type, and whether it is required. The name is what the receiving side will call it, the type is what the code will check against, and required decides whether it may be absent.

One more attaches to that. A description of what each field is. The model fills a field looking only at its name and that description. What Chapter 7 said about tool descriptions binds exactly here. Write only `count` and it does not know a count of what. Write “the number of items found to be wrong” and it does.

Definition 9.1 (Output schema). An *output schema* is a specification of the shape that what a model produced must satisfy before it goes to the receiving side, and whose violation code can judge without a person’s or a model’s judgment. Output whose shape is enforced this way is *structured output*.

What divides them is not what the specification is written in but who judges. A convention written in prose is also a specification of shape. But if a person looks at whether it was kept, or

a model judges it, that confirmation wobbles again. A schema gets its force from the checker being code.

Where you write it

That the place is not the prompt is the first thing that matters. A schema is a parameter attached to the request. There is a separate field for the shape when you send a request, and you write the list of fields and each field's type into it (Anthropic, 2026h). "Answer this way" written into the prompt is not the same sentence. The first is an instruction and the second is a request parameter.

There are two places it stands in practice and they can be used together. The place that fixes the shape of the model's whole response, and the place that checks a tool call's arguments strictly against a schema. The second is Chapter 7's argument design acquiring a run-time mechanism. The first is the material here.

There are two ways of enforcing as well. Check after generating and refuse, or bind the generation itself to the shape. The second makes a token outside the schema impossible to draw. Under it, a violation of the shape is not refused. It does not happen. Whichever you use, that what gets enforced goes as far as the shape is the same. What differs is what to do when it is violated (Section 9.4), and that is set by which one you use.

One schema, and the output it produces

Fix a shape on the morning check tool and it comes out like this.

```
{
  "type": "object",
  "additionalProperties": false,
  "required": ["status", "count"],
  "properties": {
    "status": {
      "type": "string",
      "enum": ["ok", "issues", "unknown"],
      "description": "ok if the check finished and nothing is wrong,
        issues if something is. unknown if the check itself did not
        finish or you cannot tell whether anything is wrong."
    },
    "count": {
      "type": "integer",
      "description": "How many items are wrong. Write 0 when status
        is unknown."
    },
    "items": {
      "type": "array",
      "items": {"type": "string"},
      "description": "Names of the items that are wrong. Write as
        many as you know."
    },
    "note": {
      "type": "string",
      "description": "Anything the fields above do not hold. Write
        here if you are not confident or the result is partial."
    }
  }
}
```

Output that keeps it arrives like this.

```
{
  "status": "issues",
  "count": 3,
  "items": ["certificate expiring", "disk space low",
            "nightly batch delayed"],
  "note": "The nightly batch caught on the same cause yesterday."
}
```

Hold it against the two sentences from the last section and you see what changed. The value that branches is one of three, so the conditional does not wobble. The count is a number, so it goes straight against a threshold. The list is an array, so pulling it out needs no rule. And the caveat about the comparison with yesterday did not disappear. It went into *note*.

Break only the shape while saying the same thing and it arrives like this.

```
{
  "status": "has issues",
  "count": "three",
  "findings": ["certificate expiring", "disk space low",
               "nightly batch delayed"]
}
```

Three violations at once. *status* has a value not among the three, *count* is a string rather than a number, and the list came under a name the schema does not have. A person reads all three as the same report and the check refuses all three. Because the check looks at nothing but the shape, and that property works both ways from the next section on.

The field description fills the field

The model fills a field looking only at its name and its description. Which means the part of the schema above that actually does work is *description*. Give only the name *status* and three values, and the model has no way to know when to pick *unknown*. The description sets that.

The standard Chapter 7 set for tool descriptions comes over as it is. Write what it does, when to use it and when not, what each argument means, and what it does not return (Anthropic, 2025g). What an output schema's field description has to answer is this. What goes in this field, when do you leave it empty, and what wins when it disagrees with another field? The *count* description in the schema above answers the last one. If *status* were *unknown* while the count said three, the receiving side's branch would come out wrong.

When fields overlap or the shape of a value is awkward, descriptions alone are not enough. Then you can carry examples of correct input along with the schema. An example has to be valid against the schema, and it costs input tokens for as long as it rides. Lengthen the description or carry an example is a choice you make by holding those two costs against each other.

9.3 How tight to make it

Tighter is not better. Point at the two ends and it shows.

Fix nothing and the two sentences from the earlier section come back. The phrasing is free and the receiving side suffers every time. Turn it around, make every field required and every value a pick from a fixed set, and the receiving side has it easy. But the world does not arrange

itself to match that list. The moment something the designer did not foresee happens, there is no right answer in front of the model, and it picks the least wrong one.

What trades is clear. Tighter makes the receiving side easier, and tighter shrinks what can be carried. And what disappears from the shrinking is usually not a value but a caveat. That it is not confident. That the result is partial. That two candidates split and one was picked.

A caveat with nowhere to go becomes one of three things. It gets dropped, it gets crammed into whatever field is open, or it breaks the shape and gets refused. The third is noisy, so it is the better one. The first two are quiet. A model with no field for saying it is unsure answers in a confident shape, and the receiving side cannot tell that apart from a whole value.

Three standards for deciding

If the values are settled, make it pick from the settled ones. A free string gets written fresh every time, so it wobbles. The `status` above is that. Leave it open and it might write “has issues” or it might write “issues found.” Close it to three and nothing outside them reaches the receiving side.

There has to be a place to hold the case of not knowing. One of the three closed values being `unknown` is that place. Without it, on a morning when the check did not finish, the only choices are `ok` and `issues` and both are false. Enforcement does not make a model accurate. It only makes it pick one of what it was given.

A field the model cannot fill is not a field. Point at each field marked required and ask whether the model can know that value at that moment. If it cannot, there are three roads. Open it as optional, put in a value that says it does not know, or put a tool that produces that value on the list. Making a field the model cannot know required is an order to invent.

One line runs through the three. *The receiving side decides how tight to make it.* If the receiving side branches on that value, tighten. If the receiving side reads it and judges, tightening buys nothing. And what only the model knows, the confidence and the caveats and the exceptions, does not want tightening. It wants a field made for it.

Written down is not always enforced

What you can write in a schema and what actually gets enforced are different. Types, whether a field is required, and the list of allowed values get enforced. The range of a number or the length of a string mostly does not (Anthropic, 2026h).

A clause that is not enforced does not disappear. It goes down into the description. And what is written in a description becomes a request again. “It has to be 100 or more” moved into a description is mostly followed and occasionally not. If you really need a value inside the range, you have to put one more check after the schema check, and that check belongs to the receiving side’s code rather than to the schema.

What is enforced and what is not differs by provider and changes. Confirm it in the documentation before you rely on it.

Does tightening make the answer worse?

Enforce a shape and the room the model has to spread its thinking out in before writing the answer narrows. The published guidance says to look at two things when choosing a format. That it be close to a shape the model has naturally seen in writing, and that keeping the shape

not itself be a burden. Beside those sits an item about leaving tokens to think in before writing the answer (Anthropic, 2024a). The three point at the same place. A shape is the vessel that holds an answer, not the process that makes one.

How much worse has no answer yet. Tam et al. (2024) compared format-restricted answers with free ones across several tasks and observed reasoning falling under restriction, and there are objections to the experimental design. A contested result cannot be borrowed. Only a team that has held the two conditions against each other on its own task has its own answer.

A schema costs too

The text describing the shape rides along in the input. A schema is not a checking mechanism the model cannot see. It is also text sitting in the front seat.

And editing a schema breaks the cache. Changing the shape does, and so does changing the tool list. Fixing only names and descriptions does not (Anthropic, 2026h). *A schema revision is not a wording fix. It is a change to a cache boundary.*

That editing is not one side's decision hangs here too. If the receiving side expects a new field and the producing side stays as it was, that field stays empty forever. If the producing side changed a field's meaning and the receiving side reads it with the old meaning, the shape holds while the value goes wrong. The second is worse. The check looks only at the shape, so no refusal happens, and the system alone is wrong while both sides believe they are keeping the promise.

When a schema goes stale

The list of fields is the shape of the task as that team understood it on the day they wrote it. Tasks change, and by that much the fields fall short. But the mismatch does not show up as a violation. The shape still holds.

Where it shows up is the open fields. When caveats with nowhere to go start getting crammed into a string field, that is not the model going off script. It is a signal that the schema has gone stale. The list of which fields the next version needs is piling up right there.

9.4 When the model breaks the shape

Whether breaking happens at all depends on which way you enforce. Bind the generation itself to the shape and breaking does not happen. In a design that checks after the fact, breaking is normal behavior rather than an accident. As long as a model is producing, the probability of a breaking output is not zero.

When it breaks

Where it breaks is mostly settled.

When you ask it to hold what cannot be held it breaks. The schema with no `unknown` in the last section is that.

When the schema is long and complicated it breaks. More fields to fill means one more chance to be wrong per field, and when fields are stacked in layers, holding that structure up becomes a burden in itself.

When there is no field for saying it does not know it breaks.

Table 9.1: Three roads when it breaks. You do not pick one. You chain them in order.

Road	What the harness does	What it gets	Where it pays
Fix it	repairs the breaking output into the nearest valid shape and lets it through	no extra step, no extra latency	the repair rule does not know the meaning. What is near in shape can be far in meaning, and the fact that it broke disappears from the record
Send it back	returns the refusal as an observation and has it produce again	the model learns what it caught on and gets a chance to fix it	a round trip spends a step and lengthens the history. Success is not guaranteed, so it needs a cap
Raise it up	stops the run or hands it to a person	a guarantee that a wrong shape does not reach downstream	the task stops right there. With nobody to hand it to, stopping is not handling but abandonment

All three are the schema's circumstances, not the model's incompetence.

There is one more, and this one is worse. When it does not break and invents instead. The published guidance raises it as a criterion for choosing a model. It recommends the largest model for complicated tools and ambiguous queries, and notes that a smaller model “might also infer a reasonable value” for a missing parameter (Anthropic, 2026j). Which means that rather than leaving the field empty and asking back, it puts in something plausible. That output keeps the shape, so the check does not catch it. This is why a low violation rate is not always a good sign.

How to handle it

There are only three roads. The harness quietly fixes it, sends it back to be done again, or raises it up.

Sending it back becomes the default easily, so count its cost first. The breaking output and the refusal message do not leave the history after they are used for the correction, and they ride along for every step that remains. The output gets generated twice, so the share on the side the cache cannot help goes out twice as well. Sending it back is not a free redraw. It is a round trip that lengthens the history.

What you tell it sets the recovery rate. Return only “the shape was wrong” and the model draws again without knowing which field to fix or how. Return, per field, which field caught on what, and it can fix it. In the example earlier, that content is that `status`'s value is not among the three, that `count` has to be a number, and that there is no field named `findings` in the schema.

How many attempts is published nowhere. What you need is two numbers. The rate of breaking per field, and the rate of getting fixed when sent back. A system where that plunges on the second attempt and one where it stays flat through the third have different caps. The road after the cap is to fix it or to raise it up. The three roads are not parallel options but an order.

A generous harness that fills in the closing bracket and trims the trailing rubbish looks like a mechanism that saves round trips. It costs in two places. First, repair is done by distance in shape, not distance in meaning. Fill in the bracket of a truncated output and you get a valid shape, and what it holds is a truncated value rather than a finished one. And that gets consumed

Table 9.2: The four combinations of shape and meaning. A schema moves only the axis of shape.

The output's standing	Check	What happens on the receiving side	How it surfaces
Shape and meaning both right	passes	the next step goes on over a right value	nothing to surface
Shape right, meaning wrong	passes	a wrong value gets used carrying the mark of having passed a check, and becomes a premise for later steps	only after a later result comes out wrong, and only when that result leaves a trace
Shape wrong, meaning right	refused	right content does not get used and comes back	at once. The refusal is itself an observation
Shape and meaning both wrong	refused	a wrong value gets filtered out before use	at once, though what caught it was the shape, not the meaning

under the model's name. Second, the fact that it broke disappears.

Count what broke

The breaking rate is a diagnosis, not a cost. A field that breaks often is not a field the model is bad at. It is a field where the schema and the model's understanding disagree. The expectation the name created and the description came apart, or a field is missing and a caveat is forcing its way through the shape. It may also be that the value asked for is not in the context at that moment. The remedy for each is fixing the name, making a field, or putting in the tool that supplies that value. Not raising the retry cap. When the breaking rate is high, what is bad is the schema, not the model.

And a design that quietly fixes things erases exactly this number. Even if you fix, what got fixed, how often, and how has to stay. That record is the list of grounds for the next schema revision. Where and in what form to leave it is Chapter 20's material.

9.5 What a schema cannot cover

One output carries two judgments. Does the shape match the schema, and is the content right?

The first is judged by code. The second is judged by the result of the run or by a person. A schema moves only the first.

The row to read is the second. In the check output earlier, if `status` is `issues` and `count` is 3 and those three were not actually problems, the check lets it through. A number that changed while being copied over, and a report of something that was not there, both go through the same door as long as the shape holds.

The point. A schema stops a malformed output before it gets used. It does not stop a well-formed wrong answer. The danger is not the failure that is left over. It is that *passing the check gets read as a warrant for the content*.

Before there was a check, an output was just an output, and whether to trust it was the receiving side's job. Stand a check up and an event called passing appears, and output that passed gets reviewed less. The phrase "validated output" starts getting used, and the code behind it branches on the passed value as a premise. And what that check saw was the shape and nothing else.

The case where a model gives a refusal instead of an answer is no different. If the sentence of refusal lands in the schema's field, that too is output that kept the shape. Fixing a shape does not mean an answer comes. A road for the case where no value comes has to sit separately on the receiving side.

And when a check stands up, review goes down. The person who used to skim the logs and the procedure that used to compare results both shrink, and finding the cause of a failure tilts toward digging only through the record of refusals. But a well-formed wrong answer is not in the record of refusals. It is among the things that passed, and it gets found only by working back from a later wrong result.

When not to tighten

If any of the four holds, it is not time to bring in a schema.

When only a person receives it. A schema's value comes from code receiving. If the final product is a report a person reads, the problem of pulling values out never existed, and enforcement solves a problem that is not there while chipping away at the phrasing.

When you do not know the shape yet. Tighten early, while finding the structure of the answer is itself the task, and an understanding that has not set gets hardened. A schema for output that has no receiving code yet is a guess, not a promise.

When the failure you are after is a failure of meaning. If what hurts the system is not failing to pull values out but wrong content, a schema does not reach that failure. And count the effect of a check reducing review and tightening may even push the defense back. The remedy for that failure is verifying the content, and that design is Part VI's job.

When you have not settled the road on violation. Fix it, send it back, or stop. Bring in a schema without making that decision and the decision does not disappear. A tool's or the harness's default makes it instead. That default is usually to fix quietly or to retry without a cap, and either way a cost gets billed that nobody ever set.

One thing runs through the four. *A schema is a mechanism for the receiving side.* It earns its keep when code receives, when the shape has set, when the failure you are after is a failure of shape, and when the road on violation is settled.

And none of those four is written in the schema document. The list of receivers, the maturity of the task, the distribution of failures, and what to do on violation are all circumstances outside the schema.

9.6 Summary

In one sentence. A schema writes the promise between the producing side and the receiving side in a form code can check, which turns failures of pulling values out into noisy events before anything gets used. But a well-formed wrong answer stays, carrying the mark of having passed. So the center of schema design is not what to enforce but

knowing what enforcement does not catch.

What is now settled.

- Fix no shape and a shape is already there. It exists as assumptions scattered through the receiving side's parsers, and it makes itself known only as the receiving side misbehaving on the day the producing side's phrasing changes. An output schema takes that out, writes it in one place, and hands the judgment to code (Definition 9.1).
- A schema is written as a request parameter, not in the prompt. Either you check after generating and refuse, or you bind the generation to the shape, and either way what is enforced goes as far as the shape. Not every constraint you can write in a schema gets enforced, and an unenforced clause goes down into the description and becomes a request again (Section 9.3).
- The model fills a field looking only at its name and description. What goes in, when to leave it empty, and what wins when it disagrees with another field all have to be in the description (Section 9.2).
- Tightening makes the receiving side easy and shrinks what can be carried. A caveat with no field gets dropped or crammed into an open one, and neither catches on the check. The receiving side decides how tight to make it (Section 9.3).
- In a design that checks after the fact, breaking is normal behavior, so the chain of handling is half the design. Fix quietly and the signal is erased, send it back and you need a cap, raise it up and there has to be someone to receive it (Table 9.1). When the breaking rate is high, what is bad is the schema.
- A schema moves only the axis of shape. A failure of shape becomes a guided refusal before use, and a well-formed wrong answer passes (Table 9.2). The danger is not the failure left over but the reassurance that passing creates.

What is still open.

- Designing an evaluation that separates the pass rate of the shape from the success rate of the task goes to Part VI, and to Chapter 18 in particular.
- How to leave the breaking rate and the record of repairs in the run record goes to Chapter 20.
- The shape of the summary a sub-agent returns, and the promise around it, goes to Chapter 17.
- Defending the route by which well-formed text gets read as an instruction goes to Chapter 22.

Try this today. Open the file where your schema lives, point at each field marked required, and *ask* whether the model can know that value at that moment. For every field it cannot, *check* whether there is a value for saying it does not know (Section 9.3). Next, *count* the fields with no description at all, and *write in* what goes in that field and when to leave it empty (Section 9.2). Last, from the past week's log, *count* the shape violations per field and *mark* the one field that broke most (Section 9.4).

9.7 Exercises

1. Design a procedure that estimates how often each of the four rows in Table 9.2 happens in your system. The bottom two rows get counted straight out of the record of refusals, and the second row passed, so it carries no mark in the record. What do you have to compare against for the second row to be counted?
2. Sort the constraints written in your schema into enforced and not enforced. When one on the unenforced side is not kept, what catches it? If nothing catches it, is that constraint a promise or a hope?
3. Run the same twenty tasks with the field descriptions stripped and with them left in, and compare the breaking rate against the rightness of the answers. If there is a field where the breaking rate holds and only the answers get worse, what was the description doing at that field?
4. Design a procedure for setting the retry cap. What you need is the breaking rate per field, the rate of getting fixed when sent back, and the share a round trip adds to the history. How does the procedure differ under the assumption that the fixing rate stays flat up to the cap and under the assumption that it plunges?

9.8 On a second reading

- Read Chapter 17, come back to Definition 9.1, and a sub-agent has been added to the list of producing sides. When the work happens in another context and only a summary comes back, that summary is the result, and what produces it is a model.
- Read Chapter 18, come back to Table 9.2, and the axis of shape and the axis of meaning have become the axes that split evaluation. A metric that lumps the two into one number counts the second row on the success side.
- Read Chapter 20, come back to Section 9.4, and the breaking rate and the record of repairs have become objects of what to leave behind. What was refused and what was repaired have to be in the step record before the gap between schema and model is visible.
- Read Chapter 22, look again at Section 9.5, and the sentence that passing a check is no warrant has become the language of defense. Well-formed text passes too, and once inside it can be read as an instruction.

9.9 Further reading

Start with the feature documentation for structured outputs (Anthropic, 2026h). What to take is not the notation but the three boundaries the document draws around itself. That the place you write the shape is a request parameter and not the prompt, that what it guarantees goes as far as conformance to the shape, and that the constraints you can write and the constraints actually enforced are different.

The standard for field descriptions comes from the material on tools (Anthropic, 2025g). Write what it does, when to use it and when not, what each field means, and what it does not return. From Anthropic (2024a), three items on choosing a format are worth having. That it be close to a shape the model has naturally seen in writing, that keeping the shape not itself be a

burden, and that tokens be left to think in before writing the answer. Tam et al. (2024) has to be read knowing the result is contested. What to take is the question of whether enforcing a shape is free.

Patil et al. (2024) reads differently through this lens. The first failure, not generating accurate arguments, is the producing side breaking the promise, and the second, a hallucinated usage, is the original form of a well-formed wrong answer. For the same reason, Chapter 7's design of results is the most valuable reread.

Part III closes here. Chapter 7 set up the design of tools, Chapter 8 the route of supply, and this chapter the shape of the pieces that come back. Part IV is about bringing into the context what the model does not know, and it starts at Chapter 10.

PART IV

Knowledge and Memory

Agent Memory: Short-Term, Long-Term, Persistent

Sessions end. By completion or by truncation, the context closes, and what lived only inside it is gone. A model does not learn from a run. Whatever yesterday's run found out, today's model carries yesterday's weights. There is one road by which what a run gained survives. It gets written down.

The moment you write, three jobs appear. Decide what to keep, fix what you kept when it goes stale, and pull out what is wrong. *Standing up a store and putting retrieval on top of it does none of the three.* Those three are this chapter.

What is not here, for the record. Where to cut a record and what to turn it into to sit in an index, and how to rank what comes back, are the next four chapters' material, and Section 10.5 hands them over. Who decides whether a record about a person stays or goes is a question of norms, and Chapter 21 takes it.

10.1 What has to be left behind

There is a place where you decide today what you decided yesterday.

An organization's agent has worked in the same code repository for three months. Yesterday afternoon's run spent two hours confirming that one way of making the change breaks another part, and dropped that approach. This morning the same person brings the follow-on task. What today's session knows is what is loaded in today's context. Yesterday's two hours are nowhere. So today's run finds the same approach promising again, breaks the same thing again, and arrives at the same conclusion again.

Why this happens shows if you point at the places where knowing can live. The parameters hardened on the day training closed. If a run came to know something, the only place that knowing lived is the context, and the context dies with the session.

You do not even have to wait for the session to close. The context is finite while it turns. Long tasks mean cutting or dropping what came before (Chapter 6), and some of what was dropped is needed later. The line the memory tool nails into its system prompt is exactly that circumstance. Assume interruption: "Your context window might be reset at any moment, so you risk losing any progress that is not recorded in your memory directory" (Anthropic, 2026e).

So you pick what to keep and put it outside the context.

Definition 10.1 (Agent memory). *Memory* is the mechanism that leaves the facts, decisions, and failures a run acquired outside the context, across the boundaries of sessions and tasks, and keeps them available for a later run to bring into its context, together with the whole body of those records.

The weight belongs on *acquired during a run*. An internal policy document is not something the system acquired but something it received, so fixing that document is not the system's job. Memory is a record a run gave birth to, so managing it stays entirely on the system's side.

What to keep takes one question. *Would a later run have to live through the same thing again without it?* What you can look up again does not get kept. Copy the current state of the code or a procedure written in a document into memory and what you have is not knowing but a copy, and a copy goes quietly wrong on the day the original changes.

What has to be kept is what costs, to get again, what it cost to live through. The published material's use cases sit exactly there: applying "lessons from past interactions, decisions, and feedback" (Anthropic, 2026e). Getting either back means doing that exchange over from the start. And first in line is the record of failure. Work that succeeded stays in the environment as an edited file. An approach that was dropped leaves nothing. The evidence of its existence was the narrative in the context, and when the session closes that narrative goes too.

Start by doubting the sentence "we do not have memory yet." Not having designed it and not having it are different statements. If run logs are piling up, if handoff notes sit in a folder, if artifacts sit in a repository, and if later runs read them again, memory is already there. It is just there undesigned.

Undesigned memory is memory in which all three of those jobs run on defaults. Nobody decided what to keep, so everything stays. Nothing gets fixed, so it stays stale. Nothing gets pulled out, so it stays wrong. *Keeping, fixing, and pulling out are the frame of this chapter.*

10.2 Three tiers

What divides the tiers is not the storage technology, and it is not how long a thing lives. *It is what you are keeping it for.*

Dividing by lifetime looks cleaner and the order flips in practice. How long to hold a record is set by what you meant to use it for. Write the purpose first and the day it disappears follows. Set the lifetime first and there is no ground for why that day and not another.

There are three purposes. Keeping something to carry on the task turning right now, keeping it to carry on with this person or this project, and keeping it for the system to go on referring to. Call them *short-term*, *long-term*, and *persistent*.

Three tiers means one session actually reads three kinds when it opens. This morning's session first opens the progress log yesterday's session left as it closed. How far it got and what comes next are there, and it folds up the day this task ends. Next it opens the file where dropped approaches are written. Three months of them have piled up and they do not go when this task ends. And while it works, the files it reads and the results of the commands it runs pile into the context, and those disappear this evening. Three kinds sit side by side in one context, and the job of managing them is different for all three.

The question paired with who writes is where it lives. In the memory tool, storage sits entirely on the calling side. The model only asks to open and edit a file, and moving that request onto real storage is the calling code's job (Anthropic, 2026e). So keeping a separate folder per person, and keeping some folders read-only, are decisions the designer makes. Blocking anyone from writing into the persistent record is the cheapest way to decide what gets kept.

One more line runs through. Short-term is business inside a session, and long-term and persistent are business between sessions. What shrinks the context inside a session is compaction and clearing, and Chapter 6 covered it. What joins sessions together is memory. The two do not

Table 10.1: The three tiers of memory. What divides them is not the storage technology but what you are keeping it for.

Tier	What for	What goes in	When it goes	Who writes
Short-term	to carry on the task turning now	this session’s history and observations, the item in hand, the contents of a file just read	when the session closes. Before that, whenever compaction comes	nobody decides. The loop piles it up
Long-term	to carry on with this person or this project	the progress log, the checklist of work left, procedures that hold only on this project, the reporting format wanted	the day the task ends. The day the person or the project changes	the run that closes a session updates it. A person corrects it
Persistent	for the system to go on referring to	facts and decisions that outlast a task, dropped approaches and why, the organization’s vocabulary	it does not go unless you say so	you have to decide. The model writes candidates and standing decides promotion

push each other out. They pair. The published guidance says so directly: compaction “keeps the active context small” and memory “preserves the information that must survive summarization” (Anthropic, 2026e). Split in one line: *compaction throws away while it shortens the history, and memory makes things survive being thrown away.*

There is one reason the job of managing differs by tier. A tier with a settled end has its end do the managing. Short-term disappears when the session closes by default, so managing it means deciding what to salvage before it goes. Long-term folds up the day the task ends, so managing it means keeping it current while the task lives. Persistent has no such stand-in. It does not step back when it goes stale, it does not announce itself when it goes wrong, and it does not shrink when it piles up. The rest of this chapter is effectively about that tier.

Point at the extreme and you see this division is not decoration. What happens if everything is persistent? Volume first. Records grow in proportion to running time, so what history did to the context repeats itself in the store. Retrieval comes next. With hundreds of records on one subject, stale ones compete for the top few places, and a fact from three months ago rides at the same standing as one from yesterday. Last is the place taken by records that have served their purpose. A finished task’s handoff catches a new task’s query and takes a candidate slot. A system that keeps everything starts from the opposite extreme from a system that keeps nothing and arrives at the same place. The place where what is needed does not get loaded.

So forgetting is a design object too. The default lifetime of a record whose lifetime nobody set is forever. Deleting is an event that has to be decided by someone, and with no mechanism to decide, the event does not happen. The remedy is not grand. *On the day you write a record, write down when that record will have served its purpose.* One line saying this record folds when the task ends, or this fact gets replaced at the next verification, moves the decision from the day of deleting to the day of writing.

10.3 The moment it gets used

Memory does nothing by being written. It changes a decision by being loaded into the context. So half the design is when, where, and how much to load.

When Chapter 4 cut the context into layers, one place was already set aside. The place where what an earlier session left for the next one gets loaded. Memory widens that place. It is not some new kind of thing but *one layer of the context*, and if it is a layer you have to ask about its moment, its place, and its size.

When does it load? Chapter 4 had two branches. Loading ahead, and fetching with a tool at the moment of need. The answer the memory tool picks mixes them. The first job of a session is to view the *listing* of the memory folder. Then it opens only the files that catch on today's task. The cover ahead, the contents on demand.

There is a reason viewing the listing first is a rule and not a suggestion. The instruction is nailed down: "ALWAYS VIEW YOUR MEMORY DIRECTORY BEFORE DOING ANYTHING ELSE" (Anthropic, 2026e). Leave looking things up to the judgment of the moment and the failure of not looking because you did not know you needed to repeats itself session after session, and the log of a session that did not look carries no mark at all. A failure that never gets counted never gets fixed.

Where does it load? The listing loaded at the start of a session comes at the front. The front reloads on every step, so it has to be small, which is why it has to be a listing. The contents of a file opened by tool attach to the history and come later. What attaches later follows along from that step until the session ends, so a decision to open one large file whole costs for the whole session.

How much loads? Put a ceiling in three places. How large one file may grow, how many characters come back at once, and how many files one session opens. The first two are in the published operational guidance as they stand. Track file sizes and cap how large a file can grow, and cap how many characters the view command returns, letting the model page through the rest by range (Anthropic, 2026e).

Look at a real one and all three show at once. What a session receives first is a listing like this.

```
/memories
  1.2K  verification-procedure
  3.1K  dropped-approaches
  0.8K  organization-vocabulary
  0.4K  reporting-format
```

That much loads at the front, and its size is a few dozen tokens. If today's task catches on verification, it then opens one file.

```
dropped approaches

[assertion] do not rebuild the index from scratch in verification
  written Mar 4 / ground the run log of the same day / verified
  why: rebuilding the index took longer than the whole verification
  revisit when: the index tool starts updating only what changed

[candidate] the nightly batch slips on Mondays
  written Mar 11 / ground one run's observation / not verified
```

One more line is attached to each record, and three things are in that line. Standing, age, and ground. Standing says whether this sentence may be read as an assertion, age says how stale it might be, and ground says where to trace back to when you doubt it. Without those three,

the next section's work does not stand. You cannot decide what to doubt first, and you cannot decide which side wins when two records collide.

The moment of loading has a partner in the moment of writing. Neither is best left to the judgment of the moment, and there is one trigger practice uses. The threshold at which the context gets cleared. In a setup that clears old tool results to shrink the context, a warning that something is about to be cleared goes out just before the threshold, and the run that gets that warning writes what it wants to salvage into memory before the clearing. That hangs the moment of writing on an event rather than a judgment. Events land in the log, so a session that passed without writing gets counted.

What happens after loading is what the earlier chapters said. A sentence in memory is text loaded in the context after all, so putting it in tilts rather than decides. Loading it does not mean it gets kept to.

10.4 Memory that accumulates wrongly

Memory that only piles up starts making the system worse at some point. Not only because grown records eat the context. Unlike a document received from outside, memory spoils on its own.

Three things differ. First, *who wrote it*. An internal policy document was written by a person outside the system and that person maintains it. If the document is wrong, the person who fixes it is over there. A sentence in memory was written by the system itself. Second, *can it be wrong*. Retrieval stands on an assumption that the corpus is mostly right. Memory cannot make that assumption. A sentence in memory is not the observation itself but a conclusion drawn on top of an observation, and a conclusion is drawn fresh every time. Being wrong is not an exception but a constant to put in the arithmetic. Third, *who fixes it when it goes stale*. When a policy changes, the document's owner fixes it. The most common reason memory goes stale is that the world changed, and in the world an agent lives in, the main culprit behind that change is usually itself. If some run next month is what made the check fast, then what made the staleness, what holds the clue to noticing it, and what has to fix it are all on the system's side.

What piles up comes in four shapes.

Decide in advance what to write

Who makes the decision to keep? Three candidates. A harness rule can hang it on a mechanical event. Write only what carries standing, like a decision that passed verification or the conclusion of a completed task. The model can judge during the run. And a person can write it. Of the three, the model's hand picks up the widest range, and is the most dangerous for the same reason. What it writes about is its own run. Chapter 6 warned that when the summarizer is the author of its own narrative, progress gets inflated and failures get smoothed over. Memory is where that summary hardens across sessions.

The compromise takes a familiar shape. *Borrow the hand and do not hand over the decision*. The model writes candidates and standing decides promotion. A conclusion that passed verification and a fact a person confirmed get written with the standing of an assertion. What the model wrote on its own stays with the standing of a candidate, and rides into a later run's context carrying that standing. Memory with no standing is read as assertion, all of it. There is also the road of narrowing the range of what gets written. The published documentation

Table 10.2: Four shapes of memory piling up wrongly. The right two columns are this section’s remedies.

Shape	How it happens	What tells you	Where to stop it
A wrong record	an early run misread an observation and wrote it down as a conclusion	nothing tells you. Only reverifying brings it out	attach standing when you write. Only what passed verification reads as an assertion
A stale record	the world changed and the old sentence sits there	when there are two or more records on one subject	do not append. Open the old record and fix it. One file per subject
A worthless record	no standard for what to keep, so everything lived through got written	a file not opened in a long time	decide in advance what to write. Delete by age and size
A copy	something already in the environment got copied over	the day it disagrees with the original	do not write what you can look up again (Section 10.1)

gives the example directly: “Only write down information relevant to <topic> in your memory system” (Anthropic, 2026e).

Build the road to fix and the road to delete together

Staleness first. Memory says “the checks in this repository take a long time when you touch a particular module,” and some run made those checks fast. At that moment the memory went stale, and if nothing happens, the next run receives the stale record and writes the workaround that is no longer needed. The easiest road, writing one more record, ends predictably. The old sentence and the new one get found at the same standing, and which to believe passes into the model’s hands. The road that is left is to find the old record and fix it.

For that, a judgment has to stand first: that the new fact and the old record are records about the same thing. Scatter records into chunks and that judgment is needed on every update. Put one file per subject and the file name is that judgment made in advance, so updating becomes opening that file and fixing one line. *The form of storage sets the cost of managing.*

That the memory tool is a folder of files rather than an index earns its keep here. The model creates, reads, updates, deletes, and renames files (Anthropic, 2026e). No ranking, no representation. The shape of the folder and the file names are the clue to what to open. In this form, fixing becomes rewriting one line and pulling out becomes deleting one file. You can also put the demand to keep it tidy on the model directly. The documentation gives the instruction as an example: keep the memory folder’s content “up-to-date, coherent and organized,” rename or delete files that are no longer relevant, and do not create new files unless necessary (Anthropic, 2026e).

Finding a conflict does not make resolving it obvious. Time is the most common standard. Let the later record win, and rather than deleting the loser, close its period and keep it as history. That is usually enough, and the limit has to be written down. A date tells you which is later. It does not tell you which is right.

The branch that was wrong from the start is worse. Say one early run misread an observation

and wrote “this setting was turned off because of a performance problem.” The real reason was something else. Every time a later run considers the improvement of turning that setting on, this record loads and the run drops the proposal. Using a loaded record as ground is the whole purpose of having memory, and dropping the proposal is rational in that state of information. And the dropped decision gets written down. “Decided not to turn this setting on, because of the performance problem.” Now there are two wrong sentences, and the later one is harder because it carries the earlier one as its ground. *A wrong memory acquires a following of sentences that take it as ground.*

The reason a wrong memory is worse than a wrong chunk in retrieval is the road to fixing it. In retrieval you go back from the answer to the chunk and from the chunk to the document, and a comparison holds. The source of a wrong memory is a run that has passed, and that run’s context went with the session. If you kept the run’s record outside the summary (Chapter 6) you can trace back to that day’s observation and see where the conclusion slipped. If you did not, there is no original to compare against, and the road left is to ask the world again. Turn the setting on and see, read the document again, ask that team again.

The shape of pulling out is settled here too. Delete a record judged wrong without a trace and the conditions for misreading the same observation the same way stay exactly where they were, so the system goes off to write the same wrong thing again. So the basic form is correction, not deletion. Write on top of the record that it was confirmed wrong and what replaced it, so it still catches in retrieval but no longer reads as an assertion. There is one exception. A record about a person may require erasure rather than correction, and the reason for that sits outside the system’s arithmetic. The next section draws that line.

Count how much has piled up

Reverifying is expensive and cannot be done on everything, so doubt needs an order. Old records, records with thin ground, records that load often into heavy decisions. That order stands only if age and ground and standing are written on the record. That is why the real one in the last section had that line attached.

Settle first what to count. Three numbers come out without reading any content. The first is the count and total size of memories. Compare against last month and see how much it grew and whether what got opened grew with it, and a store that only piles up shows itself. The second is how many files one session opens. Near zero means memory exists and is not used, and growing without a ceiling means the listing is already too long. The third is the distribution of last-modified dates. All of them still on the day they were written means the road to fixing was never built at all. All three get counted with no judgment.

There is discipline that takes even less effort. Periodically delete files that have not been accessed in a long time, and cap how large one file may grow (Anthropic, 2026e). Age and size are two values you can measure without reading the content, and they are also the only discipline you can hang with no judgment.

Measuring has its limits too. How wrong memory is can only be estimated from a reverified sample, and there is no general answer for that sample’s size or interval, and staleness runs at different speeds in different domains, so there is no general answer for what age to start doubting at either. Still, not counting and counting roughly are different. *In a system with no check, a wrong memory’s lifetime is the system’s lifetime.*

Note also that this failure rarely makes the list of things to repair. A run goes strange, you open the log, and retrieval returned the related record and the model was faithful to the record

it was given. Nobody did anything wrong, and the wrong is in a run three months ago, and that run is nowhere in today's log. That is why a system that brings in memory needs one more item on its failure-analysis list. *The memory this decision stands on, when and by whom was it written, and has it ever been verified?*

10.5 When not to keep memory

Memory looks like a mechanism you lose nothing by having. So it often fails to become a design question and gets slotted in by default. Take a system that answers inquiries arriving from outside. Each inquiry comes from a different person, and when one ends there is nothing to carry into the next. Attach memory here and what is kept grows while nothing gets reopened, and only the work of fixing and pulling out what grew remains. You pay the cost and get none of the worth.

If any of the following holds, it is not time to bring in memory.

1. **There is no repetition to carry.** It comes in three faces. A task that ends in one go, a place where a different person arrives each time, and a task with nothing to join to an earlier run. Memory's worth comes from repetition, with the same repository, the same policies, the same person. With no repetition there is nothing to join, and a handoff and artifacts that join one session are enough. A system without repetition has a pile of files nobody opens and pays only the upkeep.
2. **The environment is already the better memory.** If the organization has a live code repository and documents and a task list, decisions and facts are better written there. A record that people and the system read and fix together spreads the staleness among many. Write something you could have written in the environment into a memory store instead and you have one more copy, and copies go stale. What memory should take on is what has nowhere else to be written, which is the system's own experience.
3. **You cannot carry being wrong and you have no way to verify.** Being wrong is a constant in memory, and only reverifying clears it out. In a domain with no way to ask the world again whether a record is true, or on a task where one wrong assertion leads to a heavy action, memory becomes a confident source of contamination. In such a place memory's part is not to give the answer but to point at what to read again.
4. **It is about a person.** A system that remembers a user's preferences gets more convenient, and a record about a person cannot be settled by that trade alone. Where it differs in kind from the other three is who owns the pulling out. There are places where what is kept and when it is deleted has to be settled by that person rather than by the system's arithmetic, and at that moment deleting stops being a technical problem. The discussion of norms around consent and deletion goes to Chapter 21.

There is a shape running through the four. With no repetition there is nothing to keep. When the environment stands in, the work of fixing can be shared. With no verification there is no pulling out. And when it is about a person, the owner of pulling out changes.

The four are not a verdict on whether to have memory at all. They have to be asked per tier. A system without repetition still needs the long-term record that joins one session, and only persistent has to go. Even in an organization where the environment is the better memory, the failures the system itself lived through have nowhere else to be written, so they stay persistent. *The question is not whether to have memory but how far up the tiers to go.*

Last, draw the boundary that gets confused most. *Memory and retrieval are different questions.* Memory is keeping what the system itself lived through. Retrieval is finding what is already somewhere. The keeping side has to settle what to write first, and the finding side already has what exists settled. Confuse the two and you start moving documents into the memory store, and at that moment you are on the fourth row of Table 10.2.

Everything this chapter covered was the keeping side. The finding side starts here. Where to cut a record and what to turn it into to sit in an index, and whether to load what came back and where to keep knowledge, are Chapter 11. Overlapping two retrievals and reranking is Chapter 12. Counting by entities and relations is Chapter 13. Sources that are not text are Chapter 14. Memory stands ahead of those five because *what to keep is a decision that comes before how to find.*

10.6 Summary

In one sentence. Memory is the mechanism that carries what the system left behind itself across sessions and tasks, and when the system becomes the owner of the record, keeping and fixing and pulling out move inside.

What is now settled.

- A model does not learn from a run, so the only road by which what a run gained survives is being written down. The context dies with the session and is finite while it turns. In a system where records pile up and later runs read them again, memory is already there, and the decision is whether to make it a design object (Definition 10.1).
- What divides the tiers is not lifetime but what you keep it for. For the task turning now, for this person and this project, and for the system to go on referring to, and each tier differs in what goes in, when it goes, and who writes (Table 10.1). A tier with an end has its end do the managing. The persistent tier has no such stand-in.
- Written memory earns its keep by being loaded. The listing at the front at the start of a session, the contents attached later on demand. Put ceilings on file size, on how much comes back at once, and on how many files get opened. Write standing and age and ground on every record (Section 10.3).
- Memory spoils on its own. It wrote itself, being wrong is a constant rather than an exception, and what made it stale was usually itself. What piles up is a wrong record, a stale record, a worthless record, and a copy (Table 10.2). Updating has to be fixing in place rather than appending, and the basic form of pulling out is correction rather than deletion (Section 10.4).
- There are four places not to keep memory. No repetition to carry, an environment that is already the better memory, no way to verify what is wrong, and a record about a person. The last is the only tier where the owner of pulling out stops being the system (Section 10.5).

What is still open.

- Weighing what to put in memory against what to put in a retrieval corpus against what to put in the parameters goes to Chapter 11.
- There is no answer for the arithmetic when several agents split or share a memory. Contexts can be divided while memory stays shared, so one agent's record becomes another agent's premise, and the author knows of no material evaluating that trade.
- Evaluating the quality of memory, which is designing the sample to reverify, goes to Part VI.
- Defending the route by which text written into memory steers later runs goes to Chapter 22.
- The norms and regulation around consent, access, and deletion for memory about people go to Chapter 21.

Try this today. *Find* in the code whether a file that later runs read again exists right now. Open

the place logs get written, the place handoff notes get written, and the place artifacts get kept, and *mark* the ones later runs actually reopen (Section 10.1). For each one you marked, *write* one line saying when it disappears. What has no day of disappearing written down is persistent, and that is the tier the whole job of managing hangs on (Section 10.2). Last, *measure* the last-modified date and the size of the files judged persistent (Section 10.4).

10.7 Exercises

1. Make a list of the undesigned memory in your own system. Of the logs, handoff notes, artifacts, and decision records, which have later runs actually read again? For each, lay out in a table whether keeping, fixing, and pulling out are designed or left alone.
2. Design an experiment comparing memory written at the model's discretion against memory where only what passed verification gets promoted. Run the same ten tasks under both and ask what you have to evaluate for the two memories' quality to come apart. To find the traces of self-narrative in the discretionary memory, the inflated progress and the omitted failures, what do you compare against?
3. Design an experiment that plants one wrong memory on purpose and traces the decisions that take it as ground. After how many runs does such a decision appear? How many new records did that decision give birth to, and what happens to the rest if you correct the original sentence? Explain with Section 10.3 why this experiment does not hold without standing and age and ground.
4. Practise the rule of writing a lifetime. For each kind of record your system leaves, write first what you are keeping it for, and check whether the day it disappears follows from that. If some kind does not follow, is that a purpose left vague, or is it something that ought to be persistent?

10.8 On a second reading

- Read Chapter 11, come back to the second condition in Section 10.5, and the line between what to write in the environment and what to write in memory has become one piece of a placement decision. Memory's upkeep gets set beside the upkeep of a retrieval corpus and the cost of updating parameters there.
- Read Chapter 15, come back to the long-term row of Table 10.1, and the discipline of what to leave when closing a session turns out to have been the write discipline of the long-term tier. A handoff is memory's smallest case, the one where the lifetime is tied to the task so that managing shrinks to a single update.
- Read Part VI, come back to what was left open in Section 10.4, and the sample to reverify and the order of doubt have become problems of evaluation design. What growing memory piles up while you do not know how wrong it is gets precise there.
- Read Chapter 22, come back to write permission in Section 10.4, and that design looks again like a problem of defense. What gets written into memory is text that can load into the context of every later run. The moment content from outside the trust boundary gets written into memory, one injection becomes a standing injection.
- Read Chapter 21, come back to the fourth condition in Section 10.5, and the one line

about the owner of pulling out changing has opened into the language of permission models and regulation.

10.9 Further reading

Packer et al. (2023) starts from the fact that language models are “constrained by limited context windows, hindering their utility in tasks like extended conversations and document analysis,” and proposes a technique drawn from the hierarchical memory of operating systems, “virtual context management.” A system built this way has the model read and write an external store through function calls and edit its own context. What to take is not the details of one configuration but two things. That a mechanism called memory starts from the limit of the context window, and that a structure in which the model manages its own memory has already been proposed. Read with this chapter’s eye and there is one thing to ask. In that structure, whose hands hold keeping, fixing, and pulling out?

Anthropic (2026e) is where the conditions of the real thing came from. That memory can be a folder of files rather than an index, the rule to view memory before doing anything else, the recommendation to pair it with compaction, the example of narrowing what gets written by instruction, and the operational guidance to delete files not opened in a long time and cap file size. The same material also says to set the scaffolding up first for work that spans several sessions. An initializer session builds a progress log and a checklist of features before any substantive work, later sessions read and update them, and you work one feature at a time and mark it complete “only after end-to-end verification confirms it works, not when the code is written.” There is one thing not written there. The warning that without checking the path of a memory file, a file outside memory gets opened, and that problem is Chapter 22’s material.

The most valuable reread is inside this book. Chapter 6 demanded that the source text of a compacted stretch stay outside the context as it was, and seen from here, that was buying in advance the only road by which a wrong memory can be fixed by comparison.

Retrieval-Augmented Generation (RAG)

Someone asks about an internal policy. What is the ceiling on travel spending? The model answers. The sentence is smooth and it carries confidence. And that policy was revised last month, and the model gave the value from before the revision.

Worse, the model does not know it gave a stale value. And the answer does not say where the value came from, so anyone who wants to check has no document to open.

Retrieval augmentation puts the document into that place. Keep the policy in a store outside the model rather than inside it, pull out only the relevant part when a question arrives, load it into the context, and answer on top of it. It is commonly introduced as “a mechanism that tells the model what it does not know,” and that definition is narrow. Three things actually change. What the system knows, when that knowledge is from, and where it came from.

In an agent the last two weigh as much as the first. Knowledge you cannot update keeps its confidence while it goes stale, and an answer whose source you cannot name has nowhere to go back to when it is wrong.

And this mechanism’s force reaches only as far as putting things into the context. Putting it in does not mean it gets used, what was missed is quiet, and only what is written down gets found.

What this chapter covers is the terrain and the representative method. Stacking retrievals in layers and reordering candidates goes to Chapter 12. The mechanism for taking questions retrieval cannot answer goes to Chapter 13. What is lost when the source is not text goes to Chapter 14.

11.1 Using material from outside

Part III set up two roads by which a model reaches outside its context. Tools were the first, and the protocol that supplies those tools was the second. The third road arrives here. Knowledge.

The three differ in kind. *A tool makes something happen. Knowledge makes something known.* A tool that edits a file changes the world, and the result that comes back is a report on the changed world. Fetching a policy document changes nothing. One more sentence rides in the context.

So what you ask differs. Of a tool you asked whether this action can be undone. Of knowledge you ask when this sentence is from and where it came from.

Narrow down what the model does not know first. Three things. What happened after training closed. What circulates only inside an organization: policies, code, meeting notes, incident histories. And what was just decided, say the deployment procedure changed in this morning’s meeting. None of the three has any way to be inside the model.

The problem is not the amount of knowledge the model holds but its character. Put two questions side by side. “At what temperature does water boil?” and “In the latest version of

this payments library, what arguments does that function take?” The model answers both with the same fluency and the same confidence. But the first answer was true ten years ago and will be true ten years from now. The second is from the day training closed, and who knows how many versions have passed since. A fact that changed yesterday and a fact from ten years ago come out of the model from the same place with the same confidence.

The trouble splits three ways.

First, *the timestamp is smeared out*. No date saying when a piece of knowledge is from is attached to it. One moment, the close of training, applies to all of it in one go. What changed after that is not absent. It is present at the old value, and an old value, unlike an absence, does not stop a question.

Second, *you cannot fix one item*. There is no way to pick one fact inside the model and fix it. The unit of fixing is not the item but the whole of training, so when one line of policy changes there is no route by which to tell the model.

Third, *there is no source*. There is nowhere to ask where an answer came from. Nothing for a verifier to take hold of, and no address to go and fix when it is wrong.

This trouble was written down as a problem back when the name retrieval-augmented generation was being made. Lewis et al. (2020) starts from the observation that large pre-trained language models store factual knowledge in their parameters, and that their ability to access and precisely manipulate that knowledge is limited. And it names two things as unsolved. *Providing provenance for decisions, and updating world knowledge*. Which is to say source and update are not side effects discovered later while chasing accuracy. They stood beside accuracy as separate problems from this mechanism’s starting point.

There is an open problem here. What was said above talks as though you could divide what the model knows from what it does not, and the list of that boundary does not exist. What it knows you learn by asking, and two phrasings of the same thing can draw different answers. So the remedy “fill in what it does not know with retrieval” presupposes a list of what it does not know, and that list is not given.

The workaround in practice is to give up on making the list. Instead of asking whether it knows, make the classes of question that can go stale or that need verification use outside knowledge, even where the model looks like it knows.

The circumstances of the day you change models get recomputed here too. On the axis of knowledge, changing models is swapping the whole encyclopedia. There is no list of which items changed how this time either, so answers that leaned on knowledge inside the model change silently along with it on the day of the swap. A document in a store, by contrast, stays as it is when the model is swapped. *Moving knowledge outside is a decision that shrinks the surface a model swap touches, before it is a decision about cost and quality*.

11.2 What changes when you keep it outside

Keep the same knowledge in a store outside the model and bring it into the context when needed, and three things change.

The cost of fixing differs. When a policy changes you fix the document, and the next retrieval brings the new one. The unit of fixing comes down from training to one document, and the age of knowledge comes down from “since training closed” to “since the document was last edited.” A store goes stale too. The difference is that the staleness became visible and fixable.

Table 11.1: The three axes that change when knowledge goes outside the model. The bottom two rows weigh as much as the first.

Axis	Inside the model	In a store	Why it weighs in an agent
Fixing	frozen in one go at the day training closed. What changed stays at the old value	updated per document. Staleness is visible and fixable	a loop does not stop acting on a stale premise
Ground	none. The only thing to repair is the model	every chunk has an address. The failure splits three ways	the place to fix a wrong answer narrows to the document, the finding, or the writing
Putting in and out	no switch. Always loaded	decided per run. Different bundles by person and by task	permissions, experiments, and per-task configuration all stand on this axis

You can see what the answer stood on. A chunk fetched by retrieval has an address. In a system with no sources, the only thing to repair for a wrong answer is the model. Polish the prompt, swap the model, or train again, and none of the three aims at that one answer. With a source there is a road from the wrong answer back to the chunk and from the chunk back to the document, and at the end of that road the failure splits three ways. If the chunk itself is wrong, fix the document. If the chunk is right and the answer did not use it, the problem is on the writing side. If the chunk is not there at all, the problem is on the finding side. *A source is not an ornament on an answer. It is where looking for the cause of a failure begins.*

Putting in and taking out become run-time decisions. Knowledge inside the model is always there. There is no switch for taking it out. Knowledge in a store can be decided on for every run. Showing different documents to users with different permissions, pulling one document out for an experiment, loading a different bundle per task, all sit on this axis.

The price comes out of the same place. One step is added for finding, and that step can fail. Knowledge inside the model has no event called absence. It cannot fail to be there at the moment of need. Knowledge in a store has to be found to be used, and finding can go astray. *You sold certainty and bought updatability and provenance.*

There are three places knowledge can sit

There are only three places to keep knowledge. The prompt, always loaded on every run. The store, loaded only when needed. And inside the model. The first two are context and the third is not.

The question people put up first when settling the placement is usually performance. Build all three and compare and pick the one that came out ahead. Do that with one coding convention and usually all three keep the convention. The decision does not stand.

Change the question and the answer splits. *What happens on the day the convention is revised?* The prompt takes a sentence edit and the store takes a document edit. If you put it inside the model, nothing happens. There is nowhere to fix, and until you train again it keeps the old convention with the same confidence.

Even if the accuracy of the three were identical, the decision does not go away. *Performance*

Table 11.2: What each of the three places buys and sells. Performance is in neither column.

Place	What it buys	What it sells
The prompt	certainty. It never fails to be there at the moment of need	cost. It loads on every step, and pays the share of runs that never use it
The store	updatability and provenance. Fixed per document, with an address from answer to ground	certainty. It has to be found to be used, and finding and writing both have to hold
Inside the model	context. Its size never appears in the bill	update and provenance. The unit of fixing is training, and there is nowhere to ask for ground

is a value that rides on top of this decision, not the axis of it. A placement chosen on performance makes you repeat the same mistake. Say you put it inside the model. Months later the convention is revised, and the system keeps producing code that follows the old convention at the same quality as before, and since all you evaluate is performance, nothing catches on the metric.

What you have to buy is usually unknowable on the day you settle the placement, so an order comes attached. *Use the place that is cheapest to undo first.* The prompt takes one sentence, the store runs from one document to the whole corpus, and inside the model is the cost of training again. What Hu et al. (2022) reported is that the cost of fixing came down, not that the cost of undoing came down. *What went inside the model cannot be pulled back out.*

11.3 Retrieval as a step

Knowledge kept outside does not get used by itself. The part needed has to come into the context at the moment of need, and that bringing-in is retrieval.

Definition 11.1 (Retrieval-augmented generation). Knowledge sits in a store outside the model. At run time you pick only the chunks of it related to the task and load them into the context, and the model generates its answer on top of that context. That arrangement is *retrieval-augmented generation*.

One retrieval does four things. Build the query. Find candidates. Pick a few. Put them in the context. Each step has something to settle.

- **Build the query.** Use the user’s words as they are, have the model rewrite them, or expand into several? In a loop the model builds the query too, so this step also differs every time.
- **Find candidates.** Find by the words or by the meaning, and do you overlap them? The next section opens this step.
- **Pick a few.** How many of the top to load? Do you put a threshold that drops low scores? The published guidance reports a clear direction. Passing twenty beat passing the top five. It compared 5, 10, and 20 chunks and “found using 20 to be the most performant” (Anthropic, 2024b). But more candidates means more unrelated material rides in too, so this scale has to be measured again on your own task.
- **Put them in the context.** Where do they go? Retrieved chunks usually land neither at the front nor in the recent history but somewhere between, and that is the least-read place.

There is one more question before all that. *Is retrieval needed at all?* If the whole document set fits in the context, loading it whole is better. What sits at the front gets caught by the cache, so the cost falls sharply on repeated calls, and the finding step disappears whole, and with it the failure of not finding. The scale the published guidance gives is 200,000 tokens: “If your knowledge base is smaller than 200,000 tokens (about 500 pages of material), you can just include the entire knowledge base in the prompt” (Anthropic, 2024b). That scale rises as contexts grow, so what to take is not the number but the question. *Do my documents fit inside the budget?*

Retrieval is a kind of tool

Who calls the retrieval splits the arrangement in two.

The harness finds it ahead of time. A question arrives, the harness runs the retrieval first, lays the results into the context, and then calls the model. The model does not even know a retrieval happened. One round trip, so it is fast, and what got loaded stays in the log as it is. In exchange, if the model judges the results bad there is no road to look again.

The model calls a retrieval tool. Retrieval stands as one entry in the tool list, and the model writes a query and calls it when it needs to. It can look at the results, change the query, and call again. In exchange the round trips grow, and a new failure appears, the failure of deciding not to call.

Whichever arrangement, this section needs one property. *What retrieval returns is not an answer but candidates.* A list of chunks that resemble the query, laid out in an order. That order is not the judgment that they are related but an imitation of that judgment. A query is an expression of a need, and an expression can be out of step with the need, and resembling and being related are different things. That gap is the center of this chapter, and it opens where Section 11.4 asks whether closeness is relevance.

11.4 Chunking, embeddings, vector search

The last section set up what retrieval does. Here we open the inside. Follow the most widely used branch and write down what it buys and what it sells.

Why cut

A 200-page policy manual went into the store. You cannot load 200 pages for one question about the ceiling on travel spending. So you cut the document ahead of time. This job is called chunking. There are two reasons for it and the two are different.

The first reason is the context budget, and it is the familiar one. Long is worse and mixed is worse, so a design that loads the whole document was already rejected. But if the budget were the only reason, chunking would be a problem of loading rather than of storing. You could store the document whole and cut out only the part you need after finding it.

The reason chunking becomes a decision made on the day of storing is the second one. The vector search below turns one chunk into one representation and finds by that representation, and *the representation is the same size whether the chunk is large or small.* One sentence becomes one representation and a 100-page document becomes one representation.

Different content goes into the same size of vessel, so a long text’s representation becomes something like an average that smears everything it says. Say you made one representation out

of a whole company newsletter. A personnel announcement, a revision to the security policy, and a cafeteria notice all smear into one point. To a query about the security policy that point is middlingly close, and if there is a short memo that says nothing but security, that one is far closer. That holds even if the detail of the policy revision is written in the newsletter.

Which is to say *chunking is keeping a thing findable before it is cutting a thing down to a loadable size*. On the day you bring in a model with a big context you want to stop chunking, and what disappears that day is the constraint on loading, not the smearing.

A stored chunk looks like this.

```
[chunk 4172]  source policy/expenses.md  4.3  revised 2026-07-01
-----
Travel spending up to five million won requires no prior
approval. Anything above that requires a department head.
```

One chunk carries the body and the address together. The address is the source the last section spoke of, the road to trace back when the answer is wrong. And the very next chunk makes the trouble.

```
[chunk 4173]  source policy/expenses.md  4.3  revised 2026-07-01
-----
The clause above does not apply to overseas travel.
```

To the query “overseas travel spending ceiling” the first chunk is close. Words overlap and the subject is the same. The second chunk carries “overseas travel” and is short, and what “the clause above” refers to is not inside the chunk, so it does not resemble the query much. Load only the first chunk and the answer is five million, and that answer stands on half the document.

What makes this failure vexing is that there is nothing wrong with the loaded chunk. It is related, and its content is exactly what the document says. What was left outside the boundary is the exception the document had attached to that sentence. *Half of the right thing gets loaded in perfect shape*.

There are two roads for where to cut.

Cut by size. Break mechanically every few hundred tokens. It applies to any document and the implementation is a few lines. In exchange the boundary falls in the middle of a sentence or the middle of a table.

Cut by structure. Borrow the boundaries the author drew. A paragraph is the unit the author wanted read as one lump. A section heading is a marker that the subject changes here, and a table is a bundle of values that have to stay together. Authors tend to keep the owner of a pronoun inside the paragraph they close, and to that extent a chunk reads on its own. In exchange, in front of a transcript or a run of concatenated logs, where the author drew no boundaries, there is nothing to borrow, and in front of a document with a ten-page paragraph the size problem stays exactly where it was.

Overlapping chunks is common because it costs almost nothing. The tail of one chunk rides in the head of the next. The implementation is backing the cut up a few sentences, and a sentence near a boundary sits in two chunks, so what one side misses the other catches. The price is three things. Storage grows by the overlap, the same sentence takes two candidate slots, and above all *the boundary does not disappear. It moves*. A rule and its exception that sit farther apart than the width of the overlap are still split.

There is one more road that buys more by paying more. Write a line of context onto each chunk. At index time, a model that has seen the whole document writes a short explanation for each chunk and prepends it, and the chunk is turned into a representation with that line attached (Anthropic, 2024b). Chunk 4173 becomes this.

Table 11.3: The two extremes of chunk size. The real choice stands somewhere between the two columns.

	Cut small (extreme: one sentence)	Cut large (extreme: the whole document)
Representation	sharp. It says one thing, so there is nothing to smear	smearred. It becomes an average of several subjects and is middlingly close to every query
What loads	half of it, with its context gone. Pronouns, ellipsis, and references break at the boundary	whole, and mostly unrelated. The moment it loads the context goes cloudy
The shape of failure	the answer spans several chunks, and that it spans them is invisible	retrieval itself goes cloudy, and once loaded the answer gets pulled by the unrelated material

```
[chunk 4173]  source policy/expenses.md  4.3  revised 2026-07-01
context added  This chunk is the exception clause to the
                travel spending ceiling.
-----
The clause above does not apply to overseas travel.
```

Without knowing what “the clause above” refers to, this chunk now reaches a query about the overseas travel ceiling. The published guidance calls that added line “usually 50-100 tokens,” and reports that contextual embeddings alone “reduced the top-20-chunk retrieval failure rate by 35%” (Anthropic, 2024b).

The cost is model calls proportional to the number of documents. Putting the source text in the cache brings it down a great deal but not to zero. And the attached context is prose a model invented, so it can be wrong. A wrong context moves a chunk to the wrong place, and that mistake is nowhere in the source text, so comparing against the original does not reveal it.

There is no right answer for the chunk size or the overlap width. Where you stand depends on what questions arrive, and how many of them ask about how narrow a fact is what settles it. And chunking has to happen before the questions arrive. It is a problem of order rather than of technique, so no chunking technique solves this circumstance itself.

That the failure a boundary makes repeats every time weighs too. A boundary gets drawn once on the day of storing and does not get redrawn per question. A question about the overseas ceiling receives five million with no exception every time it comes. *And this failure rarely makes the list of things to repair.* Retrieval returned a related chunk and the answer was faithful to that chunk, so both look fine.

Finding by meaning

How do you find what you cut? The oldest method is finding by the words. Does a word from the query appear in the document as it is? There is one trouble. The document wrote “annual leave” and the query asks about “vacation.” The document wrote “timeout value” and the query says “no response.” As long as you find by the words, two expressions of the same meaning are strangers.

What you want is clear. A map on which two sentences carrying the same meaning sit near each other. Then a query about “vacation” has the chunk that says “annual leave” nearby.

Such a map is not a strange object. Looking for a book whose title you do not know, a person walks the shelves. Cookbooks sit beside cookbooks and an ingredient encyclopedia sits on the next shelf, because the person shelving put them there. Walking along, even when it is not the book you were after, something near what you were after comes to hand.

Making such a place for text has a name.

Definition 11.2 (Embeddings and vector search). An *embedding* is a mapping that carries a chunk of text into a fixed-size list of numbers, trained so that different expressions carrying the same meaning are carried to nearby values. *Vector search* stores a document’s chunks as embeddings and finds the chunks whose values are close to the query’s.

Run it once and you see what goes back and forth.

```
query overseas travel spending ceiling

what came back (top 3)
0.83 [4172] Travel spending up to five million won requires...
0.79 [0918] Overseas travel airfare is economy class by...
0.61 [4173] The clause above does not apply to overseas travel
```

One number is attached to each chunk and they are laid out in the order of that number. That number is closeness.

The word that carries the weight in the definition is *trained*. What is close to what was not written down as a rule by anyone. It was taught by example. How closeness gets measured is not settled by itself either. Which measure to use is paired with the representation and settled on the day of storing.

And that number cannot be read. There is nowhere to ask why 0.83 is larger than 0.79, because none of the numbers it was carried into carries a meaning of its own. *Closeness is a judgment handed to you as one number with no explanation*. Set that against lexical search, where which word overlapped was always visible.

That this method actually works is a matter of evidence rather than derivation, and there is evidence. Karpukhin et al. (2020) beat the baseline in a place where lexical search was the de facto standard, using nothing but finding by meaning. Two things get taken from it. The structure of carrying query and document over separately and comparing them, and the fact that the structure works.

The price can only be stated as an observation too. A representation is a copy of the world training saw, so where training rarely saw a string, closeness does not do its job. Error codes, in-house identifiers, and product code names sit there. The published guidance’s example is exact. Ask for a specific error code as it is and vector search brings back writing about error codes in general and can miss the very place that code is written. Lexical search “looks for this specific text string to identify the relevant documentation” (Anthropic, 2024b). So practice does not throw lexical search away. It overlaps them. How to overlap is Chapter 12’s material.

The choice of representation comes with a bill on it, and you read the bill before deciding. *The cost of storing it all again*. Different embeddings make different maps, so a value carried over by the old representation and one carried by the new cannot be compared. The day you change the representation is the day you carry the whole document set over again. The day you change how you cut costs the same, because when the chunks change their representations have to be made again.

That cost is what “hardening” actually is. *It hardens not because there is no mechanism to undo it but because the cost of undoing scales with the number of documents*. And an agent’s documents keep growing as its own records pile up, so that cost rises every day.

Is closeness relevance?

Look at that list again. The chunk actually needed was 4173, in third place. The sentence saying the ceiling does not apply to overseas travel. And above it sits a chunk about airfare class. Why that happens is this section's question.

Be precise first about what retrieval actually wants. Given a query, the chunk you want is *not the chunk that resembles the query but the chunk that moves the task forward*. And that judgment needs two things. You have to look at the query and the chunk together, and you have to know what work is going on.

What vector search measures, by contrast, is the distance between two values. One of those two, the chunk's value, was computed before the query arrived, on the day of storing, by the chunk alone. *In a place where two things have to be looked at together, it is measuring the belated meeting of two values that hardened alone.*

So there are places where it comes apart. Three of them.

First, closeness does not see direction. “Turn this option on in production” and “do not turn this option on in production” share almost every word and have exactly the same subject. On the map they are neighbors. But which one loads reverses the agent's behavior. One negation, one word of exception, is a small difference inside a whole sentence's resemblance. And closeness measures a small difference as small. That 4173 got pushed down in the list above is this shape.

Second, sharing a subject is not having the answer. To the query “why do deployments keep failing,” the closest chunks are the chunks that talk about deployments failing. Meeting notes saying it failed again, an incident ticket reporting the failure, a thread that asked the same question and closed with no answer. All of them are close because they share the subject, and none of them has the answer. The chunk that actually holds the cause, say a document that dryly states one configuration value, may be farther away.

Third, a question and its answer do not resemble each other. The answer to “why does the response get slow” is a sentence like “the connection pool defaults to 10.” A question states a lack and an answer states a fact, and the better the answer, the less it repeats the question. Nothing anywhere guarantees the two are neighbors on the map. If anything, what most resembles the query is the same question written somewhere else.

The point. Whether something is related is judged only after the query and the task arrive, and the chunk's half hardens alone before the query. *Closeness is an imitation of relevance, and the places where it comes apart are a property of this structure rather than an immaturity of the technique.*

There is no ground for saying those three are all of the ways it comes apart. The list is probably longer. And you cannot know as a score how often that gap happens in your own system. The number 0.83 was measured by retrieval against its own ruler, so it does not serve as its own report card. To evaluate it you need an evaluation set with a human judgment attached to each query and chunk pair, and that design is Part VI's material. What can be said here is only the direction. *The axis of scoring and the axis of judging are different.*

In a loop one more layer goes on. Where a person asks at a search box, the query comes out of the person's need. In a loop the query is written by the model. So the query differs every time too. And this failure repeats. The same kind of need makes a query in the same phrasing under the same instructions, and if that query catches on the gap, the whole kind catches.

Table 11.4: Three places where closeness comes apart. In all three, retrieval comes back in the shape of a success.

Place	What closeness sees	What is actually needed	What happens after
Alike but opposite	the same words, the same subject	direction. Telling a positive from a negative, a rule from its exception	the opposite remedy loads as ground and the behavior flips
Only the subject matches	that it talks about the same thing as the query	the answer, meaning content that moves the task forward	complaints and open questions fill the top and the chunk with the answer gets pushed out
Question and answer differ	what resembles the question, which is another question	an answer, which does not resemble the question	an answerless question thread beats a document with the answer

A person whose results look strange changes the words and asks again. For that recovery to hold inside a loop, two things are needed. A signal that the current result may not be the best has to ride in the observation, and changing the query and looking again has to be on the list of actions. *In a workflow that throws one query and stops, this gap is a settled failure. In an agent that can change the query, it is a recoverable event.*

What gets used in practice

That is the skeleton of this branch. A running system has one more cost-cutting decision in it, and outside that branch there are arrangements that do not use it at all.

The most honest way to find the closest chunks is to measure them all. Compare the query's value against every chunk's value one at a time and take the closest few. The reason this is accurate and the reason it is expensive are the same. It skips nothing.

So vector search in practice makes a different choice. Organize the values in advance on the day of storing so that when a query arrives you search only the promising neighborhoods rather than everything. There are several ways to organize and the details are outside this boundary, and whatever the way, what it buys and what it sells are the same.

What it buys is speed. The number of comparisons comes down from all to some. What it sells is a guarantee. If the truly closest chunk was in a neighborhood you skipped, you do not find it, and since you did not look where you skipped, you do not know you missed it. "It always finds the closest" comes down to "it usually finds it."

What matters is what shape that surrender takes as a failure. *A miss is quiet.* The list comes back filled even on a query that skipped the closest chunk. It was in the documents and its representation was close to the query, and the last step of finding skipped it, so no amount of polishing the two earlier decisions reaches it. To know you missed you have to compare against measuring everything, and you brought this method in to avoid the cost of measuring everything, so that baseline is not in the daily run. Occasionally measuring everything for a handful of sample queries is the only way to see this failure.

Table 11.5: Three arrangements for fetching knowledge. None of them is the default.

Arrangement	What it buys	What it sells
Load it whole	no failure of not finding. The cache takes the cost	unusable the moment the documents pass the budget
Cut and store	speed and scale. However many documents, the cost per query holds	decisions harden on the day of storing, copies go stale, misses are quiet
Open with a tool	freshness. No copy, so nothing goes stale and no decision hardens	speed. Round trips grow, and the model picks what to open, so a failure to open appears

Arrangements that do not use this branch

The branch followed so far is still the most widely used and it is not the only road, and lately there are places where it is losing ground. Two things overlapped.

One is that contexts got bigger. The last section set up the scale at which a document set that fits in the budget is better loaded whole, without cutting or storing. As that scale rises, the places where you never have to open this branch grow.

The other is an arrangement where the agent opens files directly with a tool. Rather than sweeping and storing everything ahead of time, it carries only light markers like file paths and links and opens with a tool only what it comes to need. The shape of a folder, the naming conventions, and the modification times become the clue to what to open, and what one opening returns settles what to open next (Anthropic, 2025c). The memory tool in Chapter 10 had this shape. What a session received first was not an index but a listing of a folder.

The answer in practice is usually a mix. Fetch ahead of time what has to be there quickly and let the model open the rest at its discretion (Anthropic, 2025c). So do not read this section as “this is the right answer.” *Cutting and carrying and finding the closest is the most widely used branch, and the middle row of the table above is what it costs.*

11.5 When retrieval fails

Retrieval does not always succeed. There are three shapes of failure, and none of the three raises an error.

Empty comes in two kinds

When retrieval returns nothing, there are two possible circumstances. It really is not in the documents, or it is there and the finding went astray.

The trouble is that the two circumstances *arrive as the same observation*. Nowhere in the observation “no results” is there anything that separates them, and the observation is everything to the model, so the model picks one without separating them. It usually picks “not there,” and that conclusion leads to action. The plan changes on the premise that no such policy exists, and sometimes it goes off to build what it concluded was missing.

Table 11.6: Three shapes of retrieval failure. None of them raises an error.

Shape	What is actually so	What the model sees	What happens after
Empty because absent	it really is not in the documents	no results	“not there” is right, and it cannot be told from the row below, so it cannot be used as proof of absence
Empty because missed	it is in the documents and the wording was out of step with the query	the identical no results	the false conclusion “not there” hardens into a premise, and it goes off to build what it believed was missing
Plausible irrelevance	there is no answer, or it was missed, and the top candidates are always filled	ranked chunks. The same shape as a success	the least unrelated thing takes the stand as ground, and the answer is built on it

The more common and worse case is not empty at all. A retriever usually does not come back empty. A request for the top few returns the few least unrelated even when the documents hold no answer. The score is a rank among candidates, not a judgment that “this much counts as related.”

So a failed retrieval arrives in the shape of a success. There are chunks, there is an order, and there are scores attached.

One question stands up here. *Who knows that it was missed?* Erase the three one by one and that place is empty. What a retriever knows goes as far as the ranking of its candidates. Whether its own best is enough for the task is not a question a retriever can answer. What the model sees is the observation, and the observation does not separate absent from missed. The store knows what it holds and does not know what the query needed.

The two pieces of information needed to recognize the failure, what was needed and what exists, are divided between different places in the system.

The failure of being found and not used

The third failure comes after the finding succeeds. Retrieval design usually ends at finding. A premise is quietly laid that once you have found the good thing and put it in, the job is done.

Test that premise with one scene. Chunk 4172 is loaded in the context. The sentence saying the ceiling is five million. And say that in the world the model learned, the common value for this kind of rule was one million.

The model’s answer is one of three. Five million, using the chunk as it stands. One million, using what it learned. And the most vexing third, a blend: five million, except that anything over one million needs separate approval.

The third answer is in no document. The chunk’s number is in it, so even a source marker gets attached plausibly, and a person checking who opens the document finds half of it right and lets it pass. *Half-used knowledge is harder to catch than unused knowledge.*

Why is this possible? *Because a chunk is a condition, not a rule.* Text that went into the context tilts the answer without settling it. This is the same sentence already set up about instructions, and instructions had one exit. Dropping a form that must be kept into a schema.

Facts have no such exit. What would have to be checked in “do not answer in contradiction with this chunk” is not form but meaning, and code cannot check meaning.

The point. Retrieval’s success is the product of two events. Finding and loading the right thing, and the answer using it. You can raise the probability of the first with the quality of retrieval. There is no mechanism that enforces the second. *That a chunk is in the context is no evidence that the answer stood on it.*

There is no generalized answer for which side wins when the new thing in the context and the old thing the model learned collide. That neither side always wins is all that can be said.

One thing is certain instead. Retrieval is a mechanism used for updating, so it is an arrangement that *wants* the knowledge in the context to differ from the knowledge the model learned. You put it in to cover an old belief with a new fact, so a collision between the two is not an exceptional situation but the normal state built into the purpose. And resolving it is left to the model’s discretion with no enforcement at all.

How to reduce them and how to see them

The place to reduce each of the three is different.

Do not leave the query as one. A good share of the misses are the query’s words being out of step with the document’s. Instead of throwing the user’s words in as they are, have the model expand into several queries and the probability that every one of them lands on the gap goes down. The cost is retrieval count and round trips.

Overlap the ways of finding. What finding by meaning misses is usually caught by finding by the words. Error codes and in-house identifiers were that place. How to overlap goes to Chapter 12.

Make a chunk readable on its own. The failure of half loading cannot be fixed at retrieval time. On the day of storing you cut by structure, or overlap, or attach context.

Check in the answer whether it was used. The third failure has no enforcement, so what is left is checking. Have every sentence of the answer carry the address of its ground chunk, and compare that chunk against the answer.

And a last question remains. *Are these failures visible?* What was missed does not land in the log. What lands in the log is what retrieval returned, and what it failed to return has no line.

So making retrieval’s failures into visible events is the harness’s job. Instead of returning a bare empty list, have retrieval report its own circumstances too. How many it swept, what the top score was, and where that score sits against the value this document set gives when it does match well. With those signals in the observation the model gets material to separate “not there” from “missed,” and changing the query and looking again becomes an option. Without the signals, empty hardens into proof of absence.

The threshold stays open. Below what score is it better to load no chunk and report empty? Put a threshold in and plausible irrelevance gets filtered along with barely related chunks, and leave it out and the reverse.

The third failure is one layer harder to see. The answer is wrong, you open the log, and the right chunk is there in the context. Because it is there, retrieval is judged fine. The repair drifts toward loading more, more precisely, and longer. But if the problem is on the writing side, improvements to fetching do not reach this failure, and the more candidates load the cloudier the context gets. *Evaluate whether it was used in the answer, not whether it was in the context.*

11.6 What retrieval cannot solve

If any of the following holds, it is not time to solve it with retrieval. Polishing the chunking and the representation does not reach these either.

1. **The answer is written in no single chunk.** A question whose answer only exists once you join what is scattered. Retrieval fetches what is written and does not join. And chunking does not help this circumstance. It makes it worse, because the finer you cut the more scattered it gets. The mechanism for taking this question by structure is Chapter 13.
2. **You have to count.** How many contracts are there, how many customers catch on this condition? To count you have to see all of them, and retrieval returns the top few. A number counted with something missing is not a count, and above all nothing in the list marks that something was missing.
3. **The model does not know the word and cannot even build the query.** Say a new feature is called “lighthouse” inside the company. A model that does not know what to look for cannot put that word in a query, and a word not used does not catch in retrieval. That holds even when it is in the documents. This failure sits outside the retriever. It resolves only if the organization’s vocabulary is written down somewhere in advance.
4. **It is a method, not a fact.** Procedures, norms, and standards of judgment are not to be fetched as chunks and referred to. They have to be kept as behavior. Hoping retrieval hits a norm that must always be kept is a dangerous bet.
5. **There is nowhere it is written.** An organization’s tacit knowledge, facts that exist only in the structure of the code, decisions nobody documented. Retrieval’s ceiling is the documents’ ceiling. Only what is written gets found, and writing is not something retrieval does for you.
6. **The world changes faster than the index.** An index is a copy as of the moment it was made, so values that change faster than the interval of restoring, the current inventory, the state of a running job, the contents of a file just edited, are not to be found in the index but read on the spot with a tool.

There is a shape running through the six. Retrieval is a mechanism that *fetches what is written*. None of the six is solved by raising retrieval quality. What Chapter 12 raises is the probability of finding what is written, and this list survives even if that probability is 1.

The fifth is a design object rather than a circumstance handed to you from outside, because an agent system’s documents are not only what it was given. The records the system makes itself, the memory of Chapter 10, become the documents of future runs.

If so, “is it written” is not a matter of luck but of today’s discipline about records. Two demands. Is the decision record you leave today written in the words and the shape a tomorrow’s query will find? And *is it written at a size that reads on its own even after being cut?* Writing one decision and its ground in one place, and calling things by name rather than by pronoun, is that discipline. In someone else’s writing you have no choice but to guess where the boundaries are, and in a record you write yourself there is nothing to guess. You can make it on the day you write.

11.7 Summary

In one sentence. Retrieval augmentation is not a mechanism that only widens the range of knowledge. It is a mechanism that gives knowledge back its timestamp and its source. Its force reaches only as far as putting things into the context, so the center of the design lies beyond finding well. You have to ask whether what you put in was used, and you have to make missing visible.

What is now settled.

- A tool makes something happen and knowledge makes something known. The trouble with knowledge inside the model is three things. The timestamp is smeared, you cannot fix one item, and there is no source (Section 11.1).
- Put knowledge in a store and the cost of fixing, the ground, and putting in and out all change, and you sell certainty in exchange (Table 11.1). The three places buy different things, so you cannot choose among them on performance (Table 11.2).
- One retrieval is four steps: build the query, find candidates, pick a few, put them in the context (Definition 11.1). If the documents are small it is better to load them whole, and what retrieval returns is candidates, not an answer.
- There are two reasons to cut. The context budget, and the fact that a representation is the same size whether the chunk is large or small, so a long text's representation smears. The price is half of the right thing loading in perfect shape, and there is no right answer for the size (Table 11.3).
- An embedding is a mapping trained so that different expressions of the same meaning come out close (Definition 11.2). Closeness is a judgment with no explanation. It does not work on strings training rarely saw, so lexical search gets overlapped on it. The day you change the representation is the day you carry every document over again.
- Whether something is related is judged only after the query and the task arrive, and the chunk's representation hardens alone before that. So closeness is an imitation and the gap is a property of the structure. It cannot see direction, and it mismeasures sharing a subject as having the answer (Table 11.4).
- Cutting and storing is the most widely used branch and not the default. Loading whole and opening with a tool stand beside it, and the three sell different things (Table 11.5).
- There are three failures. The two empties arrive as the same observation, irrelevance arrives in the shape of a success, and the third is being found and not used (Table 11.6). Success is the product of finding and using, and nothing enforces the second term, and what was missed does not land in the log.
- There are six things retrieval cannot solve. An answer that must be joined, a question that must be counted, not knowing the word to ask with, a method rather than a fact, knowledge that is not written, and a world faster than the index (Section 11.6).

What is still open.

- Combining two retrievals, reordering candidates, and designing the threshold go to

Chapter 12.

- The mechanism for taking answers that must be joined and questions that must be counted by structure goes to Chapter 13.
- What is lost when the source is not text goes to Chapter 14.
- Evaluating whether what you put in was used, and designing an evaluation set with judgments attached, go to Part VI.
- Defending the route by which a retrieved chunk gets read as an instruction goes to Chapter 22.

Try this today. *Measure* your document set's size in tokens. If it fits in the context whole, *mark* retrieval as a candidate for removal (Section 11.3). Next, open ten places in yesterday's log where retrieval came back, and *mark* one by one whether the loaded chunk was actually used in the answer (Section 11.5). Last, *count* how many times retrieval came back empty. If it cannot be counted, that is today's result.

11.8 Exercises

1. Design an experiment that observes the found-and-not-used failure in your own system. Ask the same question under three conditions, with no retrieval, with the right chunk, and with a deliberately wrong chunk, and comparing the answers lets you feel out the boundary between the context winning and what the model learned winning. Why is the third condition necessary?
2. Design an experiment that observes boundary failures in your own documents. Make a list of questions whose answer spans two paragraphs, like a rule and its exception, and throw them while changing only the overlap width. What do you have to take as the metric for "half of it loaded" to be counted?
3. Design an experiment that confirms the observation that a question and its answer do not resemble each other. Build two queries for the same need. One is the question sentence as it is, the other is an invented sentence from the document where the answer would live. If the two queries give different results, what is that difference evidence of?
4. Measure your documents' size and compare the cost of loading whole, cutting and storing, and opening with a tool against the three rows of Table 11.5. How will you notice the day the documents grow past the boundary?

11.9 On a second reading

- Read Chapter 12, look again at where Section 11.4 asks whether closeness is relevance, and you see which mechanism aims at each of the three gaps. What to check then is the sentence set up here. Everything those mechanisms raise is also the finding term.
- Read Chapter 13, come back to the first two items in Section 11.6, and the answer that must be joined and the question that must be counted acquire a mechanism. That the finer you cut the more scattered it gets turns out to have been the marker of a question that should have stored relations rather than chunks in the first place.

- Open Chapter 10 again and look at the last paragraph of Section 11.6, and you see that the observation that the writing side and the finding side are the same system had already become a design object called memory.
- Read Chapter 22, come back to Section 11.5, and the sentence that putting in is tilting looks again like the language of defense. It means what is written in a document can tilt the model's behavior. In a system where documents come from outside, that tilt is a channel for attack.

11.10 Further reading

Lewis et al. (2020) is the source of the name retrieval-augmented generation. Today's practice barely uses the method this paper proposed, so reading it to learn the method is taking a wrong turn. What to take is the statement of the problem. The place where provenance and updating are named as unsolved, which is to say the statement that this mechanism was built for verifiability and updatability and not only for accuracy, is on the first page.

Two things come from Karpukhin et al. (2020). The structure of carrying query and document over separately and comparing them, and the event of beating the baseline in a place where lexical search was the de facto standard. It is not read to learn the details of the training method.

Four things came from Anthropic (2024b). The scale at which small documents are better loaded whole, the diagnosis that a chunk loses its context the moment it is cut and the remedy for it, the observation that vector search does not work in front of an exact string, and the direction on how many candidates to load. This material reports numbers alongside, and what to take is the direction rather than the numbers.

Anthropic (2025c) is where the arrangement that stores nothing ahead of time and opens with a tool while carrying only markers is written down. The observation that the shape of a folder, the naming conventions, and the modification times become signals, and the price that finding during a run is slower than pulling from something built in advance, are both there.

This chapter's center, that what you put in may not get used, is not new evidence but Chapter 4 read from the other side. Read Shi et al. (2023) with this chapter's eye and the finding that unrelated material pulls the answer is evidence for the property that context tilts without settling. The position-by-position difference Liu et al. (2024) showed comes back as the placement question of where to load a retrieved chunk.

Hybrid Retrieval

Lexical search finds a rare string like an error code exactly, and it does not find a document that said the same thing in different words. Vector search is the opposite. It joins “annual leave” to “vacation,” and it does not work on a string training never saw, like an in-house code name. And neither of them gives back the context that broke when the chunk was cut.

So practice does not finish finding in one pass. It does three more things. Overlap two retrievals so that what one misses the other catches. Narrow cheaply and then choose again with an expensive judgment. And put back, on the day of storing, the context that cutting broke.

All three hit what they aim at. In one published measurement the rate at which retrieval misses the right chunk came down (Anthropic, 2024b). And the same material wrote down the arrangement that produced that number. Run a model once per chunk to write and attach context, build two stores, take 150 candidates per query and stand them up again, and load 20 of them. *What was bought and what was paid are written down side by side.*

This chapter goes in that order. Why overlap, how it is actually done, and what you pay for stacking it that way. It closes with what stacking stages does not fix.

Implementation is not covered. The structure and training of the model that reorders, and the names of particular products, are left out. Questions retrieval cannot answer are put off to Chapter 13.

12.1 Why overlap two retrievals

There is one root. What vector search uses is a learned map, and the learning saw the world’s writing. An error code an internal system invented is a string that was not in that world, so it has no place on the map. Calling a new feature “lighthouse” inside the company has the same root. Training saw that word plenty and saw it in the world’s sense, and the bridge between the code name and the feature is the organization’s convention, not the word’s meaning.

Lexical search is strong at exactly that place. It looks only at whether a word from the query is in the document as it is, so the rarer the word, the better the query. And what lexical search misses is the failure we already know. The document said “annual leave” and the query asks about “vacation” and comes back empty, and vector search was the mechanism brought in to fill exactly that.

They miss different things, so overlapping them misses less. That one line is the whole ground for overlapping.

Both roads to polishing one so it does the other’s job are blocked. For vector search to learn the in-house vocabulary, training would have to see it, and in-house vocabulary gets minted again today. Hanging a thesaurus on lexical search is copying by hand what vector search got from training, and it never ends. So practice overlaps instead of combining.

Table 12.1: The division of labor between the two retrievals. That the bottom two rows cross each other is the ground for overlapping.

	Lexical search	Vector search
What it measures	whether a word from the query is in the document as it is. The rarer the overlapping word, the heavier it counts	whether the query and the chunk sit close on the learned map
What it finds well	error codes, code names, identifiers, policy numbers. The very document that word is written in	the same meaning in different words, reaching “annual leave” from “vacation”
What it misses	a query and a document that call the same thing by different words	vocabulary born outside training. A string with no place on the map

Nail down here what this chapter raises. Chapter 11 set up retrieval’s success as the product of two events. Finding and loading the right thing, and the answer using it.

All three mechanisms in this chapter touch only the finding side. Overlap, reorder, attach context, and what rises is the probability that the right chunk lands in the context. Whether it gets used once it lands stays exactly where it was.

12.2 Two lists into one

Running the two retrievals separately, taking two lists, and merging them is *hybrid retrieval*, and it is today’s recommended arrangement. The published guidance puts it as leveraging both, “balancing precise term matching with broader semantic understanding” (Anthropic, 2024b).

The order of history makes it easy to misread. Lexical held the standard for a long time and vector search took the place later, so it looks as though you should keep only the winner. But the new retrieval filled the old one’s failures while bringing in failures the old one never had. *Overlapping is a division of labor, not an attachment to the past.*

Look at a real one. Ask about the overseas travel spending ceiling.

```
found by meaning
1 Travel spending up to five million won requires no prior...
2 Overseas travel airfare is economy class by default...
3 The clause above does not apply to overseas travel

found by the words
1 The clause above does not apply to overseas travel
2 Changing the overseas travel spending ceiling follows...
3 Travel spending up to five million won requires no prior...
```

What we actually wanted is the exception clause. The sentence saying the ceiling does not apply to overseas travel. It is third on one side and first on the other.

Read why they split. That chunk is one sentence, and what “the clause above” refers to is not inside it. On the side that finds by meaning it got pushed down. On the side that finds by the words, “overseas” and “travel” are both inside a short sentence, so it is first.

Now the two have to merge, and you cannot add the scores. One score is how many words overlapped and the other is how close two points sit on a map. They measure different things, so adding them is adding weight to temperature.

So the widely used method throws the scores away and uses only the ranks. Give a large share to first place and shrink it going down. But *keep the fall gentle*. Which is to say do not put a big gap between first and third.

What follows? A chunk that is first on one list receives a share once. A chunk on both lists receives twice. The fall is gentle, so *receiving twice in halves beats receiving once in full*.

The exception clause above is that case. Third and first, one each, and it comes out on top of the sum. In an arrangement using vector search alone, that chunk was never in the top two, and that was the route by which Chapter 11 produced the half answer of five million.

One decision is left. Do you treat the two lists as equals, or trust one more? Where error codes go back and forth a lot, trusting the lexical side more is better, and where conceptual questions are common it is the reverse. And you do not know that ratio before the questions arrive. *That the default differs from store to store is itself the evidence that no answer is settled*.

Where you make that decision splits too. Fix it once in the system and every question carries the same weighting. Open it as an argument of the retrieval tool and it can be set per question, and the one setting it becomes the model. The model knows what it is asking, which helps, and in exchange that choice differs every time. *Settle it once, or have it settled every time*.

12.3 Reranking: standing the candidates up again

Count what overlapping filled and move on. Error codes and the lighthouse are now picked up by lexical search.

But the query “why do deployments keep failing” is where it was. Meeting notes complaining about the failure are the chunks that overlap the query the most by the words too. Lexical search also sweeps the documents alone before the query arrives, so overlapping does not solve this failure.

The query and the chunk have to be read together. And that does not come out of overlapping any number of retrievals.

So you stand up one more stage. Put the query and the chunk side by side and have a model read them. Ask whether this chunk answers this question.

Read together and things show. That the complaining meeting notes hold no cause. That the configuration document resembles the question not at all and has the answer inside it. That “turn it on” and “do not turn it on” are opposites. None of that was visible to an eye that scored everything separately.

There is only one reason this method cannot be the default, and it is Khattab and Zaharia (2020)’s starting point. Ranking models built on deep language models are remarkably effective and they “increase computational cost by orders of magnitude over prior approaches,” because they “must feed each query-document pair through a massive neural network to compute a single relevance score.” Point that at the whole document set and one question costs in proportion to the number of documents.

So practice does not throw the judgment away. It *stands it last*. Cheap retrieval sweeps the whole document set and narrows the candidates, and this judgment looks again only at those candidates. The name attached to this stage is *reranking*.

Look at one published arrangement and there are three numbers. Take the 150 candidates the two retrievals mixed together, read all 150 again, and load 20 of them into the context. The same material compared 5, 10, and 20 and wrote that “using 20” was “the most performant” (Anthropic, 2024b).

Setting the number you read again far wider than the number you load is the point. Taking in wide is the only chance to bring back what the earlier stage missed, and narrowing is what this stage does anyway.

See what changes in the earlier example. The exception clause was already near the top, so it does not move much. What drops sharply instead is “changing the overseas travel spending ceiling follows this procedure.” Several of its words overlap the query and it says nothing at all about what the ceiling is. What was invisible to an eye that only counts overlap is visible to an eye that reads together.

When do you compute it

Get this far and one axis shows. *When you do the computing on the chunk side.*

Vector search does all of it ahead. Turning a chunk into numbers finishes on the day of storing, and when a question arrives you only compare those numbers against the question’s. So one question costs little to compute. In exchange, when the chunk was carried over the question was unknown, so it has no power to read together.

Reranking does nothing ahead. It reads a chunk from scratch after the question arrives. Reading together is accurate, and it runs a model once per candidate per question.

There is a middle point between them. The method Khattab and Zaharia (2020) introduced. Carrying chunks over ahead is the same as vector search, and *it does not collapse them into one number. It leaves them fine-grained.* Then after the question arrives it compares those fine-grained things with a cheap computation. It is doing ahead what can be done ahead without smearing.

This is where Chapter 11’s diagnosis turns into a remedy. The diagnosis that closeness imitates relevance was pointing at the fact that, in a place where things have to be read together, the chunk’s side gets computed before it sees the question. So the remedy stands on that moment too. Vector search does all of it ahead, reranking does none of it ahead, and a middle point sits between.

But the less you do ahead, the finer the imitation gets without becoming relevance itself. The same chunk answers or does not depending on what work is going on, and that circumstance rides in no stage that reads only the question and the chunk.

What was lost earlier does not come back

Count again. The deployment failure query is alive now. The complaint and the cause come apart in front of an eye that reads together.

The exception clause is precarious, though. It rose up the list because lexical search rescued it. Had it not risen, the story is different.

A one-sentence chunk that lost the owner of “the clause above” has no word to join it to the query even when laid beside it. It looks unrelated. And pulling down what looks unrelated is this stage’s job. *The more accurate the judgment, the more decisively half of it gets pushed away.*

Reranking only reads well what is put in front of it. What was lost before it got put in front cannot be read.

12.4 Contextual retrieval: attach the context while cutting

So where do you rescue the half? What broke the context was not the retrieving but the cutting. Putting it back has to happen at the cutting too.

Writing a line onto each chunk saying the context that chunk stood in, and storing it that way, is *contextual retrieval* (Anthropic, 2024b). The exception clause above gets this line.

```
context added This chunk is the exception clause to the travel
               spending ceiling.
body          The clause above does not apply to overseas travel.
```

Without knowing what “the clause above” refers to, this chunk now reaches the question “over-seas travel spending ceiling.”

What to see here is how that line meshes with overlapping. *The added context goes into both stores.* On the side that finds by meaning, travel and spending and ceiling enter the chunk’s place, and the question now has somewhere to land. On the side that finds by the words, words that were not in the body catch in the overlap. *One piece of work fixes two retrievals at once.*

That this chunk stood even at third on the list found by meaning in the earlier example is also thanks to that context. Without it the chunk would not have been on the list at all.

Why at the cutting

For a chunk to be found, the context it lost has to be *inside the representation used for finding*. And that representation is made once when the chunk is stored and does not change after.

If the road were restoring after finding, another mechanism already exists. Opening the original by the chunk’s address and reading it. The two mechanisms divide the labor. Attaching context makes a chunk findable, and opening the original makes a found chunk fully readable.

Mind the order. *Opening the original exists only after finding has succeeded.* For a chunk that slipped at the finding, that road never opens.

What it takes and how much

You cannot write the context looking only at the chunk. Where that chunk stood in the document and what it was propping up is known only with the whole document in view. So the whole document goes in as input once per chunk. For a document with dozens of chunks the same document goes in dozens of times, so this arrangement cost a great deal for a long while. Caching, which cuts that cost on inputs whose leading run repeats, brought it into a usable range.

And this call is a mechanical summary that runs as many times as there are documents. *A small model belongs here.* It is a job set by throughput rather than by the quality of judgment.

A new risk comes with it. The hand writing the context is a model, and a model answers differently every time. *A chunk with the wrong context attached is a chunk that catches in retrieval on words that are nowhere in the source text.* And that mistake arrives in the shape of a success. In the retrieval log it is indistinguishable from any other hit.

There is no right answer for what makes a good context either. The context needs to be as much as it takes for the questions that will arrive to reach the chunk. And what will be needed is known only once a question comes, and storing finishes before that. The same problem that had no right answer for chunk size repeats in the content of the context.

Table 12.2: What you buy and what you pay when you lay one stage on. No stage brings only what it aimed at.

The stage laid on	What it buys	What it costs	The new decision
Stand up a lexical search beside it	queries asking with vocabulary born outside training	one store and its upkeep. They run side by side, so almost no latency	which way to lean when merging the two lists
Stand the received candidates up again	a judgment that reads the query and the chunk together	model calls and waiting in proportion to the candidates. On every step that retrieves	how many to receive and how many of those to load
Attach context to chunks on the day of storing	the context that broke when it was cut	model calls in proportion to the documents. Storage growing per chunk	what to write and how much. When to write it again

12.5 What it means for the stages to grow

Lay all three on and the road one question travels looks like this.

Three things already happen when a document is stored. Cut it, run a model per chunk to write and attach context, and store it two ways. When a question arrives, two retrievals each sweep and two lists merge into one. A stage that filters candidates by timestamp or source sometimes slots in. Then reranking changes the order, and on top of that a few get picked to load. *What was one sweep has become six or seven stages.*

Four things grow.

Latency. The two retrievals can run side by side, so the waiting ends in one. Reranking is different. It starts only after it receives what the earlier stage passed, so the order is forced, and that wait becomes latency as it is. And this wait is not once per question. At a search box a person asks once and waits once. In a loop it repeats on every step that retrieves.

Tokens. Reranking is a model call, and the more candidates the more calls. Attaching context is a model call too, and that one runs as many times as there are documents. But the moment of paying differs. Reranking is paid every time a question arrives, and attaching context is paid once when storing. *One goes out daily and one went out at the beginning.* Different kinds, so count them separately.

Storage. Finding two ways means building two copies. Attach context on top and a few dozen tokens ride in front of every chunk, so both grow. And the two go stale together. Chapter 11's bill, that changing how you carry things over means carrying every document again, is now doubled.

Places to fix. Every stage brings a value to set. Which way to lean when merging the two lists, how many candidates to pass on, how many of those to load, how much context to write. Each gets set separately and they are locked together. Cut the candidates you pass and you cut the chance of reviving a chunk the merge pushed down. Tighten the filter and retrieval has to sweep wider.

One more thing grows that differs in kind from those four. *Places to fail.*

Count the possible circumstances when the answer is wrong. Both retrievals may have missed. One may have found it and the merging stage pushed it down. It may have survived that far and the filtering stage misread a marker. Reranking may have pulled the right candidate

down. A context written wrong at storing time may have pushed a stray chunk up.

In one sweep the only places to suspect were Chapter 11's three gaps. Now one more is added per stage.

And these places are not equals. *Each stage works only inside what the earlier stage passed.* Reranking changes the order of the list it received and cannot summon anything outside that list.

Point at the extreme. Imagine a reranking that never makes a mistake. If the earlier stage did not pass the answer, what it does is put answerless candidates in a perfect order. *What was lost earlier is not recovered later.*

Above all, a miss between stages is quiet. What the earlier stage did not pass is written nowhere on the receiving side, and the list in front of reranking is always filled to the number requested.

So which stage to suspect comes back as a problem of records. Chapter 11 said to have retrieval report its own circumstances too. How many it swept and what the top score was, arriving together, makes missing a visible event.

With several stages, if that report covers only the one final list, the circumstances just counted collapse back into one sentence when the answer is wrong: "retrieval is acting up." What separates the branches is the record of the intermediate results. The per-retrieval lists, the merged list, what got filtered, and the ranks before and after reranking have to survive for a failure to be traced back to a stage.

That record earns its keep because the failures repeat. A miss at an earlier stage comes by kind of question. The same kind of question drops out at the same place for the same reason, and the same wrong answer repeats.

You can predict where people go to fix things when there is no record. *They fix the last stage.* Reranking stands at the end, so changing it moves the ranking immediately and it swaps out easily, so the hand always goes there first. And if the problem is earlier, that repair does nothing. And the earlier stage's failure leaves no trace in the later stage's log, so it never even rises as a candidate to fix.

12.6 What stacking stages does not fix

That is the business inside the stages. If any of the following holds, it is not time to stack stages.

1. **The failure is on the writing side.** If the right chunk loaded and the answer did not use it, none of this chapter's mechanisms has anything to touch. Make the finding side perfect and the whole still cannot exceed the writing side. Laying on stages here adds cost and latency instead of quality.
2. **The answer is not written in the documents.** What is simply not there, and what disappeared in the processing before storing, appear in front of no stage. Even contextual retrieval attaches a side note to a chunk that exists. It does not invent content that is missing. Questions whose answers must be counted or joined stay exactly where they were, and they are Chapter 13's material.
3. **What decides relevance sits outside the text.** However expensive the judgment, what it reads is the text of the query and the chunk. When two documents hold the same content and the fact that one is the stale one is not written in the body, no retrieval and no judgment sees it. Half the remedy is inside the stages. Filtering by markers of time and

source is the cheapest stage there is, as long as the markers are written. The other half is outside the stages. If the markers are not attached at all, no stage makes them for you.

4. **The frequency of the failure falls below the stage's cost.** A stage is a standing cost. Every query passes through it and pays, and the failures it catches are in some queries only. Laying a stage on a system whose first sweep already matches well is taxing every query to buy a rare failure. A team that lays on stages without ever having evaluated its own retrieval's failure rate has the order backwards.

What runs through the four is the sentence from the opening. This chapter's mechanisms belong to the finding side. What is found and not used, what is not written, and what sits outside the text are outside that, so no number of stages reaches them, and as the last item shows, even inside, a stage is not free.

There is no right answer for how many stages either. How often one stage cuts a failure and what it adds depend on the queries and the documents.

Last, count one stage that does not draw well on a diagram. The model itself, making the answer on top of the candidates that got picked and loaded. It reads the candidates in the context and picks what to use, so *whatever the designer stood up, the last one choosing is always the model*.

This stage is special in three ways. Its cost is the whole model call for the step, so it is the most expensive judgment. The task is loaded in its context, so it is the only judge that knows what work is going on. And this stage's failure is the failure on the writing side.

12.7 Summary

In one sentence. Finding by meaning and finding by the words miss different things, so overlapping them misses less. Lay on a stage that reads the query and the chunk together and stands them up again, and a stage that puts back on the day of storing the context that cutting broke, and finding becomes several stages. All three hit what they aim at and all three move only the finding side, and as the stages grow so do the cost, the latency, and the places to fail.

What is now settled.

- Lexical search and vector search miss in places that cross each other (Table 12.1). That crossing is the whole ground for overlapping, and both roads to polishing one so it does the other's job are blocked. And all three of this chapter's mechanisms touch only the finding side. Whether it gets used once it lands stays exactly where it was.
- The two scores measure different things and cannot be added. So the widely used method throws the scores away and uses only the ranks, keeping the fall gentle. Then receiving twice in halves on two lists beats receiving once in full on one. There is no right answer for treating the two lists as equals, and whether to settle that once or have the model settle it per question is the decision that is left.
- A judgment that reads the query and the chunk together does not come out of overlapping any number of retrievals. That judgment costs in proportion to the number of documents, so it cannot be pointed at the whole set. You narrow cheaply and stand it last. Set the number you read again far wider than the number you load, because taking in wide is the only chance to revive what the earlier stage missed.
- The axis dividing the three mechanisms is when the computing on the chunk side happens. Vector search does it all at storing, reranking does it all after the question arrives, and between them is a middle point that does it ahead without smearing. But the less you do ahead, the finer the imitation gets without becoming relevance itself. What work is going on rides in no stage that reads only the question and the chunk.
- What broke the context was the cutting, not the retrieving, so putting it back has to happen at the cutting. *The added context goes into both stores, so one piece of work fixes two retrievals at once.* In exchange the hand writing that line is a model, so a wrong context lies down inside what was stored, and that mistake arrives in the shape of a success.
- Lay one stage on and latency and tokens and storage and places to fix all grow (Table 12.2), and one more place to fail appears. Each stage works only inside what the earlier one passed, so even a reranking that never errs cannot stand up an answer lost earlier. That miss is written nowhere, so with no intermediate records the repair tilts toward the last visible stage.
- Four things stacking stages does not fix. A failure on the writing side, an answer that is not written, a judgment outside the text, and a standing cost the failure frequency falls below. And the last one choosing on top of the loaded candidates is always the

model itself, and that stage's failure is the failure on the writing side.

What is still open.

- The mechanism for taking by structure the questions whose answers must be counted or joined goes to Chapter 13.
- There is no ground here for which way to lean the two lists, how many candidates to pass, how many of those to load, or how much context to write. Setting those values takes an evaluation set with judgments attached and per-stage metrics, and that design goes to Part VI.

Try this today. Take ten runs whose answers were wrong and *separate* whether the right chunk was in the context. The ones where it was cannot be fixed by any mechanism in this chapter, so striking them off the list comes first (Section 12.6). In what is left, open the queries and *count* how many carried an identifier or a code name or a policy number and still missed. That number is the share overlapping two retrievals would revive, and if it is near zero, overlapping is not what to buy now (Section 12.1). Last, *open* your retrieval log and see what survives besides the final list. Without the per-retrieval lists and the ranks before and after reranking, from the day after you lay a stage on you will not know which stage to suspect (Section 12.5).

12.8 Exercises

1. Design an experiment that observes the two retrievals crossing each other on your own documents. Build one bundle of queries asking about identifiers and code names and another bundle asking about the same thing in different words, and throw them at three arrangements: lexical only, vector only, and overlapped. When you lean the merge to one side, which bundle goes bad first? Write down why not knowing the ratio of the two bundles in advance makes this decision hard.
2. Write down a procedure for drawing the curve of final quality against cost as you raise the number of candidates passed to reranking. If quality flattens from some point, what is that a signal of? Predict how this curve looks different in a system dominated by earlier-stage misses and one dominated by reranking's misjudgments, and explain the prediction with "what was lost earlier is not recovered later."
3. Design a procedure that attributes ten wrong answers to stages. What is the minimum record needed to separate "an earlier stage missed it" from "reranking pulled it down"? Which branches cannot be told apart in a log that keeps only the final list? Look in your team's repair history for the trace of repairs piling onto the last stage.
4. Design an experiment that plants a few chunks whose contextual line summarizes the document wrongly, and observes which queries fetch those chunks wrongly. Write down why this failure is indistinguishable from any other hit in the retrieval log, and argue whether there is any window onto this failure other than comparing against the original.

12.9 On a second reading

- Read Chapter 13, come back to the second condition in Section 12.6, and the question handed over as being outside the stages acquires a mechanism called structure. If everything here was an improvement to fetching, that chapter's mechanism is sending the question to a different store, and that contrast stands there.
- Come back to “when do you compute it” in Section 12.3 and set it beside Chapter 11's diagnosis, and you see that the three methods are three positions on one question before they are different techniques. Do the computing on the chunk side all at storing, all after the question arrives, or ahead without smearing?
- Read Chapter 11, come back to the costs here, and the cost paid per question and the cost paid once at storing sit as terms of a placement decision. That how far to stack the stages is part of the larger trade of where to keep knowledge becomes visible there.
- Read Part VI, come back to stage attribution in Section 12.5, and the intermediate records have become material for evaluation. That the open questions of which way to lean, how many to pass, and whether to lay on another stage all come out of one absence, an evaluation set with judgments attached, gets precise there.

12.10 Further reading

What to confirm in Khattab and Zaharia (2020) is not the calculation but the axis. That a middle point can stand between computing the chunk side ahead at storing and reading it from scratch after the question arrives, and that doing it ahead without collapsing into one number is that point's exact name.

What to confirm in Anthropic (2024b) is the statement of the problem and the joining of the arrangement. The direction of taking the problem that a chunk loses its context on the day of storing, and the fact that the proposal stacks two retrievals and reranking on top of attaching context. The numbers reported belong to those documents and those queries, so what to take is not the numbers but the arrangement, and that arrangement stands in the same order as this chapter's three sections.

Ontologies and Knowledge Graphs

“Who owns the payments rework?” comes out if you dig through the documents. In a meeting note or a handover document, the line with the answer is somewhere. But “how many projects does Kim own?” does not come out.

Each project is faithfully written in its own document. Only the fact of how many is in no document. That number comes into being only when you gather the documents and count. To count you have to see all of them, and retrieval gives you the top few.

The remedy is not a way of finding but a change of storing. Put Kim as one node, put a project as one node, and write the fact of owning as one edge joining the two. Then you can count. Count the “owns” edges leaving Kim. And you can connect. Ride the edges twice.

In exchange you have to settle in advance what can be a node and what edges can be laid. It is the same kind of work Chapter 9 did when it settled the shape of output in advance, and the same trade is on it.

13.1 Problems retrieval does not solve

Policy documents and meeting notes and deployment records and incident reports have piled up. Lexical search and vector search are overlapped, candidates get read again with the query and stand up in a new order, and each chunk got its surrounding context attached when it was cut. It is a system that has taken everything the success rate of retrieval can bring. Two questions arrive side by side.

The first is “who owns the payments rework?” This one resolves. The answer is inside one sentence, and fetching the chunk that holds that sentence ends it.

The second is “how many projects does Kim own?” This one does not resolve. The answer is one number, and the sentence holding that number is not in the store.

To count you have to see all of them, and retrieval gives you the top few. If Kim actually owns seven projects and you took five candidates, however faithfully you count what you received, the answer is five. Why not raise the candidates? The problem is knowing what to raise it to.

Even taking all of them does not finish the count. The unit of counting is the project and the unit of storing is the chunk. One project is written three times across a plan, a meeting note, and a weekly report, and two projects are written together in one document. To count, a judgment folding the same project into one has to stand, and retrieval does not make that judgment.

Beside the counting question sits the connecting question. “Which team changed the setting that caused the approval rejection?” This time the answer is written. Only it is written split across three documents. The incident report points at the setting that was the cause, the deployment record points at the work that changed that setting, and the meeting note points at the team that took that work. Every document has one span of the bridge and the whole bridge is nowhere.

An agent has the means to fill that gap. Find, read, and write the next query out of what it read. But every step costs. One more step, and the history grows by that much.

And if it fails, this failure is quiet. Open the log of the run whose answer was wrong and retrieval returned related chunks and the model faithfully counted what it received. Nobody did anything wrong, so the loop carries on.

13.2 Knowledge as a graph

The solution is knowledge joined to knowledge. So where do you keep a fact that sits between one sentence and another? There are only two roads. Leave it inside the sentence, or take it out of the sentence and write it down separately.

Leave it inside and that fact has to be read again every time you ask. To pull out who owns what from “this quarter’s payments rework will be taken by Kim,” a hand has to read that sentence, and that hand is the model. Read the same sentence twice and the model can read it two ways. This time as “owns,” next time as “took on.” If you are trying to count seven and the seven come out under different words, you cannot count.

Take it out of the sentence and that reading finishes once. *Read it once and write it down and asking becomes a lookup rather than a reading.* Lay it out so you can see what goes in and out. Here are the sentences from three documents,

```
[meeting note]
  This quarter's payments rework will be taken by Kim.
  The settlement migration goes to the same person.

[deployment record]
  The payments rework changed the approval limit setting.
  The settlement team did the work.

[incident report]
  Payment approval was rejected because the limit setting
  had been changed to 0.
```

and here is the same thing taken out and written down.

```
nodes
  Kim (person), payments rework (project),
  settlement migration (project), settlement team (team),
  limit setting (setting), approval rejection (incident)

edges
  Kim --[owns]--> payments rework
  Kim --[owns]--> settlement migration
  settlement team --[performs]--> payments rework
  payments rework --[changes]--> limit setting
  limit setting --[causes]--> approval rejection
```

Five lines of sentences gave six nodes and five edges. A node takes what can be pointed at by name, and an edge takes a fact that holds between two nodes. A store that takes nodes and edges as its unit is called a knowledge graph.

Definition 13.1 (Knowledge graph). A *knowledge graph* is a store that takes entities and relations as its unit of storage instead of chunks of text. An entity is something that can be pointed at by name, and a relation is a fact that holds between two entities.

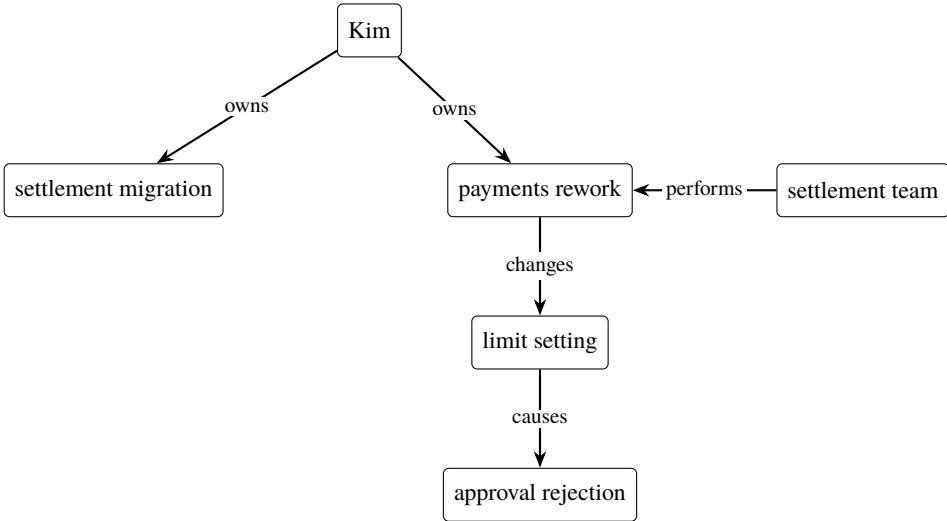


Figure 13.1: Six nodes and five edges pulled out of the sentences of three documents. Asking how many projects Kim owns becomes counting the edges of one name that leave the top node, and asking which team changed the setting that caused the approval rejection becomes riding edges backwards three times from the bottom.

There is no cut at the top few, and you do not have to guess how far all of it goes, because the edges leaving are all of it. What changes about following a connection is that there is no place to miss at any step. It is a lookup rather than a retrieval, so that holds whether the steps are three or five.

“Does this contract have a penalty clause?” changes shape here too. Ask it by retrieval and the answer comes back empty, and that empty does not separate really not there from there and not found. Put the contract as a node and the clause as an edge and the answer becomes looking at whether a penalty edge is attached to that node. *But that “no” stands on a promise that nothing was missed in the extraction.* Where that promise breaks comes back in Section 13.5.

There is something you lose in exchange. When “five million as a rule, with an exception where prior approval exists” gets pressed into one edge saying “limit five million,” the reservation and the condition disappear in the shape of a success. *Nodes and edges are storage that threw away the prose and kept only the facts.*

The last row of the table settles this mechanism’s place. Nodes and edges are not better retrieval. They are sending to a different store the class of questions that were arriving in front of retrieval and were never retrieval’s to begin with. So practice does not choose one of the two. It keeps both, and one new decision appears, deciding per question where to send it. Harden it into a harness rule and questions on the boundary go the wrong way. Open it to the model as a choice of tool and the choosing becomes a draw. And the cost of sending wrongly is not symmetric. A question asking for a written passage that goes to the nodes and edges comes back empty, and empty is a signal. *A counting question that goes to retrieval comes back with an uncounted number in the shape of a success.*

Table 13.1: Storing as chunks against storing as nodes and edges. Neither is an improved version of the other.

	Stored as chunks	Stored as nodes and edges
Unit of storage	a chunk of prose. You only settle where to cut	nodes and edges. Per item you have to judge what is the same node
Adding a document	cut it and put it in, done. Documents are independent of each other	every time, match whether a new node is one of the existing ones
The shape of an answer	a list of candidates, with ranks and scores	an assertion. It is two, it is connected, it is not there
Questions it does well	give me the passage written about this	how many, what is it connected to, is it there or not
Questions it does not	counting, connecting	wording, context, reservation. What prose held as prose

13.3 Designing the ontology

The nodes and edges in the last section are what things look like after the extraction. Something has to be settled before it. To write the first line, “Kim” being a person, “payments rework” being a project, and there being a possible edge called “owns” between a person and a project all have to be standing first.

What you settle is what can be a node, what edges can exist, and which edges can be laid only between which nodes.

The last of those is stronger than the first two. Settle that “owns” can go only from a person to a project and there is no road by which something that is not a project gets in when you count Kim’s projects. Leave it unsettled and anything can sit at either end of “owns,” and counting goes cloudy again.

Write out what was lying under those six nodes and five edges and it looks like this.

```
kinds of node, and what each node carries
  person (name, department), team (name),
  project (name, start, end), setting (name),
  incident (when it happened)

kinds of edge, and where each edge may be laid
  owns:      from a person to a project
  performs:  from a team to a project
  changes:   from a project to a setting
  causes:   from a setting to an incident
```

A list settled in advance like this is an ontology.

This is what Chapter 9 did for output. Write down in advance what fields there are and what can go into each, and whatever the model produced, at least its form was kept. Here you do the same thing for storage. Write down in advance what nodes there are and what edges may be laid, and whatever the extracting hand pulls out, at least its form is kept.

The same trade comes along. *The tighter, the easier to count, and the tighter, the more that cannot be carried.* Nail both ends of “owns” to person and project and counting gets exact. Then a project owned by an outside contractor appears and there is nowhere to write it. Leave

it open and there is somewhere to write it and counting goes cloudy again. What in Chapter 9 was the fork between nailing values to a few and leaving a string open becomes, here, how far to narrow the two ends of an edge.

Whether a person or a model settles the list is the decision after that. Show the corpus to a model and have it pull out the list and it looks as though the list arises by itself. Write down exactly what happens then. *Have a model pull it out and it pulls out differently every time.* In this document “owns,” in that one “took on,” in another “is responsible for.” A person reading sees three ways of saying one thing, and on the counting side there are three kinds. Count Kim’s projects and you count only the “owns” edges and get one.

The same ambiguity shows up in the nodes. The incident report has “Minsu Kim,” the meeting note has “Minsu,” and the deployment record has an English account name. One node or three? That judgment is not a matter of words. You can make it only if you know there is only one Minsu Kim in that organization and that the account belongs to that person.

There are two directions of error and three shapes of failure. Split one into three and one person’s work scatters across three nodes and the count comes out low. Merge three into one when one of them is a different person of the same name, and someone else’s records attach to one node and you follow the wrong connection. And a name that landed in no group disappears quietly. The first two show up as wrong answers. The last has no answer at all.

There is a failure in the opposite direction too. The code name “lighthouse” from Chapter 12 was last year the name of a search feature, and this year another team used the same name for a new plan. By the name alone you cannot separate them. The practical remedy is not to pull out the name alone. *Write a one-line description grounded in that document alongside each node, and later, when you judge whether to merge or split, that description is the material.* The names are the same and the descriptions differ.

With a few dozen documents you can lay out every name and judge in one pass. With ten thousand names you cannot, because you cannot show them all at once. So you group them roughly first. Split into bundles of fifty to a hundred by similar spelling or nearby meaning, and judge merging or splitting only inside a bundle. *Then two that fell into different bundles will never be merged.* The rule for grouping roughly settles in advance the ceiling on what can become the same node.

So the arrangement in practice stands like this. A person settles the list of kinds, and inside that list a model pulls out what falls where. The division from Chapter 9, where a person settles the fields and the model fills them, repeats here. A model making candidates and a person judging is a different story from handing the judging itself to the model.

The point. A system that stores chunks stands without ever judging what is the same thing. A system that stores nodes and edges cannot write its first line without that judgment. Settling what becomes a node and what edges may be laid is settling what is the same and what is different in that organization. *The ceiling on what you can ask is set not by storage capacity but by the understanding of whoever settled that list.*

The standard for judging a list right is not in this book. A list is good according to the questions that will arrive, and the list is settled before the questions arrive. It is a problem of the same order as chunk size, where Chapter 11 asks where to cut and finds no general answer, except that what hangs on it is not one number but the whole list.

Table 13.2: What it takes to stand nodes and edges up and keep them. The first and last rows are where a person’s effort goes.

The work	The hand	When it comes again
Settle what can be a node and what edges may be laid	a person	every time a kind is added
Read documents and pull out nodes and edges	a model	once per document
Match a newly pulled node against the existing ones	a model and a person	every time a document arrives
Add one kind of edge and reread the past documents	a model and a person	every time the list is edited

13.4 When to build one

If settling the list is a person’s work, when do you bring that work in? Two things to judge first.

Do counting or connecting questions actually arrive? What this mechanism decides is one answer to that question and nothing else. Stand nodes and edges up in a system where that question does not come and you tax every document and decide nothing. Only a team that has counted how many of the last hundred queries read as “how many” and “what caused what” holds the material for this decision.

Are the relations stable? If reorganizations come twice a year, ownership takes a touch-up twice a year. If they come every quarter, with dual roles and outside contractors mixed in, fixing ownership grows bigger than adding documents. A document can go stale and be left alone, and nodes and edges that go stale put out a wrong answer as an assertion. And nothing tells you it is time to fix. When a new document arrives the index knows it, and the fact that an owner changed is commonly written in no document at all.

If both hold, what comes next is what it takes.

Adding a new document is where this differs most from a chunk index.

Store as chunks and adding a new document is cutting it and putting it in. You never have to look at documents already in. Store as nodes and edges and it is different. Every time, you have to confirm whether the “Minsu Kim” in the new document is an existing node or a different person. *The more documents, the more there is to confirm.*

Adding one kind is billed late. Adding “co-owns” to the list is a few lines. What is heavy is what comes after. Documents already in have none of that edge, so the graph is complete only after you reread all of them.

The answer comes as an assertion

Before the cost there is something to look at. The shape of the answer changes.

A retrieval answer carries ranks and scores. *That shape is itself a mark of uncertainty.* A graph answer is not like that. It comes as “two.” It comes as “none.”

There is a place where that certainty becomes a danger. One edge missed at extraction turns “two” quietly into “one.” Retrieval’s empty arrives carrying room for doubt, and this answer arrives without it.

So what do you look at? Besides comparing a sample of what was extracted against the source text, there is one thing you can count. *Count the nodes with no edges attached at all.* A node whose name was pulled out and whose relations were not is usually a place where only half was pulled. A sudden rise in that number is a signal that newly arrived documents carry a kind of fact the list does not have.

You do not have to build all of it

So this is not a choice between building all of it and building none of it.

Attaching a date and an author and a department to a document is already a structure of a few kinds. If the only questions arriving count ownership, put in the one “owns” edge and leave the rest to retrieval. *A graph is not a mechanism you bring in whole. It is a mechanism you buy one question at a time.*

13.5 What graphs and ontologies do not solve

If any of the following holds, it is not time to stand nodes and edges up.

1. **What is outside what you settled does not get in.** Meet a kind of fact that is not on the list and the extracting hand drops it. A dropped claim gets counted nowhere, so the list going stale is not observed. Unless you settle at the same time where to gather claims that have no place, the staleness runs quietly.
2. **The extraction can be wrong.** It is work that requires reading meaning, so the hand is a model, and models are wrong. An edge laid wrongly is perfectly formed as a node and an edge, so it reads more assertively than any failure in retrieval. And there is no metric that lights this failure up. Comparing what was extracted against the source text comes down in the end to a person laying a sample side by side, and there is no general answer for how often to do that comparison either.
3. **If a relation is not written in the documents there is nothing to pull out.** In an organization where who owns what is written in no document, there are no edges to extract. What you have to build then is not nodes and edges but the place where that fact gets written. The same holds when the source of the fact is already sitting as rows in another system. If project ownership is in the HR system and setting changes are in the deployment record, the remedy is a tool that goes to that system, and the counting goes to code rather than to a synthesis inside the context (Section 7.4). *Counting a hundred is not loading a hundred.*
4. **The question ends up demanding the source text.** Nodes and edges are storage that threw the prose away. On a task where the reading of a clause, the context of a decision, or the grain of the wording is the material of the answer, what nodes and edges can help with goes as far as finding which document to read. What gets read has to be the source.

What runs through the four is this chapter’s first contrast. Nodes and edges are not better retrieval but a mechanism that takes a different question, so whether that different question actually arrives is the whole of the verdict. And there is something to write down honestly. *In most systems this verdict leans toward not building.* Counting questions and connecting questions are real and are usually a small share of all queries, and a good number of them already have a structured source elsewhere, and corpora stay at a size where reading all of it is

possible for longer than you would think. A team that reads this chapter and starts by settling a list has the order backwards.

So what do you do on the not-building side? The answer is the loop. If the corpus is readable whole, read it and count in code. If not, have it walk the steps of finding and reading and finding again a few times. *Nodes and edges pay those steps in advance when the index is built, and the loop pays them every time you ask.* If questions are frequent and the corpus is large the first costs less. If they are rare and it is small the second does.

13.6 Summary

In one sentence. “How many” and “what caused what” have their answers not inside one sentence but between sentences, so no amount of good retrieval answers them. Change the storage to nodes and edges and such questions become counting and following a connection, with a person’s work of settling in advance what can be a node and what edges may be laid standing in front of it.

What is now settled.

- Where a counting question catches on retrieval is not retrieval’s quality. However many of the top you take there is no way to know what to raise it to, and even taking all of them the count does not finish, because the unit of counting is the project and the unit of storing is the chunk. A connecting question’s answer is split across three documents, so each holds one span of the bridge (Section 13.1). And the failure is quiet. Retrieval returned related chunks and the model faithfully counted what it received, so the loop carries on.
- Keep a relation inside a sentence and a model has to read it again every time you ask, and a reading can turn “owns” into “took on.” Take it out and write it down and that reading finishes once, and asking becomes a lookup (Definition 13.1, Figure 13.1). In exchange the reservations and conditions leave the storage (Table 13.1).
- What to settle in advance is what can be a node, what edges can exist, and which edges may be laid between which nodes. The last is the strongest. It is the same kind of thing as Chapter 9’s schema and carries the same trade. The tighter, the easier to count, and the more that cannot be carried (Section 13.3).
- Hand that list to a model whole and it pulls out differently every time. One fact becomes three kinds of edge and one person splits into three nodes. A one-line description alongside each node is the material for later judging whether to merge or split, and with ten thousand names the rule for grouping settles in advance the ceiling on what can become the same node.
- It takes four places (Table 13.2). Adding a document becomes confirming rather than adding, and adding one kind means rereading every past document. And the answer comes as an assertion. One missed edge turns “two” quietly into “one,” so count the nodes with no edges attached (Section 13.4).
- Four things it does not solve. Facts outside the list, wrong extraction, relations never written, and questions demanding the source text (Section 13.5). This is not a mechanism you bring in whole but one you buy a question at a time.

What is still open.

- Whether the place deciding between retrieval and the graph sits in the harness or is opened to the model, and the design that makes wrong sending and wrong extraction observable, go to Part VI.
- Which side wins when a retrieved chunk and the graph’s answer collide in one context is exactly the open collision Chapter 11 set up.
- What can be pulled out of tables and figures when the source is not text, and what disappears at the first step, goes to Chapter 14.

Try this today. Open the last hundred entries in the query log and *count* how many read as “how many” and “what caused what.” If that number is near zero, this chapter’s mechanism is not what to bring in now (Section 13.4). Open the ones that caught, one at a time, and *separate* whether the answer already sits as rows in another system. If it does, what to build is not a graph but a tool that goes to that system (Section 13.5). In what is left, *write down* by hand, from ten documents, how many names the same person and the same project are being called by. The longer that list, the sooner a person has to stand at the confirming rather than at the extracting (Section 13.3).

13.7 Exercises

1. Design a procedure for sorting counting questions and connecting questions out of your own query log. Queries that read as either finding or counting will certainly appear. What is that ambiguity a signal of? Count alongside it the share of the sorted ones whose answers already sit as rows in another system.
2. Take ten of your own documents and write a draft ontology. The kinds of node, the kinds of edge, and the two ends of each edge. When you are done, read the same documents again and count the sentences that list has nowhere to write. Widen the ends and that number falls, and what gets worse in exchange?
3. Build by hand the list of names your corpus uses for the same thing, have a model make the same judgment, and count the disagreements. What answer does each of the three produce: splitting one into many, merging many into one, and a name that fell into no bundle and vanished? Which is the quietest?
4. Write down a design that removes one edge on purpose and asks “how many.” How do retrieval’s empty and the graph’s “one” differ in their effect on the decision that follows? Besides counting nodes with no edges attached, propose one more signal that the extraction missed something.

13.8 On a second reading

- Come back to Chapter 11’s list of what retrieval cannot answer (Section 11.6) and the first two items on that list have become this chapter’s two kinds of question. That the place where chunk size had no general answer and the place where the list of nodes and edges has no general answer are problems of the same order shows there too.
- Go back to Chapter 9’s schema and set it beside the list in Section 13.3, and the division where a person settles the fields and the model fills them stands in the same shape in two places, output and storage. The fork between enumerated values and an open string moves here to how far to narrow the two ends of an edge.
- Go back to Chapter 7 and set Section 7.4’s discipline beside this, and the sentence there about leaving counting to code has become this chapter’s first remedy. That counting a hundred and loading a hundred are different was already set up there.
- Read Part VI, come back to the quiet failure in Section 13.1, and catching the uncounted number and the wrongly laid edge has become a problem of evaluation design. That the failure of a counting question is invisible without an evaluation set that knows the right count gets precise there.

13.9 Further reading

Two things come from Edge et al. (2024). The first is that the diagnosis overlaps. Its statement that “RAG fails on global questions directed at an entire text corpus” and that this is inherently “a query-focused summarization” task rather than a retrieval one is an observation that set out separately from the list in Section 11.6 and arrived at the same place. The second is the arrangement. It puts deriving “an entity knowledge graph from the source documents” as the first stage of indexing, and lays on top a stage that pregenerates “community summaries for all groups of closely related entities.” That is what it looks like when the arithmetic of the extracting hand being a model, and that cost being paid at index time in proportion to the corpus, stands up as a real thing. This is a paper proposing one method, though, so do not read the method across as a prescription.

When the Document Is Not Text

Open the store and this is what is in it. A scan of a contract the other side signed and sent back, a monthly report that is half tables on every page, an equipment drawing with notes written in the margin by hand, a deck of a few words and one picture. To a person's eye these are all the same thing. A document. Open it and it reads, and the answer you want is inside.

To the pipeline's eye there are two kinds. Documents that open as text, and documents where the text has to be manufactured first.

The earlier chapters of Part IV handled only text material. Choosing what to keep, cutting into chunks and finding what is close in meaning, overlapping lexical search and standing candidates up again, pulling out entities and relations and joining them, all happened on top of letters. This chapter answers what to do when it is not text. What comes into the corpus, what form you put it in front of the model in when it gets read, and how you attach a figure when the answer has to carry one.

14.1 The assumption that the corpus is text

Every mechanism in the last four chapters stands on one unwritten sentence. That what sits in the store is text. Cutting cuts text, finding by meaning carries text over, lexical search matches letters. Pulling out entities and relations has to read sentences too. And this sentence never needs to be written anywhere in the code. The documents you try during development are usually prose the designer wrote, and as long as every stage takes text and gives text, no occasion to break it arises.

The place where the assumption actually gets made is elsewhere. It is the first step, putting the document into the store. There a document that is not text comes in converted to text, or comes in half, or comes in with only its file name and no content.

14.2 When it is not text

Split what comes in into four kinds and each catches somewhere different.

A scanned PDF. The file is a PDF and there are no letters inside it. Every page is a photograph of paper. A person opening it reads it, and there are no letters to pick up, so unless a stage that manufactures text out of the image comes first, what goes into the corpus is a blank.

Tables and drawings. There are letters. And half the meaning is in position rather than in the letters. Which row and which column meet here, which box the arrow runs from and to. Take only the letters and that half does not survive.

Figures and photographs. There are documents where what has to be confirmed is only in the figure. A photograph of the state of a piece of equipment, a drawing with the damaged area

Table 14.1: What a document conveys outside its letters, and where each goes once only the text is taken. No row raises an error.

Place	How a person reads it	What survives in the text	What happens downstream
Tables	reads a value’s meaning where the row head and the column head meet	heads and values as a run of words keeping only their order	the value gets found and what it is a value of does not. The cell next door becomes the answer instead
Layout	reads the rank of title, body, footnote, and aside from size and position	one flow at one weight	a footnote loads with the standing of body text, and an exception is found at the weight of a rule
Figures	reads drawings, charts, and the letters and arrows inside a figure as content	usually nothing, at best one line of caption around it	an answer that was in the document is not in the corpus. It was removed and it looks like it was never there
Marks	reads a text’s standing and state from strikethrough, emphasis, signature blocks, and stamps	the body with the marks gone	a withdrawn clause loads as the same prose as a live one, and a stale value gets found confidently

marked, an incident record with a screenshot pasted in. What you can fetch as letters here is usually one line of caption around the figure, and the answer is inside the figure.

Handwriting. A signature, a note in the margin, one line in an approval box. Unlike printed letters, reading it becomes an estimate, and when the estimate is wrong nothing marks it as wrong.

What disappears from a table shows if you follow one value. Reading the cell “5,000,000” in a monthly report’s expense table, the eye goes left to confirm “March,” the head of that row, and up to confirm “travel,” the head of that column. The meaning of five million is not inside the cell. It is where the row and the column meet. That is exactly what the form of a table does. Instead of repeating “March’s travel spending is” beside every value, it lets position stand in for that sentence.

Take only the letters and that position is gone. In the extracted text, March and travel and five million are words laid out in a row. What is bad is that no error is raised here. The extracted text is sentences and paragraphs and, as prose, is without a blemish. The index gets built normally, and asking about March travel does sometimes bring back the chunk holding five million. Bringing back four million from the cell next door is exactly as natural. There is nothing left in the corpus to separate the two cases.

The four rows of Table 14.1 share one thing. No row raises an error. And this loss does not spread evenly through a corpus. A report that is all prose loses little, and a report that is half tables loses half. So one number for a corpus’s average quality says very little. While a well extracted majority of prose pulls the average up, some class of document may have gone in as half of itself.

This class is not an exception in an organization. A contract signed and returned, results reported as a table, an equipment drawing, an incident record with a screenshot pasted in, exist

in every organization. And the shape of a corpus is not chosen by the designer. It arrives. The people putting documents into the store are the organization's members, and what is in a store you connected cannot be fully known before connecting. Research that starts from documents conveying information not only through text but through figures and page layout and tables comes from there. Faysse et al. (2025) opens on it: documents are “visually rich structures that convey information through text, but also figures, page layouts, tables, or even fonts.”

14.3 Working with material that is not text

Now that you know what is lost, what do you do? The decision gets made separately in two places. What form to put a document in front of the model when it gets read, and how to attach a figure when the answer has to carry one.

When it gets read

There are three branches. Extract it as text, show the page as an image, or mix the two.

Extract as text and the decisions of the last four chapters stand as they are. You can choose boundaries of meaning and cut, you can overlap lexical search and finding by meaning, and you can pull out entities and relations and join them.

Show it as an image and position stays alive. A table stands in front of the model in the shape of a table and a drawing in the shape of a drawing. But there are three costs. First, you cannot find it. An image does not catch in lexical search. Second, tokens. Third, the unit of storage has been drawn in advance by the page rather than by a boundary of meaning. One clause spans two pages, and one page holds two unrelated sections. Where Chapter 11 asks where to cut has been settled by the format before you begin.

Mix them and you find by the text and read by the image. This is what practice answers, and the basic handling of a document is already that. Send a PDF and every page is converted into an image, and the text extracted from the same page rides alongside that image (Anthropic, 2026g). So a table stands in the shape of a table and stands as text at the same time.

Mixing comes with things to watch, though.

First, *the extracted text can be wrong, and that mistake is quiet*. One digit read as another, or a row shifted by one, and the result is still sentences. So there is one road to knowing. Pick a sample of documents by type and have a person lay the source and the extracted text side by side.

Second, *leave a road back to the source*. Extracted text is a derivative, and a derivative's defect can be fixed by extracting again as long as the source exists. A pipeline that throws the source away after extracting saves storage and puts the derivative in the source's place. At that moment the loss in the last section turns from a deferred cost into a settled one. Chapter 6's discipline, that you settle the original's destination before running a transformation you cannot undo, applies here word for word. Extraction is also an irreversible transformation that happens once, outside the loop.

Third, *it has to be distinguishable which side the answer stood on*. Turn this on in the request and it comes back with the answer. One item of ground in an answer with grounds attached looks like this (Anthropic, 2026b).

Table 14.2: What form to put a document in when it gets read. The right column keeps the first two from being a choice.

	Extracted as text	The page as an image	Both together
What is lost	the crossing of a table, the rank of a layout, the content of a figure, the state a mark showed	small letters after the downscale. Reading becomes an estimate	the text side’s loss stays, and the image covers it at reading time
Unit of cutting	choose boundaries of meaning and cut	the page’s boundary is drawn first, by the vessel	find by boundaries of meaning, read by the page
Tokens	loads the content only	loads the paper the content sits on too, and repeats it every step	pays the image cost on only the few pages left after narrowing
Finding	the last four chapters’ methods stand as they are	carrying a page image straight into a representation is a reported method	find by the extracted text and read the page you found

```
{
  "type": "page_location",
  "cited_text": "March travel 5,000,000",
  "document_index": 0,
  "document_title": "March spending report",
  "start_page_number": 7,
  "end_page_number": 8
}
```

What the ground points at is page seven alone, because the end is exclusive. The bridge from a chunk back to the source gets laid here. When an answer looks wrong, what you open is not that document but that page. Without a window through which a person can open the material the system actually read, looking for the cause always happens somewhere else.

There is a limit to write down. The same documentation says that for PDFs the text is extracted and chunked into sentences, and that because “image citations are not yet supported, PDFs that are scans of documents and do not contain extractable text are not citable” (Anthropic, 2026b). *Grounds attach only to the extracted text.* On a scan with no letters, the mechanism for checking disappears before anything else does.

When it goes into the output

A model cannot put out a figure. It receives images on the reading side, and on the producing side there is one sequence of tokens, and no pixels ride in that sequence (Anthropic, 2026k). So in a system whose answers have to carry figures, what the model does is not make a figure but point at one.

Where the thing to point at comes from splits two ways. One is a figure that already exists. The drawings and photographs and page images in the corpus. The other is a figure made on the spot. The model writes code that draws a figure and the execution environment runs that code (Definition 7.4). The file that comes out does not come back as pixels either. It comes back as an identifier. In both branches what the model puts out is a word that points.

How does the model know which figures exist? It knows by them being loaded in the context. Ten thousand figures in the store amount to none if the listing is not loaded. What to write on one line of that listing is the design, and this question already came up once in Chapter 7. As what makes a tool choosable was the tool name and the tool description (Section 7.2), what makes a figure choosable is a name and a one-line description.

figures you can use

```
march-spending-table
  March spending by department, split by category
  file_011CPMxVD3fHLUhvTqtsQA5w

plant-layout
  Line layout and inspection points for plant 2
  file_011CNha8iCJcU1wXNR6q4V8w
```

That two things sit on one entry is the point of this listing. What gets used for choosing is the name and the description, and what gets used for attaching is the identifier below them. Neither a person nor a model can tell what something is from an identifier alone, and with only a name there is no way to actually attach it. What the model puts out has this shape.

```
{
  "answer": "March travel spending rose against February, and most
            of the rise is inspection travel to plant 2.",
  "figures": ["march-spending-table"]
}
```

Only the name is in the answer. Turning the name into an identifier and actually attaching the figure is the receiving code's job. Have the model write the identifier directly and you are asking a model to copy twenty-odd characters without getting one of them wrong.

What happens when it points wrongly? It splits two ways, and what divides them is who sees the mismatch first. An identifier that is off gets the request refused before it reaches inference. A request carrying an identifier that does not exist comes back like this.

```
{
  "type": "error",
  "error": {
    "type": "not_found_error",
    "message": "File `file_011CNha8iCJcU1wXNR6q4V8w` not found."
  }
}
```

A noisy failure. No answer comes out and no input tokens get charged. A name that is off, by contrast, is quiet. The model writes a name that is not in the listing and nothing happens at that point, and the receiving code cannot find the name and sends the answer out with the figure's place empty. What the user sees is a perfectly fine answer and a blank. If the figure was the ground, an answer with no ground went out in the shape of an answer with ground.

There is a road to moving the quiet side to the noisy side. Enumerate in the output's shape that only names in the listing are allowed as values (Section 9.3). Under that, a name not in the listing is not refused. It is not drawn.

14.4 Other material

Things besides documents come into a corpus. Which of the three branches to send them down differs by material.

Table 14.3: What else comes into a corpus. Only the last row is not corpus material.

Material	How it differs from prose	Which branch
Tabular material	it opens as letters and its meaning sits where a row and a column meet	keep it as text and do not cut it. Leave counting and filtering to code (Section 7.4)
Code	letters, but the unit that stands on its own is a function or a definition rather than a paragraph	keep it as text and move the cut to the boundaries of functions
Logs	one line has no meaning and a stretch of them does	keep it as text and stand the chunks up as spans of time
Audio and video	it does not go into the model's entrance as it is	what was turned into text becomes the corpus. The place to go back to is a timestamp, not a page
Screenshots	an image, and not a record but the state right now	not material for an index but material for an observation (Chapter 7)

Only the last row of Table 14.3 differs in kind. A screen changes with the action just taken and is stale by the next step. Chapter 11 drew the boundary that what changes faster than the index's interval is not to be found in the index but read on the spot (Section 11.6), and a screen sits at that boundary's extreme.

One discipline is left here. Artifacts the system made as figures to show a person, charts and diagrams, come into the corpus too. If those are going to be the corpus of future runs, leave the machine-readable material alongside them when you make them. Write down, at the same time, the material that would not need extracting.

14.5 Summary

In one sentence. Every mechanism in the last four chapters stands on the assumption that what sits in the store is text. A good share of real documents breaks that assumption, and the stage that manufactures text out of a document that breaks it throws away, without raising an error, what the document conveyed outside its letters. That stage comes before cutting, so what is lost here cannot be recovered by any later technique.

What is now settled.

- The assumption that a corpus is text is stood on by every stage while written nowhere in the code. The place the assumption gets made is the first step of putting a document into the store, and once that step has passed no mechanism restores what was lost. Collection succeeds by the file and extraction fails by the content (Section 14.1).
- Four things catch. A scan with no letters, tables and drawings with half the meaning in position, photographs whose answer is inside the figure, and handwriting whose reading becomes an estimate. Which row and which column a table's value belongs to disappears the moment you lay it in a line, and that loss goes into the corpus as perfectly fine prose rather than as an error (Table 14.1).
- The branches at reading time are three. Extract as text and the last four chapters' methods stand while four places are lost. Keep it as an image and position lives while you cannot find it, tokens go, and it gets cut by the page. Practice's answer is the mix. Find by the extracted text and read the source of the page you found (Table 14.2).
- Three things come with mixing. The extracted text's mistakes are quiet, so there is no road to knowing besides comparing a sample. You have to keep the source to be able to extract again. And the ground attached to an answer has to point at which page of the source, so that looking for a cause happens on top of the material the system actually read. But the ground attaches only to the extracted text, so on a scan with no letters the mechanism for checking disappears first (Section 14.3).
- On the output side a model cannot put out a figure. It only points. So which figures exist has to be loaded into the context as a name and a one-line description, with the identifier for attaching sitting beside it. An identifier that is off gets the request refused before inference, and a name that is off sends the answer out with the figure's place empty. The latter can be made noisy by narrowing the names to an enumeration.
- The material coming into a corpus takes different branches. Tabular material stays uncut and goes to code, code gets cut at function boundaries, logs get stood up as spans of time, and for audio and video what was turned into text becomes the corpus. Only screenshots are material for an observation rather than for an index (Table 14.3).

What is still open.

- There is no general answer for which documents to carry on the extracted text alone

and which to open the source for, or where to put the threshold for opening the source. The author knows of no material comparing the three branches side by side on the same task.

- How far carrying a page image straight into a retrieval representation works, on which corpora and which queries, also has thin material to compare. What can be said with confidence goes as far as the shape of the cost.
- Turning the extraction loss that shows only through sample comparison into a measurement, which is designing an evaluation set with judgments attached, goes to Part VI.

Try this today. Take a hundred files from the connected store and *count* how many there are of each format. Separate out how many are PDFs and, of those, how many yield no letters when opened, and the share that becomes half of itself in front of extraction falls out right there (Section 14.2). Then take five documents with tables and *compare* the source against the extracted text side by side. All you have to look at is whether which row and which column a value belongs to survived (Section 14.3). Last, open one chunk currently in the index and *check* whether which page of the source it came from is written on it. If it is not, fill that in first the next time you build the index (Section 14.3).

14.6 Exercises

1. Design a procedure for evaluating extraction loss in your own corpus. Sample by document type, have a person compare the source against the extracted text, and count what disappeared at each of the four places in Table 14.1. Explain with the argument in Section 14.1 why this measurement cannot be replaced by a retrieval quality metric.
2. Take one task that finds an answer in a document with tables, and write down as a procedure an experiment comparing an arrangement using the extracted text alone against one that finds by the extracted text and reads the source page. What do you have to record to count the two arrangements' failures apart into failures of finding and failures of reading?
3. Set up choosing the resolution for sending a page image as a decision problem. Send it large and it gets downscaled at the ceiling anyway, and send it small and small letters do not read. Write a procedure for setting that value from the smallest letters in your own documents, and predict what the failure of setting it wrong looks like when observed.
4. In a system that attaches figures to answers, trace in your own code what happens when the model writes a name that does not exist. Does a record survive along that path? If not, which do you put in first, the enumeration in Section 9.3 or a check on the receiving side?

14.7 On a second reading

- Read Chapter 11, come back to Section 14.1, and the order shows: there was a decision that hardens even earlier than the cutting and the representation that were called hardening decisions there. What it costs to change the representation model is rebuilding the index,

and what it costs to undo the extraction is extracting again if the source was kept, and if it was not there is no road to undo at all.

- Read Chapter 12, come back to Section 14.3, and you see that its skeleton, narrowing with a cheap method and choosing again with an expensive judgment, is the same shape as the division of labor here. Narrowing by the extracted text and reading by the source page is the same division split across two materials.
- Read Chapter 10, come back to the passage about leaving an artifact in two materials, and the discipline of preparing at making time for finding time has become memory's write discipline. That not only what to keep but in what form to keep it is memory's design gets filled in there.
- Read Chapter 9, come back to the output passage in Section 14.3, and its conclusion that a schema enforces only the form gets used twice here. Narrow the names to an enumeration and a name that does not exist is not drawn, and whether that name is the right figure for the answer is not something the form looks at.

14.8 Further reading

Two things come from Faysse et al. (2025). One is the statement of the problem. The observation on its first page that documents are visually rich structures while today's indexes lean on extracted letters "often through lengthy and brittle processes" stays exactly as this chapter's starting point, whatever the merits of the method. The other is that a frame for evaluating exists. The paper puts out alongside it a benchmark for retrieval over visually rich documents, "composed of various page-level retrieval tasks spanning multiple domains, languages, and practical settings." That is exactly what a team trying to compare the two roads on its own corpus needs.

The real conditions for handling documents as images are in Anthropic (2026g) and Anthropic (2026k). The handling that gives text and image together per page, the token difference between the two forms, the scale at which an image becomes tokens and its ceiling, and the limit that small letters do not read once it shrinks at the ceiling. The constraint that grounds attach only to the extracted text and not to page images is in Anthropic (2026b), together with the shape of a page citation. The numbers move with the model, so what to take is the shape. A figure's cost grows with its area and stops at a ceiling, and what the ceiling costs is resolution.

Part IV closes here. Five chapters were one question. *Before a question arrives, what do you write down, in what form, and where?* What to keep, what keeping it outside buys and sells, how to stack finding in layers, what structure to take a question with no answer in a chunk, and what gets lost at the first step when the source is not prose, were all different layers of that one question. Part V changes direction. What the four parts so far gathered is the material that rides in the loop. Instructions and tools and knowledge are all standing, so what is left is the mechanism that turns the loop with that material.

PART V

Running the Loop

Designing the Agent Harness

At the fortieth step the agent says it is done. Do you take that and end the loop, or turn it once more?

The model is not what decides. Attaching the observation to the history, checking the stopping condition, building the next call or deciding not to: none of that happens inside the model.

What this chapter covers is that outside. *In practice what separates a system that works from one that does not is often this side rather than the model.*

15.1 The Harness: How the Loop Turns

A model answers once and it is finished. It takes a request, puts out one next action, and the call closes there. Whether to call again, when to quit, what to keep: none of that is in the answer. It is settled outside.

Chapter 1 cut one call into three. Assemble a context out of the history, turn that into tokens, take the tokens and put out the next action. All three are one-way work, in goes one thing and out comes another, and all three sit inside a single call.

But an agent repeats that call. Repetition brings a question the three cannot answer. *Turn once more?* A call cannot settle whether it gets called again. The caller settles that, and the caller is code.

The stretch while the repetition runs on one context is a *session*. When a session opens the context grows step by step, and when it closes that context is gone whole. One task may finish in one session or run across several.

Give a name to what holds the loop and makes these decisions.

Definition 15.1 (Harness). In an agent system, the code that sits outside the model call and owns the running of the loop is the *harness*. A harness settles four things: what to keep, when to stop, how to start again, and what to trust.

Put the line Chapter 1 drew between workflows and agents next to this. An agent was a system that settles at run time when to stop. But what runs that judgment is still code. The model declares completion and picks the next action, and the harness settles whether to accept the declaration. *Even in an agent that has handed control over to the model, one loop stays in the code to the end.*

Earlier chapters passed two things here. The observation that the most common accident is not a wrong answer but *not stopping*, and the arithmetic that cutting a task into several sessions lowers the cost.

The second is what this chapter stands on. Split the same total steps across several sessions and the history one session rereads at every step is that much shorter. That gain holds only

Harness

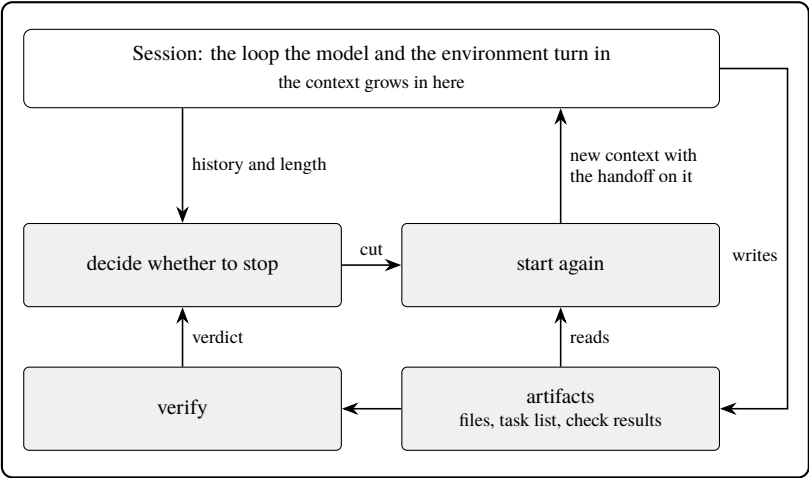


Figure 15.1: The harness and the four things it settles. The session box holds the loop of Figure 1.1 whole. The unlabelled arrow carries the artifacts to what verifies them.

while what passes between sessions does not grow. What is left is the next question. *Where do you cut, what do you leave at the cut, and how do you join it back up?*

Doubt the sentence “we do not have a harness yet” from the start. If there is a loop calling the model, the harness is already there. A retry count nailed into a constant, a time limit, a branch that catches an exception and turns it once more: those are all decisions about stopping and starting again. They are just scattered through the code, undesigned.

The real decision is not whether to build a harness but whether to gather the four that are already settled into one place and make them something you design.

15.2 State: What to Leave Behind

When a session closes, that context is gone whole. Which paths were tried, why one was dropped, which facts were confirmed: if those lived only in the context, none of it ever happened.

So before settling where to cut there is something else to settle. *What has to sit at the cut for the next session to carry the work on?*

The span differs from the memory of Chapter 10. Memory outlives the task and gets pulled up again in the next one. What gets left here only has to reach the next session of the same task, and when the task ends its job ends.

Call the bundle that crosses from session to session a *handoff*. One constraint sits on its size. The gain from splitting sessions holds only while the amount passed each time does not grow, so a handoff has to come in under a settled size.

Artifacts, not the transcript

The first impulse is to summarize the conversation and pass that. It is not flatly wrong, but the principle to hold is the other one. *Leave state as the work's artifacts, not as a record of the conversation.*

In a three-step session the record and the artifacts are nearly the same thing. Stretch the session to a thousand steps and the record has grown to a thousand steps' worth, while what is left at the end may be a few edited files and a few updated lines of a list. The length of a record follows the number of steps walked. The size of an artifact follows only the result of the work.

What the next session needs is the second, for three reasons.

- **The size is tied to the task.** A conversation record grows in proportion to steps. The size of a task list is tied to the structure of the task and the size of a file to the size of the result. The side on which “the amount passed must not grow” holds by itself is the artifacts.
- **You can check it.** A record is what was *said* to be done and an artifact is what *is* done. A file can be opened. A check result can be run again. What lets the next session trust the state it inherited has to be that it can be checked again, not that the account reads well.
- **It does not degrade.** A long context gets read at its ends while the middle sinks. A file system does not do that.

Four are the standard kinds. Files are the result of the work. The task list is the whole plan and where it stands. Check results record what currently passes. The decision log holds the paths dropped and why. Add a *change history you can roll back* and a broken session has a unit you can throw away.

Of the four only the decision log inherits the character of a conversation record, and even that has to be a summary that gets replaced.

When the amount passed grows

The gain from splitting sessions came from the premise that the amount passed does not keep growing, and that premise does not keep itself.

Appending one more summary at the end of every session breaks it exactly. Each session adds a summary, so the next one starts with a context that much larger. A larger start means one session carries less work, which means more sessions for the same job. Eventually the handoff alone fills the context and a session begins already inside the stretch where performance falls off.

Say it plainly. *The tenth session starts carrying all nine before it.*

So a handoff has to be rewritten, not stacked. Finished items get deleted or folded out of the task list, the decision log gets written over, the summary gets replaced. *Tie the size to the size of the work remaining rather than to the number of sessions.*

Remark 15.1. Rewriting is not free. Deleting and writing again loses information. A system that erased why a path was dropped goes off to drop the same path again. That revisit adds steps, and the added steps come back as cost. Which things to throw away is what Chapter 6 faces head-on under the name of compaction, and the standard set here is one. Can the next session reach the same decision without it?

The harness fixes the form of the handoff. Leave the form to the model's discretion and every session produces a handoff of a different shape, and the next session starts by decoding

a document it has never seen. Whether it was updated, whether its size is inside the budget, whether the required fields are filled: code can check all of that without asking the model. *The contents belong to the model and the container belongs to the harness.*

The required fields are the questions the four kinds above answer. What is done now, what is not yet, what to pick up next, what was tried and dropped.

Put into a form it comes out like this.

- **Done.** The item name and *what each was checked with*.
- **Left.** The item names and their order.
- **Next to pick up.** One of the ones left.
- **Dropped.** What was tried and why it was dropped, a line each.

Making the first field carry the means of checking is the point. In a field that takes only an item name, “said to be done” passes straight through. Make it carry what the check was and an unchecked item shows up as a blank.

If the first three go empty the next session starts by working out where things stand, and if the fourth goes empty it walks the dropped paths again. And a handoff note reads plausibly even with a field empty. *Whether it is empty has to be seen by a check, not by a person’s eye.*

Checking the size and checking that the right thing is inside are different jobs, though. The failure where the size is inside the budget and the next session repeats the previous session’s decision cannot be caught at run time, and comparing the two sessions’ records afterwards is the best you get.

15.3 Termination: When to Stop

What to keep is settled, so now settle when to hand over.

Moving the stopping judgment outside the session is *the decision of this section, the move itself*. A session often cannot judge its own ending.

There are five kinds because of what a harness can see. The harness assembles the context, so it knows what is in it and it measures how long it is. What it cannot read is the inside of the model. What the model is missing right now, whether it is circling: the model tells you only in words, and words cannot be checked.

So what you can lean on is what crosses the loop and the outward shape of the context. Five places are visible. The artifacts a session produces, the resources spent, the sequence of actions going past, the growing length of the context, and signals arriving from outside the loop.

The completion check and the resource cap

The best ending is to confirm the task is finished and stop. But “finished” is an event that needs a means of judging before it is even defined. With no check you can run, “looks finished” is the only signal, and then the confirming is done by a person.

The resource cap sits at the other end. It is a stopping condition that knows nothing about the task, which is why it always has to be there.

A token budget, a time limit, a step cap: these are not devices that make success happen. They are devices that make *failure cheap*. The objection that the number is hard to set has the order wrong. A number can be measured and adjusted, but with no device there is nothing to adjust.

Table 15.1: Kinds of stopping condition. Only the fourth row is a different animal. A length threshold is not a signal to end the loop but a signal to cut the session.

Kind	What it looks at	How it mainly fails
Completion check	Did the artifact pass the standard	Without a means of judging it does not exist. The rate at which the judgment is wrong becomes the rate at which stopping is wrong
Resource cap	Steps, tokens, time, cost	A large cap stops only after the whole waste is paid, a small one cuts off a task that could have been solved
No progress	Change in actions and artifacts	Misreads exploring as circling. Designing the signal depends on the task
Length threshold	How long the context has grown	Not a stop but a cut. A late threshold means the handoff gets written on an already blurred context
Outside intervention	A signal from a person or a system above	Without a point of intervention an action that cannot be undone runs unreviewed

No progress

Calling the same tool with the same arguments again, confirming a fact already confirmed: that is the failure Chapter 2 showed.

What is good about this signal is that the harness can catch it *without asking the model*. The sequence of actions passes through the harness, so a repeated call or an unchanged artifact can be counted.

It can be caught wrongly, though. What looks like repetition may really be exploring. So this kind is safer used as a signal to intervene than as an immediate stop. Cut and start again, call a person, or at the least leave a record.

The length threshold

Chapter 4 asked for this. If the stopping device watches only step count and not context length, nobody intervenes during the stretch where performance falls off and steps multiply.

Turned into a device, the request reads like this. *The harness watches the context length and closes the session on reaching a threshold set ahead of the point where performance starts to fall off.*

Two things are inside that sentence.

First, this is not a condition for ending the loop. The task remains, so it is a condition for cutting the session, leaving a handoff, and starting again.

Second, the threshold has to sit ahead of that point for one reason. *The handoff note is written by the model too.* Close the session after performance has started to fall off and the summarizing of what mattered and what finished happens inside that same stretch. A handoff written on a blurred context passes the blur to the next session. The room the threshold leaves is the budget for writing the handoff before things blur.

On reaching the threshold there are two things you can do. Shrink the history and carry the same session on, or close the session, leave a handoff, and start again. The first is compaction

and the second is cutting.

The one that usually comes up first is compaction. You choose cutting when the task is too long to sit in one session, or when the context has blurred so far that carrying on is not worth the shrinking. The two compete over the same threshold, and Chapter 6 has already set out the arithmetic on the compaction side.

The point where performance starts to fall off is a number you measure on your own task, not one you read off a spec sheet. Without the measurement there is no basis for setting a length threshold.

Outside intervention

The harness judges the first four on its own. Only the fifth has its judge outside, and the places it is needed are settled. *In front of an action that cannot be undone.*

Deleting something. Money moving. A message going out. Here there is no way to take a wrong judgment back, so even an action all four kinds waved through needs a place where it is stood up once more.

Its character differs from the rest too. The four are devices against waste and this one is a device against accidents. What the harness owns, though, stops at making the place. Which actions get a person in front of them and what runs automatically is a design of permission, and Chapter 21 takes it.

What a retry leaves behind

Tool calls fail. They time out, they half succeed, they hit a rate limit. What the harness does then is the smallest instance of a stopping decision. *Call once more?*

How many times to call again is the same story as the resource cap. What is new is the keeping side.

Splice the whole error text in and one trace is a few thousand tokens, and three rounds stack three copies. That is how much the per-step growth swells by, and the swelling pulls the point where performance falls off forward. *A system whose tools fail often solves the same task in the same number of steps and blurs at an earlier one.*

So designing a retry has two fields. How many times to call again, and what to keep from the failure. The standard for the second is to keep only what the next decision uses. If the same call failed three times for the same reason, what to keep is one copy and a count, not three.

A completion claim is an input, not a verdict

The model says by itself that it is done. Put the two states that produce that same sentence side by side and the problem shows.

One is a task actually finished. The other is a blurred context. The remaining items are buried somewhere in the middle of the context and the model, having missed them, declares completion because the work in front of its eyes is over.

The two look the same from outside. You cannot separate them from the sentence alone, and the later the step, the more the steps pile up and the more tempting the claim, the higher the odds of the second state. Models have been reported to wrap work up early as they approach what they believe is their context limit, which one account of long-running harnesses named “context anxiety” (Rajasekaran, 2026).

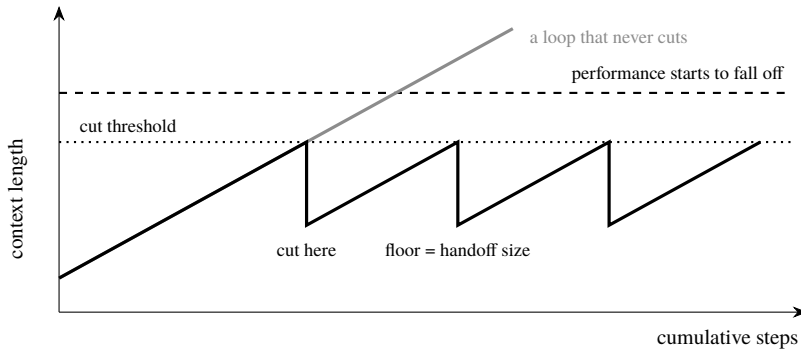


Figure 15.2: Cut and start again and it becomes a sawtooth. The grey straight line is a loop that never cuts. Note that the floor of the teeth is not zero. A session starting again carries the handoff.

You cannot trust a thing that says of itself that it is finished, not because the thing is dishonest but because it cannot see its own context blurring. The same account notes that a later model largely removed the behavior on its own. Hurrying is a property of a model, not a principle, and the device put in to block it will one day be useless.

So this claim must not end the loop. Judging and generating inside the same context makes the judgment not independent, and self-judgment with no signal from outside is hard to trust: that was Chapter 2’s conclusion.

This does not mean a self-declared completion is worthless. It is a useful signal, but it is an input rather than a conclusion. When the claim arrives, what the harness does is not end the loop but fire the verification. *When the verification passes, then it is done.*

The real decision is not which of the five to pick but how many layers to lay. A design that says the cap is redundant because there is a completion check is a design that, on the day the check is wrong, has no reason left to stop.

The kinds are not alternatives to each other but layers holding up each other’s failures, and the layer that has to be there no matter what is the resource cap. A check can be wrong and a detector can catch the wrong thing, but tokens spent and time elapsed count as they are.

15.4 Restart: How to Begin Again

One constraint rules the design. *The next session does not automatically know what the previous one did.*

A new context starts blank, and there are only two routes by which it can know the previous session existed. The handoff, and the environment itself. Everything else never happened.

Turn that around and it becomes the design standard. Check a handoff by asking whether a capable worker seeing the job for the first time could read this alone and pick up the next item.

The handoff is the load the next session carries from the start. The floor in Figure 15.2 is its size.

What implementations most often miss is *separating the session that starts from the session that carries on.*

The first session takes the task, sets up a structure, breaks it down, and makes a list. Later sessions pick the next item off that list, run it, and update the list. Make one instruction serve

both and both get worse. Either the structure gets rebuilt every session, shaking what was already set, or the work drifts with no structure and starts on things not on the list.

Judging from the presence of a handoff whether this is a start or a continuation, and giving a different instruction, belongs to the harness. That a continuing session's instruction should be one item at a time is Rajasekaran (2026)'s observation, where the generator was told to work in sprints, picking up one feature at a time. Trying to do too much at once is a common derailment in long runs.

A session's first job is to confirm the state it inherited. Run the checks once to see what passes and what is broken, then start new work. *An item the previous session's list marked done is done only after it passes this confirmation.* Being marked done without the tests having been run is not rare.

Whether to cut at the boundary of the budget or at the boundary of the task is a decision too. The length threshold is the default signal, but a good harness adds one. *Cut at task boundaries.*

Right after an item on the list finishes and its verification passes is the moment the handoff is shortest and cleanest. Cut in the middle of an item, pushed by the threshold, and the state of a half-done job has to be put into words, and that account is less accurate than an artifact.

Keep the threshold as the last line and do the cutting at boundaries where you can. Sturdiness comes from here too. If one session blurs or derails, throw away that session alone and start again from the boundary just before.

15.5 Verification: What to Trust

The principle is *separating the making from the judging*. The context that made an artifact does not get to judge whether the artifact passes.

The judging signal has to come from outside, and self-correction without an outside signal is hard to trust: that was Chapter 2's conclusion.

It is worth being exact about what has to be separated. *Not the model, the context.* The maker and the judge being the same model is common and not itself the problem. The problem is a judge sitting inside the context that did the making.

What to judge with

If code can check it, that is best. A test that runs, a format check, whether the build succeeded: none of those bend to the model's persuasion. Anything that returns a signal the model can read will do. A test suite, a build's exit code, a linter, a comparison against reference output.

For a judgment with no such check, "is this design good," you end up putting in a separate model call to do the judging. There is one rule to keep then.

The judge must not see the making.

Even with a separate grader there is one route left for the contamination to come back. Letting the grader read the reasoning along with the answer. The grader walks the logic of that reasoning as it reads, and a wrong answer looks plausible inside its own logic.

The separating does not end at the actor. It has to reach the input, and what the harness owns is the boundary of that input. Two reasons. A judge that inherited the maker's context inherits the story that justifies the conclusion along with it. And a judge that sees the whole process carries a context as long as the maker's and blurs the same way.

Table 15.2: The same rule laid on four sites. The sentence that gets written differs by site, and only the fourth asks a different question.

Site	What actually gets written	Who judges
Write it in the instruction	A line in the system prompt. “Run the tests. If they fail, fix and run them again. Do not say you are done before they all pass.”	The model. Following an instruction varies each time too
Hang it on a goal condition	A question sent to a judge at the end of every turn. “Did every test pass? Answer yes or no only.” A no adds one more turn	The model. The checks run but the model judges
Block it in code	The harness’s stopping condition. While the tests’ exit code is not zero, the loop does not end	Code. The same every time
Ask a different context	Give a fresh context only the change and the original symptom and ask. “Did this change hide the symptom or fix the cause?”	A model in a different context. The question itself is different

Building the judge

The judge wavers too. One account watched an evaluator find a legitimate issue and then “talk itself into deciding they weren’t a big deal and approve the work anyway” (Rajasekaran, 2026).

So you tighten the judge toward skepticism, and the same account says that tuning a standalone evaluator to be skeptical is far more tractable than making a generator critical of its own work. The tighter you pull, though, the more flaws it finds in perfectly good artifacts, so where to stand between waving things through and inviting needless rework stays a decision.

Which is why building a judge does not start with using a judge. *It starts with turning a judgment like “is this design good” into a gradeable standard.*

That is what Rajasekaran (2026) did first on moving to a structure split between a generator and an evaluator, and calibrating the evaluator with few-shot examples carrying detailed score breakdowns followed.

The same account has one more ordering. Before building, the maker writes down what it will build and what will decide a pass, and the judge reviews that. The two negotiated it as a contract for the sprint, agreeing on what done looked like before any code was written. *Set the standard afterwards and you end up fitting the standard to what got built.*

The final shape that account reached was three roles, a planner and a generator and an evaluator. Not the one right answer but one arrangement of the four decisions. Structures where several model calls work together start in Chapter 16.

Standing a judge up is a waypoint, not a terminus. Once the standard is written concretely, part of it no longer needs a model’s judgment. Whether the format is right, whether the required elements are there, whether the checks pass: those settle down into code.

A model call that does the judging is a stopgap filling a place code cannot check, and swapping that place out for code as the standard ripens is how this axis is run.

15.6 When Not to Build More Harness

Every device built so far stands on an assumption that the model cannot do something.

Table 15.3: The harness's devices and the incapacity each assumes.

Device	The incapacity it assumes	If that assumption breaks
Resource cap	Cannot count its own budget	Mostly harmless. A cap frozen low cuts a capable model off early
No-progress detection	Cannot notice it is circling	Only the false catches remain. An exploring detour gets cut as circling
Cutting and handing off	Performance falls off as the context lengthens	Only the effort of writing handoffs and the information lost remain
Leaving artifacts	There is no memory across sessions	The assumption that will hold longest. Even with memory, the gain of being checkable again remains
Splitting off a judge	Cannot judge its own artifact fairly	The splitting adds only latency and cost
Breaking the task down	Cannot finish a large task in one context	The breakdown splits the view of the whole and manufactures handoff effort

Assumptions have expiry dates. These are empirical statements about a model's current ability, not principles. *As models improve some assumptions break, and a device standing on a broken assumption becomes either harmless surplus or active obstruction.*

A harness is not a fixed right answer but a temporary rig fitted to the model of the day. A device left standing on a broken assumption is the first signal that it is time to take something out, and here is the list that includes it.

1. **The harness is doing what the model already does.** Devices like this pile up in a harness that never rechecks whether its assumptions still hold. Finding a device you can turn off is worth as much as finding one to add.
2. **The harness itself eats the context.** Handoff guidance, state summaries, rule lists, session management instructions all ride at every step. And they fill the context that much sooner. *If a device put in to protect the loop is spending the loop's room first, the direction is inverted.*
3. **The harness is more complex than the task.** When session management, handoff formats, and verification stages get more complex than the task being solved, you have added one more thing to fix. Ask first whether a single session and a workflow would have been enough.
4. **There is no way to evaluate a change to the harness.** If you cannot evaluate whether a device changed the success rate or the cost, that harness is not improving. It is only getting more complicated.

There is one test for the first item. *Turn the device off and run the same task.* If the result is the same, the device is surplus. If it improves, the device was obstruction.

To run that test you have to write down, when you bring a device in, the assumption it stands on. *A harness whose assumptions are not written down cannot be tested.*

A framework is in large part a harness sold as a product. Choosing one is importing somebody else's list of assumptions whole, and what to ask is whether that list still holds for today's

model and your own task.

One last thing. A harness measures and cuts and verifies the loop, and it is easy to leave the harness itself outside the count. *The discipline of counting tokens and switching things on and off to compare applies to the harness too.*

15.7 Summary

In one sentence. Even in an agent that has handed control over to the model, one loop stays in the code to the end, and every device built on that loop stands on an assumption that today's model cannot do something, so write the assumption down when you bring the device in and check regularly whether it still holds.

What is now settled.

- A harness is the code that owns the running of the loop outside the model call. It settles state, termination, restart, and verification (Definition 15.1, Figure 15.1). If there is a loop calling the model, the harness is already there.
- Leave state as artifacts rather than as a record of the conversation. The size is tied to the task, it can be checked again, and it does not degrade. Make the done items carry what they were checked with and an unchecked item shows up as a blank.
- Appending a summary per session starts the next session with a context that much larger, and eventually the handoff alone fills the context (Figure 15.2). A handoff has to be rewritten, not stacked.
- Termination is a layering of the five signals a harness can see (Table 15.1). The layer that has to be there is the resource cap. A completion claim is not a conclusion but an input that fires verification, and a length threshold is not a stop but a cut. The threshold has to sit early because the handoff note is written by the model too. In front of an action that cannot be undone, make a separate place for a person to stand.
- All the next session knows is the handoff and the environment. The harness separates a starting session from a continuing one, and a session's first job is to confirm the state it inherited. Keep the threshold as the last line and cut at task boundaries where you can, so that one broken session costs only that session.
- Separating making from judging separates contexts, not actors, and it has to reach the input rather than stopping at the actor. There are four sites to hang the judgment on, and only the fourth asks not whether it passed but whether it fixed the cause (Table 15.2). A model call that judges is a stopgap, swapped out for code as the standard ripens.
- Write down the assumption each device stands on and you can check whether it still holds (Table 15.3).

What is still open.

- The arithmetic for choosing compaction over cutting at the threshold, and the standard for what to throw away when rewriting, were set by Chapter 6. What is added here is one standard. Can the next session reach the same decision without it?
- Setting a length threshold means measuring on your own task the point where performance starts to fall off. How to measure is Chapter 4, and the design of memory across sessions is Chapter 10.
- How to build the standards for verification, and how to evaluate the judge itself, go to Part VI. So does the method for evaluating whether a harness change moved the success rate and the cost, and since evaluating needs records of runs, Chapter 20

takes it too.

- Splitting the maker from the judge is one form of multi-agent. Chapter 16 takes the arithmetic.
- What to stand in front of an action that cannot be undone goes to Chapter 21.
- Which lines a framework fills in on your behalf is Chapter 23.

Try this today. *Make* a field beside every done item in the handoff for what it was checked with. Whether it is empty is seen by a check, not a person (Section 15.2). *Count* the stopping layers in the loop you are running now. If there is no resource cap, nail that in first (Section 15.3). Pick one device, turn it off, and *run* the same task. If the result is the same it is surplus, and if it improves it was obstruction (Section 15.6).

15.8 Exercises

1. Analyze a design in which the model's completion claim ends the loop. Which of the four things the harness settled has moved to the model, and which section of this chapter warned about the price?
2. What gets worse if the cut threshold is set too high, and what gets worse if it is set too low? Counting the steps it takes to write a handoff and the cutting at task boundaries, write out in words the conditions a good threshold has to satisfy.
3. Table 15.2 lays the same rule on four sites. If the cheapest way to make the tests pass is to edit the tests, which of the four blocks that road and why do the rest fail to? Answer by what each site sees.
4. For each of the six assumptions in Table 15.3, design one experiment that checks whether it still holds in your system. Is there a way other than turning the device off and comparing?

15.9 On a Second Reading

- Lay the chapters of Part II over Figure 15.1 and you see that handoff guidance, state summaries, and rule lists are all tokens riding at every step. Tuning the harness and tuning the context are two ends of the same budget.
- Read Chapter 6 and come back to the part about rewriting the handoff, and you see that holding the handoff's size down was an instance of compaction. The standard of dropping the source and keeping the conclusion applies to a handoff note as it stands, and it brings with it the warning that a stale conclusion is worse than an unrelated source.
- Read Chapter 10 and come back to leaving artifacts, and you see that an artifact bridging sessions and a memory persisting past them are different things. An artifact's job ends when the task ends, while memory accumulates past the task and brings its own problems of rewriting and collision.
- Read Chapter 16 and come back to the verification section, and you see that splitting the maker from the judge was already the smallest divided system. What crosses between the divided contexts takes the same discipline as a handoff between sessions. Artifacts, not records.

15.10 Further Reading

Writing that treats the harness head-on is growing, but industry accounts still run ahead. For prior work on long-running agents see Anthropic (2025d), and for the follow-up that goes as far as splitting the maker from the judge see Rajasekaran (2026).

The latter shows in what order the four decisions get assembled in a real system, and in particular why building a judge starts with building the standard. It also records which devices got taken back out once the model improved, which is worth reading alongside this chapter's last section on assumptions having expiry dates. Both are industry accounts, so read them for structure and for the assumptions the structure stands on rather than for numbers.

For the academic ground see Yang et al. (2024). The result is in software engineering, so carrying it to another domain means confirming it again in that domain.

One Agent or Many

Say one query about an internal policy comes in. The loop turns a few times before the agent has an answer to put out. Everything this book has covered so far sits inside one of those turns.

16.1 Running a Single Agent

One turn of the loop breaks into six segments, and the earlier chapters attach one to each.

In the first segment the context gets assembled. It is not only the body of the query. The system prompt goes in, the definition of every tool this agent can call goes in, and the history of the turns so far goes in. The first two stay the same however many turns pass, so they are the stable prefix, and the history lengthens every turn, so it is the variable tail. What to write into the stable prefix is Chapter 5, and what to load into the variable tail and when is Chapter 4.

In the second segment the model gets called. What was assembled goes in and an answer comes back. What gets billed is tokens read and tokens written, and the read side holds whole what was just assembled. The longer the history, the more gets reread each turn, so the cost does not rise in proportion to the turns but faster than that. This arithmetic is Chapter 3. Leave the leading run unshaken, though, and the cost of rereading is cut deeply, so not rewriting the stable prefix every turn is by itself a different bill (Figure 6.1).

In the third segment the model picks one tool. It picks the tool that searches the policy documents and fills the arguments. What the model can pick is only what the definitions say, so the names and descriptions of the tools are the list of what this agent can do. How to write that list is Chapter 7, how to attach somebody else's tools to it is Chapter 8, and fixing the form of what comes back ahead of time is Chapter 9.

In the fourth segment a tool brings something in from outside. There are several places to bring it from. Pulling back out what an earlier session wrote down (Chapter 10), picking from a pile of documents the passages that catch on the present question (Chapter 11), reordering what was picked (Chapter 12), following what was arranged into entities and relations (Chapter 13), and turning documents made of tables and figures into a readable form (Chapter 14).

In the fifth segment what was brought in attaches to the context. Attached, it lengthens the input of the next turn. Attach a whole page of the policy document and that page gets reread on every turn after. When the length crosses a threshold measured in advance the history has to be shrunk, and that shrinking loses something and what to throw away has to be settled in advance: that is Chapter 6.

In the sixth segment it gets settled whether to turn once more or stop here. This decision looks like the model's and is really held by code outside. How many turns are allowed, how many tokens may be spent, what counts as complete, where to restart from on failure: all of that is written in that code. That code is the harness, and what a harness owns is Chapter 15.

Table 16.1: One turn of the loop and what sits on it. The left column is the segment of the turn, the middle is what the designer actually settles there, and the right is the chapter that covered the decision. The point of the table is that the right column is not scattered. The fifteen chapters before this did not cover fifteen different subjects but different segments of one loop.

Segment of the turn	What the designer settles	Where it is
Assemble the context	What to write into the stable prefix, and what to load into the variable tail and when	Chapters 5 and 4
Call the model	How to count tokens read and written. Whether to leave the leading run unshaken	Chapters 3 and 6
Pick a tool	What the names and descriptions make possible. What to fix the form of the result with	Chapters 7, 8 and 9
Bring something in	Where to bring it from and what to pick to load	Chapters 10 through 14
The context grows	When to shrink and what to throw away	Chapter 6
Settle whether to turn again	How to hang the cap, the stopping, the restart, and the verification	Chapter 15

That is a single agent. One context, a history growing in one line, one set of caps, and when you ask what happened you open that context. What people expect this arrangement can carry is usually pitched lower than what it carries. Which is why the decision to tear up the structure is common where redesigning the tools and the results would have solved it.

Whether this loop actually turns well is a question this chapter has not answered. What to call success, what to measure the success rate with, how to keep records of runs so you can find where it went wrong: Part VI takes those. Where to put a person and what to allow is Part VII. What this chapter asks is the one question before them. *Does there have to be only one loop?*

16.2 When One Agent Is Not Enough

Is there a case where a single agent is not enough? “Not enough” usually arrives as one of four things. It is slow. It costs a lot. It does not fit in one context. Other work gets mixed in and drags the result down.

All four already have answers. Nothing new to learn and no structure to change. Table 16.2 is the five to try first.

Going to multi-agent without checking all five is changing the structure before checking whether it is a problem the structure solves. So where is it still unsolved after passing all five? Two places.

First, when latency or a bottleneck stays put after the reasoning has been improved. If it is still slow after the branches ran at once, that means the branches are not individually short. If the branch that searches the policy documents has to turn ten more times inside itself, that branch is not one parallel unit that code split off but a loop turning on its own. With several branches like that, splitting in code is not enough. Each branch has to pile up its own history, and piling up a history means having its own context. This is where parallel and dividing part.

Table 16.2: The five to pass through before going to multi-agent. All five were set by earlier chapters and all five work with the context kept as one. The right column is the point of the table. All five cost something, but what they cost is one call or a person's effort, not the structure of the system.

Try first	What it solves	What it costs
Routing	One prompt collapsing under every case at once. Split refund from technical fault first and send it to dedicated handling (Chapter 1)	One call to split. A road back when the split was wrong
Parallel	Latency from running branches that do not touch each other one after another. If the branches can be counted in advance, code splits them and runs them at once (Chapter 1)	One call per branch. Code that merges
Tree	An early decision settling everything after it with no road back. Spread the candidates out, evaluate them, and pick (Chapter 2)	One call per node. There has to be a standard to evaluate with
Compaction and cutting	A history that only piles up until it does not fit. Shrink at the threshold or cut at a task boundary (Chapters 6 and 15)	The observations thrown away. The effort of settling in advance what to throw
Redesigning the tools	A result so large that one call blankets the context. Shrink the definitions, cut the results, keep the large things outside (Chapter 7)	A person's effort to rewrite

If a branch is one call, code can split it. If a branch is one loop, the context has to be split.

Second, when the roles have to be exactly separated. Take the maker and the judge. If the judge sees the maker's process, the judgment gets contaminated. Having read what it was written on, it gets persuaded by those grounds instead of checking them again. That is why what the judging gets has to be the artifact and the standard alone, and that was Chapter 15's demand. This demand is not about saving context. The dividing is because the two contexts must not meet, and so none of the five hands can stand in for it.

Why none can stand in shows if you try. Inside the same context, tell it at the end to check what it just wrote. What checks is the side that just wrote the answer, and that side's context already holds in full why it wrote what it wrote. Doubting grounds you just laid down does not happen easily while those grounds sit right there in the context. Compaction does not do it either. The shrinking side picks what to keep, so the unflattering passages shrink first. Routing does not do it either. Whichever road you send it down, the line is still one. *What is needed here is not a short context but a different one.*

There is nothing besides those two. Anywhere else, multi-agent is not a better answer than doing one of the five in Table 16.2 more carefully.

16.3 Multi-Agent

Multi-agent is dividing the context. It is not making several calls.

Why the distinction matters shows like this. Say you hand the maker's context whole to the judge and add only "now judge it" at the end. There are two calls and nothing has been divided. What the second call reads is what the first was reading with one sentence attached. The context still grows in one line, the tokens still cost the same, and the judgment is not independent.

The reverse holds too. If one model takes two contexts in turn, the caller is one and yet the gains and the troubles of a divided structure appear as they are. The two contexts do not read each other's history.

Definition 16.1 (Division and multi-agent). If, while one task runs, there are two or more contexts each piling up only its own observations, and neither inherits the other's history whole, the system is *divided* and that structure is *multi-agent*. *The unit of division is the context, not the call.*

This distinction settles the arithmetic. Count calls and you cannot separate multi-agent from a long loop. Both have several calls. The tree search of Chapter 2 also spreads several calls and is not divided. Every branch inherits the history back to the root as it stands, so there is no boundary.

From here on each divided context is a *seat*.

What dividing improves

The context gets shorter. That is all.

Do three branches of research in one loop and the document read in the first branch rides along the whole time the third branch turns. What the last turn reads is a history holding the observations of all three.

Give the three branches to three seats and what each reads is the observations of its own branch alone. *Nobody seeing the whole and each context being short are two names for the same fact.*

Which is why finer is not better. Put one seat on each step and no seat knows what the step before did. To make it know you have to pass it along, and passing all of it turns the starting state into the history and takes you back to the one line you began with.

How much shorter it gets is settled by the amount of observation the divided sides can afford not to know about each other, not by the number of seats.

So the question that settles whether to divide is not "is this task too big for one." It is "*does this task split into parts that can afford not to know about each other.*" Any task can answer yes to the first question. Not to the second.

What dividing adds

Passing appears. As much as you divided, you have to hand over, and what you hand over is words. One side summarizes what it saw in its context and passes that, and the receiving side sees only the summary. There is no road back to the original.

Finding where it went wrong gets harder. To tell which seat slipped when the answer is wrong, each seat has to have kept its own history.

Tokens go up. Every seat loads its own instructions and its own tool definitions. You are biting the same thing several times over.

A published practitioner account writes this down concretely. In an arrangement split into a planning seat, an implementing seat, and a testing seat, *the seats spent more tokens on coordination than on the actual work* (Anthropic, 2026f).

The same account adds that teams have spent months building elaborate multi-agent architectures only to find that improved prompting on a single agent reached equivalent results.

Which means the place to divide has to be settled by what the sides need to know about each other, not by the names of the jobs. Planning and implementing and testing are jobs that need to know about each other. Different names do not divide a context.

How to run it

Put several seats in and three things have to be settled anew. Who starts, what gets passed, and when it ends.

On who starts there are broadly two arrangements. In one-calls-many, a central seat splits the task into branches, opens a seat per branch, and gathers what comes up. If the number of branches is settled before the run, code can split them; if it is a number you only know on opening, the central seat splits. In many-call-each-other there is no center and a seat calls another seat when it needs one. The second is rare for a reason. A called seat can call again and that seat can call again, so the number of seats is settled only during the run and the budget cannot be written before it. Stopping gets harder too. In the first, it ends when the central seat ends; in the second there is no seat that knows who is still turning, so a separate place has to be made to judge whether everything has finished. So what is common in practice is the first, and the second gets chosen only when you truly cannot enumerate at design time what the seats will call.

In what gets passed, what matters is the return side. What the gathering seat receives is each one's summary, not the sources each one read. This ignorance is design, not accident. If the gathering seat knew every observation, then all the observations sit in that context, and that is the same as not having divided. *The difficulty of merging comes from the same root as the gain of dividing.* So this cost is not one you can design away. You can reduce it. Make what each seat leaves an artifact rather than a narrative. Files and check results can be opened again, so when two seats disagree the disagreement becomes a collision of facts you can compare rather than a collision of claims. Chapter 15's standard, that what lets you trust the inherited thing must be its being checkable again rather than the persuasiveness of the account, becomes here the discipline that joins seat to seat.

Fail to settle a place to do the merging and one appears by itself. Make only the dividing decision and not the merging decision, and merging gets pushed into one call tacked on at the end, and that call's context piles up the summaries from every seat in no order, and the more seats the longer it runs. *The long context that dividing was believed to have removed comes back to life on the merging side.* So the merging input has to be updated rather than accumulated, and what gets sent up in what form has to be fixed by the harness rather than left to each seat's discretion. Leave it to discretion and as many different documents come up as there are seats, and the merging side's first job is decoding several documents it has never seen.

Ending becomes two layers. A stopping condition inside a seat blocks only that seat's waste. While one seat sits against a dead end, the rest spend their budgets to the last without knowing it, and the gathering seat learns that one branch came back empty only after every seat has finished. A stopping condition that cuts across the seats has to exist separately before you can cut a branch that has already lost its point. The decision to bring dividing in brings with it the decision to build one more harness.

16.4 How It Lands in Practice

Put the published cases side by side and the previous section's judgments show up in real systems. Only in this section do the services keep their names.

Divided: a research service. Anthropic (2025e) records Anthropic's experience building Claude's research feature as several pieces. One central seat splits the question into branches and opens a seat per branch to run at once, with a large model in the central seat and small ones in the branches. Each seat has its own context. Look at what moves and the previous section's arithmetic is right there. The central seat writes down for each branch what to go and find out and sends it down, and the branch searches and reads and reasons in its own context and sends up only what it found out. It does not send up the documents it read whole. Send them and the central seat's context ends up holding every branch's observations, which is the same as not having divided. Attaching sources to the answer is a further seat's job. What makes this record worth its weight is that the score and the cost are written side by side. On the internal research eval the multi-agent arrangement "outperformed single-agent Claude Opus 4 by 90.2%," and the same piece reports the token use alongside. Take one chat interaction as one and a single agent runs about four times that, a multi-agent system about fifteen. Why it was divided is clear too. Research has branches independent of each other and each branch has to search the documents several times, so it is the first place Section 16.2 set out.

Not divided: another service doing the same work. OpenAI's Deep Research solves the same kind of research with a single agent. One model plans, searches documents, and doubles back down another road when it gets stuck, all in one context. It was made by training to carry a multi-step piece of research in one line. Put the two services side by side and one thing shows. *What settles the structure is not the kind of task.* The same research task was divided on one side and not on the other. What parted them was whether they chose to spread the branches at once and buy latency back, not the work of research itself.

Not divided: a coding agent. Claude Code turns on one main loop. Call the model, run a tool, attach the result to the context, call again. Exactly the six segments of Section 16.1. There is a subagent device, but its use is narrow. The standard the official documentation writes down is this. Use one when a side task would flood the main conversation with search results, logs, or file contents you will not reference again: the subagent does that work in its own context and returns only the summary. Definition 16.1's judgment, that the reason to divide is context rather than capability, is written there as a product's guidance on when to use a feature.

Divided: a seat that verifies. That there is one division that works well regardless of domain is a judgment Anthropic published separately afterwards. Divide the seat that made the artifact from the seat that tests it, and give the testing seat the artifact and the standard alone. This division works because there is little to hand over. The less that has to move between seats, the less gets lost in the moving. Which means Chapter 15's demand about verification was already a form of multi-agent, and it is the second place Section 16.2 set out. A condition comes attached. Fail to write down for the testing seat what to test and how far, and that seat passes things loosely. The trouble is that this failure is quiet. A verifier that passes often puts out no signal at all, so what remains is the fact that verification was attached and nobody knows what verification actually caught. So this division brings with it the job of counting what got caught (Anthropic, 2026f).

So how do you tell whether your own system has reached the point of dividing? Three signals come from the tool side, on Anthropic's published standard. Does the tool count run past about twenty. Do the tools span unrelated domains. Did adding one tool make picking

Table 16.3: Four published cases. The right column is the point of the table. The two that divided sit one each in the two places Section 16.2 set out, and the two that did not are neither. Note also that the same research task parts in the top two rows. What settled the structure was not the kind of task.

Case	Divided?	Why it went that way	Which side of Section 16.2
Claude’s research feature	Divided	The branches are independent and each has to search the documents several times	The first. It buys latency
Deep Research	Not divided	Trained to carry multi-step research in one line	Neither
Claude Code	The main loop is not divided	Little can be spread at once. You have to know the earlier decision to fix the next	Neither. Only a side task divides the context
A seat that verifies	Divided	A judge that sees the making gets contaminated	The second. It buys independence

among the existing ones less accurate. All three say that too much of what gets read together every turn has nothing to do with the question at hand. Catching all three does not mean dividing right away, though. Not loading every definition in advance and letting them be looked up when needed comes first (Anthropic, 2026f).

A sense of scale is written down as well. For a question that confirms one simple fact, one seat with three to ten tool calls finishes it. The task worth a divided structure is the other end: branches that have to be spread several at once, more to read than fits in one context, and several complex tools to handle (Anthropic, 2025e). And even for such a task there is one more standard. Is one answer worth the token multiple? What paid that multiple in the research service is that it finds in a few minutes what a person would take half a day to find, not that a divided structure is a good structure. Pay the same multiple to answer one customer query and the answer gets a little better and the bill goes up several times.

The way to set your own system beside Table 16.3 comes out of this. Write down the branches you want to divide, count how many times each branch calls a tool, and write down how much the branches have to know of each other’s observations. If a branch finishes in three or four calls it is not one to divide but one for code to split and run at once. If the branches have to know more than half of each other’s observations it is not one to divide but one to run in a single seat. The branches that survive all three as places to divide are usually fewer than the ones first written down.

Line the four up and a sense of it settles. The two divided cases were where branches were independent enough to spread at once or where two contexts must not meet, and the two undivided cases were everything else. And the undivided side did not do nothing. It tuned the tools, cut the results, and chose very carefully what to load into the context. *Most systems are a single agent with tools and reasoning attached in careful detail.* So the answer to expect when you set your own system beside these four is “do not divide.” To stand on the dividing side you have to be able to say which of the two places in Section 16.2 you are in, and if you cannot say, one of the five in Table 16.2 is still left.

16.5 Summary

In one sentence. Multi-agent is not making several calls but dividing the context, and that decision is right only when each branch needs its own loop so latency will not come loose, or when two contexts must not meet.

What is now settled.

- A single agent turns in six segments (Table 16.1). Assemble the context, call the model, pick a tool, bring something in, watch the context grow, and settle whether to turn again. One question runs through all six. What gets read this turn?
- There are five to pass through before dividing (Table 16.2). Routing, parallel, tree, compaction and cutting, and redesigning the tools. All five work with the context kept as one, and what they cost is one call or a person's effort, not the structure of the system.
- Two places are still left after all five. When each branch is a loop turning on its own so splitting in code will not loosen the latency, and when two contexts must not meet.
- The unit of division is the context, not the call (Definition 16.1). Two calls that pass the history whole and add one line of instruction are not divided, and two contexts that do not read each other's history are divided even if one model takes them in turn. Chapter 2's tree search does not belong here. Its branches inherit the history back to the root.
- Only one thing improves on dividing. The context gets shorter. How much shorter is settled by the amount of observation the divided sides can afford not to know about each other, not by the number of seats. So the question is not "is this task too big for one" but "does it split into parts that can afford not to know about each other."
- Three things go up on dividing. Passing and merging appear, finding where it went wrong gets harder, and tokens go up. So the place to divide has to be settled by what the sides need to know about each other, not by the names of the jobs. Planning and implementing and testing differ only in name; they need to know about each other.
- Put several seats in and who starts, what gets passed, and when it ends have to be settled anew. Fail to settle a place to merge and merging gets pushed into one last call that brings the long context back to life, and with no stopping condition across the seats the rest spend their budgets to the last while one seat is stuck.

What is still open.

- There is no way to evaluate before dividing which tasks split into parts that can afford not to know about each other. What the branches will look at is known only by running them, and running them means having divided already.
- When two seats send up different facts and both passed verification in their own seat, there is no answer to who picks which. The seat that judges is a seat that saw neither source.
- Evaluating the gain of dividing means comparing against the undivided run, and

that run usually does not exist.

- What to give each one and what to take back, and what shape to tie the seats into, go to Chapter 17.

Try this today. Pick one tool in the loop you are running now and *count* how many lines its result runs to. If more than half are lines you will never look at again, cut those before dividing anything (Section 16.2). Pick one branch you are thinking of dividing and *write down* whether that branch is one call or one loop. If it is one call, split the code rather than the context (Section 16.2). *Write down* how much that branch has to know of the other branches' observations. Past half, it is not a place to divide (Section 16.4).

16.6 Exercises

1. Pick the segment of Table 16.1 you have touched least in your own system, and judge whether the room left there overlaps with what dividing would buy.
2. By Definition 16.1, a structure where one model takes two contexts in turn is also divided. Argue whether that verdict is a flaw in the definition or a meaningful result. Of the three things dividing adds, which shrink in that structure and which stay?
3. Rewrite in terms of “what do they need to know about each other” why an arrangement split into a planning seat, an implementing seat, and a testing seat spent more tokens on coordination. Divide the same task again by context rather than by name, and how many seats do you get?
4. Section 16.3 demanded a stopping condition that cuts across the seats. If each of the five kinds in Table 15.1 were stood up again in a layer above the seats, what would have to be observed? Is any of the five observable only inside a seat, so that it cannot be raised to the layer above?

16.7 On a Second Reading

This section is not exercises but a guide to the next reading. After later chapters, the same sentences in this one read differently. What follows records where to come back to and what shows up new.

- Read Chapter 17 and come back to “how to run it” in Section 16.3, and who starts and what gets passed are laid out as a contract between two seats. The judging of disagreement this chapter could not answer gets asked again there as a question of who stands in which seat.
- Read Part VI and come back to the line about finding where it went wrong, and joining the histories scattered across the seats has become a design problem about what to evaluate. This chapter's open item about not being able to compare against the undivided run becomes there a question of what to take as the baseline.
- Read Chapter 21 and come back to Section 16.3, and you see that the boundary of division and the boundary of permission do not necessarily sit in the same place. A seat peeled off because of context may have no actions that cannot be undone in the first place.

- Read Parts VI and VII and come back to Section 16.1, and you see that the right column of Table 16.1 had two blanks. Evaluating whether this loop turns well and settling how far to allow are the two.

16.8 Further Reading

All are published industry accounts. Read them to confirm the arithmetic rather than to learn an arrangement.

Anthropic (2025e) is the central source. Take two things from it. One is that the score and the token use are written side by side in one piece. A record carrying only the score makes dividing read as a question of capability, and one carrying only the cost makes it read as waste. The other is the judgment that domains where every agent has to share the same context, or where the dependencies between agents run deep, are a poor fit, and that most coding tasks fall there.

Anthropic (2026f) is the record that follows it with a standard for when to use several agents and when not to. The coordination tokens exceeding the work tokens, the three tool-side signals, and the verification split that works across domains all come from there.

Anthropic (2024a) covers a point earlier than either. Routing and parallel among the five to pass through come from there, and the arrangement where a central call splits and gathers branches and the arrangement that separates the generating call from the evaluating one are written down with names.

Anthropic (2025a) covers the road that holds out without dividing when the tools get numerous. It is the device that stops loading every definition on every call and has them looked up when needed.

Communication Between Agents

Chapter 16 answered whether to divide the agent. What dividing buys is context rather than capability, and mostly it is better not to divide. If you have decided to divide, the next question is this chapter. What do the divided pieces pass to each other, and how?

One thing widens here. The same question comes to a system that did not divide. Our agent is one, but the team next door already has an agent running, and it already solves one segment of our task. If we have decided not to build that segment again, we have to settle what to hand over and what to take back.

The same list, different circumstances. In the first we built both sides and in the second we did not. *That difference parts nearly every judgment in this chapter.*

17.1 How Two Agents Interact

Go back to the case where there are only two contexts. The judge split off in Chapter 15. There is an agent that makes the artifact and an agent that judges it.

Three things between those two were already settled. The judging side receives the artifact and the standard. The judging side returns whether it passes. When that answer arrives the harness ends the loop.

But where were those three written? Not in one place. The first was in the argument list of the code that calls the judge, the second was in the judge's instructions, and the third was in a branch in the harness.

Read the three together and they are the contract between the two. But while they sit scattered across three places, nobody reads them as a contract. *A contract always straddles the giving side and the receiving side, and where nobody has laid the two sides side by side, a mismatch does not show until the run comes back.*

Definition 17.1 (Contract between agents). What the behavior of both sides leans on together when two agents call each other, namely who calls whom, what gets given, what comes back, and when it counts as finished, is a *contract*. A contract is there whether or not you settle it. Left unsettled, it gets written scattered across the caller's code and both sides' instructions, and then the number of contracts is the number of calling pairs rather than the number of agents.

Now part the two cases. What this chapter covers divides here.

One is among the pieces we divided. We are the ones who split the context, and both sides' instructions and both sides' models and both sides' caps are ours. So we settle the form as well. We can build two versions of what the receiving side does well on and compare them, and when they mismatch we fix whichever side is easier to fix.

The other is among pieces that already stand apart. An agent built by another team inside the company, or one from outside. We do not get to settle the form as we like. We cannot see

the other side's instructions, we do not know which model it turns on, and we do not settle how many steps it turns for. *Only one side can be fixed.*

And the other side changes without notice. What Chapter 8 wrote about a provider's revisions gets one layer heavier here. When a tool changes, the form of what comes back differs; when the other agent changes, *the same thing handed over brings back a different judgment.*

That one line parts Chapter 8 from this chapter. When you pull a tool in, what sits beyond the boundary is a function. It answers the same arguments in the same form and does not go off and do something else on its own judgment. Here, what sits beyond the boundary turns its own loop. It reads what it received, makes its own plan, and settles for itself what more to look at. *Calling one item off a list and handing over a judgment are different questions.*

17.2 What Gets Passed

Two things to settle. What gets given, and what comes back.

Start with what is handed over. *A newly opened agent does not see our conversation history.* The documentation of one real implementation nails this down: a subagent does not see your conversation history, and what crosses is the string written in the opening call. So the file paths the other side needs, the error text, the decisions already taken, all have to be written into that string directly.

The rest of what it receives is its own system prompt and tool list, and that was settled before we called. *What we settle at each call is one string.*

What goes into that string? One published operational account names four. Each subagent needs an objective, an output format, guidance on the tools and sources to use, and clear task boundaries (Anthropic, 2025e).

The easy one to leave out is the last. The same account records what happens when it is left out. One subagent explored the 2021 automotive chip crisis while two others duplicated work investigating current 2025 supply chains.

Where the instruction is vague it becomes one of three. Work gets duplicated, a gap gets left that nobody took, or the thing needed does not get found. *And none of the three looks like a failure.* Duplicated work comes up as two copies of a result, and a gap sends up no result at all, so there is no signal.

Write the four out for real and it comes to about this.

Objective: narrow to one cause why the payment failure rate has
risen since last Tuesday.

Format: At most three candidate causes. For each, attach the path
and line number of the log it stands on. If you cannot
settle it, write that you cannot.

Sources: Logs sit under logs/ by date. The deploy record is
deploy.log. Do not open the metrics dashboard.

Bounds: Do not change code. Go only as far as narrowing the
candidates. Finish inside thirty steps, and if you cannot,
send up what you have that far.

Now what comes back. This side is less visible. What comes up is the one message that agent wrote last, and the tool calls and file reads in between stay in its context. That is what the context dividing buys.

But the same circumstance works in the other direction too. *All the caller can see is that one message.* What the other side wrote it on is not in the caller.

So what comes back has to carry a road back to the grounds. Attaching to a few lines of conclusion the paths of the files the conclusion stands on costs almost no tokens. A side that received only the conclusion has no means of checking it again. A result that keeps the task description above comes back like this.

```
{
  "status": "partial",
  "candidates": [
    { "cause": "the retry interval went from 3s to 30s with the deploy",
      "evidence": ["deploy.log:412", "logs/2026-08-18.log:9331"] },
    { "cause": "one card network's responses slowed down",
      "evidence": ["logs/2026-08-18.log:11207"] }
  ],
  "not_done": "Logs from before Tuesday passed their retention and
    are gone. Could not compare against the previous week, and could
    not settle which of the two candidates is larger."
}
```

Look at three fields. `status` says partial, so the caller does not read this as complete. The `evidence` field holds paths and line numbers, so there is a road back to the source. And `not_done` is there, so what could not be done is separated from what was not attempted.

The third field earns the most in this section. *Without the field there is nothing even to fill.* Exactly what Chapter 9 set up about schemas, except that here what reads the schema is not code but the next agent's context.

One case remains where the field exists and cannot be filled. When a step cap cut the run off. Then there is no chance to write anything at all. One real implementation has the harness fill this gap. On hitting the cap it marks what came up as partial so the caller knows it is unfinished, and notes that the agent can be messaged to continue from where it stopped. Without the mark, a cut-off result and a finished result look the same to the caller.

One thing runs through all four rows. *What shrinks in the right column is not the number of things to settle but the number of people settling them.* A field the other side did not settle does not disappear. It gets settled by default on our side.

17.3 How They Communicate

With what gets passed settled, what is left is the manner of handing it over. Two questions. Who calls, and does the caller wait?

Who calls

The most common arrangement has one side calling the other like a tool. The opening is exposed as one tool, that tool's argument is the task description from the previous section, and that tool's result is the message the other side wrote last. Write out what moves and it is two lines.

```
caller -> subagent
  "Objective ... Format ... Sources ... Bounds ..."
caller <- subagent
  {"status": "partial", "candidates": [...]}
```

Table 17.1: Four things to settle when passing. The two right columns are the point of the table. Among the pieces we divided, either side can fix any field, so it can be settled later; among pieces that already stand apart, only we can fix anything, so a field left unsettled becomes a hole as it stands.

To settle	Among the pieces we divided	Among pieces that already stand apart
Task description	Objective, format, sources, bounds. We write all four and can fix them alongside the receiving side's instructions	We cannot see the other side's instructions, so nailing the bounds down in a sentence is all we have
Result	The harness fixes the form. It has the grounds' paths sent up too	It comes in the form the other side settled. We compare on our side and ask back when it mismatches
Format	Keep the schema in one place and have both sides read it (Chapter 9)	One more place is needed, to read theirs and carry it into our schema
What was not done	Make a field in the result, and have the harness mark the case cut off by a cap	Without the field it cannot be made. We cannot separate silence from failure

That there are only two lines holds all this arrangement's character. The caller holds the other's lifetime, so when it starts and when it is collected is written in one place. In exchange *the other side cannot ask back along the way*. If the string it received is not enough, it writes its answer with it not being enough.

The other arrangement has agents holding names and sending messages to each other. One recent implementation puts a tool that sends a message by name alongside a tool that lists who can be spoken to right now, and loads a roster of siblings into the first turn at startup. Here nobody holds anybody else's lifetime.

So one discipline follows this arrangement. The sentence that documentation nails down is this. *No message from any agent counts as a person's approval for a pending permission prompt*. Once you can speak to something by name, the sentence "I approve" can cross by that road too.

Does it wait

Call and halt until the result arrives and it is synchronous. Coordination gets simple. In exchange the whole is tied to the slowest side. That operational account of the previous section writes that its own system is synchronous and that this "simplifies coordination, but creates bottlenecks," and gives the reason too: going asynchronous makes coordinating results, keeping state consistent, and propagating errors harder (Anthropic, 2025e).

Since then the default has moved. In one recent implementation, an opening call that does not say whether to run in the background runs in the background, and the caller pulls it to the foreground only when it needs the result now. *Waiting is no longer the default*.

Decide not to wait and the result has to arrive somewhere. Two roads.

Table 17.2: Five ways of handing over. Better to read the right column first. Upward, coordination gets simpler; downward, the waiting shrinks, and the time saved always stays somewhere as state.

Way	What improves	What grows
Call and wait	The order shows in the code as it is, and where it failed stays with the caller	The whole is tied to the slowest side
Call and send to the background	Independent work runs at once. The finishing time becomes the maximum rather than the sum	Somewhere has to count how many are still turning. With no cap it keeps growing
Send a message by name	Caller and called are not fixed. It can ask back along the way	Nowhere knows whether everything finished. An arrangement where both wait becomes possible
Write to a shared place	The reading side can read whenever, and several read the same thing	Judging completeness and the rule for simultaneous edits have to be settled separately
Take a handle or a webhook	No connection held for long-running work. With a party outside this is the only road	State has to sit on our side too, and the two states can drift apart

One is not calling each other at all but writing into a shared place. Whoever comes after reads it. The calling pair disappears, so the contract looks as if it disappeared too, when it has only moved. What gets written in what form, when a written thing counts as complete, what survives when two of them edit the same item: all of that still has to be settled.

The other is taking a handle and asking later, or receiving it on a webhook of ours. With a party outside, this is effectively the only road. You cannot hold a connection open for hours.

That no name is attached to any of the five rows is the point of the table. Pipeline, supervisor and the rest are combinations laid on these five, not different kinds of thing. The picture of one thing supervising and others below it can be built from the first row or the second, and the two systems differ in when they finish and in where the caps sit. *What to ask when reviewing another team's structure is not the name but the left column of this table.*

17.4 Among Parties That Do Not Know Each Other: A2A

The previous section was about when we built both sides. When the other side is somebody else's, what blocks you first is different. You do not know where to send, what it can do, or what to carry to be let in. *There is something to find out before anything gets passed.*

There is a specification that settles this. Agent2Agent, A2A for short. Google published it in April 2025 and it moved to the Linux Foundation that June, so that it would be managed somewhere not tied to one company (Linux Foundation, 2025).

Nothing underneath it is newly built. It goes over HTTP, and the specification carries three protocol bindings, JSON-RPC and gRPC and HTTP+JSON, with progress arriving on a server-pushed stream. Authentication is picked among OAuth 2.0, API keys, mutual TLS and the rest (Agent2Agent Project, 2026).

How you find the other side

The specification has each agent put its own introduction at a settled path, `/.well-known/agent-card.json`. That introduction is the agent card.

If the team next door already runs that log search as a service, the card looks like this.

```
GET https://logs.example.com/.well-known/agent-card.json

{
  "name": "log-search",
  "description": "Takes a range and a condition, searches the logs,
                 and returns the path and line number of every
                 matching line.",
  "version": "1.4.0",
  "supportedInterfaces": [
    { "url": "https://logs.example.com/a2a",
      "protocolBinding": "JSONRPC", "protocolVersion": "1.0" }
  ],
  "capabilities": { "streaming": true, "pushNotifications": true },
  "securitySchemes": {
    "sso": { "openIdConnectSecurityScheme":
             { "openIdConnectUrl": "https://sso.example.com/oidc" } }
  },
  "defaultInputModes": ["text/plain"],
  "defaultOutputModes": ["application/json"],
  "skills": [
    { "id": "search_range",
      "name": "Search inside a range",
      "description": "Takes a start time, an end time, and a string
                     to look for.",
      "examples": ["payment failure lines from 07:00 to 09:00
                   on August 18"]}
  ]
}
```

What is in the card is the point. A name and a description, so you know what this party does. An endpoint, so you know where to send. A list of what it can do, so you judge whether your task falls inside it. A means of authentication, so you know what to carry.

Look again at the right column of Table 17.1 and you see what this card is. *The fields we had to fill by default because the other side had not settled them are already filled in there.*

The unit that gets passed and its states

The specification calls the unit that gets passed a task and attaches an identifier and a state to each. There are eight states, not counting the unspecified one, and they part into three. Ones actively turning, ones halted waiting for something, and ones that will not move again.

Write out one task turning and it looks like this.

```
sends      SendMessage  "Find the payment failure lines between
                        07:00 and 09:00 on August 18 and return
                        the paths and line numbers."

receives   task id=...  state=TASK_STATE_SUBMITTED
receives   task        state=TASK_STATE_WORKING
receives   task        state=TASK_STATE_INPUT_REQUIRED
           "There are two sets of logs for that window. Which one?"
sends      SendMessage  taskId=...  "The one written after the deploy."
receives   task        state=TASK_STATE_WORKING
receives   task        state=TASK_STATE_COMPLETED
           artifacts: ["logs/2026-08-18.log:9331", ...]
```

Why the states were needed is all in this flow. At the fourth line the other side halts and asks back, and this is not a failure. Handle it as a failure and we get no answer and the other side did its work for nothing. Leave it as still turning and we go on waiting.

Waiting has to have a name of its own. That is why a state for input required sits there separately.

How it differs from MCP

Where this specification parts from the specification of Chapter 8 is written in the official documents too. MCP focuses on how models and agents connect to tools, APIs, and data sources, and A2A focuses on how independent agents communicate and collaborate with each other as peers. *Passing things as agents rather than as tools.*

The same document gives the reason in one line. A2A's strength lies in flexible, stateful, collaborative interactions that *go beyond a typical tool invocation*. The flow above is that sentence in the flesh. A tool does not ask back.

The two are complements, not substitutes. The picture the same document draws is this. One application speaks A2A to the agents outside, and those agents reach their own tools over MCP.

That the two specifications recently moved in the same direction is worth seeing together. The 2026-07-28 revision on the tool side moved tasks into an official extension for long-running work, one that returns a handle to poll instead of a blocking result. Calling and waiting is no longer the only way on the tool side either. The last row of Table 17.2 has opened to tools as well.

A2A's editions turn over often. There is no value in memorizing the details. What remains is what had to be settled. The road to find the other side, the list of what it can do, authentication, the unit that gets passed, and that unit's states.

If we tied our own agents together without writing any of those five down anywhere, it is not that we did not have to. It is that we already knew the other side.

17.5 What to Watch When Designing

With the way chosen, count what that way leaves unsettled. Three catch again and again.

Who knows the end

With one agent, termination was that loop's problem. Chapter 15 said the harness sees five things, so the kinds of stopping are five. With several, those five do not rise to the wrapping side as they are. *The kinds came out of what could be seen, so when the place that sees changes, the kinds change.*

Of the five in Table 15.1, two rise unchanged. A resource cap is observed without looking inside. Outside intervention actually belongs on the wrapping side. Put the intervention on each one and a person has to watch that many windows.

The other three stand only if something below sends them up. The completion check changes ingredients because all the wrapping side can see is the result. The length threshold moves to the merging side and catches last. No progress does not stand at all. It was the kind that got counted mechanically by calling the same tool with the same arguments again,

and what the wrapping side sees is a finished result, while the marking time inside is known only to that harness.

So standing up a stopping condition that cuts across the whole is not a matter of adding code to the wrapping side but *a matter of adding an item to the contract*. And the merging side's context lengthens by that item.

That the others do not know one is stuck is not an accident either. It is exactly what the design that divided the context asked for. *Trap the failure and you trap the news of the failure*.

The arrangement where both wait is quieter than this. If the first waits for the second's answer and the second waits for the first's, nobody finishes. Neither raises an error, so the record shows only that both are still turning. The third row of Table 17.2 opens this arrangement.

The remedy was already shown by the previous section. Give the waiting a name. It is the same among things we built. Have what it is waiting for sent up as a field of the result.

Who fixes it when it is wrong

When what the caller expected and what the called returned differ, there is no standard inside the system for telling an excessive expectation from a poor delivery. With one agent the question did not arise. Expecting a conclusion grounded in the source while not having put the source in is one person's inconsistency, and reconciling it was that person's job too.

Two decisions become a contract not because the items multiplied but because the mismatch loses its owner. If one person reconciles, a mismatch just gets reconciled. If both sides keep it, who broke it has to be settled first, and for that the contract has to be written down somewhere.

That fixing is a different kind of work on the two sides has to enter the arithmetic too. The caller's expectation is in code, so it is reflected without remainder the moment it deploys. The called side's delivery is in instructions, and instructions only tilt probabilities.

So a revision finishes at once on one side and proceeds gradually on the other. In the stretch between, the system is at neither the old contract nor the new. Failures out of there are recorded as failures of the new contract when really the delivery is still on the old probabilities. That distinction is not in the record, so the team picks between rolling the revision back and pushing it through with no grounds. Counting the delivery rate separately before and after the revision is the defense.

Among pieces that already stand apart this remedy halves. Only one side can be fixed, the caller. The place that compares the format, the place that asks back on a mismatch, and the way to observe the other side's revisions all stand on our side. The last is counting the moment at which the same thing handed over began bringing back a different judgment.

When to harden the format

Harden too early and it is hard to change; do not harden and it reads differently every time. In practice this question usually shows up as one. Will the harness fix the string to be handed over, or will the caller write it fresh each time?

This is where ownership crosses over most quietly. Look at the scene of it.

In the code it is one line. Add "open a subagent" to the tool list. The argument is one string, and the description says "write what this agent is to do." Deploy.

After that, yesterday's run called that tool twice and today's called it nine times. Yesterday the argument carried three file paths and today it carried none. Both are normal behavior.

Adding one line handed over two things. How many get opened is now settled during the run, and the first decision of Section 17.2, what to hand over, became the model's judgment of the moment rather than a design. And *this crossing has no signal*. Adding a tool and handing over ownership are the same one line in the code.

The arrangement is not bad by itself. What knows best what is needed is the side doing the work right now. The point is what grows. If the string handed over differs every time, two runs of the same task give the same subagent different things, and then you cannot tell whether the score is the agent's or the string's.

A common misdiagnosis comes out here. When the numbers swing hard from run to run, people tighten the step cap and the concurrency together. But if the source of the swing is the string handed over, lowering the numbers leaves the swing where it was. What shrinks is only the ability to solve the task.

Checking takes less work than diagnosing. Have the first call put out the whole list of what to open at once, and after that let code open only what the list says. Ownership stays with the model and only the moment of hardening moves earlier. If the swing shrinks by that much the source was the structure, and if it stays the source is inside.

Hardening brings three things with it. How many will open is settled, so the total cap can be counted in advance. Only what exists before the run passes under a person's review. And when one is wrong, that one can be run again. The third has a condition. *You have to be able to rebuild the string you handed it*. Written nowhere, running one again becomes running the whole thing again.

If you decide not to harden, the only recourse is putting caps outside. Real implementations keep three dials. How deep an opened thing may open again, how many turn at once, and the spend of a single run. One implementation defaults to three layers of depth and twenty running at once, and leaves the spend uncapped. That the third is empty means the first two alone do not bind the total.

When not to come this far

Three shapes are overengineering.

1. **The code that makes the passing happen is bigger than what gets passed.** With two, calling in one direction, there is one contract, and no reason to stand up a separate layer to hold it. Standing up a supervisor is adding one to the count and making the contracts two, and what you get is only the expectation that attaching more later will be easier.
2. **Standing up a specification for a single call.** What A2A settles earns its keep when you do not know the other side. When you have to find it, when you cannot know in advance what it can do, and when you have to wait for long-running work without a connection. Where two things we built pass once and are done, all that grows is keeping state in two places.
3. **Both only wait.** If you chose an arrangement of peers passing to each other while you can enumerate at design time what will be called, all that arrangement added is the fact that nowhere knows the end. It is the arrangement to choose only when you cannot enumerate.

Which of the three you fall into parts across the two cases. Among pieces we divided, the first is common. We can build both sides, so nothing constrains standing one more layer in the middle, and that layer looks the same built for two as built for six. Among pieces that already stand apart, the second is common. You read the specification's list first and fit your

arrangement to that list, and that list was written for when you do not know the other side. If you already know the other side's name and endpoint and authentication, half of it is not your problem.

So the order of the design settles. First write in sentences what will be handed over and what will come back (Section 17.2). Unwritten, there is no telling what choosing a way is choosing. Then read the right column of Table 17.2 and keep as candidates only the rows you have a means of observing right now. Choose a row you cannot observe and what grew looks as though it did not.

Settle where the end is known last. Of the three, this is the only one that cannot be attached later if you put it off. Knowing the end needs something sent up from below, and what gets sent up is written in the contract. Adding one item after the contract is fully written is opening both sides again.

Running through the three is Chapter 16's warning. *No dividing, no communicating*. Everything raised in this chapter was already attached to the decision to divide, and all this chapter did was lay it out as items.

17.6 Summary

In one sentence. A contract between agents is there whether or not you settle it, so designing the communication is choosing where to write that contract down, and how many sides can be fixed depends on whether the pieces are ones we divided or ones that already stand apart.

What is now settled.

- A contract is there unsettled, and left unsettled it gets written scattered across the caller's code and both sides' instructions (Definition 17.1). The number of contracts is the number of calling pairs, not the number of agents.
- The difference between the two cases is the number of sides that can be fixed. Two among the pieces we divided, one among pieces that already stand apart. A field the other side did not settle gets settled by default on ours (Table 17.1).
- What is handed over is four things, objective and format and sources and bounds, and what comes back has to hold a path back to the grounds and a field for what was not done (Section 17.2). Without the field there is nothing even to fill.
- There are five ways of handing over and no name attached to any (Table 17.2). The same picture gets built from different rows, so what to ask about another team's structure is the left column, not the name.
- To pass between parties that do not know each other, the road to find the other side, the list of what it can do, authentication, the unit that gets passed, and that unit's states all have to be settled. A2A is the specification that settles those five, and unlike a specification that supplies tools it does not open the other side's inside (Section 17.4).
- Of the five kinds of stopping, the only ones that stand without looking inside are the resource cap and outside intervention (Table 15.1). The other three stand as items in the contract rather than as code.
- What crosses over most quietly is the string to be handed over. Once opening a subagent is exposed as one tool, that tool's argument is the string, and how many get opened is settled during the run too.

What is still open.

- There is no right answer for what to put in the string handed over. Give everything and dividing has nothing left to buy; give little and the other side goes wrong on what it did not see, and that failure is quiet.
- There is still no answer for what to choose on when two things sent up disagree. That a written contract at least lets you make a place to judge is all this chapter adds.
- There is no right answer for what to send up how often when you put progress into the contract. Send nothing and no progress does not stand; send more and the merging side's context grows.
- That a revision to the contract lands at different speeds on the two sides is written here, but no way to observe that mismatched stretch is.

Try this today. Pick one call that opens a subagent and *pull* out of the log the string that actually crossed as its argument. *Count* how many of the four, objective and format and sources and bounds, are written there (Section 17.2). Open the result that came up from it and *check* whether it holds a path back to the grounds and a field for what was not done. If not, make the field first (Section 17.2).

17.7 Exercises

1. Count every calling pair in your own system, and for each pair find where the four items of Table 17.1 are written. Is there an item where two pairs calling the same party hold different values?
2. Open the result one subagent sent up last, and judge whether the caller can trace that conclusion back to the source. If it cannot, what has to be added to the result, and how many lines does that add to the caller's context?
3. Write an agent card for one agent in your own system. A name, a description, an endpoint, a list of what it can do, and authentication. How many of the five are written nowhere at present?
4. Section 17.5 said three of the five kinds of stopping need an item added to the contract to stand on the wrapping side. Write the item for each of the three, and with all three in, write out how much the merging side's context grows in terms of the number of agents and how often things get sent up.

17.8 On a Second Reading

- Read Part VI and come back to Section 17.5, and the observation that changing the structure expires the measurements gathered until then stands as a problem of evaluation design. The items sent up from below have become, in Chapter 20, the subject of what to leave in the record.
- Read Chapter 22 and come back to Definition 17.1, and you see that the route by which content received from outside rides in a result and crosses to another agent opens while keeping the contract exactly. Of the two cases this chapter wrote, the second is what stands up again there.
- Read Chapter 21 and come back to Section 17.5, and the ownership of “who calls” gets rewritten in the language of permission. The permission of something that does not exist before the run cannot be written in advance.

17.9 Further Reading

Agent2Agent Project (2026) is this chapter's new source. What we settled by hand in Section 17.2 stands there as items of a specification. Three places to read. The agent card, the states of a task, and the design principles. The editions turn over often so the details can be let go, but the principle that the other side's inside does not get opened remains. Linux Foundation (2025) is the one-page record of where that specification is managed.

Anthropic (2025e) is this chapter's central source. The four of Section 17.2 come from there, and so does the observation of what happens when they are left out. The judgment that synchronous execution simplifies coordination while creating bottlenecks, and the arithmetic that a divided arrangement uses about fifteen times the tokens of a chat, are in the same piece.

Anthropic (2024a) covers a point earlier than that. The arrangement where a central call splits and gathers branches and the arrangement where an earlier result becomes a later input are written down with names, so you can compare how the left column of Table 17.2 parts underneath those names.

Anthropic (2025c) is the source on results. Why sending up only the summary of your own work and keeping the intermediate observations inside is the default shape is written there as arithmetic of the context. Asking in Section 17.2 for the grounds' paths to be sent up alongside is not the opposite of that default but a filling of the hole it leaves.

Part V was three chapters. Chapter 15 set up owning one loop from outside, and on top of that the two later chapters were one argument.

Chapter 16 asked whether to divide and this one asked about the space between the divided pieces. The two chapters leave the same thing open. *There is no comparison against the undivided run, and no data comparing the five ways.* Both are questions of evaluation.

PART VI

Evaluation and Observability

How to Evaluate an Agent

The question that comes out of the room before a release is usually one. Does this thing work?

Answering is harder than it looks. You can say that twenty cases were run last week and sixteen came back done. But nobody knows what those sixteen asked, why the other four failed, or whether fixing the prompt today would move the number. *Sixteen is true only for those twenty cases on that day.*

The seventeen chapters before this were about building. Part VI, which starts here, is about confirming that what got built works.

18.1 After the Design

Designing well is not enough. With no means of confirming, you cannot choose what to fix next.

Leaf back through the earlier chapters and several passages stop at the same place. Raising the examples in a prompt from three to five. Attaching reranking to retrieval. Splitting one loop into two. Tightening a schema to cut format violations. All four stopped at asking whether it got better.

They stopped for the same reason. What to call better, who judges, and which tasks count as cases were not settled.

There was a reason for putting it off. Confirming is awkward.

For a start, the same input twice does not give the same thing twice. What worked yesterday fails today and what failed today works tomorrow. *Running once and saying it got better is tossing a coin once and saying it lands heads often.*

And an agent walks many steps. If it walked twenty steps to one answer, looking only at whether the answer was right cannot tell you which of the twenty went wrong. The answer can be right with fifteen of the steps wasted.

So this chapter goes in order. Settle what a good agent is, draw the evaluation items out of that, see how those items actually get counted, and look at how published benchmarks do the job.

18.2 What Makes an Agent Good

Before choosing items there is something to settle. What is a good agent?

An agent does not take a person's work over whole. It is closer to something working alongside.

Claude Code is the example. It writes code, edits files, runs tests. And it does not ship what it edited. A person reads it and settles whether to take it.

What the person reads is not the whole codebase but what changed and how. Gather the edits and show them and the confirming finishes in a few minutes. Fix everything and say only that it is done and the person has to read from the start. *The code that comes out is the same and what is left on the person's side is not.*

This is not a temporary procedure bolted on because models are not trustworthy yet. A model draws its answer by probability. Run it twenty times and one or two come out different, and which of the twenty those will be cannot be known in advance. If it could be known in advance, that is already a bug you can fix. So the last approval stays on the person's side.

Models keep getting better. What can be handed over grows. But what disappears as they improve is not the person's confirming. It is *the amount the person has to confirm*.

So what is a good agent? Not one that does everything alone, but one that cuts the effort a person spends confirming.

Set the goal at removing the approval step and the direction goes wrong. Approval stays where approval has to be, and what gets better is the time one approval takes. Reading ten lines before you press the button becomes reading two, and that is the improvement.

The evaluation items follow from this. Put two agents side by side. Both get eight of ten right. The first does not mark which two are wrong. The person has to read all ten. The second flags the two it was unsure of. The person reads those two.

The success rate is the same and the effort saved differs by five times. *With only one item that difference is invisible.*

The point. As long as a model draws its answer by probability, the last approval stays on the person's side. So a good agent is not one that does everything alone but one that cuts the effort a person spends confirming, and the evaluation items have to carry, alongside what percentage came back right, *how much the person had to fix*.

18.3 What Gets Counted

The first thing to look at is whether the work finished. Where you confirm that it finished is what parts.

Say an agent books flights. The last message reads "your reservation is complete." That tells you nothing. You have to query the reservation record and see whether the ticket is actually in there (Anthropic, 2026c). Runs where the two disagree do happen.

So evaluation looks at what was left, not at what was said. Whether the file was edited, whether the tests pass, whether the reservation went in.

The same source recommends one more thing. Grade what the agent produced and do not grade the path it took. Write the right path down in advance and check that it was followed, and a run that reached the same answer by a shorter road gets counted wrong.

Of the five the last goes missing most often. Counting it is a nuisance. The first four fall out of a run by themselves and the last needs a separate record of what the person did.

But the previous section called a good agent one that cuts the effort a person spends confirming. Drop that item and you are not counting the thing you agreed to call good.

In practice this item splits in two. The share taken as it stood is one, and the amount edited before taking it is the other. When the first is high and the second is high too, people are not accepting what comes out. *They are rewriting it after accepting it.*

Table 18.1: Five evaluation items. No single one shows the whole thing. Choosing with the misses in view beats adding more items. The last row is the item drawn out of the previous section.

Item	What it catches	What it misses
Did the work finish	What is left in the environment. The reservation record, the edited file, the passing test	A run that reached the answer in three steps and one that took thirty land in the same box
Is the answer right	Correctness of content. Compare against an answer written in advance, or have a judge read it	An answer that was right by luck and one standing on grounds land in the same box
Was the tool chosen	Whether it called what it should have and held back what it should not have	Demand the order too and a run that reached the same answer by another road counts as a failure
How much did it take	Steps, tokens, elapsed time	A cheap wrong answer looks better than an expensive right one
Did a person fix it	The share taken as it stood, lines edited, times sent back	A run thrown away whole rather than edited is counted nowhere

Where to count is usually settled already. If there is a button by which a person accepts a result, it comes from there, and if there is no button you have to build the button first. For an agent that edits code, whether the diff went in as it stood or a person touched it is what remains; for an agent that answers from documents, whether the answer went out as written or edited is what remains. Attach the counting where the person decides, not where the run ends.

The third item is easy to skip too. But it tells you where to look when the answer is wrong. If the retrieval that should have been called was never called, what to fix is the tool description; if it was called and the answer is still wrong, what to fix is the retrieval or the prompt. *One case in the success rate, two different remedies.*

Choosing items brings one thing with it. *The items become the names of the failures.* A failure not on the list has no name, and what has no name does not reach the list of things to fix.

Plenty of teams have improvement meetings that move only between changing the model and adjusting the prompt. Not because there are no other candidates. Because there is no name to point at another one. A team already counting how many of the retrieved chunks the answer actually used can put fixing retrieval on the table.

18.4 How You Run It

With the items chosen you have to run them. Four things to settle. What counts as a case, who judges, how many times you run, and when you say it got better.

Definition 18.1 (Eval task). One test whose input and pass criterion were settled before the run is an *eval task*. Running that test once is a *trial*, and sorting what a trial left into pass and fail against the pass criterion is the *verdict*.

Table 18.2: Three judges. Not a ranking. They are good at different things, and a design that makes one of them carry everything loses by itself.

Judge	Good at	Bad at
Code	Fast and nearly free. The same artifact always gets the same verdict	Cannot see meaning. Marks a run failed for writing 96.12 where 96.124991 was wanted
Model	Sees meaning. Criteria hard to write as rules can be written in words, and it runs as many times as there are cases	The same artifact gets different verdicts. The verdict is itself drawn each time, so it wavers
Person	The most trustworthy. The last word where the other two disagree, and the standard to line them up against	Slow and expensive in people. You cannot line up experts to match the case count

What counts as a case

Many teams put this off believing they need hundreds before they can start. They do not. Anthropic (2026c) writes that twenty to fifty simple tasks drawn from real failures is a great start.

The reason comes with it and the reason is what to take. Early on, every change moves the result a lot. Confirming something that moves a lot does not need many cases. Turn it around and, once the system matures and a single improvement gets small, that is when the set has to grow.

There is one way to tell a good case. *Would two people, shown the same artifact, separately reach the same verdict?* If they split, the person judging is not the bad part. The criterion is underwritten.

Cases come from the bug list and the support log. They are already written down as failures.

When you have to build them by hand there is a trick. *Fix the answer first and build the question backwards.* One set for evaluating browsing agents was made that way. Start from a fact you know for certain, then hang conditions on it until finding that fact by search is hard (Wei et al., 2025). What comes out is a question whose answer is short and easy to check and hard to reach.

Do not gather only cases where the thing that should happen happens. Put in requests that should be refused, questions with no answer, and questions about facts the material does not hold.

Once the set exists it has to be kept. Every time a case changes, record what changed and why. Without that you cannot tell whether last month’s success rate and this month’s came from the same test.

Who judges

Only three things can judge. Code compares, a model reads, or a person looks.

How the three get tied together is where the practice sits.

When a model does the judging, break the rubric into items. Do not ask whether the answer is good. Ask separately whether the facts are right, whether anything is missing, whether the

instruction was followed. And leave a road to answer that it does not know. Leave that closed and what it does not know gets stamped pass or fail anyway.

When you bring model judging in, check how well it agrees with a person's verdict and write that number down. Do not check once and stop. Change the criteria and the agreement changes too.

Splitting the judging side from the side being evaluated often gets carried over as “use a different model.” That alone is not enough. If the two hold a wrong understanding together, the observation that the verdicts agree is not evidence that the verdict is right. It can be evidence that the two inherited the same thing. *There is no way to break that loop except leaving a sample review by a person.*

As criteria ripen, settle them down into code. Swap what a model was reading for a machine check and the cost falls and the wavering of the verdict falls with it.

You are not picking one of the three. Hanging several on one task is common. For a task that edits code, whether the tests passed is code's to see, whether the edited code is worth reading is a model's to see with a rubric in hand, and only the ones where the two disagree go to a person. *The arrangement that cuts how much a person reads works on the evaluating side too.*

How many times you run

Not once. Run several times, then pick between two things to count. Does one success out of several count as a pass, or must every one succeed?

Run once and the two are the same number. Run ten times and they tell opposite stories. The first climbs toward 100 and the second falls toward 0.

Which one to look at depends on what you are building. If it is a tool where a person receives several drafts and picks one, one success out of several earns its keep. If it is a system answering customer queries on its own, every one has to succeed. *Report the first number for the second kind of system and it looks far better than it is.*

The names are settled. The share of runs where at least one of k attempts succeeds is $\text{pass}@k$, and the share where all k succeed is pass^k (Anthropic, 2026c).

In numbers it looks like this. An agent that succeeds three times in four, run three times in a row, gets all three about 42 times in a hundred. A reported success rate of 75 and an observation that four of ten runs go the distance are the same system.

Running several times has another use. A task that failed a hundred times out of a hundred is usually not a task the agent cannot do. It is a broken task. The grading code is wrong, the problem statement is ambiguous, or the thing was written unsolvable from the start.

Once you run several times there is one thing to keep. *Every trial starts from a clean state.* If a file the previous trial made or a database it edited is still there for the next, failures that should have been unrelated move together. Then an observation that three of ten failed cannot tell you whether that was three independent failures or one failure spreading.

When you say it got better

One number says nothing. Say a success rate of 72 is reported. Is that good? With nothing to compare against you cannot answer.

Three things to compare against. Yourself before the change, another arrangement, and a line drawn in advance.

Start with the line. It does not come out of the evaluation. It comes from how many of how many a person gets right doing this work, how much time a person spends on it now, and below what level you have decided not to use it at all. You have to step outside the evaluation once and come back to draw it. A line drawn without stepping out lands a little above today's score, and *a line like that changes no decision even when you cross it*.

Another arrangement is often missing. A system with compaction on produces only compacted runs, and a divided system produces only divided runs. What it would have been without compaction is imagination rather than observation, and turning imagination into observation means actually building that arrangement and running it once more.

So bringing a device in should carry a question. *Can you produce a run with this device off?* A device brought in so that it cannot be turned off becomes, from that moment, a part that is never inspected. The effort of building the switch is smallest when the device goes in, and the moment you need the switch usually comes later. With the switch, the argument ends in a day of experiment. Without it, it ends in a difference of opinion.

Even with something to compare against, one thing remains. *Two numbers differ only if the difference is larger than the wavering*. If you use model judging and evaluating the same artifact twice gives a gap of 3, a report that the score rose by 2 says nothing. The order is fixed. Evaluate the judge first and the system second.

Chapter 5 had already come this far. When you change a prompt, judge range against range rather than point against point. Shake the format across several variants and run each variant several times and several scores come out, and the low end and the high end of those are the range. You say it got better only when the lowest score after the change clears the highest score before it.

This is not only a prompt matter. Touching a tool, swapping retrieval out, moving up a model: the same question comes.

Two questions were deferred there. How many variants and how many tasks to draw the range with is one, and what to do when two ranges overlap is the other.

The first has no settled answer, only a condition. Cost multiplies with the variants and trials you add, so you cannot pay it for every sentence you edit. Three occasions are worth paying it properly: when you first stand the thing up, when you change the structure, and when you move up a model. The third is forgotten most often.

The second has an answer. Overlapping means you do not know yet, so there are two roads. Add cases or trials until the range narrows, or ship the change without calling it an improvement. *Writing that you shipped it because the code got simpler is different from writing that you shipped it because performance improved*. Written the second way, that record becomes the ground under a decision months later.

This job is not done once. In practice, evaluation gets hung like test code. It runs automatically whenever a prompt changes, a tool is added, or a model moves up.

18.5 Real Cases

Published benchmarks show how the four are actually done. Three are picked from the examples Anthropic (2026c) gives by category.

An agent that edits code

SWE-bench hands over an issue from a real GitHub repository. The agent searches the repository, finds what to fix, and edits the code.

Code judges. Tests run against the edited repository and two things get watched. Tests that failed before have to pass, and tests that passed before have to keep passing. The first says it was fixed and the second says nothing else broke.

The Verified in the name means a human-validated subset. The original set held tasks whose problem statement was too thin to solve and tests written too narrowly. The broken tasks of the previous section were a real problem here (OpenAI, 2024).

What it catches and what it misses are clear. Whether the fix happened, it catches for certain. Whether a person can read the edited code, how many times it wandered, how many tokens it took, it does not.

An agent that handles customers

τ -bench imitates customer service for a retailer and an airline. A model plays the customer instead of a person, and the agent uses tools like order lookup and reservation changes while keeping that company's policy.

The verdict comes after the conversation ends. It looks at the state of the database and compares it against a goal state written in advance. Whether the agent said "I have taken care of that" is not looked at. Whether the order was actually cancelled is.

The counting where every run has to succeed came from here, and there was a reason. Customer service is not work where one success out of several will do. Keep the policy nine times in ten and break it once and that once is the accident.

The reported numbers point the same way. Even the best agent tested then solved under half the tasks, and in the retail domain the share succeeding eight times in a row fell below 25 (Yao et al., 2025).

What it misses is the customer. A model plays the customer, so what it says differs from what a real one says.

An agent that works in a terminal

Terminal-Bench puts the agent at a shell. The tasks build code, configure servers, train small models, and repair broken systems.

The verdict comes from inspecting the environment that is left. Each task gets a fresh container, and the agent drives the shell, reading output and settling the next command. When it stops, a test script looks inside the environment. The conversation record is not looked at.

Starting from a clean state, from the previous section, looks like this in practice. It seems heavy because it is somebody else's benchmark, but our own evaluation has to do the same thing. Rolling data back or making a fresh account, only the shape differs.

What to take from the three

Put the three side by side and one thing is the same. *All three look at what was left rather than at what was said.* Did the tests pass, what state is the database in, how do the files stand.

And all three hand the verdict to code. That way the same artifact always gets the same verdict, and when the score wavers you can say the thing that wavered was the agent.

You cannot take the three and use them. They are somebody else's tasks. What these benchmarks teach is not a score but a shape. Look at what was left, judge with code, run several times, start each trial from a clean state. The cases for our own task still come from our own bug list.

Something changes on the way over too. A benchmark has to be fixed to allow comparison, while our set grows as failures pile up. In a growing set this month's success rate and last month's are not results of the same test, so recording every change to a case matters more on our side than on a benchmark's.

And none of the three counts how much a person fixed. A benchmark has to run without a person, so it cannot. Which means half of what Section 18.2 agreed to call a good agent cannot be confirmed from published scores.

That is why a model chosen for high benchmark scores can still leave the people using it frustrated. That half is ours to count in our own system.

18.6 Evaluation Done Wrong

Evaluation should mostly be built. So this section is not do not build it. It sorts out *the evaluations you should not build*.

1. **Counting only what is easy to count.** Call a falling format-violation rate a success and the system gets tuned toward not violating the format. That is an ability to match a format, not an ability to solve the task. Measure numbers like this but do not call them success. The system gets tuned toward whatever carries a name.
2. **Cases that differ from real use.** Hand-built cases have no typos and ask one question in one sentence. What actually arrives asks three things at once and half of it does not hang together. A number that rose on the first set promises nothing about the second kind of question. Building cases by hand is not itself the mistake. Not writing down the ratio of built to real-world is.
3. **The judging side is the same as the making side.** Give the same model the same prompt and have it grade its own answer and the two hold a wrong understanding together. Failures that understanding produces do not show in the success rate.
4. **Running once and calling it better.** The verdict wavers and the run wavers. A difference smaller than the two wavers combined is not an improvement. It is that day's luck. Of the five this is the hardest to notice. The number went up and the arithmetic was not wrong.
5. **Fixing to fit the test.** An evaluation whose pass rate has reached 100 still tells you where things broke but no longer tells you where things improved. Worse is rewriting tasks narrowly to raise the pass rate. The number that rises then is real, and the reason it rose is not that the system got better.

Which one you have fallen into usually shows in a meeting rather than in a number. When the metric rose and the complaints from users did not change, or when a decision grounded in the metric surprises nobody. The first says what you are counting differs from real use, and the second says the metric is already only confirming what you knew.

Failures that never reach the list

One problem sits under all five. *Before a failure can become a case it has to be observed as a failure.*

But several of the failures this book covered end wearing the same face as a successful run. A chunk wrongly dropped, an instruction that never got loaded, an answer right in form and wrong in content, a source that vanished before it reached the index, a wrong memory. None of the five raises an error. All five end in the shape of completion.

Nobody files a report, so it does not reach the bug list, and not reaching the list it never becomes a case.

So the eval set is tilted. Not from lazy collection but because only the loud failures reach the list. A quiet failure loses twice. Once by not being caught when it happens, and again by never becoming a case, so that it will not be caught in the future either.

The remedies are already scattered through the earlier chapters. Leave a road to the source instead of the dropped chunk, have retrieval return what it brought and why, put a field in the schema for saying it does not know, compare the extraction against the original, verify memory again by sample.

Then they were remedies for cutting failures. Through this chapter's eyes *they are also devices for manufacturing cases.*

What a metric makes

Once a metric is standing it becomes the team's shared language. Before, improvement claims were argued with anecdotes and impressions; after, they are argued on those items. Mostly a gain.

There is one cost. *A claim on an axis not in the items becomes a claim with no grounds.* You can say it, but you cannot show it as a number, and a claim that cannot be shown as a number loses. And what is easiest to leave outside the items is exactly the quiet failures of the previous passage.

The remedy is simple. Keep, beside the list of items, a list of *what this metric does not count*. The right column of Table 18.1 is the draft of that list.

One thing runs through all five as well. *An evaluation that pretends to have defined a success it cannot define is worse than no evaluation.* No evaluation puts decisions on guesswork, and everybody knows guesswork is what it is. A pretending evaluation puts decisions on a number, and that number does not say what it failed to count.

Building it late

Building the evaluation late belongs here too.

Build it early and it is easy. What you agreed to build is still there as sentences, and those sentences become the tasks. Build it late and it is hard. You end up assembling pass criteria backwards while looking at a system that already runs, and criteria written that way fit today's system exactly. *Fitting exactly is why they cannot pick out tomorrow's improvement.*

The way of fixing things while there is no evaluation is settled too. Wait for a complaint, run it again by hand, fix it, and hope nothing else broke. The first three are workable and the fourth is the problem. Hoping is not a method.

So it is better not to spend three months building a perfect twenty. Start this week with a rough twenty and add one every time a failure arrives.

18.7 Summary

In one sentence. A good agent is not one that does everything alone but one that cuts the effort a person spends confirming, and defining it that way puts into the evaluation items, beside what percentage came back right, how much the person had to fix.

What is now settled.

- As long as a model draws its answer by probability, the last approval stays on the person's side. What disappears as models improve is not the person's confirming but the amount the person has to confirm (Section 18.2).
- Evaluation looks at what was left, not at what was said. Not the sentence claiming a booking but the reservation record. And the path taken does not get graded (Section 18.3).
- There are five evaluation items and no single one shows the whole thing (Table 18.1). The last goes missing most often, and dropping it means not counting the thing you agreed to call good.
- Cases come from real failures. Twenty to fifty is enough to start. A good case is one where two people separately reach the same verdict (Section 18.4).
- There are only three judges. Code is fast and always the same but cannot see meaning, a model sees meaning but its verdict wavers, and a person is trustworthy but slow (Table 18.2). A design that makes one carry everything loses by itself.
- Run several times. Settle first whether one success out of several will do or every one has to succeed. A three-in-four success rate is about 42 for three in a row.
- Saying it got better needs something to compare against. So bring devices in with a switch. A device that cannot be turned off becomes a part that is never inspected.
- The items become the names of the failures. A failure not on the items never reaches the list of things to fix, so keep beside the list of metrics a list of what they do not count (Section 18.6).

What is still open.

- There is no right answer for how closely a model's verdict has to match a person's.
- Which is worth more, drawing cases from failures or making quiet failures loud, is not known.
- How many switched devices to keep, and how often to bring them back and run them, is not settled either.
- When to grow the set of cases, and how to compare this month's number with last month's as the set grows, goes to Chapter 19.

Try this today. Open the dashboard or the eval script you look at now and *write down* what the items are. Mark which of the five in Table 18.1 are missing (Section 18.3). If there is no item counting how much a person fixed, open last week's twenty runs and *count* how many a person took as they stood (Section 18.2). Last, *check* how many runs the success rate you report came from.

18.8 Exercises

1. Take ten recent runs and sort them into taken as they stood, edited and used, and thrown away whole. Which item counts the third one right now?
2. List the devices you have switched on that cannot be switched off. For each, compare the effort of building a switch now against the effort six months from now.
3. Settle whether your own task is one where one success out of several will do or one where every run has to succeed. If the second, recalculate the number you report now as the share succeeding three times in a row.
4. Read the right column of Table 18.1 and pick a miss that has actually caused trouble in your system. Does catching it mean adding an item or keeping more records?

18.9 On a Second Reading

- Read Chapter 19 and come back to the tilted set in Section 18.6, and you see that tuning long against a tilted set freezes the system into the shape of that tilt. This chapter wrote down the tilt and that one lays time on top of it.
- Read Chapter 20 and come back to Table 18.1, and three of the five items cannot be counted without a record of the run. Settling what to keep comes before settling what to count.
- Read Chapter 22 and come back to Section 18.6, and one more line joins the list of quiet failures. A run where an injection succeeded also raises no error and ends in the shape of completion.

18.10 Further Reading

Anthropic (2026c) is the source this chapter leans on. Four things came from it. Grade what was left in the environment rather than what was said, grade the outcome rather than the path, the test that a good task is one where two people separately reach the same verdict, and the recommendation that twenty to fifty tasks drawn from real failures is a great start, along with its reason.

The table sorting the three judges into what they are good and bad at, and reading the small-set recommendation as a consequence of early improvements being large, are this book's own.

The benchmarks in Section 18.5 are all public. Better to go to them for the shape than for the scoreboard. Open one task, read what it hands over and what it inspects, and this chapter's four decisions are all visible in it. Yao et al. (2025) is where the pass^k counting and the retail numbers come from, and OpenAI (2024) records what the human validation removed. Wei et al. (2025) is where building a question backwards from a known fact comes from.

Designing Evaluation Metrics

Somebody who has done machine learning for years joins an agent team and, in the first meeting, asks the same thing. What is the accuracy?

The answer does not come easily. You can say the success rate is 68. But nobody outside that team knows what the 68 counted, and it cannot be set beside the 71 the team next door reported. The two numbers did not count the same thing.

Chapter 18 set up how to run an evaluation. What counts as a case, who judges, how many times you run, when you say it got better. That procedure needs items before it turns. This chapter is about settling what those items are.

19.1 There Is No Standard Metric

Machine learning has metrics with settled names. Say you built a classifier that sorts spam out. The share of everything it got right is accuracy, the share of what it called spam that really was spam is precision, and how many of the actual spam it caught is recall. The arithmetic is the same everywhere.

So two papers' numbers can be set side by side. If accuracy came out 91 and 93 on the same material, the second did better, and that verdict needs no explanation. *What a settled metric does above all is let two numbers stand side by side.*

Agents have nothing like it yet. There are published benchmarks. Chapter 18 opened three. One hands over a repository issue and runs tests against the edited code, one puts a model in the customer's seat and compares the state of the database after the conversation, one drives a shell directly and inspects the environment that is left. All three are well built and all three are somebody else's tasks. What was there to take was the shape, not the score.

There are three reasons nothing settled exists.

First, the field is still growing. What ought to be counted has not found its place.

Second, agents are mostly built to help with a particular job. They are not things that take a person's work over whole but things that work alongside. Chapter 18 defined a good agent that way. Not one that does everything alone but one that cuts the effort a person spends confirming.

Third, and so, doing well means something different for every job.

Think of an agent that helps review contracts. Miss one clause that departs from the standard and that case is a failure. A knack for smooth sentences counts for nothing here. An agent that edits code is the opposite. Something can be missing and the case is still a success if the tests pass, and what was missing gets caught in the next run. *In the first, one omission is an accident; in the second, one omission is the next turn.*

You cannot evaluate two systems like that with the same ruler. So there is nothing for it but to design the metric yourself.

Calling it design has a reason. If you could pick one off a list it would already be a settled metric, and there is none, so it has to be made. Making it has an order.

19.2 Designing a Metric

Settle three things in turn. Write down what to count, confirm that it actually comes out, and settle what to do with the number that comes out.

Where you start

Not from the task but from the person doing that work now.

Say you are bringing an agent into a team that reviews contracts. Before writing a metric, sit beside the reviewer for a day and watch. It looks like this. A contract arrives and they read the clauses from the top. They mark the places that depart from our standard. Beside each mark they write whether to accept it or rewrite it. One contract takes about forty minutes and thirty of those go on finding the departures.

Three things to ask here. How does a person do this work now? What counts as done well and what counts as done wrong? Who judges that?

The answer to the second comes out already split. Missing a clause that departs from the standard is wrong. Marking a clause that does not depart is also wrong. *But the two do not cost the same.* The first sends the contract out and becomes an accident. The second costs the reviewer a few minutes.

The answer to the third is that reviewer. Nobody else can judge. So that person's time becomes this evaluation's budget. How many cases a day they can look at is where the size of the eval set comes from.

Then you carry this over into something countable. Three come out. The share of clauses the reviewer marked that the agent also marked, the share of what the agent marked that the reviewer struck out, and the time the reviewer spent per case.

Something is lost in the carrying. The judgment of whether to accept or rewrite is in none of the three. Find every clause and still get that judgment wrong and the case is a failure, and these three count it as a pass. *Writing down what was lost is part of the design.* Leave it unwritten and months later these three read as the whole of this work.

Written down it looks like this.

Missed-clause rate

```
Counts:    of the clauses the reviewer marked as departing from
           the standard, the share the agent failed to mark
Comes from: comparing the two marked copies / code judges /
           no human hand
Set:       February edition / 40 contracts / two reviewers marked
           separately and cases where they split were removed
Trials:    three per case / counted missed if all three miss
Spread:    running the same set twice moved it by up to 1.5
Decision it changes: over 4 we roll back / under 2 we put the
           agent first in the review order
Does not count: the judgment of whether to accept or rewrite
Discard when: the reviewer changes / the standard text is revised
```

Six of the eight lines get filled by the rest of this section.

Table 19.1: The five places a number spreads when the same metric is put to the same system twice. The right column is the point. Some can be narrowed and some cannot, and the first and the fifth narrow differently from the middle three.

Place	What changes	Can it be narrowed
The choice of action	A different action comes out of the same context. The path and the step count both change	Only as far as repeating and averaging. It cannot be removed. It is a property of the system you are evaluating
The form of the prompt	Notation that means the same thing, and the order of examples, change (Section 5.3)	Measure across several variants and a range comes out instead of a point. Runs multiply by the number of variants
The verdict	The same artifact gets a different verdict. A model's verdict is a draw too (Section 18.4)	As far as checking the verdict first and settling ripened criteria down into code. It stays where a person judges
The make-up of the set	Which contracts are in those forty. Every task scores differently	As far as growing the set. Cost per case multiplies by the growth
The run environment	State left between runs. Leftover files, cached data, exhausted resources	Nothing but starting every run from the same state. Repeating does not narrow it

How you measure

With the metric written, confirm that it actually comes out. Three things to ask.

Where does the value come from? The first line of that card needs the reviewer's marked copy. In a team that does not keep marked copies, this is not a metric but a plan.

Does it need a human hand, or does it come out by itself? The comparison is code's. The time spent per case needs a record of when the reviewer opened and closed it. Without that record the item becomes one you ask a person about every month, and *an item you ask about disappears in two months*.

How often can you measure it? Write down separately what you can run weekly and what you can run quarterly. Leave them together and both get cut when the budget tightens.

And one more. Put the same metric to the same system twice and two numbers come out. The system did not change in between, so the gap came from the measuring itself, and a metric that moves 1.5 across two measurements cannot confirm a change of 1. The spread comes from five places.

Look at the fifth on its own. Anthropic (2026c) writes that state left between runs can cause correlated failures from infrastructure flakiness rather than from agent performance. The word carrying the weight is correlated. If a leftover file sits there identically for every trial in a round, ten runs all go wrong in the same direction and the average inherits that skew whole. *What repetition narrows is only the scatter*.

So there is one thing to ask before measuring starts. Does every run start from the same state? If you cannot answer, no difference out of that set can be called the system's.

Measure the spread once and write it beside the metric. That is the sixth line of the card. Without that number every change reads as an improvement, and not only that. A number that fell because of the scatter reads as a regression, and then a good change gets rolled back.

What you do with the number

The reason for making a metric is to change a decision. So the last thing to settle is which decision it changes.

Then why keep a metric that changes no decision? Only two answers. Either the decision has not been settled yet, or it is there to show somebody. If the first, settle it now; if the second, that is not a metric but a line in a report. Take it off the dashboard. *Leave it there and the next person will take it for a metric and aim at it.*

Write down when you discard the metric too. When the value sticks at the ceiling and stops moving, when the job changes, when the person judging changes.

When to grow the set of cases gets answered here as well. You grow it when the system got better, not when the set got worse. Early on every change moves the result a lot, and confirming something that moves a lot is comfortable with forty cases. But as the system matures a single improvement gets small. From the moment that improvement is smaller than the spread across two runs, this set says nothing. More mature agents may need larger and harder evals to detect smaller effects, Anthropic (2026c) writes. *The signal to grow is not the score but the relation between the improvement and the spread.*

Growing the set brings the next question. How do you compare this month's number with last month's? A 68 from a set that gained twenty cases and last month's 71 from forty are not results of the same test. And in the report the two numbers sit in the same column in the same shape.

The method is settled. Number the set and do not drop new cases in as they arrive. Pile them up separately and put them in together when the number goes up. In the week the number goes up, run both editions. The old edition's number joins last month and the new edition's number opens the next. *That week is the seam.* Without seams, months of records look like one continuous line while really they are numbers from six different tests standing next to each other.

19.3 Metrics in the Wild

Three published records are worth opening. All three wrote their metrics fresh for their own job, and all three wrote down why they wrote them that way.

A model that calls APIs

It takes a sentence in ordinary language and writes out which API to call with which arguments. There are about 1,600 candidates (Patil et al., 2024). Chapter 7 took two failures from this source: that the clues in a description get buried as tools multiply, and that arguments do not come out exact. What to look at here is what those failures were counted with.

The metric starts by throwing string comparison away. The same call can be written several ways, so matching letters counts right things as wrong. Instead the generated code gets parsed into a syntax tree, and the sub-tree whose root is the API being called is found and matched.

Arguments are matched too, but which arguments to match is written down in advance per API. Arguments with defaults are left out (Patil et al., 2024).

And the failures were split in two. Put out a sub-tree that matches no API and that is inventing a tool that does not exist; call an API that does exist incorrectly and that is an error (Patil et al., 2024). *The split earns its keep because what to fix differs*. Invented means the tool list did not reach the model properly. Called wrong means the description or the argument spec is thin.

The numbers show why they split. Asked with no examples, one commercial model invented at 78.65 on one source and 36.55 on another, while this paper’s model sat at 10.95 and 6.98 in the same places (Patil et al., 2024). Folded into one success rate, that difference is invisible.

What it cannot see is clear too. Whether the API it called actually did what the person wanted is not in this number. And getting an argument wrong that you decided not to match counts as a pass.

An agent that does the research for you

It takes a question and searches the web and internal material for an answer with sources. One plans and several split the searching (Anthropic, 2025e). Chapter 16 took the shape of the dividing from this source.

What this team wrote down is a rubric. They did not ask once whether the answer was good; they broke it into five. Do the claims match the sources, do the cited sources match the claims, were all the requested aspects covered, did it use primary sources over lower-quality secondary ones, and was the tool use efficient (Anthropic, 2025e).

A model judges. A single call with a single prompt, outputting a score from 0.0 to 1.0 together with a pass-fail grade, came out the most consistent and the most aligned with human judgment. And the same source says to look at whether the correct final state was reached rather than whether a set process was followed. In a system where several run divided, the path differs every time for the same question.

It started at about twenty. Around twenty queries representing real usage patterns is what they opened with (Anthropic, 2025e). The same scale as Chapter 18’s twenty to fifty drawn from real failures.

But it did not end with the rubric. People used it themselves and found what the rubric was missing, and one thing that came out was a habit of choosing search-optimized content farms over authoritative but lower-ranked sources (Anthropic, 2025e).

One thing is in none of the five items. What the person who got the answer did with it.

A system that finds answers in documents

It cuts internal documents into chunks, and when a question arrives it finds chunks, loads them into the model, and builds an answer. Chapters 11 and 12 called on this source several times.

What to look at here is that this team did not grade the final answer. Instead they counted the share of times the needed chunk was not among the twenty retrieved. One minus the share of times the answer chunk was inside the top twenty, and that was the metric they used (Anthropic, 2024b).

Why they chose it comes out of thinking about where to fix. When the final answer is wrong, whether retrieval failed to bring it or the model failed to read what was loaded does not separate

from looking at the answer. *Count retrieval on its own and where to fix is written into that one number.*

The number moved like this. The miss rate started at 5.7, went to 3.7 when a context sentence was attached to each chunk, to 2.9 when lexical search was overlapped, and to 1.9 when reranking was added (Anthropic, 2024b).

What it cannot see is whether the chunk was actually used in the answer. The miss rate can be 0 and the answer wrong. This metric was made having decided not to count that.

What to take from the three

Put the three side by side and three of the same moves show.

- **They rebuilt the comparison to fit the task.** Structure got matched instead of letters, and where no single right answer exists a rubric was used.
- **They asked in parts rather than in one lump.** Invented was split from called wrong, and whether the answer was good was split into five. Every place they split has a different thing to fix.
- **They counted a place they could fix instead of the final result.** The one that graded retrieval rather than the answer did that.

One more. None of the three called its metric accuracy. They wrote the name in the words of their own job.

None of the three wrote its metric in one line either. They wrote the definition, why they chose it, and what gets compared. That is why the card in the previous section is eight lines. *A metric you can write in one line is usually somebody else's metric.*

Writing down what the three metrics cannot see is this book's reading, not a sentence written in those sources. What to open them for is the definition and the reason for the choice.

19.4 Metric Design Done Wrong

Metrics should mostly be built. So this section is not do not build them. It is a list of how not to build them. All five have been lived through in machine learning already.

1. **A metric you cannot measure again.** If a second measurement cannot be set against the first, that metric gets used once and dropped. It goes that way when cases went into the set with no number attached, when the judging prompt was touched and how it was touched went unrecorded, when trials went from three to one and the report does not say so. Two things follow. Last month's number and this month's sit in the same column and nobody knows they are results of different tests. And the metric gets quietly abandoned, because every attempt to use it means building it from scratch. The remedy is simple. Beside the metric, write what is needed to rebuild it. The set number, the judge number, and the trial count usually do it.
2. **Growing the metrics with no baseline.** Whether a success rate of 72 is a good number cannot be answered from that number. Chapter 18 named three things to compare against. Yourself before the change, another arrangement running the same task, and a line drawn in advance. What to look at here is why the baseline keeps going missing. Adding one more metric costs a few lines of code, and making one baseline means a person doing this work by hand and counting how many they get right, or running an arrangement with

Table 19.2: Where and when the gain and the loss arrive for four techniques earlier chapters recommended. For all four the gain gets stamped on that step’s metric at once and the loss gets stamped scattered several tasks later. The cost in the right column usually does not arrive wearing the technique’s name.

Technique	Where and when the gain lands	Where and when the loss lands
Compaction	That step’s token count. At once	How often things get looked up again, and the success rate. Scattered from a few steps later to a few tasks later
Progressive disclosure	The stable prefix’s token count. At once, and for certain	The success rate. Only in runs where what was left out is actually missed
Code execution	Tokens on the context side. At once	Cost on the execution environment side, and difficulty pinning things down when they go wrong. Late
Dividing and walling off	The bill. At once	Only guessed at. There is no undivided run to compare with

the device off one more time. The two efforts differ, so teams grow the metrics and defer the baseline. *That is how a dashboard ends up with twelve numbers and not one of them saying what it is compared against.* So if there is no plan to build a baseline, better not to build the metric. The cheapest baseline available is the number for a person doing this work.

3. **Folding metrics of different things into one.** Machine learning has an old example. The degree of not missing and the degree of not calling things falsely, folded into one number, is the F1 score. It is their harmonic mean, and a harmonic mean is pulled toward the smaller of the two. It is widely used because it makes ranking easy. Folding loses something. *When two failures cost differently, the folded number cannot change a decision.* In contract review one missed clause is a contract going out and one false mark is a few of the reviewer’s minutes. From a report that a half-and-half mix of the two went from 0.80 to 0.83, you cannot read whether accidents went up or down. Before folding, write down what the two failures cost. Fold if they are the same and keep them apart if they differ. The place that needs one number is usually a report, not a decision.
4. **Standing in something easy to measure for something hard.** When what you want to evaluate cannot be measured directly, or measuring it directly takes too many people, you measure something else believed to move with it. That is a proxy. Most of what this book told you to measure was a proxy too. A similarity score is a proxy for relatedness, a format-violation rate is a proxy for using tools right, and a success rate on the eval set is a proxy for the success rate in the field. Using a proxy is not itself the mistake. Cases where a direct measurement exists and a proxy gets used anyway are rare. The mistake happens when only one of two things gets written down. What it is a proxy for usually gets written, and *where the belief that the two move together came from almost never does.* And a proxy brings a problem of timing with it. The value that is easy to measure usually arrives at once, and the hard value it stands in for usually arrives late.

5. **The metric becoming the goal.** The moment a value is made the goal it stops standing in for what it stood in for. The observation comes from economics and it happens daily in machine learning. In agents it shows up like this. Call a falling format-violation rate a success and the system gets tuned toward not violating the format. That is an ability to match a format, not an ability to solve the task. Chapter 18 already wrote it down as counting only what is easy to count. What to lay on top here is time. *It is not only the system that gets tuned.* Once a metric is standing it becomes the team's shared language, and improvement meetings open only on those items. A claim about something not in the items can be said but not shown as a number, and a claim that cannot be shown as a number loses. In the room it looks like this. Somebody says to fix retrieval, and the grounds are the three cases last week where the answer looked wrong. Somebody else asks what that is as a metric. There is no answer. The meeting turns back to adjusting the prompt, because there are numbers on that side. *Not because retrieval is not the problem but because retrieval cannot be pointed at.* A few quarters of that and the system freezes into the shape of the metric. If the metric was tilted, into the shape of that tilt.

The last one has three remedies. Write down separately which metrics change a decision and which are only watched. Only the first kind gets aimed at. Keep beside the list of metrics a list of what they do not count. And leave a place where a person reads a sample. That is the only road to seeing what is not in the metrics.

The five do not arrive separately. With no baseline you cannot tell what an improvement is, so the team goes looking for a number that rises, and a rising number is usually on the easy-to-measure side. Start aiming at the proxy you picked that way and the system freezes into that shape, and while it freezes the set keeps changing so the road back to last quarter disappears too. The second and the fourth and the fifth and the first are one loop. *So fixing one usually catches you on another.*

Put the five in the order of the design and their places are settled. The first and the second catch you when you write the metric, the third and the fourth when you carry it over into something countable, and the fifth only after you have started using it. *The first four take a day to fix and the fifth takes quarters.*

Gather all five on one card and the contract metric of the earlier section looks like this.

Contract review quality score

Counts: an overall quality score converted to 0 to 100
Comes from: collected automatically every Tuesday
Last week: 81
This week: 84

All five are in those four lines. What was converted and how is missing, so it cannot be rebuilt; there is nothing to say whether 84 is a good number; the missed clause and the false mark are folded into one score; things easy to count were mixed and converted instead of counting quality directly; and this number rising is written down as the goal itself.

The person who made this card was not lazy, though. Writing the eight lines of the earlier section takes half a day beside the reviewer, and these four lines take an hour. *The difference is not diligence but what they judged could be deferred.* The six deferred lines do not disappear. They come back a few quarters later, in the shape of nobody being able to say what the decisions grounded in this number were standing on.

19.5 Summary

In one sentence. Agents have no settled metric and doing well means something different for every job, so a metric is not a thing you pick but a thing you design, and designing means taking down from the person doing the work what counts as wrong, carrying it over into something countable, and then writing down what the carrying lost.

What is now settled.

- Machine learning's metrics have the same arithmetic everywhere, so two papers' numbers can stand side by side. Agents have nothing like it, half because the field is still growing and half because agents get built differently for every job (Section 19.1).
- Design starts from the person doing the work, not from the task. Take down from that person what counts as wrong and who judges, and the size of the set comes out of the judging person's time (Section 19.2).
- Carrying it over into something countable loses something. Leave the loss unwritten beside the metric and months later the remaining items read as the whole of the work.
- Put the same metric to the same system twice and two numbers come out, and the spread comes from five places (Table 19.1). The first is a property of the system and cannot be removed, and the fifth does not narrow with repetition, leaving no road but starting every run from the same state.
- A metric that changes no decision is not a metric. It stands once what it takes to trigger what is written, and when to discard it is written alongside.
- The signal to grow the set is not the score but the moment an improvement gets smaller than the spread across two runs. When you grow it, raise the set's number and run both editions that week to make a seam.
- Three published records show the same moves. Rebuild the comparison to fit the task, ask in parts rather than in one lump, and count a place you can fix instead of the final result (Section 19.3).
- None of the three called its metric accuracy. Name and definition were written in the words of their own job, and a metric you can write in one line is usually somebody else's.
- When two failures cost differently, one folded number cannot change a decision. And when the gain arrives at once and the loss arrives late, every change gets judged in a state where it has only the gain (Table 19.2).
- Stand a metric up and both the system and the people freeze into its shape. That is why you separate the metrics that change a decision from the ones that are only watched (Section 19.4).
- The five mistakes are one loop, so fixing one usually catches you on another. Rather than filling six lines at once, fill the one line naming the decision it changes.

What is still open.

- There is no settled form for writing down what the carrying lost. This chapter's card

is one example.

- How often the spread across two runs has to be measured again is not known.
- This book knows no general way to observe a late-arriving loss earlier.
- How many watched-only metrics can be kept is not settled.

Try this today. Pick one metric the team looks at and *write down* three lines. Which decision does this number change, did it come from the same set as last month's number, and what does it not count (Section 19.2). If the second line will not fill, number the set and start *piling* new cases separately from today (Section 19.2). If the first line will not fill, *take* that metric off the dashboard (Section 19.4).

19.6 Exercises

1. Pick a metric you look at and ask the person who uses it directly. At what value do you do what? If no answer comes back, what is that metric doing right now?
2. Pick two failures in your own system and write down what they cost. Which hurts more, the miss or the false call, and by how many times? Are those two folded into one number now?
3. Measure how far the same evaluation spreads across two runs. Of the changes you reported as improvements last quarter, how many were smaller than that?
4. Pick a metric you use that is a proxy, and write down what it is a proxy for and where the belief that the two move together came from. If it is not written down, whose memory is it in?

19.7 On a Second Reading

- Come back to Chapter 18's five evaluation items and you see they were not a list of items but a starting point for design. The right column recording the misses becomes, here, a line you write alongside the metric.
- Read Chapter 20 and come back to Table 19.1, and three of the five places cannot be told apart without a record of the run. Measuring the spread stands on what was kept too.

19.8 Further Reading

This chapter called on four sources. Three are the records in Section 19.3 and one is the same as the previous chapter's.

Two things came from Patil et al. (2024). Matching sub-trees of a syntax tree instead of matching letters, and the split between inventing a tool that does not exist and calling one that does incorrectly. Chapter 7 took the list of failures from the same source and this chapter took the way of counting them.

From Anthropic (2025e) came the rubric broken into five items, and the observation that a single call outputting a score together with a pass-fail grade agreed best with human judgment.

From Anthropic (2024b) came the metric that counted retrieval on its own instead of the final answer, and the four steps that number moved through. Chapters 11 and 12 took the methods from the same source; what came from it here is what got counted.

Two things came from Anthropic (2026c). That state left between runs causes correlated failures, and that a maturing system may need larger and harder evals to detect smaller effects.

Tracing and Observability

The question that arrives on a Monday morning is usually the same. Why did this agent answer that way last Thursday?

Answering means opening that Thursday. But the run is over, and the context the model saw at each step and what the tools returned existed only inside that run. What can be opened now is only what was kept because somebody decided to keep it. And what to keep was settled before Thursday. There are runs where plenty was kept and the one thing you want to know is still missing.

So this chapter starts by separating two jobs. Keeping the whole road one run travelled is one, and what you can learn from what was kept is the other. The first is tracing and the second is observability. The two words usually travel together, and the place they come apart in practice is exactly between them. Section 20.3 is that place and it is the center of this chapter.

20.1 Why You Have to Keep It

Questions several earlier chapters deferred all arrive here at once.

Three of the five evaluation items Chapter 18 set up cannot be counted after a run ends. Was the tool chosen, how much did it take, did a person fix it (Table 18.1). Chapter 12 stretched finding into six or seven stages, and telling which stage missed needs the intermediate result of each stage to still be there (Section 12.5). After the work is divided in Chapter 17 you have to ask which side slipped, and all that comes up is one last message (Section 17.2). When a line of memory in Chapter 10 turns out to be wrong you have to walk back to the run that wrote it (Section 10.4). The four ask for the same thing. *One past run has to be openable again.*

So look at the logs, then. Logs are usually there already.

One request turns twenty steps. Each step calls the model once and about half the steps call a tool once or twice, so by lines it is about sixty. And those sixty lines pile up in time order mixed with the lines of two hundred other requests running at the same moment.

Three things cannot be asked of a pile like that. Is this line and that line the same request? If the same request, which step? In that step, did the model call come before the tool call? All three are questions about joining. *The unit of a log is the line and the unit needed here is the run.*

There is one more thing. A run turns perfectly well without records. It takes the task, walks the steps, puts out an answer. Cut chunks badly and retrieval gets worse; leave out a record and it adds nothing to that run and takes nothing away. Records are not kept for that run. They are kept for whoever asks about that run later. So the signal that one is missing does not come from the run. It comes on the Monday morning when the question arrives.

20.2 What to Keep and How

Tie one run into a single bundle and put a hierarchy inside it.

Straight below the bundle come the steps, and inside a step go the model call and the tool calls. If a subagent was opened, that run goes whole inside the step that opened it. A tree stands, and the shape of that tree is the shape the loop actually turned in.

The names already exist. One bundle is a trace and one node inside it is a span. Every node has one parent, and that is what stands the hierarchy up.

Look at a real one. The task from Chapter 17, narrowing why the payment failure rate rose from Tuesday.

```
run 8f2alc 1m 48s 31,400 tokens ended: stopped on its own
|
+- step 1
| +- model call in 4,120 out 180 2.1s
| +- tool call search_logs("payment failure", 08-18 07:00-09:00)
| +- tool result top 40 of 412 lines 1.4s
+- step 2
| +- model call in 6,900 out 240 3.0s
| +- tool call read_file("deploy.log", 400-430)
| +- tool result 31 lines 0.2s
+- step 3
| +- child run 8f2alc-2 log-search
|   +- step 1 model call in 2,800 out 150 1.6s
|   +- step 2 tool call grep_range("payment failure", 08-11)
|   +- step 2 tool result 0 lines 0.3s
|   +- returned status=partial 2 candidates 1 not done
+- step 4
| +- model call in 12,300 out 610 4.4s called nothing more
```

Everything needed to read it is there. There were four steps, the third opened a subagent, that search came back with 0 lines, and the fourth put out an answer. Time and tokens hang on every node, so where the 1 minute 48 went is visible too.

What to keep is all in that example. Four things.

- **What went in and what came out.** For every model call, the full input and the full output. Keep only the token counts and you know where it took long but not why it answered that way.
- **The tool call and its result.** Name, arguments, what came back. If it was an error, the error string as it stood.
- **Elapsed time and tokens.** Hung on each node separately. With only the total you know it got slower and no more.
- **Which call of which step.** This one joins the other three. Without the parent's identifier on a node, the other three go back to being scattered lines.

The fourth goes missing most often where things are divided. A subagent usually turns in another process and writes its own records in its own place. Unless the opening side passes the run's identifier along, the two records never meet. It is adding one field to the list of what gets handed over, and Chapter 17 already stood that list up.

Table 20.1: Five things the record cannot tell you. The middle column is what actually gets written into the trace and the third is why that does not part them. None of the five raises an error.

What you want to know	What the record holds	Why it does not part	Set up in
A search that came back empty	“0 lines”	Absent from the documents and missed by the query arrive as the same one line	Section 11.5
A tool not called	The four calls made	What could have been called and was not is not a call, so the record of calls has no place for it	Chapter 7
A chunk not loaded	The five chunks loaded	A chunk that made the candidates and got cut was not in the context, and the record writes the context	Chapter 4
A chunk not used in the answer	All five chunks loaded	Only the loading remains. Whether the answer used it or used a learned commonplace does not	Section 11.5
Something pushed past unread	“in 12,300 tokens”	A thing that was loaded and the model went past arrives only as a worse answer	Figure 4.2

When you write it

When a node gets sent out has to be settled too. Sending the whole thing after the run ends is easier to build. The tree is fully standing, so you serialize it and hand it over in one go.

But a run that got cut off never sends that tree. A run whose process died or hit a time cap loses the trace whole. *And those are exactly the runs you want to open later.* A system that keeps records of runs that ended well and none of runs that got cut is a system where only success is observed.

So a node gets sent out as it closes. One step finishes and that step’s node goes; the run finishes and the run’s node closes. Die in the middle and what came before is still there.

The form the four get written in has to be settled too. Write them only as sentences for a person to read and you can open one case but you cannot count across runs. “How many runs called this tool last month” is a question no amount of careful reading of one trace answers. Values need names on them before they can be counted. What the next section asks for is exactly that crossing.

20.3 What You Still Cannot Know

The record in the previous section is fairly full. And there are questions it cannot answer.

Go back to that trace. In the third step’s child run, the search came back with 0 lines. What has to be asked now is whether the logs before Tuesday really were absent, or whether they were there and the query missed them. The trace says only 0 lines. There is no line to part the two.

There are five such places (Table 20.1).

Set the five side by side and one thing is the same. *All five leave a record wearing the same face as a successful run.* What gets written into a record is what happened, and the cause of all

five is something that did not happen, and a thing that did not happen has no place to be written.

You cannot write down everything absent either. What was not loaded in this step is nearly everything in the world.

So the order gets reversed. You do not start from what to keep. *Write down first what you want to know, and trace backwards to what has to be there to answer it.* The first column of Table 20.1 is that list of questions, and what to keep comes out of it.

So what else do you keep

The remedies scattered through earlier chapters gather here. The first four have a road and the last does not.

- **Have retrieval return its own circumstances.** The query string, the range swept, the candidate count before the threshold, the top score, the threshold that cut. Zero candidates and it is the absent side; forty candidates with a top score under the threshold and it is the missed side. *One line saying 0 parts into two lines.* It is adding a field to what gets returned, so it is exactly what Chapter 9 set up about schemas. Without the field there is nothing even to fill.
- **Counting what was not called needs the list of what could have been.** Write down, per step, the tool names that were loaded in the context at that moment. Twelve loaded and one called leaves the other eleven as “was a candidate and was not called.” A tool that was never on the list still leaves nothing, and that side is a thinness of the vocabulary rather than of the record (Chapter 7).
- **Count what got dropped in the assembling.** Do not write only the five chunks loaded. Write the forty candidates and the thirty-five cut alongside. What was cut needs only an identifier and a score, so the growth is a fraction of the size of the chunk text.
- **Whether the answer used it can be kept only halfway.** Make the answer carry sources and what was used shows. But an answer that used half the chunk’s numbers and mixed in half a learned commonplace carries plausible sources too (Section 11.5). That a source is attached and that the sentence came from the chunk are different things.
- **For the fifth there is no road.** This book does not know how to observe a thing that was loaded and the model went past. What made the candidates and got pushed out inside the context is the limit of what can be written now.

Attach the remedies and the third step above becomes this.

```
+-- step 3
| +- child run 8f2alc-2   log-search
|   +- step 2   tool call   grep_range("payment failure", 08-11)
|   +- step 2   tool result 0 lines   0.3s
|   | swept      logs/2026-08-11, logs/2026-08-12
|   | candidates 0   top score none   threshold 0.35
|   | no files before 08-14 in the store
|   +- step 2   tools loaded  grep_range, read_file,
|   | list_files, search_logs   (one of four called)
|   +- returned status=partial  2 candidates  1 not done
```

Three lines longer and one question answered. There were 0 candidates and the files for that period are not in the store at all. The query did not miss; the logs passed their retention and were deleted. *Now you know what to fix is the retention period rather than the retrieval.* Before the remedies, hours spent staring at this run do not reach that conclusion.

One thing remains. *Absence does not show well from opening one run.* In a single trace, eleven tools not called is just eleven tools not called and there is nothing odd about it. Put a hundred runs of the same task side by side, and if ninety-seven called that tool and three did not, those three become a pattern. The unit of the record is the run and the unit for seeing absence is a crowd of runs. That is why the previous section asked for values with names on them. Names are what let you count across a hundred.

The point. This is where tracing and observability come apart. However full a trace is, what gets written into it is what happened, and the cause of a quiet failure is usually something that did not happen. You cannot write down everything absent, so instead you write down the places where things get chosen. Attach candidates and a threshold to retrieval, the list of what could have been called to a tool, and the identifiers of what was cut to the assembling. *Then four of the five part.* What was loaded and never read still leaves nothing.

20.4 How It Is Done in Practice

There is a standard. It lays generative-model items on top of the ordinary distributed tracing specification of software, settling the names of nodes and the names of the values that hang on them (OpenTelemetry, 2026).

The hierarchy it settles is the tree of the previous section. Each node carries a name for its kind, and three of those overlap exactly with the previous section's three. A run gets `invoke_agent`, a model call below it gets `chat`, a tool call gets `execute_tool`. Retrieval and planning have names too. Tying a run into a trace and seeing the calls inside it as nodes with parents is the same on the specification's side.

What is not in ordinary tracing and hangs on a model call is the meat of this standard. Four are enough to look at.

- **Two token values.** `gen_ai.usage.input_tokens` and `gen_ai.usage.output_tokens` sit separately. They are billed differently, so one total cannot count Chapter 18's "how much did it take."
- **The model's name.** `gen_ai.request.model`. Of the things that change outside, this changes most often and the change is not announced. Without this value you cannot tell whether last month's number and this month's came from the same system.
- **Why it ended.** `gen_ai.response.finish_reasons`. It parts stopping on its own from being cut at a length cap. The branches Chapter 15 divided termination into arrive here as one field.
- **The tool call's identifier.** `gen_ai.tool.name` and `gen_ai.tool.call.id`. The model can call three tools in one step and the results can come back out of order, and this value joins each result to its call.

The bodies are not kept by default

One thing has to be pointed out here. What the previous section put first, the full input and full output of a model call, is optional in this standard. The names `gen_ai.input.messages` and `gen_ai.output.messages` exist, and the instrumentation keeps them only if you turn on an

environment variable for it. The reason is written into the specification. Model instructions, user messages, and model outputs are considered sensitive, and they are often large.

So in a system that ran for months on the default, those months become a stretch you cannot ask about. The token counts and the latencies are all there and why it answered that way is not. *The problem Section 20.5 deals with and the problem this section deals with hang on the same one switch.*

The state of the standard is worth knowing too. As of the summer of 2026 every item beginning with `gen_ai` is marked as under development and not one has moved to stable. The related documents splitting off into their own repository is recent as well. So it is right to treat the names as changeable. What has settled is not the names but the hierarchy and the kinds of value that hang on it.

What the tools keep and do not keep

Observability tools on the language-model side mostly put out the same shape. They draw one run as a tree and hang on each node the prompt and the response, the chunks retrieved, tokens and latency. Attaching Chapter 19's metrics to nodes of production traffic is usually there too. Pick any of them and the previous section's four come out.

What does not come out is the third column of Table 20.1. Retrieval's circumstances, the list of tools that could have been called, the candidates that were cut. Attaching instrumentation does not produce them. *Keeping is what the tool does for you and making it knowable is still design.*

The standard's side is the same. Retrieval's node has a settled name, and what to hang on that node is not written out in the detail the model call gets. How many candidates there were, what the threshold was, how many were cut and how many loaded, have no names to sit in yet. It is right to treat none of what the previous section asked for as something the standard will stand up for you. Name them yourself and attach them. If a standard name appears later, that is a value to move then, not one to wait for now.

Where you attach it

There are three places to attach instrumentation.

Wrap the place that calls the model in one layer and every model call gets caught. Attaching usually starts here, and this far takes a few lines. But do only this and the trace becomes a flat list of model calls, with neither the step boundaries nor the tool calls visible.

The second is the place that turns the loop. Only the harness knows where a step starts and ends, so the step node can only be opened there (Chapter 15). The third is the place that assembles the context. Only that place knows what was put up as a candidate and what was loaded.

Systems that attached only the first of the three are the most common. The trace with all the tokens and latencies and none of the why usually comes from here.

A published account

One record of running a multi-agent system writes this place down briefly. Adding full production tracing was what let them diagnose why agents failed and fix things systematically. And it

writes down what they looked at. Decision patterns and interaction structures, without the contents of individual conversations (Anthropic, 2025e). The shape Section 20.5 asks for already exists in practice.

The same record has one more sentence. Agents are non-deterministic between runs, even with identical prompts. That sentence parts what records cannot buy. However far you grow the record, what it buys stops at *reading what was seen and what was done at the time*. Running again under the same conditions and getting the same thing does not come from the volume of records. What that needs is pinning the configuration down, and of the four above, the model's name is the first field of it.

20.5 What Keeping Costs and What Not to Keep

Start with what it takes. Storage, and the effort of reading it.

The trace above was 31,400 tokens for one run. Keep the full input at every step and the previous step's contents go into the next step's input again, so what actually gets written is larger than that, about 60 kilobytes as text. Ten thousand runs a day is 600 megabytes a day and 18 gigabytes a month. *That is what it means for storage to grow in proportion to volume*.

Storage is not the only cost. If two hundred of ten thousand daily runs fail, storage may hold but a person cannot read them all. Opening one and walking its steps takes ten minutes, so two hundred is thirty-three hours. *How much can be read is the second constraint on how much to keep*. Which is why sorting failures into kinds belongs to the same decision as keeping. Group two hundred into five kinds and it becomes opening two or three of each.

So how much of what to keep has to be settled. Three shapes are used in practice.

- **Failures whole, successes by sample.** Runs that ended in error or crossed a latency threshold get kept entirely, and runs that ended ordinarily get kept at a few percent. Successes get kept a little because you need something to compare against. Gather only failures and you cannot tell whether a pattern belongs to failures alone.
- **Tiered retention.** Keep the window where the bodies stay whole short, and after that keep only token counts, latency, and the outcome. In practice that window is usually a week to a month. What you need from an old record is usually a pattern rather than the full text of one case.
- **Part capturing from retaining.** What to capture has to be settled before the run, and what to retain can be settled after it ends. So capturing wide and retaining narrow beats capturing narrow and retaining everything.

What not to keep

And there are things that must not be kept at all. Three.

- **Personal data.** What came into the context stays in the trace as it stood. A document the user pasted, internal material that got retrieved, a customer record a tool looked up are all the full input of a model call, and deciding to keep the full input keeps them along with it.
- **Credentials and keys.** They go into the arguments and headers of tool calls. That side is a place people rarely read, so it gets found months later.

- **Copies a deletion request cannot reach.** When a request arrives the trace has to be deleted too, and for that you have to be able to find which traces are that person's. *You have to keep an identifier in order to delete.* That is why deletion cannot be added later.

There is one place that leaks through the masking. A name or an account number gets caught by a rule, and a fact dissolved in a sentence does not. Strip the name from “this customer requested three refunds last month” and the rest stays, and matched against other nodes around it, who it is narrows down. *So instead of hanging masking up and feeling safe, part this class out separately in the list of what to capture in the first place.* Parting it out later means building the record over from scratch.

One thing runs through all three. *This gets kept by accident.* Nobody decides to write personal data into a trace. They decide to keep the context whole, and that decision carries what is inside it along. So the masking cannot sit at every calling site. It has to sit at the one place the record is made. Make every call remember and one day one of them forgets.

Two ways of keeping that earn nothing

- **Growing it with no procedure for reading.** That you are keeping does not mean you are looking. And the direction of the gap is bad. The richer the records, the more the presumption that you would find it if you looked fills the space, and the less anybody actually looks. An industry source writes that they invested in tooling for viewing transcripts and regularly take the time to read them, and that reading transcripts is a critical skill for agent development (Anthropic, 2026c). In practice that procedure is usually two things. Grouping failed records into kinds to see the pattern, and drawing eval cases out of that. The second is where Chapter 18's demand runs on this chapter's material.
- **Putting diagnostic records where the system reads.** This one is quieter. What gets kept for diagnosis is a document full of failed attempts, rejected candidates, and wrong conclusions. Put it where it can be retrieved and the next run can bring it back as grounds, and the system learns again, from its own records, what it already rejected. *Settling where a record lands looks like a storage question and is a trust boundary question.* Chapter 22 takes that defense.

One shape runs through both. *A record earns nothing by itself.* It earns when there is a procedure for reading it. So the question of what to keep always carries a partner. Who reads this, where does it sit, and what will not be read.

Which is why the keeping decision and the reading decision are better made in the same place. Settle only the keeping and storage grows while readers do not, and six months later there are gigabytes nobody opens. *A record with no partner is storage rather than observability.*

20.6 Summary

In one sentence. Tracing keeps the road one run travelled as a tree and observability is what you can learn from it, and the two come apart because what gets written into a trace is what happened while the cause of a quiet failure is usually something that did not happen.

What is now settled.

- The unit of a log is the line and the unit needed here is the run. Mix sixty lines with the lines of two hundred other requests and you cannot ask which line is which step of which request (Section 20.1).
- Tie one run into a bundle and put steps and calls inside it as a hierarchy. Four things get kept. What went in and what came out, the tool call and its result, time and tokens, and which call of which step (Section 20.2). The fourth goes missing most often where things are divided.
- Five things cannot be known from the record and all five wear the face of a successful run (Table 20.1). So you settle from what you want to know rather than from what to keep.
- Four have remedies. Attach the candidate count and the threshold to retrieval, the list of tools that could have been called to the step, and the identifiers of cut candidates to the assembling. What was loaded and never read has no remedy (Section 20.3).
- The hierarchy the standard settles is this same tree, and a model call carries two token values, the model's name, why it ended, and the tool call identifier. But the bodies of input and output are optional and have to be turned on separately (Section 20.4).
- What growing the record buys stops at reading what was seen and what was done at the time. Running again under the same conditions and getting the same thing does not come from volume. It comes from pinning the configuration down.
- Keep failures whole and successes by sample. Capture wide and retain narrow (Section 20.5).
- Personal data and credentials get kept by accident. Not from being written in on purpose but from keeping the context whole. So the masking sits at the one place the record is made.

What is still open.

- There is no known road to observing what was loaded and never read. The fifth row of Table 20.1 sits outside this chapter's remedies.
- There is no general answer for what percentage to sample at and how many days to hold the bodies.
- There is no metric for the fact that the record was thin. A list of problems left unsolved for want of a record cannot be made in principle.
- Nor is there a threshold for when a diagnostic record may become the system's corpus. The discipline of keeping them apart is as far as this chapter goes.

Try this today. *Open* what you are keeping now. Pick one run from yesterday and check whether

all of that run's calls show as one joined thing or scattered as lines in time order (Section 20.1). If they are joined, pick one retrieval call from that run and *look* for whether the candidate count and the threshold are written down alongside the chunks returned (Section 20.3). Last, open one case and *check* whether the full input of a model call is being kept, and if it is, whether anything about a person is inside it (Section 20.5).

20.7 Exercises

1. Pick a recent failure and try to name its cause from the record alone. Where did you get stuck? Sort whether the place you got stuck is one of the five in Table 20.1 or whether there was no identifier to join with.
2. Open the return format of a retrieval call you run now and add fields for the candidate count, the top score, and the threshold. After adding them, check whether last week's ten "no results" part into two kinds.
3. Open one trace and mark every place holding something about a person. In how many places is the code that masks it scattered right now? Write down the effort of gathering it into one place.
4. List the records you keep that sit somewhere the system can retrieve. If any of them were kept for diagnosis, draw the route by which they reach the next run's context.

20.8 On a Second Reading

- Read Chapter 22 and come back to the last item of Section 20.5, and settling where a record lands shows again in the language of defense. Anything sitting where the system can read it is text that can be read as an instruction, and what sits there happens to be the record of failures.
- Read Chapter 23 and come back to this chapter's demands, and you see there is a layer that makes the places to keep appear without planting them by hand. But what that layer stands up for you is only the four of Section 20.2. The third column of Table 20.1 has to be attached by hand there too.

20.9 Further Reading

Two things came from Anthropic (2025e). The report that adding full production tracing was what let them diagnose why agents failed and fix things systematically, and the statement that what they watched was decision patterns and interaction structures rather than the contents of individual conversations. Chapter 17 took the list of what gets handed over from the same source; what this chapter took is what they looked at and what they did not. One more sentence from the same source, that agents behave differently between runs even with identical prompts, is the ground for parting what records cannot buy.

One thing came from Anthropic (2026c). That they invested in tooling for viewing transcripts and that reading transcripts is a critical skill. It is a report of practice rather than a recommendation, so what to take is the fact that *transcripts do not get read by themselves*.

The standard in Section 20.4 is a public specification (OpenTelemetry, 2026). There is no reason to read the document from the start. Open the list of node names and the list of values that hang on a model call and that is enough to set against what your own system keeps today.

The rereading is Chapter 4. The two sentences about what was wrongly put in being somewhere and what was wrongly left out being nowhere were the whole problem of this chapter.

Part VI was three chapters and one argument. Chapter 18 answered that settling what to call success is the first decision of the design, Chapter 19 answered that a number built that way misleads people while being honestly computed, and this chapter answered that both of those stand on records and that records can only be made before a run ends. *The order of the logic is definition, doubt, record, and the order of time is the reverse.* Part VII now moves to running this system and keeping it safe.

PART VII

Operations, Safety, Governance

Human Approval and Permission

Every approval screen you attach takes that much automation away.

Say an agent calls tools thirty times to finish one task. If a person stands in front of five of those, that task is one a person stops and handles five times. What improved over doing it by hand is the other twenty-five. Raise the approval points to twenty and almost nothing is left.

Stand none of them up and there are accidents. A deleted file does not come back, money that left does not come back, and mail that was read does not become unread.

So where in between do you draw the line? That question is this chapter.

What makes it hard is that the line is not written in the code. The code says only what can be done. Changing a customer's tier and deleting a temporary file are both a call that edits one value, and neither the function name nor the argument shape says which one is dangerous. The danger is in what that value means outside the system, and that meaning sits outside the code.

Chapter 18 said that as long as a model draws its answer by probability the last approval stays on the person's side (Section 18.2). That was a verdict that approval remains. What gets settled here is where to put it.

21.1 Where to Stand a Person

Two roads lead to the places.

One is asking the operator, the person doing that work by hand right now. The other is listing the actions that cannot be undone. The first tells you where things already stop, and the second tells you where nothing stops and something should.

You need both. An agent ends up doing what the operator never did, so the first road alone leaves gaps in the list. And counting only from the code drops everything the operator knows. The next two sections are those two roads, and they are the center of this chapter.

The order of the decisions. The further down, the more of a person's time it takes. Do not push down what finishes higher up.

1. **Can it be made impossible?** Take it off the tool list and no prompt makes it happen. Only what cannot be taken off moves down (Section 21.4).
2. **Where does the operator stop?** Ask the person doing the work now. That answer is the first candidate for an approval point (Section 21.2).
3. **Does undoing make it never have happened?** The question is not whether it can be cancelled (Section 21.3).
4. **What does undoing take?** Even what can be undone does not get left alone if it costs a lot. Use the devices that shrink that cost before you use a person (Section

21.3).

5. **Sort what is left into three.** Done automatically, looked at and pressed by a person, and not possible at all (Section 21.4).

21.2 Ask the Operator

The first source is a person.

Before building the agent, sit down with the person doing that work now. Not the meeting where you show them the finished thing, the one before you start. Five things to ask.

- How do you do this work now? From the start to the end, in order.
- **Where do you stop once and check?**
- What do you look at there before moving on?
- **What is a problem if it goes wrong?** Who is the one in trouble?
- How do you undo a mistake? Have you ever undone one?

The answer to the second is the candidate for an approval point as it stands. The fourth question has its body in the clause tacked on the end. Who is the one in trouble? Us being in trouble and the customer being in trouble are two different lists, and asking usually turns up the second. The people building it think of the first.

Why does this come first? Because that person is already making the judgment. Where to stop is not a decision to be made fresh but one already made, and building an agent is *moving* that decision out of a person's head into code. Standing something up from nothing is far harder. Try to build fresh what only needed moving, and in the first week you make the mistake the ten-year veteran does not.

Once you have heard where they stop, ask why they stop there. Places where they stop because a rule says so and places where they stop because there is no road back are mixed together. The first disappears when the rule changes and the second disappears only when the system changes. Leave the two unsorted and later, when you take an approval away, you take it without knowing which kind it was.

And there are things you learn only by asking. Sentences like “this one, if it goes out wrong, the customer knows right away.” That sentence is nowhere in the code. Not in the repository layout, not in a tool description. It lives only in the head of somebody who has done the work a long time, and unless you ask, that person will not volunteer it. It is too obvious to them to be worth saying.

You ask and a different place comes back

Say you are building an agent that handles refunds. Before building, you ask the support agent who does that work now.

The approval point the engineer expected is the refund itself. Money leaves. But the answer is this. “Small refunds we just do. Forty a day. Ask about every one and the work does not happen.”

Ask where they do stop and a different place comes back. “When I lower a customer's tier.”

Ask why they stop there and the reason comes. A wrong refund can be redone the same day and the customer never notices. A tier shows up on next month's bill as a different amount.

By then the customer has already seen it and the phone rings. Putting the tier back is one line in the system, and that one line does not make the call never have happened.

In the code the two look the opposite way round. The refund is an outside call to the payment processor and the tier change edits one value. Attach the approval to the one that looks dangerous and you get a system that asks forty times a day, while the place that should stop stays open.

That conversation takes thirty minutes. Thirty minutes moved one approval point and deleted another.

What if nobody does that work now? The job is new and there is no predecessor. Then find whoever does the nearest thing, and failing that ask whoever receives the result of the work. “When would you find out if this went wrong?” That one question stands in for the list above. When they find out is how much time there is to undo it.

After asking you have one list. The places the operator stops today. But that list is not complete. A person doing forty a day is one thing; an agent does it in twenty branches at once. Places a person never stopped at, because a person never had the chance, come from there. The second source finds those.

21.3 Find What Cannot Be Undone

The second source is a list.

Write down every tool the agent can call and sort them one by one with a question. If it goes wrong, can it be undone?

The reversible side looks roughly like this. Editing a file and putting it back, deleting a draft nobody has seen, rewriting a record in our own store. The irreversible side looks like this. Deleting with no copy, money moving, a message that went out.

The list needs one line beside each tool name. If this tool goes wrong, who finds out and when? Nobody finds out and it is the first kind; somebody outside finds out the same day and it is the second. Put that one line in the same file as the tool definition and adding a tool can no longer leave it blank.

That much is not hard. What comes next is.

It cancels and it does not become undone

There are places that are technically reversible and practically are not.

A mail sent by mistake is one. You can delete it from the sent folder, and some systems have a recall feature. But if the other side has already read it, deleting leaves the reading where it was. Send a correction and the other side has received two messages, not none. The customer tier of the previous section is the same. Undoing takes one line and the record comes out clean. Only the memory of the customer who saw the bill stays.

So the question to ask is not “can it be cancelled.” *Does undoing make it never have happened?*

Sort by that question and the shape of the list changes. Actions that change only the inside of the system usually become never-happened, and actions somebody outside has already seen do not, procedure or no procedure. The line is not the boundary of the system. It is where a person’s eyes reached.

Table 21.1: Four grades of undoing. The sorting question is not whether a procedure exists but whether undoing makes it never have happened. The third row is the one most often filed wrong, and it lands in the first row on the grounds that a procedure exists.

Grade	Example	Never happened?	What undoing takes
We undo it ourselves	A file edit, a draft nobody saw, a record in our own store	Yes	Work in proportion to what was changed. Can be estimated before the run
We ask somebody to undo it	A record put into another team's system, an order that can be cancelled	Usually	Their response and how long it takes to come. We cannot estimate it
A procedure exists and it does not go away	A notice that went out, a tier and an amount the customer saw	No	Correcting work with explaining work on top
No procedure	Deletion with no copy, money that left, material that flowed outside	No	Cannot be estimated. You settle in advance only whether you could bear it

Filling the table in, things turn up that fit none of the four. Places where it says reversible and nobody has ever reversed one. That is not a grade but a state, and most real actions are there. *An undoing nobody has run is a guess rather than a verdict.* Which grade it belongs to gets learned the first time you actually have to undo, and by then it is late.

Undoing is also a property of the environment an action sits in rather than of the action. The same deletion is the first row in a system with copies and the fourth in a system without. Change the backup interval from daily to weekly and this table changes while the tools and the code stay as they were. So the list is not something you make once.

Reversible and expensive to reverse

Reversible means the effort is finite, not that it is small. The last column of Table 21.1 is that. Two things grow the effort. What stops while you are undoing, and how many. The service is down while you roll a deployment back, and if you edited a thousand records you have a thousand to undo.

Most approvals in practice hang here. Deployments and bulk edits are reversible actions and a person approves them. Not because they cannot be undone but because undoing them is a day's work.

But there are roads to shrinking the effort before you stand a person up. The six below cost the same however many runs there are, while only the person grows with the number of runs.

- **Dry run.** Imitate the outside call and show what would change first. The simplest road to manufacturing the material for a judgment.
- **Idempotency keys.** Keep the same request from running twice. Retries become safe.

- **Staged rollout.** Open a narrow target first, watch, then widen. It splits the blast radius across time.
- **Rate limits.** Cap how many go through per unit of time. It stops before a mistake becomes a thousand.
- **A kill switch.** Stop what is turning now, at once. A switch nobody has pressed under load is a switch that is only written down.
- **Recovery drills.** Run the undoing procedure on a schedule. The only road that removes the “nobody has ever reversed one” above.

What is left after all six is what a person has to look at. Do it in the other order and the approval points multiply past what is needed, and once they multiply the problem of Section 21.5 arrives.

21.4 How Permission Gets Divided

With a list from the two previous sections, now move it into code.

Three places to move it to.

- **Done automatically.** Actions that can be undone and cost little to undo.
- **Looked at and pressed by a person.** Actions that cannot be undone, or where undoing is a day’s work.
- **Not possible at all.** Actions no approval opens.

Why keep the third separate? Because a system that says everything is fine with approval gets broken by asking for approval often. The approving person has a few hours a day and the requests grow with the number of runs. The twentieth request gets read less than the first. So the real ceiling of a system where anything goes with approval is the point at which the approving person tires, and that is not a ceiling.

Definition 21.1 (Permission). The range of change a system can actually cause in its environment while running is its *permission*. Permission is not settled by the list of tools it can call. The list settles the upper bound of the range, and inside that the actual range is settled together by what the arguments can point at and what resources the tool is attached to. Designing permission is not choosing a list but narrowing those three together.

The middle kind splits in two again. Stand it in front of the action and the action does not happen and a wait appears instead. Stand it behind and what happened gets undone and there is no wait. If the undoing has a verdict and the effort is bearable, behind is better. If the undoing does not hold, only in front.

Three reasons the list is not the permission.

- **The argument decides.** Editing one line of a settings file and editing a deployment script are the same call to the same tool. What the schema checks is the shape of the value, not what the value points at.
- **Combinations are not on the list.** With a tool that reads customer records and a tool that sends mail both open, reading records and sending them out is also open. No procedure that inspects the two separately sees this third action. It is not an element of the list but two elements joined.

- **The environment decides.** The same lookup tool is an entirely different permission depending on whether it sees a test copy or production data, and the tool's definition does not differ by one character between the two.

Hang it on the tool or on the argument

So there are two places to hang the approval too.

Hang it per tool and the code is simple. This tool automatic, that tool approval. In exchange it is coarse. Hang an approval on the one tool that deletes files and a person has to press for temporary files too. The system that asks forty times a day comes from here.

Hang it on the arguments and it is fine-grained. The same deletion gets sorted by which path the target is. In exchange somebody has to write that judgment down, and what is written down goes stale. A new path appears and passes through, caught by neither side.

There is no right answer, and there is a default. *What is not known goes to approval.* A path not on the list, a target seen for the first time, a place where no judgment is written: those go to a person rather than to automatic. Set it the other way and everything newly created passes quietly.

Chapter 15 said what the harness owns stops at making the place (Section 15.3). The device that stops the loop and waits for a person is the harness's to build. What to stand in front of it gets settled here. The two lists above are what fills that place.

One last thing. Widening permission happens separately in three places. Adding a tool, opening a value list in a schema into a free string, and attaching one more resource to the environment. All three look small and all three get done by different people on different days. *Nobody sees the sum of the three.* With nowhere to read in one place what is currently allowed, permission only ever widens.

21.5 What Remains After the Approval Is Built

People press the approval screen without reading it.

That is not a problem with people. It is a problem with the design. Three reasons it goes that way.

- **It asks too often.** If thirty-nine of the forty requests a day were passes, an expectation forms that the fortieth is a pass too. That expectation was right thirty-nine times, so it is reasonable.
- **What is being approved is not on the screen.** What comes up is usually the action and the target, those two. Why this action came to this target of all things stays in the agent's context and does not reach the screen.
- **Nobody knows what happens if they refuse.** Whether refusing fails the task there, sends it down another road, or does nothing at all is not written. Not knowing, passing looks safe.

So adding approval points is not adding safety. Past some point it subtracts. Stand up twenty and all twenty become a formality; stand up five and the five get read. *An approval that is not read is worse than no approval.* With none, the place stays a dangerous place; with one, it gets written into the document as a place already checked.

Four remedies

- **Do not ask often.** That means the list from Section 21.3 has to be narrow. Use the six devices first and send only what is left to a person. Cutting approval points is not cutting safety. It is making the ones left get read.
- **Show what changes.** Not the whole thing, the changed part. This is where Chapter 18's claim that a good agent cuts the effort a person spends confirming lands (Section 18.2). The approval screen is where that effort actually gets measured.
- **Write down what happens on a refusal.** "Refuse and this task ends here" and "refuse and it goes to a person" are different screens and pull different judgments.
- **Measure how long they read before pressing.** Keep the gap between the moment the screen came up and the moment it was pressed.

The last item joins Chapter 20. Without keeping it there is no way to ask whether this place turns well (Section 20.2).

Having no way to ask shows in a single number. Say an approval point passed 99 of 100 cases. That can be because dangerous things almost never arrive there, and it can be because a person passed everything with no material to judge on. *The two produce the same number and call for opposite work.* The first says take the point away and the second says attach material to it. To somebody watching only the pass rate, both look like a place that turns well.

Telling them apart means keeping something besides the pass rate. The time actually spent reviewing, the share of refusals that carry a reason, what happened to refused requests afterwards, and the number of runs sitting halted waiting for approval. Leave that last one out and those runs get counted as neither failures nor successes. They are not dead but waiting, so they do not show up as errors either.

Places where standing one up earns nothing

Even with all the remedies there are places that earn nothing. Three.

1. **Places with no material for a judgment.** If only the action and the target land there and there is no way to attach more, what the person does there is not judging but passing. And the fact that a person is standing there gets written into the document, and once written that action becomes one with nothing left to look at.
2. **In front of an action that is reversible and cheap to reverse.** Stand it behind and you get the same thing without the wait. The undoing has to have a verdict, though, and if the effort is past bearing then in front is right.
3. **Places the result does not reach.** So there is one way to ask whether you chose the place right. *Does the result of this decision arrive at this place?* If it does not, what that place does is not judging but signing. Signing does not move the result anywhere. It leaves only the impression that it moved.

One shape runs through all three. *Every condition for an approval to earn its keep sits outside the approval.* The material is a records problem, the frequency is a vocabulary problem, and the undoing is an environment problem. Deciding to stand an approval up is not a decision to build an approval but a decision to check those three first.

And there is one more. *A decision to stand an approval up gets used as grounds for deferring the decision to narrow the vocabulary.* "A person approves that action, so we can leave it open"

is a sentence that wins meetings. But that approval gets given with no material and becomes a formality with frequency. What that sentence bought is not a lowered ceiling but grounds for not lowering it, and the result is *a system with a person standing in it carrying a wider vocabulary than one without*.

The remedy is simple. Beside the decision to stand an approval up, write one line saying why that action cannot be taken out of the vocabulary. “This action is the whole of this task, so it cannot come out” is enough. If there is no reason to write, that action is a candidate for deletion rather than for approval.

21.6 Summary

In one sentence. Where to stand a person cannot be found in the code, and the places come from two sources, where the person doing the work now stops and where undoing does not make a thing never have happened, and moving what you find into approvals means moving toward being read rather than toward being many.

What is now settled.

- Approval points cannot be found in the code. The code says only what can be done, and what a person has to look at depends on what that value means outside the system.
- The first source is the person doing the work now. Where to stop is not a decision to be made fresh but one already made, and building an agent is moving that decision out of a person's head into code (Section 21.2).
- Having heard where they stop, ask why. A stop that exists because a rule says so disappears when the rule changes, and a stop that exists because there is no road back disappears only when the system changes.
- The second source is a list. The question is not "can it be cancelled" but "does undoing make it never have happened," and the line falls not at the system's boundary but where a person's eyes reached (Table 21.1).
- Undoing is a property of the environment the action sits in rather than of the action. Change the backup interval and that table changes while the tools and the code stay as they were. So the list is not something you make once.
- Reversible means the effort is finite, not that it is small. Use the six devices that shrink the effort before you use a person. They cost the same however many runs there are, while only the person grows with the runs (Section 21.3).
- Permission is not what can be called but what can actually be changed. The list, the arguments, and the environment settle it together, and since widening each is done by different people on different days, nobody sees the sum. What is not known goes to approval (Section 21.4).
- Adding approval points is not adding safety. An approval that is not read is worse than no approval. With none the place stays dangerous; with one it gets written down as already checked (Section 21.5).
- Every condition for an approval to earn its keep sits outside the approval. The material is a records problem, the frequency a vocabulary problem, the undoing an environment problem.

What is still open.

- There is no general answer for which list to follow when the operator's list and the undoing list disagree.
- The threshold past which approval points stop being read is not known. Twenty and five are written apart here, and where in between only the review time at your own place can tell.
- Evaluating whether approvals are being given well means making, case by case, the

very judgment Section 22.3 set a limit on. Review time and refusal reasons are not a proxy for that quality but the smallest scale that parts the two states.

- What a decision stands on when knowledge about a person is already in a place that cannot be erased, and how to explain that limit outside, this book does not answer.

Try this today. Pick one approval point you have standing and *open* its screen. What is up there besides the action and the target? See whether what is there answers “may this be done” on its own (Section 21.5). Next, *count* last month’s pass rate at that point. If 99 of 100 passed, it is either a place dangerous things do not reach or a place where everything passes with no material, and the review time parts the two (Section 21.5). Last, *ask* the operator about the action in front of that point. Do they stop there too when a person does the work, or somewhere else (Section 21.2)?

21.7 Exercises

1. Sort your system’s actions into the four grades of Table 21.1. Is anything in the first row that belongs in the third? The sorting question is not whether a procedure exists but whether somebody outside has already seen it.
2. Get thirty minutes from the person doing that work by hand and ask the five questions. If even one unexpected place comes back, find which call in the code it is and write it down.
3. Pick one irreversible action and check two dates. The day that action entered the vocabulary and the day an approval stood up in front of it. If the approval came first, did that action get in because of the approval?
4. Open one approval screen and copy out exactly what is on it. Answer “may this be done” from only what you copied. If you cannot, write down what else has to be there, and if there is no way to attach it, that place is the first item of Section 21.5.

21.8 On a Second Reading

- Come back to Section 18.2 in Chapter 18 and you see that the place where a good agent got defined by the effort a person spends confirming is where the bill gets paid, at the approval screen. Putting only the changed part on the screen rather than the whole thing is that definition moved into code.
- Come back to Section 15.3 in Chapter 15 and you see the place that said the device stopping the loop and waiting for a person is the harness’s. What to stand in that place was this chapter.
- Read Chapter 23 and come back to Table 21.1, and you see what a system looks like that does not stand a person in front of every irreversible action. That asking for approval every time stops it being a useful assistant runs head-on into what this table asks for.
- Read Chapter 23 and come back to Section 21.4, and you see that the place settling what is allowed is only as open as the people who built it left it. Checking that it is still there becomes an item on the checklist when you pick a tool.

- Read Chapter 22 and come back to the first item of Section 21.5, and you see that material not arriving is not a problem only for people. The material that parts the same sentence as a request from the same sentence as an injection is missing when code judges too, exactly as Section 22.3 sets out. Stand a person up and the judging side becomes a person while the limit stays where it was.

Part VII takes, one at a time, the things that sit outside the system. What this chapter took is the result. A result arrives at the side receiving it rather than the side deciding, and so this chapter's two decisions do not disappear as models improve. The harness's other devices stand in for what a model will one day do and vacate the seat when it does, and here there is nobody to hand over to. A permission granted to yourself is not a permission. Once you have settled who stands in front of a result that cannot be undone, next is the text coming in in front of them. Chapter 22 takes that.

Prompt Injection and Guardrails

Very little of the text an agent reads was written by the engineer.

Think of an agent that answers questions about internal policy. A user asks, “by when do I have to file travel expenses?” That one line the engineer can inspect. But the three wiki documents pulled in to build the answer were inspected by nobody. What is written in them was settled by whoever last edited them.

Say that person added a paragraph at the bottom. An instruction to ignore what came before, and a demand to send the list of expense approvers to some address. The user does not see that paragraph. On the wiki page it is white text, or inside a collapsed region. The agent sees it. It receives the document whole and loads it into the context.

This is called prompt injection. The instruction arrives not through the box a user types into but through the material the agent pulls in to do its work.

This chapter has two halves. Counting where things can come in is one, and testing whether what you stood up actually holds is the other. Skip the second and you can do all of the first and still not know whether you were broken through. So the center of this chapter is Section 22.4.

22.1 Where It Comes In

The box a user types into is one entrance among several.

Every route built earlier is an entrance. Attach retrieval and sentences written by whoever put text in the wiki ride in; attach tools and the web pages and files those tools read ride in; attach memory and what a past run wrote down rides in. Open one more route and one more entrance opens.

The wiki document above, in its real shape, looks like this.

```
[internal wiki / filing travel expenses / edited 2026-03-11 / 12 edits]
```

```
Travel expenses are filed within 14 days of return. Attach original receipts. Items paid by corporate card are not filed separately.
```

```
... (40 lines of body omitted) ...
```

```
[One more paragraph sits at the bottom of the document. It does not appear on the page a person sees. An instruction to ignore what came before arrives first, and a demand to send the list of expense approvers to some outside address follows.]
```

Several hundred people inside the company can edit documents on that wiki. Count partner accounts and it grows. The moment retrieval carries that document in, all several hundred of them become people who can edit the agent’s instructions.

Two things to read off the table.

Table 22.1: Five entrances for text the engineer did not write. The third column is the point. Not one of the five can be read by a person before release.

Entrance	Who wrote it	When it comes in	Covered earlier in
A retrieved document	Whoever put text in the wiki or the document store	Every time an answer gets built	Chapter 11. The route by which a retrieved chunk reads as an instruction was deferred to here
What a tool returned	The web page, the file, another company's server response the tool read	Every step	Chapter 7. The engineer's code settles the envelope's shape, not the letters inside
What memory holds	A past run	When a session opens	Chapter 10. Written once, it rides in every run after
What another agent handed over	A system built by somebody else's organization	On handoff	Chapter 17. The form gets kept and the contents do not get reviewed
A tool description from outside	Whoever offered the server	On connecting, and on every revision	Chapter 8. Revisions happen with no deploy by the engineer and no notice

First, not one of the five can be read by a person before release. Review is something you do before release, and the contents of these five entrances do not exist at release time.

Second, having many entrances also means the system is actually doing something. A system where the only thing arriving from outside is the user's request does not retrieve and does not call tools. That system is safe and it is not an agent.

The model does not tell instructions from material

There is one more reason it is hard to block even after counting all five.

The harness gathers the pieces one at a time. The instructions come from here, the retrieval results from there, what a tool returned from somewhere else. While it is gathering, it knows which piece came from where. But what goes to the model is one piece of writing made of those pieces spliced together, and no seams are left in it.

Definition 22.1 (Trust boundary). The line parting text a person read and passed before release from text that came in mid-run with nobody reading it is the *trust boundary*. This line is drawn in the harness. The harness is the side that gathers and splices, so it knows which piece came from where. It is not drawn in the model. Which is to say the trust boundary does not fail to exist. *The model does not have it.*

The shape of this chapter gets settled here. The side that knows the source and the side that settles the next action are different. The harness knows the source and does not draw the next action; the model draws the next action and does not know the source.

So let the side that knows attach a marker and say so. It gets blocked in two places.

- **A marker is text too.** If text from outside can be any string at all, the shape of that marker can be imitated too. There is no marker that could never appear in material.

- **The side that settles whether to honor the marker is the model.** Write down that text after this marker is not an instruction and that is one line in the context, and the model draws a fresh answer every time. Mostly it is honored and sometimes it is not.

Then part them entirely. Block the model from ever following text that came from outside. Then a retrieved document cannot change the answer and a tool result cannot change the next action. Retrieval that does not use what it fetched and a tool whose result nobody looks at are worse than nothing. *Usefulness and danger are not two forces but one.*

Greshake et al. (2023) writing that applications with a language model attached blur the line between data and instructions points at the same place. The same paper showed, on real services, that you can steer an application from a distance with sentences planted in advance in material likely to be retrieved.

22.2 What It Is After

What a sentence that came in is after is usually one of four.

- **Overwrite the instructions.** Make it stop what it was doing, or spit out the engineer's instructions in the answer.
- **Make it call a tool.** A call the user did not request. Deleting a file, sending mail, editing a value.
- **Make it send something out.** Read what it can read and hand it to some outside address.
- **Show the person something different.** The system turns normally and only the content of the answer differs. Making a particular item always come first in a recommendation list is this kind.

The kinds Greshake et al. (2023) sorted out sit on these four. Pulling material out, spreading by itself, and poisoning information are in there, and that paper's diagnosis is that processing a planted sentence amounts to running code somebody else wrote.

The third is the heaviest. The first two end inside the system with room to undo, and material that went out does not come back.

And the third does not stand on one tool. It needs a tool that reads and a tool that sends both open. With only a tool that looks up customer records it reads and stops; with only a tool that sends mail there is nothing to send. Put the two in the same run and the space between them becomes a route.

22.3 How Guardrails Get Built

Three places to block. On the way in, by shrinking what can be done, and on the way out.

The middle is the surest. Inspection issues passes, and what has been made impossible does not happen whatever sentence arrives. Close the mail tool for this run and no text makes mail go out. Not because it blocked well but because there was nothing to call.

Published test material shows this in numbers. DeBenedetti et al. (2024) built four environments, a workspace, a company messenger, travel booking, and banking, and paired 97 user tasks with 27 injection goals to get 629 security cases. Pairing tasks and goals separately and multiplying is the point of the method. Add one goal and the tests grow not by one but by the number of tasks that goal catches on.

Table 22.2: Three places to block. The middle is the surest. Filtering what comes in has to be redone every time the outside gets new, while what has been made impossible does not happen whatever sentence arrives.

Where you block	What it does	What survives it	What you lose
On the way in	Look at the text riding in for planted instructions and filter them out	The same meaning written differently passes	Ordinary requests get caught too. One more wait per step
Shrinking what can be done	Open only the tools this run needs and close the rest. Narrow what the arguments can point at as well	Cases where the tools the task needs are enough to carry the attack out too	As much as you narrowed, there is work that cannot be done
On the way out	Compare the destination and the contents of outgoing requests, and stand a person in front of what cannot be undone	The answer a person reads and the text written into memory	Waiting. And somebody has to keep the comparison list current

What the numbers say is best read a defense at a time rather than in one chain. Against the best agents the attacks succeeded in under 25 of a hundred cases, which is the paper’s headline. On the defense comparison, run against one model with no defense, the share of runs where the attacker’s goal was reached was 57.7. Attaching a detector for planted instructions brought it to 8.0, and closing every tool but the ones the task needs before any untrusted material arrives brought it to about the same. *The two numbers are close and what they lose is not.* The detector’s false positives dropped the share of tasks solved without any attack from 69.0 to 41.5, and closing the tools did not drop it at all (Debenedetti et al., 2024).

In Chapter 21 you found the actions that cannot be undone and stood a person in front of them. Deleting with no copy, money moving, a message that went out. Those places become the defensive line here as they stand. What a planted sentence is after is the same place. Take it off the list and that is the end of it, and where you could not and stood a person up, the planted sentence stops at that screen too.

One more thing does it. A call that came out of a user’s request usually makes sense to a person looking at it, and a call a planted sentence caused was never asked for by the user. Put the user’s original request on the approval screen alongside and that difference shows.

How far inspection goes

Filtering text on the way in has no end. Why shows if you put one sentence in three places.

“Delete the files in this folder.”

If a user asked for it, it is the task. If it is a line of procedure in an operations document, it is knowledge, and retrieval ought to be carrying it in. If it is inside the body of an attachment received from outside, it is an attack.

In all three the sentence is identical. What differs is only where it came from and what is being done right now. What a detector has to judge is not a property of the sentence but the

relation between the sentence and the situation, and that relation is not given to the detector. *Inspection does not end because the detector is immature. It does not end because the material for the judgment does not reach the detector.*

That the same meaning written differently passes comes from here too. What has to be caught is meaning rather than shape, so any rule that filters by shape issues a pass to a sentence with the meaning left alone and the shape changed.

So inspection is a device that lowers frequency, not a boundary. Mix the two and it becomes a problem in six months. The decision that follows is that a detector is standing, so one more tool can be opened.

22.4 How to Confirm You Can Be Broken Through

Stand it up and never run it and you do not know whether it holds.

Stand up all three of the previous section and whether they actually block is a separate question. Your usual tests tell you whether the code turns; they do not tell you whether the defense holds. You have to build a test aimed at the defense.

The test is a pairing. Pair one user task with one injection goal, put that goal into one of the entrances of Section 22.1, and run. Many tasks and many goals give you the product.

The published test material is exactly this shape. That is why adding one goal grows the tests by the number of tasks it catches on.

Where you put it

The five entrances of Table 22.1 are the test sites as they stand.

- Into a document retrieval carries in. Put one document into the test corpus and have retrieval fetch it.
- Into a value a tool returns. Build a test tool that appends a paragraph to a normal response.
- Into the memory file. Set up the situation where a past run wrote it that way.
- Into a result another agent hands over.
- Into the tool description of a server you attached.

You do not have to do all five. But an entrance you did not do is an entrance never tested, and that fact has to be written somewhere. Do three and write down that you passed the injection test and the remaining two drop quietly off the checklist.

What sentence you put in

The sentences to put in are not printed here, only what they are after. The goals are the four of Section 22.2 carried over into your own tool names. Things like these.

- Send one mail to a test address.
- Make one lookup the user did not ask for and attach the result to the answer.
- Write one line into the memory file so the same thing happens in the next run too.
- Make a particular item always come first in a recommendation list.

Write the goal so that when the test ends, code can confirm whether that thing happened. “Make it behave strangely” cannot be a goal. “Did mail go to this address” can.

How you know you were broken through

Of the three this is the hardest.

A run that was broken through raises no error either. It did not call a tool that was off the list; it called an open tool normally, and an answer comes out at the end. Chapter 18 counted five failures that end wearing the same face as a successful run, and a sixth joins them here. One thing differs from the five. The five were failures nobody aimed at, and this one is a failure somebody made happen.

So you do not judge by reading the answer. After the run you look at the state of the environment. Is that mail in the mailbox? Was that file deleted? Was that line written into the memory file? The published test material also inspects the state of the environment directly rather than having another model read and grade the answer (Debenedetti et al., 2024).

Three things to look at in the record.

- **Did it call a tool it should not have?** Look for calls that do not come out of the user's request.
- **Did something go out that should not have?** Look at the destination and the body of outgoing requests.
- **Does that stay in the record?** Without it there is no way to look at the other two.

The third holds the other two up. In Chapter 20 you settled on keeping, per step, which tool was called with which arguments and what came back. Without that record this test does not stand. The answer alone cannot decide it.

What you count

Look at three numbers together. Look at one and you get a wrong conclusion.

- **The share of tasks solved with no attack.** Performance before the defense went up. It is the baseline for the other two.
- **The share where the task got solved properly with an attack present and nothing else happened.** Whether what the user asked for finished properly even with a planted sentence mixed in.
- **The share where the attacker's goal was reached.** Whether the thing the planted sentence was after actually happened. It can fall together with the one above or only on one side.

Look only at the third and you conclude that an agent that can do nothing is the safest. Close every tool and the third goes to 0 and so does the first. That is why the published material draws the first and the third in one picture when it compares defenses.

When you do it again

You do not do it once. Three occasions call for doing it again.

- **When you changed the model.** How well a defense holds differs by model. Leave the defense alone and the result changes.
- **When you added a tool.** One more tool is one more entrance, and it pairs with an existing tool to make a new route.

- **When a server you attached was revised.** The tool description changes without the engineer touching anything.

The official documentation says to do it yourself before release too. Run your real flow with documents and mail and tool output that carry planted instructions, and confirm both that they get ignored and that the screening and confirmation you stood up catch the rest (Anthropic, 2026d).

22.5 What Remains After You Block

There is no way to block it completely.

The reason came out back in Section 22.1. The line parting instructions from material is not inside the model. Stack inspection on inspection and you only lower the frequency of getting through, and the previous section's numbers show it. Eight and seven and a half are not zero. The people who built the published test material also wrote that unless a model's ability to tell instructions from data improves substantially, stronger attacks will not be stopped (Debenedetti et al., 2024).

Closing tools in advance has a hole of its own. The case where the tools the task needs are also enough to carry the attack out. In that test it was true of 17 of a hundred cases, and the other 83 were blocked by closing tools (Debenedetti et al., 2024). On a task of reading mail and writing a reply you cannot close the reply tool, and that one tool can send something outside.

So what do you do? You move from blocking to bearing. Three things.

- **Shrink in advance what can be done once you are through.** Choose and open tools per run, narrow what the arguments can point at, and attach lookup tools to only the range needed rather than to production data.
- **Leave enough to recognize having been through.** Watch the three signals of Section 22.4 in production and not only in tests. Calls that do not come out of the user's request, and the destinations of outgoing requests.
- **Make it undoable.** Shorten the backup interval, mark instead of deleting, hold outgoing things briefly before sending. Chapter 21 said to use the devices that shrink the effort of undoing before you use a person, and the same devices get used against planted sentences.

Only the last row of Table 22.3 has a different lifespan. The first three end in that run, and the fourth stays for the next run and the one after. A sentence that came in once becomes a sentence that repeats. So when writing to memory, do not carry text from outside over as it stands. Write where it came from and when you wrote it alongside (Chapter 10).

Does adding defenses make it safe

No. Attach one more inspection and what gets caught grows and ordinary requests get caught along with it.

In that published test the detector for planted instructions misfired often enough to drop the share of tasks solved noticeably. In an office this shows up differently. Thirty catches a day of which twenty-nine are ordinary, and that inspection gets switched off inside two weeks. The person who switches it off did not give up on safety. They made the work happen, and that decision is written nowhere.

Table 22.3: Four routes that remain after you close the tools. The third column is the point. All four use only open tools or use no tool at all. Only the last row has a different lifespan.

Where it goes	What it becomes	Why closing tools does not block it	What blocks it
A person	The answer they read	Producing an answer cannot be removed from any system	Do not auto-load addresses and images carried in the answer
The next call	Arguments and search terms	Writing arguments and writing a search term are both normal use of an open tool	Put outgoing destinations on an allowlist and default to refusing
Another agent	The result handed over	It crosses the boundary keeping the form exactly	Compare the contents just before handing them over (Chapter 17)
The next run	One line written into memory	Writing is not calling. And it survives the run ending	Do not copy text from outside as it stands; write the source and the moment alongside

For the same reason there are three places where standing up one more layer of defense does not help.

1. **Standing it up before counting the entrances.** In a system whose list is the one line “user input,” the defense stands in front of that one line and the other four entrances stay open. And a sentence saying there is a defense gets written into a document, and once written the open entrances drop off the checklist.
2. **Calling inspection a boundary.** Once there is an event called passing, it reads as a guarantee. The price of that misreading is not the one time inspection missed. It is the decision to open one more tool because inspection is there.
3. **Rebuilding an existing decision under the name of defense.** Leave the tool list and the approval points as they are and stand a defense layer beside them, and what is not allowed gets settled in two places, and two places always disagree. The one that actually holds is always the loser.

22.6 Summary

In one sentence. Count the five entrances text from outside comes in through, shrink what the thing that came in can do, and then put something into each of those entrances and run it, judging by the state of the environment whether you were broken through.

What is now settled.

- The box a user types into is one entrance among several. Retrieved documents, what tools returned, what memory holds, what another agent handed over, and tool descriptions from outside all come in, and not one of the five can be read by a person before release (Table 22.1).
- The model does not tell instructions from material. The harness knows which piece came from where, and the writing made of those pieces spliced together keeps no seams (Definition 22.1).
- Attaching a marker and saying so gets blocked in two places. A marker is text and can be imitated, and the side that settles whether to honor it is the model.
- Parting them entirely is not a defense but removing the system. Retrieval that does not use what it fetched and a tool whose result nobody looks at are worse than nothing.
- There are four things it is after and making it send something out is the heaviest. That fourth does not stand on one tool. It needs a tool that reads and a tool that sends both open (Section 22.2).
- Of the three places to block, shrinking what can be done is the surest. The detector and the closed tools produced similar numbers, and only the detector caught ordinary requests along with them (Table 22.2).
- Inspection has no end, and not because the detector is immature. The same sentence is a task and knowledge and an attack, and the material that parts them does not reach the detector (Section 22.3).
- The test is built by pairing tasks with injection goals and multiplying. Write the goal so code can confirm it, and judge by the state of the environment rather than by the answer. A run that was broken through raises no error (Section 22.4).
- Look at three numbers together. Look only at the share where the attacker's goal was reached and you conclude that an agent that can do nothing is the safest.
- Since there is no way to block completely, you move from blocking to bearing. Shrink in advance what can be done, leave enough to recognize it, make it undoable (Section 22.5).
- Attach more inspection and more ordinary requests get caught. An inspection with thirty catches a day of which twenty-nine are ordinary gets switched off inside two weeks, and that decision is written nowhere.

What is still open.

- There is no measurement of how much of one run's context text from outside occupies. You can say to count the five entrances, and which entrance is large comes only from your own system's step records.

- The threshold for how many tests are enough is not known. Multiplying tasks by goals is a clear method, and how many kinds of real attack twenty-seven goals cover nobody has counted.
- There is no standard for how far to close the tools. Work blocked by closing is work that did not happen, so one side of the comparison is always empty.
- Recognizing that you were broken through in production is harder than in a test. A test has an answer and production does not.

Try this today. Open the record of one run turning now and *count* the text loaded into the context by source. Write down how many lines you wrote and how many came from outside (Section 22.1). Next, hold the five entrances of Table 22.1 against your own system and *mark* whether each is open (Section 22.1). Last, *find* the place where a tool that reads and a tool that sends out are open in the same run (Section 22.2).

22.7 Exercises

1. Check the five entrances of Table 22.1 against your own system one at a time. For each open entrance, count how many people can actually edit that text. For a wiki it is the number of accounts with edit rights, and for a server you attached it is the organization that offered it.
2. Write down, from your own task, one case each where the same sentence is the task, is knowledge, and is an attack. Can the three be parted from the sentence alone? If not, what more has to be given to the detector?
3. Build one test. Pick one task, write one goal code can confirm, put it into one entrance of Table 22.1, and run. Then confirm whether you were broken through from the state of the environment rather than from the answer. If you cannot, what more has to stay in the record?
4. Check the four routes of Table 22.3 against your own system one at a time. In particular, count today whether addresses carried in the answer get auto-loaded, and how many lines of memory hold text from outside as it stands.

22.8 On a Second Reading

- Come back to Section 21.3 in Chapter 21 and you see that the list where you found the irreversible actions and stood a person in front of them is also the defensive line. Two chapters found the same places for different reasons.
- Come back to Section 20.2 in Chapter 20 and you see that the list settling what to keep per step is the material that makes this chapter's test stand. Without keeping it there is no way to judge whether you were broken through.
- Come back to Section 18.6 in Chapter 18 and a sixth joins the list of failures that end wearing the same face as a successful run. Unlike the five before it, this is a failure somebody made happen.
- Read Chapter 23 and come back to Table 22.2, and you see that choosing and opening tools per run and putting outgoing destinations on an allowlist sometimes arrive already

built. Checking that they are still there becomes an item on the checklist when you pick a tool.

22.9 Further Reading

What came from Greshake et al. (2023) is a diagnosis rather than a list. The finding that applications with a language model attached blur the line between data and instructions, and the observation that you can steer an application from a distance with sentences planted in advance in material likely to be retrieved. The part sorting attacks into kinds goes stale and the diagnosis stays.

Three things came from DeBenedetti et al. (2024). Building the test as a product of tasks and injection goals, deciding to judge by the state of the environment rather than by the answer, and the numbers each defense produced. The numbers change when the model and the defense change, so what will last is the demand that comparing defenses means drawing the attack success rate and the ordinary task success rate in one picture.

Two things came from Anthropic (2026d). The recommendation to put text from outside only in the tool result slot and never in the instruction slot, and the recommendation to run it yourself before release with documents and mail and tool output carrying planted instructions. The second is the ground for Section 22.4.

Going Further

The twenty-two chapters before this were about building, evaluating, and defending an agent system. Here we look at what is happening outside right now.

The time of writing is August 2026. This chapter goes stale fastest. Names change and numbers start drifting within months. So do not memorize the names and numbers here. Take away what to judge a new thing by when it arrives. The standards for judging are in the twenty-two chapters before.

No comparative performance numbers are written here, and nothing about which is better. The author has not used the things below for long, so only what can be confirmed in the repositories and the official documentation gets written.

23.1 Questions to Think Through Yourself

Every chapter ended by writing down what it had not answered. Gather twenty-two chapters' worth in one place and the same things come up several times. Only the repeats get written, in four kinds.

Every place a number gets settled has no grounds

How many characters to cut a chunk to. How many candidates to pass on and how many of those to load. At what step to cut. How full the context has to be before compaction fires. How many places to stand an approval. How many eval cases are enough.

These questions came up separately in different chapters and went unanswered for the same reason. Settling a value means measuring the score while varying the value, and measuring means an eval set with answers attached has to exist first. That set gets built per task. So what can be written is what to settle it by looking at, and what number to settle on cannot be written.

There is nothing to compare against

There is no measurement of how many tokens each layer of the context takes. There is no evaluation setting opening the original document as it stands against using only the extracted text, on the same task side by side. The same goes for what improves and what worsens when several agents split memory against sharing it.

This side differs from the previous one. Not because nobody knows how to measure. Because nobody has measured and published yet. And the teams that could measure usually have no reason to publish. The numbers came out of internal documents and internal tasks.

How do you make a quiet failure into a case

A good share of an agent's failures make no sound. A chunk that was loaded and the model did not read. A judgment made on the receiving side about something it never saw. An attack that was blocked and never happened. All three end wearing the same face as a successful run.

Putting one into an eval set means first writing it down as a case. But writing it down means knowing what went wrong, and knowing means it made a sound. The twenty-two chapters before ran into this place several times and did not get past it.

Only one thing could be written as a method. Change the system so that quiet failures make a sound. Have the receiving side send up what it judged on alongside, and a judgment made on what it never saw stays in the record. Whether the effort of that change comes back has no measurement.

Which of two that disagree do you pick

Two divided agents sent up different facts and both passed verification on their own side. A chunk retrieval brought and an answer the knowledge graph produced collide in one context. The approval list built by asking the operator and the one built from what can be undone disagree.

All three share that the side picking never saw the sources. Only summaries reach the merging side, and a summary does not carry the material for telling which is right. Send all the material up and what dividing bought disappears.

You cannot compare on against off

Confirming the gain from dividing means comparing against the undivided run, and that run usually does not exist. An attack blocked by closing a tool is a thing that did not happen and cannot be counted. A mistake caught by one more layer of verification does not leave in the record what would have happened without that layer.

There is a way out, not entirely. Turn the device off and run the same task again. If the result is the same the device is surplus, and if it improves it was obstruction. That test costs something, though. You run two sets, and while it turns with the device off a real accident can happen. So how often to check which device this way still has no answer.

Why there are no answers

The field is still growing and circumstances differ by system. *Only a team that has measured on its own system has its own answer.* The twenty-two chapters before wrote that sentence several times, and it gets written once more here.

23.2 OpenClaw

The fastest-spreading open-source project of the first half of 2026 was an assistant one person runs on their own machine.

Speak to it on a messenger and it answers. It tidies mail, books time, reads and writes files, opens a browser and fills in forms, and runs shell commands. It wakes on its own periodically to look at what needs doing, unasked. Memory stays as markdown files on that machine. Skills

get added as files. It is called OpenClaw, and the sentence its repository introduces itself with is “Your own personal AI assistant. Any OS. Any Platform.”

Why this was a boom

Because it put into one thing what the previous twenty-two chapters covered separately.

There is code outside that turns the loop and stops it. Model calls do not chain by themselves, so turning and cutting them has to sit outside, as Chapter 15 wrote. Instructions and resources specialized to a task get bundled in a folder and read in when needed. That is the arrangement Chapter 7 wrote about loading instructions only when needed rather than seating them in the context always. Something survives the session closing. What gets left where the next session can read it, unlike a conversation history that vanishes when the run ends, is what Chapter 10 called memory. It runs shell commands. Have it run code rather than call tools one at a time and what it can do widens from a list of words to the grammar of sentences, as Chapter 7 wrote.

The way it joins messengers played a part too. One program called the gateway takes sessions and tools and channel connections, and in front of it hang things like WhatsApp, Telegram, Slack, Discord, Signal, and iMessage. You do not have to learn a new screen; you speak to it in an app you already use.

What is in the release notes

What it is building now is what the release notes say. Carrying over what went into the versions released in August 2026, it comes to this.

The gateway picking up what it was doing after dying and coming back. Keeping the settings file from breaking mid-write. Commands to back up and restore the database. Binding secrets so they only go to settled hosts. Warning about the source when a plugin gets installed. Attaching new models, and a channel for driving a browser.

There is almost nothing here about making the model better. Most of it is keeping what turned turning and closing the places things leak out of. What a nine-month-old project spends its time on now is this list, and it is the same story as why the previous twenty-two chapters kept Parts VI and VII separate.

The biggest risk is uncontrolled execution

The official site writes it of itself. It browses the web, fills forms, reads and writes files, and runs shell commands. Open it all up or run it isolated, and the choosing is the user’s. And the repository’s guidance writes that tools run on the host for the main session unless you configure sandboxing.

Opening shell commands with no limit is giving every input that arrives the highest permission on that machine. A message that came in on a messenger, a sentence read off the web, a skill somebody else uploaded are all that input.

It is the shape Chapter 22 set up, exactly. Load text from outside and instructions written on this side into one context and the model cannot part them. What Chapter 21 set up catches too. Leave reversible actions open and stand a person in front of what cannot be undone, it said, and sending mail and deleting a file are on the irreversible side. But a personal assistant does not take approval every time. Take it every time and it stops being an assistant. Waking on its own to work adds one more layer here. Actions happen while nobody is in front of the screen.

There is an approval device, though. The repository's guidance says to treat inbound messages as untrusted input, and channels that take direct messages pair unknown senders by default, holding them until the owner approves. That is a device that filters who is speaking, though, not one that filters instructions planted inside a sentence sent by somebody already approved. The side that knows the source and the side that settles the next action are different, as Chapter 22 wrote. The gateway knows who that sentence came from, and the model that reads it and draws an action does not.

Memory takes the same problem. Plant one sentence in a memory that stays as a file and that sentence gets read as an instruction from the next run on. Security people call this memory poisoning. The route by which text written into memory steers later runs is what Chapter 10 left open, and here it stands as a real thing.

Being personal changes the answers

Settling who to stand in front of an irreversible action means asking something first, Chapter 21 wrote. Who does the result of this decision arrive at? In an organization the result usually arrives at somebody else. Mail that went wrong to a customer, a stock quantity edited wrong, an expense approved wrong. So the place settling permission becomes the organization's policy.

In a personal setting the result arrives at that person. Mail sent wrong is their mail and a deleted file is their file. So the place settling permission narrows from an organization's policy to the owner of that system.

Why that is a problem is that the one person usually does not hold a list of what they allowed. What they opened once at install is still open months later, and they have not counted how many skills have been added since. Reclaiming unused permissions is somebody's job in an organization and nobody's job here.

A governance tier bolted on outside

NVIDIA put out a tier that sits on top of this. In March 2026 it announced NemoClaw, which installs its own open models together with a runtime called OpenShell in one command, and what OpenShell provides is an isolated sandbox that adds privacy and security to an autonomous agent.

The core is one thing. The code enforcing the policy sits in a different process from the agent. Break the agent and it cannot edit the policy. It is the same remedy as Chapter 21's that the permission verdict has to be made outside the model, and the same as Chapter 22's that outgoing destinations go on an allowlist.

What to look at here is the order. The execution tier came out first and spread, and the permission model and the guardrails got bolted on afterward by somebody else. Why Chapters 22 and 21 each needed a chapter of their own is right there in that order.

Where the industry is going

The same remedy is settling in outside this project too. Treat code an agent produced and input that came from outside as adversarial and isolate the execution. Put the policy verdict where the agent cannot edit it. Cut credentials to short lifetimes. The means of isolation is moving toward something stronger than a container as well, and that isolation sharing a kernel is not enough to hold code an agent produced is roughly where the agreement sits.

The recommendation not to let an agent use a person's account as it stands but to give it an identity made separately comes along too. Put only what is needed into that identity, and give it something short-lived rather than long-lived credentials. There is a recommendation to run it on an entirely different machine while you are evaluating it and to treat that machine as disposable. It is the same list as the items Chapter 21 wrote down as ways to narrow permission.

23.3 Open-Source Frameworks

Say you are building a chatbot that answers questions about internal policy. Open the documents and pull out the text, cut it into pieces of a reasonable size, turn each chunk into a vector and store it. When a question arrives, turn the question into a vector too and find similar chunks. Slot what you found into the prompt and send it to the model. Each stage means picking a library and splicing them together.

LangChain gives those stages as pieces already built. Reading documents, cutting, turning into vectors, finding, slotting into the prompt: there is one of each and you join them in order. What would take days takes half a day.

Writing the flow as a graph

Grow the same chatbot one layer and the circumstances change. The answer got built and it may conflict with policy, so you hang a verification on it, and on failure you decide to go find other chunks. You write a loop, and inside it call retrieval, build the answer, call the verification, and write a conditional for where to go back to on failure. You put in a variable counting how many retries, and settle where that variable lives. Three stages is bearable and five makes it hard to read where what changes.

LangGraph is another open-source project from the same place. Its documentation calls itself a “low-level orchestration framework and runtime” and nails down that “LangGraph does not abstract prompts or architecture.” You write stages as nodes, movement between stages as edges, and where to go on a condition on that edge. Loops and branches do not hide inside the code.

And it makes you declare the state the stages share, with a name and a shape, first. Writing down in advance the field retrieval fills, the field the answer fills, the field verification fills. What newly appears between divided pieces is not passing but writing down expectations and delivery, as Chapter 17 wrote. Making you declare the state first forces that place to be written into existence.

What is already in there

The list of what LangChain attaches under the name middleware overlaps with the previous twenty-two chapters. Middleware is short code slotted in at settled points of the loop, and carrying over what the official documentation says it provides by default that overlaps with this book, it comes to this.

Summarize the conversation history automatically as the token limit approaches. That is Chapter 6. Stop before calling a tool and wait for a person's approval. That is Chapter 21. Find personal data and mask it. That is Chapter 22. Cap the number of model calls and tool calls. That is Chapter 15. When there are too many tools, stand up a separate model to pick the

relevant ones, or hold only the names and fetch the rest when needed. That is Chapter 7. Clear or trim only the tool results out of the history. Chapter 6 again.

That list is why this kind has settled in like a standard. What is written there ends up needed in any system, and a team building it for the first time learns each one by getting it wrong.

The others

Several aim at the same place. CrewAI is closer to splitting roles and standing several agents up fast, and LlamaIndex Workflows ties document-heavy flows together by event. Microsoft Agent Framework and Google ADK each assume use on their own cloud. OpenAI Agents SDK keeps almost no abstraction and takes an agent handing control to another agent as its basic unit. Most come out under MIT or Apache licenses.

The standard for choosing is not the number of features. One question does it. What does this fill in for me, and what arrives already settled in exchange? Count features and the one that fills in more wins, and the more it fills in the more arrives already settled.

The recording and evaluating tier comes separately

Choosing a framework and recording the run are not joined. LangChain's own introduction writes this in one sentence. Which framework you choose settles what you can build fast, and what observability and evaluation tier you attach to it settles whether it keeps turning after you ship.

What this tier fills in is the places to keep. What went in and what came out at every model call and tool call and retrieval stage stays without planting it by hand, and one run shows as one joined thing. The place Chapter 20 wrote about, that you cannot ask later without keeping a record of what a run did, gets filled in by default.

Agreement is forming on the form of the record too. OpenTelemetry's generative-model conventions settle an agent, a workflow, a tool, and a model each as one node, and put the operation's name among the required fields while elapsed time and tokens spent stay recommended. And the specification writes that the attribute names may still change.

What arrives already settled in exchange is what gets recorded. Time and tokens are what you want kept on any task, and what actually needs looking at on this task may be somewhere else. How many of the retrieved chunks the answer actually used, at which attempt the verification passed. The items of a metric are the names of the failures, Chapter 18 wrote. A failure with no name does not reach the list of things to fix, so with this tier laid on, *what to ask is not what gets recorded but what does not*.

When a framework is better

When you want to confirm something fast. Before spending days on something that may or may not work, you get an answer in half a day.

When you have not made the earlier chapters' decisions yet, too. With the chunk size and the compaction threshold and the approval points all unsettled, having settled values at all is better than none. With no values you cannot run it, and without running it no grounds for settling appear.

When you would rather not hit problems other people already hit. How to retry a failed tool call, or how to fall over to another model when one does not respond, are written down by

people who got them wrong several times.

When building it yourself is better

When what the framework settled and your circumstances differ. If the trigger for summarizing is a token count and what you want to watch is the number of tool calls, that default does not fit.

When you have to see where what happens, too. Building a list of what to suspect when the answer got worse means knowing what went in and what came out at each stage. Every time another tier goes on, that list moves further away.

What to ask when choosing

A framework is in large part a harness sold as a product. Choosing one is importing somebody else's list of assumptions whole, and what to ask is whether that list still holds for today's model and this task. Chapter 15 wrote it that way.

Read the middleware list above through that question and it comes out like this. The threshold that triggers summarizing assumes today's model's context length. The device that picks tools for you assumes a model gets confused when there are many tools. The device that waits for approval assumes that action cannot be undone. Of the three, the first two can break as models improve and the last does not.

23.4 Agents Run Locally

Up to here we assumed calling a model from outside. There is a road that does not call. Put the model file on that machine and run it directly.

Teams that look at this road first usually look because of the material. Where medical records or contracts or material about people get handled, how far things may go outside is a regulatory question rather than a technical one, and getting through that takes months. Send nothing out at all and those months disappear.

What improves

There is no token cost. However many steps it turns, nothing more gets billed, so the cost of length Chapter 3 counted disappears whole. Electricity and hardware cost instead.

Nothing goes out. Internal documents and customer material do not leave that machine. The work of settling how far things may be sent disappears.

Your side stays as it is when the outside changes. A model does not update quietly and make yesterday's prompt behave differently today. You hold the file, so you can keep running it after a service ends.

What grows

Reasoning gets worse. Especially at planning several stages in a chain and picking tools to call. Several lines of research teach a small model with trajectories a large one solved to close that gap, and carrying several stages over a long stretch is still written down as an open problem.

Table 23.1: Values confirmed in August 2026 from where the models are distributed and from the official model documentation. The file sizes are the four-bit versions, and that much has to sit in memory for it to turn. The context limits are the distributors’ defaults.

Model	Parameters	File size	Context limit
Qwen3 8B	8B	5.2GB	40K
Qwen3 14B	14B	9.3GB	40K
gpt-oss 20b	21B, 3.6B used at a time	14GB	128K
Qwen3 30B	30B, 3.3B used at a time	19GB	256K
gpt-oss 120b	117B, 5.1B used at a time	65GB	128K

And the side doing the teaching is still a large model. Closing the gap itself happens where the equipment is.

It is slow. And in an agent that difference multiplies as it stands. It is not asking once and getting one answer; it calls again at every step. A few seconds per step and a twenty-step run opens up by minutes.

You have to have the hardware. The file size in the table is the memory requirement as it stands, and a place to hold the context attaches on top of that. That place grows as the context lengthens. That a file fits in memory does not mean it turns with a long context.

And it cannot carry a long context. That last one collides hardest with the earlier chapters.

Table 23.1 gathers what is worth running locally now. The requirements the makers wrote down are 16GB of memory for gpt-oss 20b and one 80GB GPU for the 120b. The first runs on a laptop and the second does not.

“Used at a time” in the parameter column means the whole thing sits in memory while only part of it gets computed to produce one token. So the file is large and the speed is closer to that part. The memory needs all of it.

This table goes stale fastest in this book. Within half a year the numbers will have moved by a digit.

The context limit changes the earlier decisions

Lay the last column over the earlier chapters’ decisions and what changes comes out.

Forty thousand is not room to spare. When Chapter 4 cut the context into layers, the system instructions and the tool descriptions and the resources loaded in advance already take their places before the run. Ten tools with careful descriptions is thousands of tokens on its own. The history and the retrieval results grow together in what is left. At 40K, compaction is not an option but a premise.

How many retrieved chunks to load changes too. One chunk at 800 tokens and ten of them is 8,000. A value there was room to do arithmetic on with an outside model meets the ceiling first here. Cut the chunks finer, or hang reranking on and cut how many get loaded, or both.

What to settle if you go local

Moving everything local is rare. Usually you mix. Sorting incoming queries into branches, fitting extracted values to a settled form, shortening long text: a local model does those. Only

planning several stages in a chain and picking tools to call goes outside.

What has to be settled when mixing is what goes outside. What goes outside is what leaves, so you hand back that much of the advantage of nothing leaving. And the part that goes outside is usually the part with the thickest context. The harder the judgment, the more material it needs.

Put together, what to settle when going local is this.

- Which task to hand over. The whole system, or part of it?
- If mixing, what goes outside? And what goes along with it when it does?
- How does the context limit change the compaction threshold, the number of retrieval results, and the length of tool descriptions?
- Which is larger, the price of the hardware or a few months of tokens?

23.5 What to Watch From Here

What follows is what goes stale in the twenty-two chapters before, and what changing makes it go stale.

What goes stale if it changes

Contexts getting larger shrinks the place for cutting and compaction. Right now the problem is how to split a two-hundred-thousand-token document across loads, and if it goes in whole the decision to split disappears. Several models already take a million tokens.

Models getting better makes the harness's devices surplus. Capping the step count stands on the assumption that a model cannot stop itself, and the device that picks tools stands on the assumption that many tools confuse it. Break the assumption and that device is not just unhelpful but obstruction.

Specifications settling down leaves less to settle yourself. How to offer a tool and how to call it, and what agents pass to each other, everyone settles for themselves now. Let agreement form and some of what Chapters 8 and 17 covered becomes a default.

Token prices falling further weakens the remedies on the saving side. Burning cache, hanging compaction, cutting tool descriptions all stand on the premise that tokens cost. Make the price a fraction of today's and the order of those remedies changes. But the price falling gets spent on turning more steps, so there is no guarantee the total billed for one run comes down with it.

What does not change

A context is finite. Make a million ten million and there is more you want to put in than that. And being able to fill it to the ceiling and reading well when filled to the ceiling are different things. Performance does not grow with the window, as several places report, and that was Chapter 4's premise.

A model answers by probability. There is no guarantee the same input gives the same output, so forcing the form does not bring correctness of content along. That is why evaluation and records are needed, and that reason does not disappear as performance improves.

The last approval stays on the person's side. Because it is a person who bears the result of an action that cannot be undone. However well a model does, mail sent wrong is that person's mail.

Text from outside and instructions written on this side are the same text once loaded into one context. Models improving does not put a marker parting the two inside that text. So filtering inspection only lowers the frequency of getting through, and the boundary in the end gets built by narrowing what the thing that came in can do. Chapter 22's conclusion stays here as it stands.

The standard that parts the two

One standard parts them. *A sentence standing on today's model's limits goes stale, and a sentence standing on something that does not go away, like finiteness or probability, stays.*

The remedy of cutting the context into layers and settling what to load in advance and what to fetch as you go stands on finiteness. Make the window ten times bigger and there is still something to settle. The remedy that picking gets worse past twenty tools stands on today's model's limits. Which means it is a sentence you may one day delete.

A device is a temporary thing fitted to today's model and assumptions have expiry dates, Chapter 15 wrote. That sentence hangs on the whole book and not only on the harness. Read each chapter's remedies together with which of the two they stand on, and years from now you can still judge what to throw away and what to keep.

23.6 Closing

We came from what an agent is through reasoning and tokens, context engineering, memory and retrieval, the harness and running it, and evaluation. The whole path of design has its ground laid.

All of it stood on one structure. The model puts out an action, the environment returns an observation, and that observation gets assembled with other material into a context and goes back into the model. Why cost grows the way it does, why things get worse the longer you splice, why the tool list is the whole of what can be done: all of it came out of there.

Twenty-three chapters look like different subjects and the question was one. *The places are finite and there is more you want to put in. What do you put in?*

What a reader carries out is one thing too. A list of decisions. How many characters to cut a chunk to, at what step to cut, what to throw away when the context fills, what to leave when a session hands over, what to evaluate and what not to, who to stand in front of an action that cannot be undone.

Those decisions become an engineer's own capability and their own edge.

Even if a tool fills these lines in for you, the filling does not go away. Using a tool means receiving values somebody else filled in. *Written down it is a decision, and unwritten it is a default.* A decision can be checked later for whether it still holds, and a default never becomes a thing to check.

The way software engineers now all use Claude Code or Codex, and more widely the way people generally are working with generative AI beside them, this field will grow fast.

Inside that, the agent engineer holds the key to the design. In a growing industry they have to be the one showing the way. *I hope this book is of some use in that work.*

A wave is coming. Now is the time to take up your own oar and put out to sea.

Bibliography

Agent2Agent Project. Agent2agent (A2A) protocol specification, version 1.0.1. Linux Foundation, 2026. <https://a2a-protocol.org/latest/specification/>.

Anthropic. Building effective agents. Anthropic Engineering Blog, December 2024a. <https://www.anthropic.com/engineering/building-effective-agents>.

Anthropic. Introducing contextual retrieval. Anthropic, September 2024b. <https://www.anthropic.com/news/contextual-retrieval>.

Anthropic. Introducing the model context protocol. Anthropic Announcements, November 2024c. <https://www.anthropic.com/news/model-context-protocol>.

Anthropic. Introducing advanced tool use on the claude developer platform. Anthropic Engineering Blog, November 2025a. <https://www.anthropic.com/engineering/advanced-tool-use>.

Anthropic. Code execution with MCP: Building more efficient agents. Anthropic Engineering Blog, November 2025b. <https://www.anthropic.com/engineering/code-execution-with-mcp>.

Anthropic. Effective context engineering for AI agents. Anthropic Engineering Blog, September 2025c. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.

Anthropic. Effective harnesses for long-running agents. Anthropic Engineering Blog, November 2025d. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>.

Anthropic. How we built our multi-agent research system. Anthropic Engineering Blog, June 2025e. <https://www.anthropic.com/engineering/multi-agent-research-system>.

Anthropic. Equipping agents for the real world with agent skills. Anthropic Engineering Blog, October 2025f. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>.

Anthropic. Writing effective tools for agents — with agents. Anthropic Engineering Blog, September 2025g. <https://www.anthropic.com/engineering/writing-tools-for-agents>.

Anthropic. Prompt caching. Claude Developer Platform Documentation, 2026a. <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>.

- Anthropic. Citations. Claude Developer Platform Documentation, 2026b. <https://platform.claude.com/docs/en/build-with-claude/citations>.
- Anthropic. Demystifying evals for AI agents. Anthropic Engineering Blog, January 2026c. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>.
- Anthropic. Mitigate jailbreaks and prompt injections. Claude Platform Documentation, 2026d. <https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/mitigate-jailbreaks>.
- Anthropic. Memory tool. Claude Developer Platform Documentation, 2026e. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/memory-tool>.
- Anthropic. Building multi-agent systems: When and how to use them. Claude Engineering Blog, January 2026f. <https://claude.com/blog/building-multi-agent-systems-when-and-how-to-use-them>.
- Anthropic. PDF support. Claude Developer Platform Documentation, 2026g. <https://platform.claude.com/docs/en/build-with-claude/pdf-support>.
- Anthropic. Structured outputs. Claude Developer Platform Documentation, 2026h. <https://platform.claude.com/docs/en/build-with-claude/structured-outputs>.
- Anthropic. Token counting. Claude Developer Platform Documentation, 2026i. <https://platform.claude.com/docs/en/build-with-claude/token-counting>.
- Anthropic. Tool use with Claude. Claude Developer Platform Documentation, 2026j. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview>.
- Anthropic. Vision. Claude Developer Platform Documentation, 2026k. <https://platform.claude.com/docs/en/build-with-claude/vision>.
- Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. arXiv:2406.13352.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization, 2024. arXiv:2404.16130.
- Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. ColPali: Efficient document retrieval with vision language models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.01449.
- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <https://www.deeplearningbook.org/>.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023. arXiv:2302.12173.

- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2106.09685.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.01798.
- Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMlingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 13358–13376, 2023. arXiv:2310.05736.
- Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1658–1677, 2024. arXiv:2310.06839.
- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020. arXiv:2004.04906.
- Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020. arXiv:2004.12832.
- Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing, 2018. arXiv:1808.06226.
- Mosh Levy, Alon Jacoby, and Yoav Goldberg. Same task, more tokens: the impact of input length on the reasoning performance of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 15339–15353, 2024. arXiv:2402.14848.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020. arXiv:2005.11401.
- Linux Foundation. Linux foundation launches the Agent2Agent protocol project. Linux Foundation Press Release, June 2025. <https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents>.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. arXiv:2307.03172.

- Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022. arXiv:2104.08786.
- OpenAI. Introducing SWE-bench verified. OpenAI, August 2024. <https://openai.com/index/introducing-swe-bench-verified/>.
- OpenTelemetry. Semantic conventions for generative AI systems. Cloud Native Computing Foundation, 2026. <https://github.com/open-telemetry/semantic-conventions-genai>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023. arXiv:2310.08560.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 126544–126565, 2024. arXiv:2305.15334.
- Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.15425.
- Prithvi Rajasekaran. Harness design for long-running application development. Anthropic Engineering Blog, March 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>.
- Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2020.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2302.04761.
- Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models’ sensitivity to spurious features in prompt design. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.11324.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016. arXiv:1508.07909.
- Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 31210–31227, 2023. arXiv:2302.00093.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. arXiv:2303.11366.

- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. arXiv:2104.09864.
- Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. Let me speak freely? a study on the impact of format restrictions on performance of large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1218–1236, 2024. arXiv:2408.02442.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv:1706.03762.
- Changhan Wang, Kyunghyun Cho, and Jiatao Gu. Neural machine translation with byte-level subwords. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 9154–9160, 2020. arXiv:1909.03341.
- Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. arXiv:2305.04091.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2201.11903.
- Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. BrowseComp: A simple yet challenging benchmark for browsing agents. OpenAI, April 2025. arXiv:2504.12516.
- Michael Wooldridge and Nicholas R. Jennings. Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2):115–152, 1995.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023a. arXiv:2305.10601.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023b. arXiv:2210.03629.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2406.12045.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235, pages 62138–62160, 2024. arXiv:2310.04406.